MUNCHIES FURKIDZ · HANDMADE PET TREATS · MALAYSIA

Munchies Furkidz - change history

Every version of the backoffice app, from v54 to v672 - newest first. The green 'Engine v##' pill on the More screen tells you which build a phone is running.

About this history: it honestly begins at version 54, when the two-phone shared-cloud era started (2 September 2026). The app grew through earlier versions before that, but they were never written down version by version, so they are left out rather than invented.

What changed in each version of the backoffice app, newest first. Each version number is the "Engine" you can see on the app's **More** screen, so you can always tell which build a phone is running.

27 Oct 2026 - engine v672: THE PRODUCT SYNC NAMES THE ROW IT REFUSED (nothing to run, no SQL, nothing to upload).

The same 400 came back on a build that already had the v671 fix, so v671 did not cover every path.

1. A SECOND ROW COULD ARRIVE WITHOUT ITS NUMBERS. One batch exists only to drop a pack's old pool columns off a product that stopped being a pack - and when that row's product and day were not already in the table, the "update" became an INSERT of two columns with nothing in slots_left, which the database refuses. That row carries its own figures now.

2. AND A REFUSAL NAMES ITS ROW. ? **"null value in column slots_left" told you nothing you could act on** - not which product, not which day. **Now, when the database refuses the batch, each row is sent on its own and the message names the ones it refused** (*"Chicken Jerky (2026-10-28) - ..."*). **This only runs after a failure**, so a healthy sync costs nothing extra.

If the message still appears, read the product and day it now names - that is the row to look at, and tell me what it says.

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v671: ONE BAD NUMBER CAN NO LONGER STOP EVERY PRODUCT'S COUNT (nothing to run, no SQL, nothing to upload).

Your report: "Sync failed: Product sync failed (HTTP 400) - null value in column slots_left violates not-null constraint".

1. WHAT IT WAS. The fault was in the sending, not in your database. A count is worked out from a product's daily limit and what is already ordered; if one of those figures had gone bad, the sum became a number the app cannot send - and the moment it is sent, JSON turns it into an EMPTY value. The database refuses an empty value in that column, and because all your products are published in ONE call, that single row rejected every product's "only N left" at once.

2. WHAT IT DOES NOW. Nothing that is not a real number can be sent: a broken figure is published as **0** - "sold out" for that one product, which can never oversell - and the rest of the shop updates normally. **A pack size of zero can no longer divide by zero either.**

3. AND IT TELLS YOU WHICH ONE. ? **"Check my system" gained a check, "Every count the shop publishes is a real number", which NAMES the product or order carrying the bad figure** - because "sync failed" without a name is a fault you cannot act on. **Open that product and type its limit again as a whole number.**

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v670: THE CHARGE IS CALLED "POSTAGE" EVERYWHERE (nothing to run, no SQL, nothing to upload).

Your instruction: "make the order card say postage".

1. WHAT WAS WRONG. Your own order card said "Courier charge (optional)" while the customer's message and the shop said something the setting could change - two words for one figure, and the word "courier" on a shop that posts parcels.

2. WHAT IT SAYS NOW. The order card's box, the refund row, the invoice's cash-on-delivery note and the shop's own card all say "Postage". ? **And the box follows the setting**: if you rename the charge in **Settings -> Delivery charge -> "What the charge is called"**, your order card and the customer's message move together. **The shop's own two sentences are translated in the shop's files**, so a rename does not reach them - that is written down where they live.

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v669: THE SHOP'S MALAY CALLS THE DAY "HARI POS" (nothing to run, no SQL, nothing to upload).

Your instruction: "change the malay to hari pos".

1. WHAT WAS WRONG. Two sentences still said "hari membakar" - the bakery's word for a BAKING day - while the rest of this shop's Malay already said "hari pos" everywhere else: the step, the calendar, the sold-out line and the "what is a posting day" note were all right, and these two were the stragglers. **A shop that dehydrates was telling some customers it bakes.**

2. WHAT IT SAYS NOW. The facts panel and the closed-day sentence both say "hari pos". A test fails if "hari membakar" OR "hari penghantaran" comes back, and it requires "hari pos" in both places - so neither can creep back in one sentence at a time.

3. WHY THIS IS A VERSION OF ITS OWN. The copy change shipped as a separate commit under the same number 668 that had already been pushed half an hour earlier - so a phone that had loaded 668 before it would read "668" and quietly hold the OLD Malay, and the update strip, which compares that number, could not tell. ? **A sticker has to move whenever what a phone can see moves**, or the sticker stops being evidence.

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v668: A COLLECTION HAS A DAY AND A TIME, A QUOTE CAN CARRY YOUR BUFFER, AND A SAVED CHARGE STAYS SAVED (nothing to run, no SQL, nothing to upload).

Synced from the bakery, v655 to v668. Munchies takes the engine's number on every sync, which is why this one jumps from 656 to 668.

1. A COLLECTED ORDER NOW RECORDS WHEN THE CUSTOMER COMES. Both order cards - + New order and the order's Edit - carry "Collection day and time". Blank means the posting day with no time, exactly as it always read, so an order you never touch is unchanged.

2. AND THE MESSAGES SAY IT IN THE RIGHT PLACE. The posting-day line used to be able to carry a time - and that time was the VAN's, left on an order that had once been posted. A real confirmation read Posting day: Wed, 28 Oct, 6-7:09 pm for a customer who was collecting. **The day is a day now, and a collection's time sits on the Collection line, where it belongs.** A collected order also stops being called a "Delivery": the second line says Collection: Thu, 29 Oct, 6pm.

3. TYPING A TIME ON A PHONE NOW STICKS. One of those boxes listened for only ONE browser event, so a phone keyboard, an autofill or a paste could save the day and lose the time. Both events write now.

4. SWITCHING POST ? COLLECT NO LONGER THROWS YOUR PICK AWAY. The collection Point you chose stays on the card, so switching back restores it. The saved order is unchanged: a posted order still carries no Point.

5. A QUOTE CAN CARRY YOUR BUFFER, AND A CHARGE YOU HAVE SAVED IS FINAL. Settings -> Delivery charge holds three answers: a **delivery window** you promise when an order carries none (leave it blank and the message says the time is still to come - which is the shipped default here, because munchies posts parcels), a **buffer in ringgit** added to a courier's own quote when you take it into the charge, and **what the**

charge is called on the customer's message and receipt. **A charge already saved is not re-priced by a later booking** - the run skips it and says so. A figure you type yourself is never buffered.

6. "CHECK MY SYSTEM" FINDS A COURIER DAY EARLIER THAN THE POSTING DAY. A day typed before the parcel exists puts a delivery on the run on that day, and a run then shows an order out of nowhere. The check names every order, and one press clears that day - nothing else on the order is touched. **And a passing check now says what it looked at** ("12 orders carry a courier day..."), because "no error" is not evidence if the check cannot say how many days it examined.

7. THE SHOP'S RELOAD PRESS RE-ASKS FOR THE FILE, NOT JUST THE PAGE (the same fix this shop got as v656, now in the shared engine's own shape, with a test that drives it).

ONE THING WHICH IS THE REAL POINT OF THIS VERSION. Three of these faults were **boxes that took what you typed and never saved it**, and every one was found by you, not by the tests. **There is now a guard that does what a person does: types into every box on the order cards, presses Save, opens it again and checks every value came back - and it fails if a box appears that it does not check.** ? **It found one more before this build shipped.**

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v656: THE SHOP'S RELOAD PRESS RE-ASKS FOR THE FILES (nothing to run, no SQL, nothing to upload).

Your words: "it fail to refresh again on browser."

1. The build was not the problem. v655 was already pushed and the site was serving it. ? **What failed was the pressing of Reload** - and the reason is worth keeping.

2. What was actually wrong. The shop page and every file beside it are fetched separately, and the browser is allowed to keep its own copy of each for ten minutes. ? **A plain reload re-asks for the PAGE and takes everything else from that ten-minute copy** - so the page came back new with **the old stylesheet still squeezing the card**, and a second press could not help either, because it does the same thing again. **v655 changed only the stylesheet, which is exactly the file that stayed behind.**

3. What it does now. Pressing Reload first re-asks for the page's own files - its scripts, its stylesheet, its fonts - and only then reloads, so the reload finds the new copies instead of the old ones. **The list is the page's own** (the browser already knows what it loaded), **never a written-out list of filenames that would go stale the first time a file is added.** ? **Photographs are left alone: a build number never names one, and re-downloading ten product photos would spend a customer's data to fix something it cannot fix. And it is raced with a two-and-a-half-second beat**, so a slow connection never leaves a customer pressing a button that appears to do nothing.

4. One thing to know. A page that is ALREADY old cannot contain this fix - the old page's button is the old one. **So the first time you update past this version, press the reload on the strip once; if the page still looks old, close the tab and open the shop again.** From then on the strip's own press is enough.

I drove the press in a browser: it re-asked for 35 files and none of them were photos, then reloaded onto the new page. The suite is 4000 tests passing.

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v655: THE PRODUCT CARD READS PROPERLY ON A PHONE (nothing to run, no SQL, no upload).

Your words, with the picture: "on phone the product card look like this, it should not be".

1. What was wrong. The card is a photo beside the words, and inside the words a second row of price and quantity. On a phone the photo took 120 of the ~380 pixels, and the price and quantity took most of what was left - so the description was squeezed into about 90 pixels and printed ONE WORD TO A LINE, and "ONLY 10 LEFT" broke into three lines. Measured, not guessed: at 412 pixels the words had 90 pixels.

2. What it does now. On a phone the price and quantity drop to their own line under the words - the same two facts, the same row shape, and the words get 252 pixels instead of 90. The photo is smaller too (88 pixels), because the words need that space more than the picture does.

3. And on a very small phone the price and quantity wrap rather than run off the side of the card - **measured at 320 pixels, where they wanted 194 and the space beside the photo was 158.**

I checked it at 412 and at 320 in a browser: no card overflows, and the page never scrolls sideways.

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v654: A COLLECTED ORDER'S OWN DAY IS CALLED A COLLECTION (nothing to run, no SQL, no upload).

Your words: "munchies also has collect date if defer from post date."

1. One field, two meanings - and one word was wrong. The same date carries "the day it reaches them" for both ways an order leaves: a van's day for a posted one, and the COLLECTION day for one being collected (a Point order gets one when the trip is booked). The line added yesterday always said "Delivery", so **a collecting customer was handed the wrong word for their own day.**

2. It follows the way the order leaves. "Delivery 30 Oct" for a posted order, "Collection 30 Oct" for a collected one - and only when that day differs from the posting day, so an ordinary order's paper is unchanged.

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v653: THE CUSTOMER'S PAPER NAMES THE DELIVERY DAY WHEN IT IS A DIFFERENT DAY (nothing to run, no SQL, no upload).

Your model, in your words: "order date = for merchant only; bake date/post date = to let customer know when is the cooking done; delivery date = if customer choose some date other than the 2nd date."

1. Two of the three were already in the app. The order date is yours and is off the customer's paper since v652. The posting day is the one the customer reads - on the invoice and on the label.

2. The third was missing from the customer's paper. The parcel label has printed both days, each named, since v429 - *"so whoever packs a bag sees when to make it and when it leaves"*. But the paper that goes to the CUSTOMER named only the posting day, so a customer whose parcel was arriving on a different day was told when it was posted and never when it would reach them.

3. It is named now - and only when it is a different day. An order with no second day prints exactly what it printed before, and a delivery day that is the same day is not printed twice under two names.

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v652: A CUSTOMER'S PAPER SHOWS THE DAY THEY CARE ABOUT (nothing to run, no SQL, no upload).

Your words: "for customer side, their concern is post date, not order date, order date is for merchant."

1. The paper had the merchant's date on it. The invoice - the one that goes in the parcel and the one that prints on the label - carried the day the ORDER was placed. That is your date, and the least useful one to the person holding the box.

2. It carries the POSTING day now - the day the parcel is going, which is the day they are waiting for. The placed day has not gone anywhere: it is on the order's row and its card, where you work.

3. And an order with no day at all still prints no date, rather than a broken one - unchanged, and now tested as "no day of either kind" rather than "no placed date".

Nothing to run, no SQL, nothing to upload.

27 Oct 2026 - engine v651: THE ADDRESS LOOKUP HAD STOPPED ASKING GOOGLE AT ALL, AND NOTHING SAID SO (nothing to run, no SQL; the two functions are already uploaded).

This one is ours, and it is the reason your addresses stopped landing on the door.

1. What was happening. A version in September moved your Google key from the web address into a request header, to keep it out of the server's logs. **But Google's older address service cannot read a key from a header at all** - it answered *"you must use an API key"* to every single request. **The lookup treats any Google refusal as "ask the free map services instead", so nothing failed on screen: every lookup since has quietly been answered by OpenStreetMap, which holds Malaysian roads and no house numbers. That is the list you were seeing.**

2. How it was found. You asked for "Google Maps autocomplete", so a small diagnostic was added that asks Google the same question the lookup asks and reports Google's own answer. It said REQUEST_DENIED - "you must use an API key" - on a key that was plainly there. One command, and the fault named itself. ? **A silent fallback is a fault that survives every test: the app worked, the lookup answered, and the answer was from the wrong service.**

3. The fix. The key goes back where Google's address service reads it. And the protection that motivated the move is kept by a different means: the key is scrubbed out of any error text before anything can be written down, and a guard now fails if any code in the function prints or returns a web address. **Your key has not been in a log.**

4. The type-ahead now uses Google's newer Places service, which *does* take a header - ported from the bakery's own working version rather than written fresh.

5. And the courier's lookup had the identical fault, silently, for the same reason. **Fixed in the same pass.**

BOTH FUNCTIONS ARE ALREADY UPLOADED, so the fix is live for your customers now. The shop page still needs your push for the live type-ahead list - until then the box behaves as it does today, but with Google's answers again.

26 Oct 2026 - engine v650: THE ADDRESS BOX NOW SUGGESTS LIKE GOOGLE MAPS (TWO STEPS FOR YOU - see the end).

Your words: "i found the the address auto complete like not working well" - and then "i want it to be like google maps autocomplete".

1. What was happening. The shop was asking Google for a DOOR, not for a list - and that question needs a nearly complete address, so a half-typed one comes back as the middle of a street. **Worse, when Google refuses (its daily cap is spent, or the key does not allow the service), the lookup silently falls back to the free OpenStreetMap services - which hold Malaysian roads but no house numbers. That is "a list appeared, but not on the door".**

2. So the box now asks the question Google Maps asks. A type-ahead list, from Google's Places service, while you type - the same rows a customer recognises from Google Maps, each with the full address. **And the pin is only set once Google has given the point for the row that was tapped - a description is not a door, and treating it as one is how a pin lands in the middle of a street.**

3. And nothing gets slower or worse while you decide. The type-ahead is only asked if Google is actually serving it - no key, **Places not enabled**, or a spent quota, and it says nothing at all: **the ordinary lookup runs exactly as it did yesterday. The free services are still never asked while someone is typing** - that is their own written policy, and the 700 ms pause is kept for them.

TWO STEPS, AND ONLY THE FIRST IS YOURS TO DO IN A BROWSER:

1. Turn on Google's Places API for the key. In Google Cloud -> project **jienluv2bake-geocode** -> **APIs & Services -> Library -> "Places API" -> Enable**. Then, on that key (**Credentials -> the key -> API restrictions**), **add Places API to the allowed list** - today it is restricted to the Geocoding API only, and a key that does not allow the service answers REQUEST_DENIED. **Until this is done the shop behaves exactly as it does today** - you will just keep getting the old list. **2. Then I redeploy the function** (shop-geocode) from the repo folder - the lookup's code has changed, and **a push does not deploy it.**

One honest word about cost: Places is billed differently from Geocoding. At your volume it sits inside Google's free monthly credit, but **the 300-a-day cap you set was on Geocoding and does not cover this** -

set a cap on Places too, so a busy day cannot surprise you.

26 Oct 2026 - engine v649: EVERY STOP ON THE DELIVERY RUN NAMES ITS ORDER (nothing to run, no SQL, no upload).

Your words: "the delivery run shown a delivery on 30 sept, which order does not exist ... there is no way for me to know which order it belongs to, there is no link up and no order number".

1. The run was right and could not prove it. A day on that screen exists only because an order points at it - the van day you type on an order wins over its posting day - **so that 30 Sep was a real order carrying that day; the screen simply had no way to say which.**

2. Every doorstep stop now wears its order code - #D4E5F6 - and it is a link: press it and that order opens. A Self collection Point carries several customers' orders, so it keeps none.

3. And the row wraps instead of crushing the name - measured while adding the code: at phone width the name column collapsed to nothing and "Mei Ling" broke to one letter per line.

Nothing to run, no SQL, nothing to upload.

26 Oct 2026 - engine v630 to v648: A NEW FACE FOR YOUR SHOP, AND A CUSTOMER CAN ASK TO BE TOLD (ONE DATABASE STEP - see the end).

v630 is last week's Force update fix coming back from the bakery, with one addition of theirs: if a phone has no worker registered at all, one is put there before the reload.

1. Your shop has the design you approved. A photo fills the first screen, the four numbered steps read as a bar, items sit in rows, and **a FULL day is now a press that answers** - *"Wednesday, 28 Oct is full. The next day you can have treats is Monday, 2 Nov"*, with a **Choose 2 Nov** press. **No count is printed on a day that is not yours to count. Your own words and rules are kept**: "Post (nationwide)" / "Collect (local)", the postal-address gate, and "posting day" throughout.

2. "Good to know" - written by you. A card near the foot of the shop carries your posting days, your cut-off, how an order is collected or delivered, how it is paid, how it can be changed, and your WhatsApp. The plan's words are yours to edit - the shop shows what you wrote, not a second copy of it.

3. "Your last order", one press. A returning customer can put their last order back in the basket in one press. It is saved on their phone only - not on your books, not shared between customers - and there is a **Forget** beside it.

4. The food facts belong to the product. Each product gains *What is in it* (its allergens - the ingredients and anything your kitchen also handles) and *Keeping it* (how long to keep it, whether it freezes). You write them; blank draws nothing at all - no line reaches a customer until you fill it in.

5. "Tell me when this day frees up." A customer looking at a full day can leave a number, and you get the list of who asked - so a freed pan reaches exactly those people instead of living in a chat.

6. The quiet nudge. A customer who has not ordered in a while can be written to - the criteria are yours and so are the words, with the day and the deadline filled in. **It is a nudge, never a campaign:** the sentence may not say discount, offer, sale or free.

7. Backups read as one list. The copies fold rather than fill the screen, and **the box shows the shop's own words** rather than the app's.

8. And a failed Wi-Fi print now says what went wrong - it used to fail silently.

THE ONE STEP: run supabase/waitlist.sql in the SQL editor. It only creates a table; nothing existing is touched, and it is safe to run twice. Everything else works without it - only the "tell me when it frees" list does not.

23 Oct 2026 - engine v629: "FORCE UPDATE" NOW ACTUALLY FORCES IT (nothing to run, no SQL, no upload).

This one is from your own screen: the strip said "this phone has v625, the site has v628", you pressed

Force update, and it came back with the same sentence.

1. What was wrong. The app's own service worker is the thing that makes the browser re-check every script instead of trusting its ten-minute copy. **Force update was switching that worker off before reloading - so the reload went back to the browser's ordinary cache, was answered with the same old files, and the strip correctly told you so again. It was a press that could not do its job, and it looked exactly like one that had.**

2. What it does now, in two stages. First it keeps the worker and asks it for a new copy of itself, clears the app's stored copy, and reloads through it - so every script is fetched fresh. If you are still behind after a forced reload, then the worker itself is the stale thing: the second press clears it out completely and loads a brand-new address.

3. And the test that pinned the old behaviour was the old behaviour. It asserted the worker was removed on the first press - so it would have kept the fault alive. It now pins both stages: the worker is updated first, and only removed when the page was already a forced one and is still behind. Watched failing before it was kept.

Nothing to run, no SQL, nothing to upload.

23 Oct 2026 - engine v627 to v628: ONE DOCUMENT ON THE LABEL PRINTER, AND IT IS THE INVOICE (nothing to run, no SQL, no upload).

Your words: "i dont have to have 2 docs, i just need one docs, that is the invoice with all info in it."

1. The invoice card now carries "Invoice over Wi-Fi". It draws the invoice at 100 by 150 mm - the shop's name and address, **Invoice #**, the date the order was placed, the customer, every item with its amount, any code or discount, **Total**, the receipt and order numbers, a COD or refund sentence, and the day it was printed - **and sends it to the label printer. That is the same page the printer driver prints onto the sticker.**

2. AND THE SEPARATE SMALL RECEIPT IS WITHDRAWN. Its button is gone; one sale, one document on the label printer. The A4 invoice and its print/PDF route are unchanged - this only replaces the small one.

3. One sale, one number, two names - which is by design. The invoice and the receipt are the same serial: *Invoice #000020* and *Receipt #000020* are one payment. **And the two dates that used to look contradictory are each labelled now** - **"Ordered 16 Oct 2026"** and the posting day - **so two correct dates can no longer read as a mistake.**

Nothing to run, no SQL, nothing to upload.

22 Oct 2026 - engine v625: REMOVE THE ORDERS WITH NO POSTING DAY IN ONE PRESS (nothing to run, no SQL, no upload).

Your words: "now remove them" - after opening each one and finding they were old test rows.

1. The red line now carries the press. "Remove the N orders with no posting day" sits on the check that names them. **It asks first and names every code in the question**, so you are answering about a list you can read.

2. And it is the same shelf as the row's x. The orders go to the Removed list for 30 days - so nothing is destroyed, and a real sale caught among them can be put back.

3. It removes only the orders that check names - no posting day, and no day of their own. **A placed order is not touched**, and an ordinary order keeps its row's x. **The per-order "Remove this order" on each card stays**, for a sale you would rather put on a day.

Nothing to run, no SQL, nothing to upload.

22 Oct 2026 - engine v610 to v624: YOUR LABEL PRINTS OVER WI-FI, AND AN OLD COPY CAN NO LONGER BRING A REMOVED ORDER BACK (ONE DATABASE STEP - see the end).

1. The label prints over Wi-Fi. A small bridge runs on the Mac and talks to your label printer directly - the phone writes the job to your shared data and the Mac prints it, because a web page cannot open a raw

socket to a printer. **Install it once** (print-bridge/ in this folder, with its own README), **and the phone needs no printer address or certificate.** ? **The same label can also be shared as an image**, which is how it reaches a printer app on a phone.

2. Two sites, one book. ? **Each device names its own printer** (Settings -> the label printer name), so **the Mac at one site prints only that site's labels** and neither bridge prints the other's.

3. And it is built to lose nothing. A sleeping printer is a wait, not a lost label - the job stays queued until it prints. **The bridge runs in the background and survives a dead connection** without wedging. **A very long label stays on one sticker**, and **the label prints the right way round** - it was mirrored. **A small customer receipt** can go to the same printer; the documents still go through the normal print/share route, because they are not for a sticker.

4. THE GHOSTS STOP. Your words: "the old data keep comming back like ghost." It was a real fault: an old copy of an order could erase a deletion and the resurrection then travelled to your other phone. The two dates now decide - a removal newer than the copy wins, and the old copy is taken off the list and its deletion sent up again. A deliberate restore is stamped with its own date and still wins the other way. A copy with no date is never guessed at, because removing a real order on a guess would be worse than one ghost.

5. One press puts the missing receipt numbers back. The check's repair press claims every missing number at once from the register - the same claim the Invoice card uses, so it can never issue a new one.

6. An order with no posting day can be removed from its own card. It had no row on any day, so the x could not be reached - the card was the only door and it had no remove. **It asks first, and the order goes to the Removed list for 30 days.**

THE ONE STEP: run supabase/print_jobs.sql in the SQL editor. It only creates a table; nothing existing is touched, and it is safe to run twice. The app works without it - only Wi-Fi printing does not - but run it first so the first label prints.

20 Oct 2026 - engine v606 to v609: YOU CAN SELL ON ANY DAY, AND THE DAY IT MAKES IS PRIVATE (nothing to run, no SQL, no upload).

Your words: *"i want to sell any day just like bake."* and then *"i dont want to announce a special bake day or sale day."* Both are in, and the second one shaped the first.

1. A day the shop does not offer now answers a tap with "sell now", beside "say what you baked". It records the sale on the day it happened and opens the order card on that day, ready for a customer. Selling still goes through the one order writer - the same card, the same receipt, the same books - **so a hand sale and a shop order cannot disagree.**

2. And the day it makes is PRIVATE. It lives in your backoffice, your orders and your books, and the shop never sees it - so a hand sale cannot put a day you are not baking on a customer's calendar. **It is not listed on the Orders calendar, the Delivery Dates screen, Home's upcoming deliveries or the purchase order, and it does not label itself. It exists as the container for that one sale, reachable from the order itself. If you later open that date as a real day, the real day wins.**

3. A day that is not yours now reads as two freedoms first. "Say what you baked" or "sell now" - neither makes it an official day, and nothing is announced - or add it in More -> Delivery Dates. The official route is the optional third, not the opening instruction. And "Baked" is still a production record, never a sale: it moves ingredients and never puts anything on the shop.

4. One press was tried and taken back. A "Sell one now" button sat on the day card for a day; the order card is already on every one of those days and makes the identical sale, so it was a third control saying what the card says. **It stays only where it is needed - on a day with no card at all.**

AND THE TWO APPS ARE LEVEL AGAIN: v601 to v604 in the bakery are OUR OWN fixes, taken back. The Edit card showing the whole sale, a receipt number that survives the other phone, the check naming the right press, and the refund line on Check my system are now in both apps - so a future sync brings

them in identical, not as a conflict.

Nothing to run, no SQL, no upload.

19 Oct 2026 - engine v605: YOUR CHECK NOW WATCHES YOUR REFUNDS TOO (nothing to run, no SQL, no upload).

The second sweep found that a rollback of an order can erase a refund mark - the same wipe that took AC8542's receipt number - and nothing on Check my system was watching for it. There is a line for the number; there was none for the money that went back.

1. The new line. "Every refund in your register is marked on its order." Your receipt register is the record that survives a rollback, so it is what this reads. It cannot cry wolf on a refund you took back, because taking one back clears the mark in the register itself - so an undone refund never reaches this line at all.

2. The way out is the press you already know. Open the order and press Invoice - the refund and its amount are copied back from your register onto the order, the same second chance that puts a receipt number back. **If you took the refund back, the register will already say so and the line clears by itself.**

3. And it does not blame you for a receipt whose order is not on this phone. That is the line above it's business - this one only speaks about refunds it can see the order for.

Nothing to run, no SQL, nothing to upload.

19 Oct 2026 - engine v604: A RECEIPT NUMBER CAN NO LONGER BE TAKEN BACK BY YOUR OTHER PHONE (nothing to run, no SQL, no upload).

The second sweep found why AC8542's number vanished twice - and it was not a press.

1. A record travels whole. When a newer version of an order arrives from your other phone, every field that version does not carry is removed from this phone's copy. ? **So a phone that had never learned the receipt number could wipe it simply by saving that order** - both of AC8542's rows arrived together, with one timestamp and no number, which is exactly that.

2. The rule that fixes it is the app's own. A receipt is money recorded, and a receipt is never rewritten - so **a number this phone is holding is carried onto the incoming copy. It only ever fills a gap:** a version that carries its own number still wins, and this can never invent one.

3. And it proves itself. Two tests drive a pull: one where the newer copy has no number (the number is kept), one where it has its own (the cloud wins). The first was watched failing before it was kept.

Nothing to run, no SQL, nothing to upload.

19 Oct 2026 - engine v603: THE EDIT CARD NOW SHOWS THE WHOLE ORDER, INCLUDING THE LINES ON ANOTHER POSTING DAY (nothing to run, no SQL, no upload).

A sale whose lines sit on two posting days had a hidden half: the Edit card listed only the day you opened it from, while its Save wrote the address, the note, the day and the tracking number onto EVERY line of the sale. So the other day's lines were changed without ever being on screen - which is exactly the "these lines disagree" state your check reports.

1. What changed. The Edit card lists every line of the order, whichever day it sits on, so what you edit is what you see. **Save settles them all together** - the promise the manual already made: the whole order moves as one.

2. And a line taken off from the other day is really taken off. Its x removes that line, and it goes to the Removed list like any other removal - before this, a line that was not on the screen could neither be seen nor removed.

3. One arithmetic pairing went with it. The card works out the order's worth from what it is holding - and the other day's lines had been added separately. **Once they were on the card they were being counted twice** (a two-day sale read RM64 instead of RM32 and the "the money is in" question never appeared). **Caught by the test for that question, not by reading it.**

Nothing to run, no SQL, nothing to upload.

19 Oct 2026 - engine v602: A PROMO CODE OR RECEIPT NUMBER ON ONE LINE IS NOW FILLED ONTO THE LINES THAT HAVE NONE (nothing to run, no SQL, no upload).

v601 settled AC8542's payment line, and the check then named a second field on the same order: promo - a discount code on one line and nothing on the other. No press re-applies a promo to an order that already exists, so that line could not be cleared by hand either - the same shape as the payment, one field over.

1. The rule is the one the app already reads by. promoOn prices the sale from its FIRST line, so the discount was already being applied; the other line simply never got the stamp. **So the repair now fills a stamp as well as a payment: where the first line carries a promo code or a receipt number and another line has none, the other line gets it.**

2. And it never overwrites an answer a line already carries. That is the half that keeps the check honest: a genuine disagreement stays visible instead of being papered over. **The same rule holds the other way too** - a code that only appears on a LATER line is never copied up onto the first, because the first line is the one that prices the sale, and copying it up would change what the customer was quoted.

3. And it runs where the data arrives - when the app opens, after a merge, and on every sync - so there is no press to find for either field.

4. And a false alarm is gone: "nothing" has three spellings and they are one answer. One line of an order carried promo as an empty value and the other as a missing one - both mean NO CODE - and the check compared their spelling, so it reported a disagreement that did not exist and no press could ever clear. It compares the meaning now. false is still an answer and still disagrees with an empty one, so a line that says "not paid" cannot hide behind this.

5. And the receipt check's own way out is corrected. It said "open the order and press Save, then Paid again" - and Paid is not offered on an order that already reads paid, so it named a button that is not there. **It names the Invoice now, which is the app's own second chance and draws the same number the register already holds.**

Nothing to run, no SQL, nothing to upload.

19 Oct 2026 - engine v601: THE ORDER THAT COULD NOT BE FIXED BY ANY PRESS NOW REPAIRS ITSELF (nothing to run, no SQL, no upload).

The order AC8542 is why this exists. Its payment had been written on one of its two lines and not the other, so your check said the lines disagree - and there was no button to press: the Pay press is only offered while the FIRST line is unpaid, and this order already read as paid.

1. Why that line could not be cleared by hand. Every money book in the app reads an order through its first line - that is how "has it been paid" is answered everywhere. **So the sale was already being counted as paid, while the second line kept a half-written copy** from before the app learned to write a payment onto every line of a sale. **No press would rewrite it, because the app believed the order was finished.**

2. So the app settles it itself. When the first line says the money is in, every line of that sale now says so too - and the account and the day the money landed come with it. It happens when the app opens and on every sync, so it needs no press at all. And it only ever works that way round: an unpaid order is never marked paid by this. Nothing on your books moves - the sale was already counted as paid through that first line.

3. Its receipt number is the one press left, and the app has always had it. Open the order and press Invoice - the number is drawn and stamped then, and it is the **same #000003 your register already holds**, never a new one.

4. And the check's advice is corrected. It used to say "press Save" for a payment line, which writes the day, the address and the rest but not the payment. A way out that sends you to the wrong button leaves the same red X after a correct action, so it names the right press now.

Nothing to run, no SQL, nothing to upload.

19 Oct 2026 - engine v600: A REMOVED ORDER THAT IS BACK IN THE LIST NOW CLEARS ITSELF, WITHOUT WAITING FOR THE APP TO BE REOPENED (nothing to run, no SQL, no upload).

Your "Check my system" page found this one, and the numbers it gave me named it exactly: two orders sitting on the live list AND on the Removed list at the same time, in your shared data itself.

1. What it was. The app has always cleared that overlap when it opens - an order in the list cannot be on the shelf as well, so the shelf row is stale and is dropped. **But the sync pulls every thirty seconds, and the pull was the one path that did not do the clearing. So the shared data handed the same stale rows back a few seconds after every open**, and the check reported them again on every run. **It also meant two records wearing one order's id**, which is why saving and re-opening an order did not settle it: a stale copy kept coming back.

2. The fix is one rule applied where the data actually arrives. The pull drops a stale shelf row the moment it comes down, exactly as the app already did on opening - and queues its removal, so the shared data stops handing it back. Nothing is lost: the row is only dropped when the order itself is right there in your list.

3. And a guard now holds it. A test builds the exact shape your book held, runs a pull over it, and fails by name if the stale row survives or its removal is not queued. **Watched failing before it was kept.**

ONE THING THIS DID NOT FIX, AND v601 DOES. Its first advice for the order AC8542 was "press Save, then Paid again" - and that was wrong: the Pay press is not offered on an order that already reads paid. **v601 repairs that line by itself; the entry above says how.**

19 Oct 2026 - engine v550 to v599: FIVE MORE SWEEPS THROUGH THE WHOLE APP, AND THE LAST OF THEM FOUND FAULTS IN THE FIXES BEFORE IT (one database step to run first, and the five uploaded functions to redeploy - both named at the end).

Fifty versions, all in one stretch: an audit of what the audits left, then a check of that check, five times over. Two of the sweeps found faults created by the sweep before them - which is worth saying plainly, because it is the reason the last one was called a freeze rather than a finish.

1. THE TRACKING PAGE NOW NEEDS THE PHONE THE ORDER WAS PLACED WITH. The code alone used to open any order, and a code is six characters. The customer now types the code and the WhatsApp number they ordered with, and the database itself refuses to answer for a code alone - so knowing a code gives nobody a name, an address or a driver's phone number. **The one press that puts the old codes back in reach** (her own screen) applies the phone to every card already out there.

2. A REFUND IS MONEY OUT IN THE MONTH IT WENT BACK, AND ONE SALE IS ONE INVOICE NUMBER. A refund used to be counted in the month the ORDER was placed, so a refund in October for a September sale left both months wrong. **And a sale whose lines sat on two posting days drew two invoice numbers** - one sale, one number now.

3. PAYING A POCKET BACK SETTLES A DEBT - IT IS NOT A WITHDRAWAL. Paying back a pocket that paid for something moved the money twice in the books: once as the debt settling and once as money she had taken out. **It settles now, and only once.**

4. The paper. Labels print on a 100 by 150 mm sheet instead of a shape no printer had. **An invoice now says who it is for** - it had a total and no name. **And nothing addressed to her prints on a customer's or a shop's sheet**: the invoice no longer tells whoever reads it to "open this app again", and the purchase order no longer prints an instruction to her for the shop to see.

5. The shop. A customer cannot place an order with a number that cannot be answered - it is marked at the box, in the same red edge as a missing name. **The day the customer tapped keeps the keyboard**, instead of the page throwing them back to the top. **A code the page has judged is judged again** whenever the basket changes. **And the next customer arrives in exactly the state a new visitor does** - which here means Post (nationwide) chosen, the way this shop opens.

6. The homepage. The price on the homepage card follows the product you pick in Settings, so a price change made once reaches both pages. **And the cut-off and the posting days are read from your settings rather than typed into the page** - so the homepage and the shop cannot say different things.

7. The stock card, the money and the trips. A stock card whose rows do not add up to the shelf is named by Check my system, with the two identical stocktakes that caused it - that is the answer to the two "+10" rows of 17 October. **Eight stock cards that could not be put right now can be. A trip can no longer be booked twice** by two presses, and **a booking that may have gone through says so rather than pretending it failed.**

8. The security. Two database functions that only exist to be fired by an order are no longer callable by name by anyone, and an order's stored data has a size limit. **None of it changes what you see; all of it closes a door.**

9. And the tests. Several guards could not fail - they checked that a screen called a thing, not that the thing worked, and one of them passed while a hard-coded number sat beside it. **Those are rewritten, and one of them caught a fault in the fix before it.**

BEFORE YOU DEPLOY THIS ONE: run supabase/tracking.sql in the SQL editor first - the tracking page asks for a phone now, and the old database function cannot answer that question. **Then run supabase/receipts.sql and supabase/shared_pool_slots.sql** (the refund month, the invoice number and the slot counts), **and supabase/advisor_fixes.sql** (the door). **Then redeploy all five uploaded functions** - courier, parcel, shop-geocode, shop-feedback (which gains a file, so it must be uploaded whole) and wish-mail. **If the SQL goes in first, the shop keeps working the whole time; if it goes in after, tracking is broken until it does.**

supabase/instant_slots.sql is RETIRED - do not run it. It refuses by itself: it checks the rule already installed in your database and stops with "This file is retired ... nothing has been changed." That refusal is the file protecting your live counts, not a fault, and there is nothing to fix. shared_pool_slots.sql is the one that replaces it, and it is the only one to run.

18 Oct 2026 - engine v549: A FULL AUDIT OF EVERY SCREEN FOUND TWENTY-TWO FAULTS, AND ALL TWENTY-TWO ARE NOW FIXED (nothing to run, no SQL, nothing to upload).

Your words: *"since there is so many error, why not you do a thorough audit on all small details then draft a plan to rectify it one off."* So every screen, every calculation and every save-and-load path was read end to end - the app, the shop, the sync, the backups, the dates and the money.

1. The part worth keeping is how they were found. All 3,525 tests were green while all sixteen were live. Every fault here was found by reading the code and measuring the number it produced - not by a test going red. A green suite proves the app does what its tests say; it does not prove the tests said the right thing.

2. The faults that put a WRONG NUMBER in front of you.

- **A recipe line's cost used the price per kg on a quantity written in grams** - a RM2.50 pouch costed at RM2,500, and Profit's cost of sales a thousand times out. **500 g plus 1 kg came to 501**, because quantities in different units were added as plain numbers. **Correcting one product's bake gave back ANOTHER product's ingredient**, and a correction the shelf could not cover invented stock that was never there. **The "amount to buy" box in the amend popup named the wrong unit** - typing 1 where it meant kg put 1,000 g on the shelf. **A blank in your saved posting-day pattern was printed as Sunday** - the app told you Monday, Sunday and Wednesday, a day you never ticked. It is cleaned when it loads now, and a blank is never named as a day. **A discount larger than the order left the printed lines not adding up.** The Total stopped at RM0.00, but the discount did not stop - so the lines under it summed to minus RM470. **The discount is now capped at what there is to take off, so every printed surface adds up to its own Total.**

3. Two phones.

- **A unit you deleted came back - and came back on your other phone too. The removal was remembered only on the phone that made it**, so the other phone pulled the unit back and pushed it in again.

A removal is now a record of its own and travels, so a delete stays deleted on both. **A change made during a sync could be thrown away** - your edit replaced by the cloud's older copy, and never sent. **Two products recorded on the same day are now two records, not one. One record per day meant the two phones could overwrite each other's day**; an old one-row day is split when it loads, moving no stock.

4. The shop.

- **A product you renamed or removed left a customer's basket stuck**: Place order did nothing at all, with no message. **The basket now empties itself and says which item went and why. A slow send could post the same order twice**, because any repaint turned the button back on. **The sending state now survives a repaint. A courier order could be placed with no address at all. The address box now marks itself and says why**, the same way the name and the number boxes do - **and a pin dropped on the map counts as an address, so a customer who marks the place is not refused. "Today has closed" was printed on a day you do not post. The heading now asks whether today is one of your posting days**, and a day that is open says so. **A refused promo code was judged once and never again** - "add RM14 more" stayed on screen after the customer had added the RM14. **A code the page has judged is judged again whenever the basket or the codes change. The menu kept the last customer's quantity boxes after an order. The menu is rebuilt with the basket now.**

5. One refund. A refund after editing an order down was taken off twice - you kept RM15 and the order read RM0. **The refund is still recorded, and it no longer moves your takings a second time. An ordinary refund, on an order you have not edited, is unchanged.**

6. The rest.

- **A stock card shows the TIME**, so two movements on one day can be told apart. **A stocktake that changes nothing now records its own +0 row**, so the day is not silently missing. **Two ingredients sharing one name are findable**, and **a hidden ingredient can be unhidden** - hiding used to be a one-way door. **Removing stock the shelf cannot cover is refused**, and says so, instead of leaving a shelf that cannot go negative with no explanation.

7. The engine number is 549 because on each sync this app takes the bakery's number, so both shops read the same sticker. **Nothing of yours was rewritten, and nothing needs running: no SQL, no upload, no database step.**

17 Oct 2026 - engine v541: A PRODUCT WITH NO INGREDIENTS CAN STILL BE RECORDED, AND IT SAYS IT MOVES NO STOCK (nothing to run, nothing to upload).

Your words: *"why a hidden product dont appear on total baked?"* - and then *"i dont see them because there is no ingredient with them"*. Both true, and both worth fixing.

1. What was happening was not a bug - it was a GATE. The baked box listed only products with a recipe, **because a box for a product with no ingredients moves nothing, and a control that does nothing is a bug in this app. But the cost was worse than the rule: you had made something, and the app would not let you record it, because a recipe was not set up. Your own rule is "guide, never a gate" - and this was a gate.**

2. So every product is listed now, and one with no ingredients carries this on its own row: **"no ingredients on its recipe - this will not change your stock."**

3. And the number itself moved from v537 to v541. It now takes the bakery's number on each sync, so **the two shops read the same sticker again - which is what this Guide has always told you they do. Numbers 538 to 540 describe changes the bakery made between syncs; this app never shipped under them**, so skipping them costs nothing.

4. And both rules are kept. You can record the bake so the day is complete; **you are told plainly that it moved no stock**, so it is not a control that quietly does nothing; and **the day you add the ingredients, the same box starts moving stock with nothing else to learn.**

Nothing to run, no SQL, nothing to upload.

17 Oct 2026 - YOUR MANUAL GAINS A "DO THIS, NEVER THAT" PAGE (the app is unchanged - engine stays v536; the manual is now v536a).

You asked for the rules, so they are page 23 of your manual rather than something in a chat you would have to scroll back through. Eleven DOs, eight NEVERS, and what to do if something goes wrong, in order.

And the page says where they came from, because that is the part worth keeping. Every rule is a fault that actually happened: a queue that said it was syncing and never emptied, a save that said it saved into a full phone, a database step that was told to you as a step and was empty. **None of it was carelessness in your shop - all of it was the app telling you something it had not checked.**

The DOs, one line each: open the app on one phone every day or two · keep one phone signed in to shared data · press Save after recording what you baked · type the TOTAL, not the extra · read the numbers on Check my system · use **Put back missing orders** rather than Restore · look at the page before Restore or Import · test on one phone first.

The NEVERS, one line each: never sign out of shared data casually · never clear the app data to fix a problem · never use Import to top up or merge · never use Restore to fix one missing order · never assume the paid stage says how the money came in · never mix up the order's Baked stage with the baked box · never delete an order you might need · never trust a number without reading it.

And if something goes wrong, in four steps: don't sign out, don't clear anything, don't press Restore · run Check my system · send the failing line word for word · then wait, because **nothing you have is only in one place.**

17 Oct 2026 - engine v536: A SAVE THAT FAILS NOW SAYS SO, AN IMPORT CAN BE UNDONE, AND A RESTORE NO LONGER HIDES RECORDS (nothing to run, nothing to upload).

Six versions from the bakery, all of them from questions you asked.

1. A SAVE THAT FAILS NOW TELLS YOU. Your question: *"is there a test that you should run to detect this systemic problem, or any risky system handling that we should eliminate before the database build up?"* - **and it found one.** When the phone is full, saving used to fail into a console you will never see, **and carry on as if it had worked. So "it saved" and "this phone is full" were both true at once, silently** - and your other phone would then hold a change this one had lost. **It says so now, and the page shows how full the phone is.** A full phone is a *growth* fault, which is why a year of use never showed it.

2. AND A REAL BUG IN THIS APP'S OWN NEW SYNC - found the same day it was built. Your question: *"any other problem that can happen?"*. **Reading only what has changed needs a bookmark. Restore and Import both throw the whole book away and put an older one back - and the bookmark was not reset. So after either, the data jumped backwards while the bookmark still pointed forward, and every record older than it would never be read again.** Both presses now forget the bookmark, at the cost of one full read. **And they forget ONLY the reading state - never the sending queue**, because that queue is exactly the unsent changes this app protects.

3. And an Import can now be undone. Restore has always saved a "Before restore" copy; **Import did not, and its own dialog said "This cannot be undone."** It saves a "Before import" copy now.

4. The backoffice is no longer offered to search engines, and your homepage and shop deliberately are. Your app is PIN-locked, but **its title and the fact it exists were public the moment Google crawled it.** The homepage and the shop now say they are findable - in their own words on the page, rather than leaving it to chance - and your **working files** (the test folder, which the site publishes) are kept out.

5. Your sixteen wording decisions are pinned, and two were still missing. Your Malay for "we use this to know who the order is for" had a verb hanging in the air, and one "Pemandu" had survived your **driver** decision - the note recorded four strings and there were five. **A rule recorded as a list stops at the end of the list.** Both are fixed.

6. One thing did NOT come across, and it is worth knowing why: the bakery's PDFs learned to print

Chinese. That work lives in its own machine's manual folder, which is never in the repository - **and your documents are written in English on purpose**, so nothing here needs it. **If you ever want Chinese in your manual, tell me and I will bring that fix over too.**

Nothing to run, no SQL, nothing to upload.

17 Oct 2026 - engine v535: THE DATABASE STEP WAS EMPTY. YOU NEED TO RUN IT AGAIN, AND THIS IS MY MISTAKE (one paste, safe to run twice).

1. What happened, plainly. v533 told you to run supabase/sync_cursor.sql before deploying. **That file was EMPTY - zero bytes.** You ran it, it did nothing, and **nothing anywhere said so. And I made it worse: because the app falls back to reading everything when the column is missing, it carried on working - so I told you the step was confirmed. It was not.** I had checked that the file existed and that it was in your commit. **I had not checked that it had anything in it.**

2. Nothing is broken, and nothing is lost. Your sync has been reading the whole book - **the careful, paged read. Slower, never wrong.** Your orders, money and bake days are untouched; no record was ever at risk.

3. So please do one thing: run supabase/sync_cursor.sql again, properly this time. It is now the real script - the same one the bakery uses - and it **prints four numbers back at you when it finishes** (rows, stamped, lowest, highest). **rows and stamped must match, and highest must be bigger than lowest.** If they do not, stop and tell me. The script is safe to run twice, changes no record, and only adds a column and a trigger.

4. And a guard now exists so this cannot happen again. A new test checks **every .sql file in the repo for actual instructions** - a step that is empty, or comments only, now fails the suite by name. **A promised step that is empty looks exactly like a step that worked**, and the fallback is what made it invisible. **The bakery found this in our own repo's history and wrote the same guard for itself** - it is their catch, not mine.

Nothing else changes. No new features. No other setting to touch.

17 Oct 2026 - engine v534: THE 30-DAY CHANGE IS NOW THE BAKERY'S, AND IT IS BETTER THAN MINE (nothing to run - the SQL you already ran covers it).

1. What happened. I made the 30-day change here as a Munchies-only fix. The bakery has now made the same change properly (their v530), and **this version replaces mine with theirs. Your decision is unchanged - the Removed list keeps orders for 30 days - and nothing you see changes.**

2. The better idea, and it is theirs. Mine typed "30 days" into each screen that mentioned it - the Removed card, and two sentences on Check my system. **Theirs keeps the number in ONE place and every screen asks for it. A number written into a sentence is a second copy of a decision, and second copies drift** - which is exactly what had happened: a note above the keeping code said "thirty days is long enough" while the setting said 48, and **that disagreement got quoted back to you as evidence.**

3. And a distinction I missed: one setting was doing two jobs. How long a removed order stays **puttable back** - which is what you answered - and how long a removal **leaves a trace** for the check that opens a week-old copy. **They are not the same question**, and 48 hours could never satisfy the second one.

4. And a guard, rewritten because the first was too weak. The first only checked that the screens *called* the helper - **and it passed while a hard-coded "30 days" sat in one of them. A check keyed to one occurrence checks one occurrence.** It now ignores comments and asserts that no screen spells a window at all.

5. One honest consequence: the two engine numbers have parted company. Munchies is **v534** and the bakery is **v530** - because this app took four Munchies-only versions (the Removed list, Put back missing orders, the record count, and the paged and incremental sync) while the bakery stood still. **The numbers are each app's own build stamp, so this is not a fault - but the two will not match by accident again.**

Nothing to run. The database step you already did is still the only one.

17 Oct 2026 - engine v533: THE SYNC NOW READS ONLY WHAT HAS CHANGED - AND THERE IS ONE DATABASE STEP FIRST.

RUN supabase/sync_cursor.sql IN YOUR SUPABASE SQL EDITOR BEFORE YOU DEPLOY THIS. A push runs no SQL. One paste, safe to run twice. **If it is not run, nothing breaks** - the app notices and keeps reading the whole book, which is slower but never wrong. **It just will not get faster until the step is done.**

1. What it does now. After its first full read, your phone asks the database for **only the records that changed since it last looked** - a handful, instead of the whole book, every 30 seconds.

2. And why it needed a database step at all - the clock. The obvious way to ask "what changed since 10:23?" is to trust each phone's clock. **Your two phones' clocks are not the same clock. A phone running behind writes its edits with a time in the PAST - and a search for "everything after 10:23" would never look there again.** With a full read that costs a little speed; with an incremental read **a missed record is missed for ever.** So the cursor is a **number the database stamps itself: every write takes a number strictly higher than every number before it**, whoever wrote it and whenever. **No clock can decide what "since" means.**

3. And the cursor only moves forward on a complete read. A half-read book is never remembered as "seen" - otherwise the next read would start after records this one never received. **And a record written while the read is running always takes a higher number than anything already read, so it cannot slip behind: the next read asks for it by name.**

4. The fallback, said plainly. If the database step has not been run, the app falls back to reading everything - **the same careful, paged read as before. Slower, never wrong.** It remembers that and stops asking.

Nothing else changes: your data, your orders, your money and the 30-second beat are all as they were. Nothing to run on the phones beyond this one SQL paste.

17 Oct 2026 - engine v532: THE SYNC NO LONGER STOPS AT A THOUSAND RECORDS (nothing to run, nothing to upload - pushing this is the whole of it).

Your words: *"what is the value of a system that can handle 1000 records only?"* - and the answer was none. That number had no business being there.

1. What was wrong. Your phone read its shared data in **ONE request, capped at 1,000 records**, and **anything past that never arrived** - silently, on every screen. **A record set with an expiry date. An order is one record per ITEM, not one per order**, and nothing removes old orders, so a working business reaches that number in months. **It was not lost - the app never deletes a record for being missing from a read - but it never arrived either.**

2. And it was worse than a cap. That read had **no order at all**, so **which** thousand records a phone got was arbitrary - **and could be a different arbitrary thousand the next time.** That is a record showing on one device and not the other, with nothing saying why.

3. THE FIX: it reads a page at a time and keeps going until the data runs out. 1,000 is now a **page size, not a limit** - the sync asks for the next page until one comes back short. **Every record reaches every phone, whatever the size of your book.** It is sorted too, so the pages cannot skip or repeat a record between them.

4. And there is still a hard stop, far out, and it says so if it is ever reached. The read gives up at **100,000 records in one pass** rather than looping for ever, and **the count on Check my system fails only at that point** - the one place a record could still fail to arrive. The number in the sentence is read from the same constant the loop uses, so the two can never disagree.

5. And you asked to see the count first, so it is still there. "How many records your shared data holds" on Check my system - under the stop it reads *"...and every one of them reaches every phone, because the sync now reads them a page at a time instead of stopping at a thousand."* **It is information now, not a warning.**

6. What is honestly next, and it is the better design. Each full read is now several requests when the book is large, every 30 seconds. **The real cure is reading only what has CHANGED since the last sync** - the app already stores the moment of the last pull and never uses it. **That cannot be trusted until the change time comes from your database rather than from each phone's clock**, so it is its own job, with one SQL script.

Nothing to run, no SQL, no upload, no setting to change on either phone.

17 Oct 2026 - engine v531: YOU CAN NOW PUT MISSING ORDERS BACK - WITHOUT REWINDING ANYTHING (nothing to run, nothing to upload - pushing this is the whole of it).

1. A new press in Backup and safety: "Put back missing orders". It puts back **only** the orders a saved copy holds that your phone can no longer account for - **not on your list, and not on the Removed list.** It **adds and never removes**: nothing you still have is changed, no product, no money, no day, no Removed row is touched, and **an order you cancelled yourself is never brought back** - if it is on the Removed list, it stays cancelled. **And it saves a "Before put-back" copy first**, so the press itself can be undone.

2. And that is what "put that copy back" should always have meant. Restore is a REWIND - it takes the whole app back to the day of the copy, here, in your shared data and on your other phone. That is the right tool for "put everything back to how it was", and **the wrong tool for one missing order.** Until now it was the only thing on offer for the second question - and the check itself was recommending it. **Restore is unchanged, and its warning is the same.**

3. So the check's advice is fixed too. Where it said *"put that copy back if these are orders you took"*, it now says to use **Put back missing orders**, and tells you plainly that leaving them alone costs you nothing.

4. And a name that was telling a lie. The function behind Restore was called mergeRestore in the code - and **it never merged anything.** The button's warning was honest, but the name was not, and a name is what the next person trusts instead of the body. It says rewindTo now.

Nothing to run, no SQL, no upload, no setting to change on either phone.

17 Oct 2026 - engine v530: THE REMOVED LIST NOW REMEMBERS 30 DAYS (Munchies only - nothing to run, nothing to upload, pushing this is the whole of it).

Your words: *"raise it and fix it"*. And it is worth saying plainly where the old number came from: two days was YOURS. You set it in September (*"the deleted order just need to keep for 48hrs"*), and thirty had been the developer's guess before that. So this changes your own decision, at your asking, and it is still yours to change again.

1. Why it needed raising. Check my system opens the **oldest saved daily copy** - about a week back - and asks whether every order in it can be accounted for. **A two-day Removed list cannot explain a five-day-old removal**, so it named **seven of your own cancellations** as orders that had "left without a trace". Thirty days covers that copy with room to spare, so **the check can now answer instead of guessing.**

2. What it also means for you: a removed order can be put back for a month, not two days. The Removed list says so on its own card now ("Kept for 30 days so a mistake can be put right").

3. And one honest limit: it cannot bring back rows already thrown away. Rows pruned under the old two-day rule are gone, so **the seven you are looking at may still be named until the oldest copy rolls past it** - a few days. Nothing is lost by waiting; the check's own words now say why.

4. And a warning about the screen's own advice, which I checked while doing this. It offers *"put that copy back"*. **That restores the whole app to that copy** - a week of orders, money and baked totals would come back as they were - and despite being called a *merge* in the code, it does not merge. It takes a "before" copy first, so it is recoverable, but **it is not a way to look something up.** Use it only if you actually want the orders back.

Nothing to run, no SQL, no upload, no setting to change on either phone.

17 Oct 2026 - engine v529: THE STUCK SYNC IS FIXED, AND THE REMOVED LIST HOLDS ONE ROW PER ORDER (nothing to run, nothing to upload - pushing this is the whole of it).

Three versions from the bakery, and they are the worst fault this app has had - the one behind every "stuck" report you have been sending.

1. ONE CHANGE THE SERVER REFUSES CAN NO LONGER FREEZE EVERY OTHER CHANGE WITH IT. Your reports, in the order they arrived: *"yes, look into the stuck records"*, *"the phone and mac having different error"*, *"more and more removed order shown on phone"*, *"suddenly the phone deleted order goes to 25"*, *"both*

grow to 27". **And here is what was happening: your changes go up together as one batch, the server refuses the batch, and a refusal sends NOTHING - so every change sitting beside the refused one was frozen with it.** Pressing Sync now could never empty that queue, however many times you pressed. **Measured: 3, then 8, then 25, then 27 - on both devices, because each phone keeps its own queue.** A refused batch now **comes apart**: each change is offered on its own, so the good ones go up straight away and the one the server will not take stands alone, **named**, instead of holding everything else hostage. And **your connection being down is not treated as a refusal** - nothing is taken apart record by record when the internet is simply off.

2. And it now remembers WHY a hand-over failed. Before, the reason was shown to whichever button pressed Sync and then forgotten - so the card said "Sync failed" only while you were looking at it, and **every other screen was blind.** It is kept now, and **Check my system prints the server's own words and names the record it refused** - by the order number you would search, or by the name you gave the thing, rather than "a removed order" or "uoms", which is a kind of record and not one you can look up. **A record with no number of its own says "with no number"** rather than having one invented for it out of the word "undefined".

3. One order in the Removed list is now one row, always. Your report: *"there is so many duplicate order number in the Removed Orders"*. **And it was a loop rather than an accident:** the shelf simply added a new row every time an order came off, and **the shared data hands an order back whenever its removal has not landed** - which is exactly what the frozen queue above was causing - so you took it off again and a second row with the same number went on. That is why the same number kept appearing and why the list only grew. **Taking an order off twice now leaves one row**, and **the duplicate rows already in your records are tidied up when the app opens** - the newest one kept, because that is where the 48 hours for putting an order back are counted from. **Nothing of yours is lost: the same orders are on the shelf, just once each.**

4. And the check stops blaming you for your own removal. It named an order (4B00A9) as having left "by no route it knows" - and the app actually held two records of what happened to it: **you removed it yourself on your other phone**, and then marked its receipt void. The copy the check opens was made before that removal, and **the Removed list keeps a row for only 48 hours**, so the row that explained it had already gone. **A receipt you have marked void now explains its order**, and the message names both innocent cases rather than one. **It still cannot hide a real loss** - a void record for a different receipt excuses nothing, and there is a test for exactly that.

Nothing to run, no SQL, no upload, no setting to change on either phone.

17 Oct 2026 - engine v526: HOW MANY YOU ACTUALLY BAKED, AND TWO SYNC FAULTS FIXED (nothing to run, nothing to upload - pushing this is the whole of it).

Thirteen versions from the bakery, and most of them are one feature - plus two faults of mine that you found.

1. A new button: Baked - how many? Your words: *"when Jien Bake focaccia, she usually bake more, not according to order"*. The day's card now has **two doors**, each with its own name: **Set day's availability** (what the shop can still sell that day) and **Baked - how many?** (what actually came out). **My first attempt merged the two into one renamed button, and you stopped it:** *"set availability and to mention how many to bake is totally different functionality, why you take way set availability?"* - **merging two jobs under one name is not tidying; it is deleting one of them from your side of the screen.** Your button is back exactly as it was.

2. The number you type is the TOTAL you baked. Type 10 with 6 ordered and only the extra 4 come off the shelf; type a smaller number and the difference goes back on. **And it works on any day** - one you do not normally post on, and one already gone. It writes no delivery date at all, so the shop never starts offering a day you opened for nobody.

3. The box is labelled TOTAL BAKED - because the words were not doing their job. You asked whether typing 15 after 10 meant fifteen in all or five more, and **that was not a near miss: it is ten items out.** Measured both ways, the app would have read "five in all" and **put five back on the shelf**. **And a line I added under the box to explain it, you deleted in one sentence** - you asked what my own sentence meant, and that was the

answer to it. A total is a total.

4. The big one: ingredients come off when you BAKE, and nowhere else. Your words: *"only take away ingredient when baked. And the order dont take away ingredient, then it can be clean cut"*. **Measured with your own numbers: two things were taking flour off the shelf** - an order marked Baked, and the day's baked total - **so a sale made after the bake was subtracted twice.** Bake 10 (5,000 g to 2,500 g), then a customer takes 4 of those ten and the order is marked Baked (to 1,500 g): **you baked ten and the shelf gave up flour for fourteen.** The old rules are **deleted, not switched off**. **The cost, named plainly: a day you do not record takes nothing off** - which is exactly why the box is filled in for days that have gone, so clearing old days is one press each.

5. What the box opens with is your rule, and it took three passes. Your three messages: *"enter a new order on a new bake date, the order qty should not be prefill into the total baked"*, *"if i open the total baked and it shows a number, that mean there is some bake already done that day"*, and *"if the pass day is not prefill, enter a number will deduct stocks"*. **So a number in the box is a STATEMENT THAT A BAKE HAPPENED, and the box may only ever show a number that is true:** a day already gone opens ready at what was ordered, **today and every day ahead open empty**, and a day you have already saved shows your own number back so you can check it or correct it. Saving the same number again changes nothing.

6. And a fault of mine that no test could see. A leftover style rule from the version I got wrong was **using a class name already taken by your Home screen's own lines** - and being declared later, it won. So **the lines on your Home screen have been wearing a green box and a green left border since v514**, while three thousand four hundred tests passed. **A dead rule is not harmless, because it can be the one that wins.** It is deleted, and the Home screen is checked back to normal.

7. Your screen no longer jumps to the top while you are reading it. Your words: *"when i look at screen, at after a while it rerender and the screen jump up"*. It was not really a jump - **it was a loss:** the view is emptied for an instant while it is rebuilt, the browser clamps the scroll, and nothing puts it back. **Measured: scrolled to 400, repainted, landed at 0.** And it was the **sync** doing it, every 30 seconds - not you.

8. And the sync fault that cost you confidence. Your words: *"the sync is a big issue, and my user lost confident"*. **A receipt you marked void on the Mac was written with no id - and a record that cannot be keyed cannot travel.** The phone never learned, so it went on showing the same error and telling you to press a button you had already pressed. **And pressing Sync made the count go away**, because the un-keyable record was **dropped, not sent** - and a count going down looks exactly like success. The repair moved from "those two lists" to **the door itself**: before anything is queued, every record in every list is given an id, worked out from its own contents so two phones make one row rather than two. **Nothing of yours is rewritten**, and the receipt already stranded is repaired when the app opens.

9. The order row now says which account took the money. Your question: *"now i can see that an order had been paid, but how do i find out whether it is pay be TnG cash or orthers?"* It reads **Paid - TNG**, **Paid - Cash**, or whatever you call your own account. An order paid with no account recorded still reads just "Paid", and an unpaid order carries nothing.

Nothing to run, no SQL, no upload, no setting to change on either phone.

16 Oct 2026 - engine v513: THE DATE ON SCREEN FOLLOWS THE CLOCK, IN YOUR APP AND ON THE SHOP (nothing to run, nothing to upload - pushing this is the whole of it).

One version from the bakery, and it is your report turned into a question that found a second one. Your words: *"it is the 1st time that i saw the app date not updated until i reload? Why?"* - and then *"will it happen on storefront?"*

1. Your app kept yesterday's date until you reloaded it. It drew the date when the screen was drawn and **never looked at the clock again**: only moving to another screen, a reload, or a data change repaints it. **So an app left open overnight went on saying yesterday.** Measured: with the clock on Saturday 17 October, the Home screen still read Friday 16 - and switching away and back changed nothing.

2. And a second thing, which made it worse: the app's "look again" alarms only started after a cloud

sign-in. If you were **not** using cloud sync - which is the state your own phone was in - the app had registered no timer, no focus and no returning-to-the-page listener at all. **Nothing could have refreshed that date except moving to another screen.**

3. And then you asked the better question, and the answer was yes. The shop **was** doing the same thing. It re-checks its numbers every 30 seconds and every time a customer comes back to the page, but **it only redraws when those numbers change - and the date changing is not a change.** So a shop left open overnight kept **yesterday's "today"**. Measured: with the clock on 17 October and the customer coming back to the tab, the green "today" ring was still sitting on the 16th, while the shop's requests were going out on every beat. **What it already caught was a day passing its 6pm cut-off - that removes a day, which is a change - so the cut-off was honest and the calendar was not.**

Both now watch the clock as well as the data: your app whenever the page is returned to, and the shop on the same 30-second beat it already runs, so the day on screen is the day it really is.

Nothing to run, no SQL, no upload, no setting to change on either phone.

15 Oct 2026 - engine v512: THE CALENDAR'S ARROWS SIT BESIDE THE DATE, AND THE GREEN CIRCLE IS EXPLAINED (nothing to run, nothing to upload - pushing this is the whole of it).

One version from the bakery, and both halves of it are yours. Your words: *"on the calendar, the 2 arrows are too far away from centre, they should appear just beside the date range, and simple note that date in green circle are bake dates."*

1. The two arrows now sit beside the date range, not at the edges of the card. They were pinned as far apart as the card allows - one at each edge, with the date range stranded between them. **That was deliberate once:** the head was built as a three-column grid so the date stayed centred whether or not an arrow was drawn, and it did that perfectly. **But nobody looks at the middle of a card - they look at the month**, and the two controls that move it were the furthest things on the card from it. **It is one centred row now - the left arrow, then the date range, then the right arrow - in your app's calendar and in the shop's**, so the two read the same way.

2. And the green circle is explained where the customer meets it. A first-time customer saw a month of dates where some wear a green ring and some do not - and **the only thing that said what the ring meant was the day they finally tapped, in a line underneath. A symbol a stranger has to tap to decode has not been explained.** There is a quiet line under the calendar now: **"Days in a green circle are posting days."**

And the note draws the symbol rather than describing it - a real green ring, the same colour and the same 2px weight as the one on a posting day, because **a legend that names a colour has to show it.** That is not a theoretical worry here: this shop spent weeks drawing a green the app never used (found last version), and no test could see it. **The test now reads the legend's ring and the day's ring together**, so the two cannot drift apart.

It is written in all three languages. The English is mine to stand behind; the Chinese and Malay are my words rather than yours, there to be corrected like every other translated line - and they follow your own shop's words (the same words your posting-day line already uses, in both languages). **Tell me if you would rather it read differently.**

Nothing to run, no SQL, no upload, no setting to change on either phone.

15 Oct 2026 - engine v511: THE CHOSEN DAY IS A TERRACOTTA FRAME, AND AN ORDER YOU HAVE CLOSED LEAVES THE RUN (nothing to run, nothing to upload - pushing this is the whole of it).

Two versions from the bakery (v510 and v511), and both came from things you spotted.

1. The chosen day is a terracotta frame now, not a filled box - and the green ball keeps its ring. Your words: *"the date change to terracotta solid. Change that to a terracotta wire frame so that the wording inside can be seen clearly, the green ball dont have to go solid."* **You were measuring rather than guessing:** the chosen day was a filled terracotta chip with white writing on it, and **white on the brand terracotta measures 4.48:1 - just under the 4.5 floor** for body text, at 14px, in daylight. It is now a **2px terracotta frame around the whole**

day, with the date in the brand's dark terracotta at 5.51:1, and it is drawn as an inset shadow so **nothing on the calendar shifts by two pixels the moment a day is chosen.**

And the green ball keeps its ring, in both the app and the shop - the rules that were eating it are **deleted rather than overridden**, so they cannot come back. **That ring is the only mark saying a day is one you post on**, and choosing a day does not un-post it: it was being taken away at the very moment you were deciding about that day.

2. And the shop's calendar is the same frame, in your words: *"i want it to be highlighted by terracotta wire frame only, the green outlined ball dont have to turn solid"* - and then *"oversize like the one at app orders"*, *"rounded recangular"*, after a first attempt that ringed the number instead of the day. **The shop's day box also grew from 38 to 44 pixels**, because at 38 the ball left only 3px and the frame read as a band against it rather than a box around the day - and **38px was under the 44 the web guidance asks for anything a thumb has to hit.**

3. And a real fault came out of that measuring: the shop and the app were drawing two different greens. The shop's calendar asked for a green **with a fallback attached** - a colour its own stylesheet never actually defined - so it quietly used the fallback, and **the day number of every posting day a customer could choose measured 3.65:1, under the readability floor.** A fallback is not a declaration, and nothing was comparing them. **Nothing changes on Munchies here: your app already declared its own green back in v369, so it has been right all along** - this is the bakery catching up.

4. An order you have marked Collected / Posted now leaves the delivery run. Your words: *"order that already mark shipped still appear in delivery run"*. **This is the third time this same shape has come up, and it is worth seeing them together: the run was asking a question that was right until the world moved on.** First it was a van sent to your own kitchen; then a van sent for an order the courier had already delivered; and now a van sent for an order **you had already closed with your own hand.** There are two ways an order "has already gone" - the courier telling the app, and **you telling it by pressing the stage on the row** - and only the courier's word was being read. **Now one function answers it for everybody**, because a third copy of one rule is how three come to disagree. **And the boundary is deliberate: only the last stage takes an order off the run** - packed treats are exactly what a run exists to carry.

Nothing to run, no SQL, no upload, no setting to change on either phone.

13 Oct 2026 - engine v509: THE RECEIPT CLEARS ITSELF, AND NOTHING ON THE SHOP RELOADS ON A TIMER (nothing to run, nothing to upload - pushing this is the whole of it).

Eleven versions from the bakery (v499 to v509), and your own question is the whole of the first one.

1. NOTHING ON THE SHOP RELOADS ON A TIMER ANY MORE. Your words: *"is automatic reloading of store a good idea? What i try is adding customer details in it, the offer still shows"* - you were entering the next customer's details with the previous order's offer still on the screen, and that offer carried a **Reload** button. **A page that reloads under someone's hands destroys what they have typed**, and there is no honest way to tell "nobody is using this" from "someone is mid-order". **The guard it had asked only whether the page had been scrolled** - a proxy, and the wrong tool for a question whose wrong answer loses a customer's details. So the automatic reload is gone - **not disabled, not guarded. And it was never needed:** the form has cleared itself since v465, and the last order's receipt now goes the moment a new order begins - the first posting day tapped, or the first thing in the basket.

2. And then you settled it yourself: the card is deleted. Your words: *"it is good to see the page cleared. So i think we should remove the order to reload. Just need to make the green Order Received! message vanish after 1min, can?"* - and then *"make it 20sec instead of 1min"*. **The whole card - "Clear the page", "Not now", and the Reload button - is gone**, from the page, the styles and all three languages, along with every test of it. **The receipt now simply goes by itself, 20 seconds after the order**, and it still goes if the phone was put down, because the page asks the clock again when someone looks at it. **The one cost, named plainly: the order number and the tracking code are on that receipt, so 20 seconds after the order they are off the screen.** That is your call, and the bakery confirms every order by hand on WhatsApp, where the customer keeps it in

writing.

3. The out-of-date line is a proper strip now, and pinned where you can always see it. Your words: *"now see the warning, it is rather small. it should be a strip like what is having in the backoffice"* - and then *"i want it to be consistant with the app"*. It wears **your app's own amber pair**, the words are 16px (they were 14 - a caption) and the press is a real 44-pixel button in your shop's terracotta. **And it is pinned directly above the Order bar, exactly where your app pins its own - with its own space reserved**, so it never covers the thing you are being told to press.

4. And three real faults came out of your screenshots. The next customer was inheriting the last one's collection centre: the after-order reset cleared the *way* and never cleared *which centre he chose* - so the next customer opened step 3 with the previous one's centre still engaged, and an order could even be stamped with a centre the customer never picked. **And the whole of step 3 now comes back:** the card folds, the address clears, and the fourth circle beside Place order goes out - it was left lit, so the shop arrived claiming a step was done before anyone had done anything. **And an order the courier has already delivered no longer appears on the delivery run** - the same shape as the kitchen fault: *a question that was right until the world moved on.*

5. And there is a machine now, because you asked for one. Your words: *"i think you alway forgot. How to make sure not to forget"*. **A test now puts two different customers through the shop and asserts the shop comes back in the same state** - deliberately not a list of things to reset, because a list is a thing somebody has to remember to add to. If anything survives, one customer's value meets the other's and they disagree.

6. And a new guard on this manual itself. The manual is built by a script whose own copy sits inside quoted text, where a stray quote stops it parsing - and it fails **quietly**, leaving the previous PDF in place looking perfectly fine. It happened three times in one day at the bakery. **A test now runs the script's own parser over the builders, so a broken manual can no longer sit there looking right.**

Nothing to run, no SQL, no upload, no setting to change on either phone.

13 Oct 2026 - engine v498: NO VAN TO YOUR OWN KITCHEN, AND A RECEIPT THAT OFFERS TO CLEAR THE PAGE (nothing to run, nothing to upload - pushing this is the whole of it).

AMENDED 13 October 2026, later the same day: the clear-the-page press described in this entry has since been REMOVED - see the entry above. The receipt now clears itself, 20 seconds after the order. Everything else here (the kitchen fix, the build-number fix, the frozen-timer fix) stands.

Five versions from the bakery (v494 to v498). Two of them come straight from things you reported, and one of those is the delivery-run item you spotted on 12 October.

1. NO VAN IS SENT TO YOUR OWN KITCHEN ANY MORE. This fixes what you reported. Your words: *"we dont do delivery run on self collection point which is the kitchen"*. You were right, and the reason is worth keeping: **the rule for what needs a vehicle was written back when your kitchen was NOT a place in the app at all** - a kitchen collection was simply an order with no collection Point, so asking "does this order have a Point?" was exactly the right question and a van could never be sent to your own kitchen. **Then the kitchen became a Point** (so you could pick it in the shop like any other place), and from that moment it had a name and an id - and the run started sending a van to your own kitchen to fetch treats that were already there. **Nothing about the rule changed; the world it described did.** A rule written when the world had one shape, and never revisited when the shape changed. It is now read off the **kitchen switch** on that Point, never off what the Point is called - so a place you happen to name "kitchen" is still an ordinary place a van has to reach, and an order collecting at a friend's shop still needs the van, because the treats have to get there.

2. After an order, the receipt offers to clear the page - and then does it by itself. The form has always cleared itself the moment an order lands; **what did not clear was where the page was sitting** - at the bottom, resting on the receipt. So the next customer picks up the same phone, sees a confirmation, and has to work out that the shop above it is already fresh. A minute later a quiet line appears on the receipt - **"All done? Clearing the page starts a fresh shop for the next order"** - with one press. **Showing it moves nothing:** it only speaks, and if nobody attends to it the page makes the same wait again and then clears itself. **And the receipt stays** - the order number and the tracking code are on it, and those are the two things the customer needs

next. **It never moves a page somebody is reading:** if the page has been scrolled at all since the offer appeared, the automatic clear does not happen.

3. And it is a press that does what it says. The first build of this was a **scroll to the top** labelled "Back to the top" - my words for my action, when what you asked for was to clear the page. So every round you were told about something that was not what you asked for, and every report back said "no reload", which was exactly right. **A button that says something other than what it does is a lie, however small.** It now genuinely clears the page - a fresh shop for the next customer, and a phone sitting on an old build picks up the new one - and it says what it costs before you press it: the order number and code on that receipt go with it, and nothing else does.

4. And it is big enough to press. Measured in the stylesheet: the sentence was wearing a caption's style and the button was the build notice's small inline link, giving it a **26-pixel target**. The floor for anything a thumb hits is 44 pixels. **So the one control this whole feature exists to get pressed was the smallest thing on the page.** The sentence is now 16px in the shop's own ink and the button a real 44-pixel target in the shop's terracotta. The fault was reusing a style that belongs to another job.

5. The shop can now tell which build it is - and this was a real hole in the warning you asked for. Your report: *"now v495, but store at v494, why no update warning at store?"* **You had found the one hole that matters.** The out-of-date line was reading **both** of its numbers out of the same little file - which build this page is, and which build the site is serving. **Two readings of one file cannot disagree about which build the code is.** So a browser that refreshed that little file made a shop running month-old code report itself as perfectly current, and say nothing - **the one fault the notice exists to catch was the one it could not see.** The number is now written into the shop's own file, so an old copy carries its own old number with it and no cache can separate the code from its build. **And a test fails the moment the two numbers disagree**, so a forgotten bump is a red suite rather than a shop that lies about which build it is. **One honest note on the same report:** a page that is *already* stale when a fix lands has no code in it to warn about the fix - it needs one reload. That is the shape of the problem, not a fault.

6. And the offer no longer waits on a clock nobody is winding. Measured on the real page: a tab nobody is looking at has its timers frozen, and that is the ordinary case here - a customer who has just ordered puts the phone down or opens WhatsApp to send it. So the one moment this is for (the next customer picking the phone up) was exactly when the timer had not run. **The shop now looks at the clock when someone looks at it** - the page coming back into view, the tab being focused, or a finger on the page - so it can only make the offer arrive, never arrive early. And the page only clears itself once the offer has itself been readable for its own minute, rather than the two happening in the same breath.

Nothing to run, no SQL, no upload, no setting to change on either phone.

12 Oct 2026 - engine v493: EVERY PLACE YOUR MONEY SITS IS AN ACCOUNT, AND THE WHOLE MONEY SCREEN IS ONE TABLE (nothing to run, nothing to upload - pushing this is the whole of it).

This is a seventeen-version catch-up from the bakery (v477 to v493), and most of it is one idea. The app kept two sets of names for the same thing - the "purse and phone" against the "pockets" - and that is why you asked twice what the difference was. **It is one word now, and it is the word you chose: an account.** Your tin, your phone, the bank, LK's hand, a loan - every place your money sits.

1. The Money screen is one shape: one block per account. What came in, what went out, what it holds, and the total under them. **Every account you have gets its own block**, in your own order, even a brand-new one sitting at RM 0.00 - and a block is a door: tap it and that account's own journal opens, saying honestly that nothing has moved that way yet rather than looking broken. Before this, an account you had just added **showed nowhere at all on that screen**, so adding one looked as though it had done nothing.

2. One Pay press, and the list it opens is yours. How a customer paid you was two fixed buttons, Paid Cash and Paid TNG, and nothing else - **a bank transfer had nowhere to go at all**, and adding an account meant changing the code. There is now one **Pay** press that opens your own list, in your own order, with **+ New way to pay** at its foot: add an account while you are recording the payment and it comes back already picked. **And the**

tag on a paid order now says one word - Paid. It used to print the method, and it read "if it is not cash, say TNG" - so an order paid into a bank account wore a TNG tag. A word that looks right and is wrong; one word cannot be wrong about which account it was, and the account is not lost, because it is on the order's own card and has a block of its own.

3. Your Net counts every account now. It used to be the tin and the phone only, so a customer's bank transfer sat on its own line and was kept out of it. You were shown the trade-off in full and chose **one figure for everything**: everything that came in this stretch, less everything that went out, through every account you have. **The honest consequence, which you took with your eyes open, is that Net no longer matches what you can count by hand** - so for the tin and the phone, read that account's own block. Every account also keeps its own line and its own **running balance** on its journey, and there is one **Reconcile** sheet with every account on it: what came in, what went out, what it holds, and the total. That is the sheet you take to the tin, the phone and the bank.

4. A rename moves everything now - including the orders. This was a real fault. Renaming a way to pay rewrote your spending and your deposits and **never the orders** - so every order paid the old way stayed filed under a name you had retired, and your figures split in silence. That is what the "Cash" block holding money you could not find on your own list was. A rename and the new **Move these** button now call the same code, so neither can move two places of three again. **And a retired account is no longer a dead end: Move these** moves every record filed under the old name onto the account you choose - orders, spending and money you put in alike.

5. Something a transaction uses cannot be deleted. A category an expense was filed under, a source a money-in row came from, or an account anything moved through: **if any record uses it, it cannot be removed** - and the refusal says how many, and names the way through. **Cash and TNG are locked outright**, with their Delete not drawn at all, because the tin and the phone figures are built on those two. Your worry about a future exception is exactly why the refusal is never just "no": the account's own Edit form carries **Move its records...**, so no case - privacy, a mistake, something we have not thought of - can ever be stuck behind this rule.

6. Making a paid order cheaper is no longer silent. You asked whether reducing an order's quantity takes paid money back out of the accounts. **It does** - the paid money is counted off the order's own lines, live, so a smaller order is a smaller payment the moment it saves. But your hand does not change, and nothing said so. **The Edit card now says it while you are editing**, with the figure, and saving asks first. **And it asks you to refund:** the second question names the amount and opens the refund on that order, rather than telling you to go and find it - a figure you have to remember and retype is a figure that gets retyped wrong. Deleting a paid order takes the money out at once, puts the order in the **Removed** list so it can be put back, keeps the ingredients consumed and keeps the receipt number claimed.

7. The accounts are one table, and the shop stopped jumping. The figures on the Money screen used to break mid-number ("RM 480.00" wrapped over two lines by one pixel) and wander left and right between rows. The heading and every row are now tracks of **one** grid, so every column is as wide as the widest figure in the whole table and nothing can break mid-number. And in the shop, after **Place order**, there were **two scrolls fighting** - one to the top and one to the notice, which is the last thing on the page - so a customer watched the whole shop fly past him upward and then all the way back down. The top one is gone.

8. The shop now tells a customer when they are looking at an old page. You already had this - you had it built at v476 - and this is the bakery's version of the same thing arriving, so **nothing about your shop changes.** What changes is underneath: one shared rule instead of two, and a wider set of tests over it.

9. And the shop now names the first step that is missing, not the step that was pressed. This was a real fault on the bakery's shop: a customer who had not chosen a posting day and pressed a delivery card was told about the basket, while the three circles beside it said step one - **the page disagreeing with itself about one order.** Every refusal now asks one question - which step is first not done - and the circles ask the same question, so they cannot drift apart. The warning sits under the field it is about, the page is carried to it only when it is not already on screen, and three banners at the foot of the page that said the same things (and fought that scroll) are gone.

10. And no sentence in the app names an account of yours any more. A caption that pointed at "the Cash and TNG blocks" was wrong the moment you renamed Cash - an account's name is yours, so nothing can name one. The little grey notes ("your tin", "your phone") are gone from the money lines too: **an account's name is the whole of what its line says.**

Nothing to run, no SQL, no upload, no setting to change on either phone.

12 Oct 2026 - the shop now says when it is out of date (same engine v476 - pushing this is the whole of it).

Why. You told me the shop was behaving differently from the bakery's. It wasn't - **your phone was running an old copy of the shop page**, and there was no way to tell. Your *app* has warned you about that since v255 (the amber "Update now" strip); **the shop had nothing.** So you spent an afternoon looking at a shop from weeks earlier.

Why the shop needs it more than the app. It re-fetches its *numbers* every 30 seconds but **never its own code** - so a stale page shows **fresh figures on old behaviour**, every number on the screen agreeing with you. That is the worst way round for something to be wrong.

What it does now. A quiet line at the very top of the shop - *"A newer version of this page is ready."* - with a **Reload** press, shown **only when the site really is serving a newer build**. It is deliberately not a warning: nothing is broken and the customer has done nothing wrong, so it reads calmly and it moves the page down rather than covering anything.

And it is honest about the second try. Pressing Reload can bring the same old page back - a browser holding the old script will serve it again. When that happens the line changes to say what actually works - *"Close this page completely, then open it again"* - and the press goes, rather than leaving a button that does nothing.

It says nothing when it cannot tell. No connection, a WiFi login page, a half-loaded file - all silent. **A shop that cried "out of date" on a bad connection would be worse than one that never spoke.**

Nothing to run, no SQL, no upload, no setting to change on either phone.

12 Oct 2026 - THE BIG CATCH-UP, PART TWO: YOUR SHOP PAGE (same engine v476, service side only - nothing to run, nothing to upload; pushing this one is the whole of it).

This is the second half of the catch-up. Part one changed the back office; **this one changes the page your customers order from**, and it is the bigger of the two.

A four-step order, said out loud. The page now explains itself: **pick your posting day, choose what you want, tell us about you, place the order** - four numbered steps, with the circles turning green as each is finished. **And it answers the question it used to leave you to work out:** what a posting day actually is, and the next day you can order. A customer no longer reads the rules and has to guess the answer.

Your calendar answers a tap. Tapping a day you do not post on, or one whose window has closed, now says so in a line under the calendar instead of doing nothing at all.

The two ways to get an order now say what they are. "Post (nationwide)" and "Collect (local)" each carry a sentence explaining what they mean and what they cost - because a customer once ordered a post when he wanted to collect. **And changing your mind asks first**, naming what it is about to clear.

Prices and quantities. You can now **type** a quantity rather than tapping + thirty times. And a saved shopping list's items show what they will cost.

Everything your customers read was rewritten in all three languages - English, Chinese and Malay - in the words a Malaysian customer actually uses, not a translation of the bakery's.

Two things are deliberately the way they were. Your shop still opens with **Post (nationwide) already chosen** - the bakery's new version makes the customer pick, and you chose to keep yours. And it still asks for an address on a posted order, your own rule.

Nothing to run, no SQL, no upload, no setting to change on either phone.

12 Oct 2026 - engine v401 -> v476, THE BIG CATCH-UP, PART ONE: your back office (ONE DATABASE SCRIPT TO RE-RUN - see the end of this entry. Your SHOP PAGE is deliberately NOT changed by this update - see "What is not in this one" at the foot).

This is the largest catch-up this app has taken: seventy-two versions at once. It is being done in two parts on purpose. **This part is everything behind the scenes - the back office and your records. Your shop page is untouched**, so nothing your customers see changes today.

The one that matters most: your syncing is now protected. The bakery hit a fault in this batch where **one unsendable record made the server refuse the whole queue** - so every phone stopped syncing, and it lost real work. Munchies had the same unguarded code. **A record that cannot be sent is now left out of the batch instead of poisoning it**, so the rest still goes, and the app tells you what is stuck rather than sitting there quietly.

Check my system - a new screen under More. Press it and it reads your records, your shared data and your oldest recent backup, and tells you in plain words what is wrong: an order that appears twice, an order on the list *and* on the Removed list, an order sitting on a day you no longer have, a coupon counted twice. **A check that could not look says so rather than counting as a pass.**

A removed order is no longer gone. Delete an order by mistake and it sits in a **Removed** list for two days, where you can put it back. The bakery added this after losing orders it could not get back.

Receipts you can void, with a reason. A receipt number stays spent for ever, but you can now mark one void and say why - so your register reads as explained rather than as a hole.

Money and Profit can look at any day, week or month. Both screens gain a picker, so you are no longer limited to today / this week / this month.

"Paid" now means the money actually arrived. You reported this yourself: a customer's card said they had paid when they had not. The stage and the money are two different facts, and the screen now says which is which.

And an order can be fixed after the fact. You can move an order to a **past** day, and changing its status now moves the **whole** order rather than only the lines still on it.

Smaller repairs from the same batch. An order's number is now fixed when it is created and never moves (it used to be possible to end up with two orders sharing a number); editing a saved shopping list removes only the lines you actually removed; a coupon survives an order being split; and a hand-written list can be back-dated.

Before you push - the one thing you must do. In your Supabase project -> SQL Editor, run **supabase/order_alerts.sql** again. One line in the phone notification now names the day honestly. It is a plain "create or replace" and is safe to run as many times as you like.

What is NOT in this one - on purpose. The same batch also rebuilt the **shop page** (a four-step order flow, and a good deal more). That is your customers' page, so it is being done as its own separate pass where it can be tested properly on your phone, rather than bundled in here. **Until then your shop works exactly as it does today.**

9 Oct 2026 - engine v400 -> v401, ONE VERSION: a shopping list you can write yourself (nothing to run, nothing to upload - pushing this one is the whole of it).

Your ask: *"I want to add a manual PO issuing, the rest of the po process follow what we already have for po processing. I think a thing only different is it dont tie to specific bake date"* - and, asked what goes on it, *"Just pick products to bake"*, with *"then i can use the amend to adjust the details"*.

The Purchase Order screen has a new press: Write a list by hand. Pick the products and how many, and the app works the ingredients out of your recipes and prices them in whole packs from your cheapest supplier - **exactly as a day's list does**. Nothing about it is tied to a posting day.

Everything else is the same machinery, so nothing new to learn: the list appears in PO history with the same per-shop groups, the same **Buy / Not buying**, the same **Undo**, the same *Print this shop*, the same **Amend** for changing any detail afterwards, and the same stock and money behaviour when you buy from it.

Saving it never marks a day as shopped. A normal list remembers which days it covers; this one records **no day at all** - so it can never tick a day off your list, and deleting it can never un-tick one. That is the entire difference, and it is an **absence** rather than a rule, which means there is nothing that can forget it.

One corner worth knowing about. With **no posting days at all**, that screen used to stop at a single sentence - so the new press would have been invisible **exactly when it is most useful**. A list that needs no day has to be reachable when there are none, so the empty screen offers it too.

And a correction to some wording. Two lines that came over from the bakery in earlier updates still called the day a **"bake day"** - on the consolidated invoice and in the notice about an order that could not be taken in. Your app says **delivery date**, so they do now.

Nothing to run, no SQL, no upload, no setting to change on either phone.

9 Oct 2026 - engine v399 -> v400, ONE VERSION: the button says Buy, and becomes Bought (nothing to run, nothing to upload - pushing this one is the whole of it).

Why. Your question: *"can the button initially say buy, after buy it become bought?"* - and it does now. A shop you have not bought from reads **Buy**; press it, answer what you paid, and the press is replaced by the record - **"Bought 9 Oct, 11:42"** - with **Undo** beside it:

Mydin RM 96.00 [Buy] [Not buying] Yen Grocer RM 34.00 Bought 9 Oct, 11:42 [Undo]

The tick means something now. *"Bought "* was doing two jobs and only one of them well: on the press, the past tense plus a tick read as *already done* - but since the day you could undo it, pressing it only opens *"What did you pay?"*, and nothing moves until you answer. The tick belongs to the record, where it is true: **"Bought 11:42" can only appear once the packs really are on your shelf.**

And the two presses are a pair now. Buy and **Not buying** are the two answers to one question - did you buy at this shop? Before, the row mixed a verdict with a decision (*"Bought / Not buying"*), which read as though one had already been settled.

Nothing else moved. The record still says Bought, the stock card still says *"Bought - Mydin"*, and the card still counts *"Bought 1 of 2 shops"*. Only the press changed its word.

Nothing to run, no SQL, no upload, no setting to change on either phone.

9 Oct 2026 - engine v393 -> v399, SIX VERSIONS: your own money, more than one owner, and your own order (nothing to run, nothing to upload - pushing this one is the whole of it).

Your own money has a name now: an investment. When you pay for something out of your own pocket, the expense form used to treat it as one thing only - a debt the business owed you. It now asks **"Whose money paid for it?"** with two answers: **"The business owes me"** (what it always did - the pocket's line shows the debt, and *Pay back a pocket* settles it), or **"It is my investment"** (the money **stays in** the business, counts as money you have put in, and **nothing is owed back** for it). The two can never both claim the same ringgit - if you mark it as your investment, the pocket stops saying it is owed. **It is not asked at all when the till paid** - cash and TNG are the business's own money, neither owed back nor invested.

And there is a card called Your investment on the Money screen. What you have put in, what you have taken back out, and what is **still in the business** - counted **from the beginning**, not for this week, because a balance only means something counted from the start. **Open** shows every row behind those figures, as a book you can Print or Share. Money you put in that has since been spent on ingredients is still counted as in: it is your money, and it bought stock rather than leaving.

You can also move money between your own pots. A **Transfer** press beside *Put money in* - cash banked, TNG taken out for the float, and so on. Nothing enters the business and nothing leaves it, so a transfer is **not** counted as money in or money out. *Put money in* also asks **where the money came from** now, from a list of your own - beside your categories and your ways to pay.

More than one owner. Each owner now has their own share of the investment account, kept on its own: what **they** have put in, and what **they** have taken back. There is nothing new to learn - an owner is just a name on

the *From* list. **Money going out names its owner too**: recording a withdrawal now asks *"Whose investment does this come out of?"*. That question is asked **only** of a withdrawal - an ordinary cost like packaging is not anybody's money going back to them. Money with no owner named is still shown, in its own section, so nothing is hidden.

And you arrange your own ingredients. Every ingredient now carries a small **grip** on its left - drag it and the order is yours to choose, and the order is written onto the ingredients themselves, so it follows you to your other phone. An ingredient you have never dragged keeps exactly the place it has today. The card is roomier than it was, too.

Nothing to run, no SQL, no upload, no setting to change on either phone.

9 Oct 2026 - engine v384 -> v393, NINE VERSIONS: your stock, and the shopping you do to fill it (nothing to run, nothing to upload - pushing this one is the whole of it).

The journal now says where your stock went. Five different things move an ingredient's stock - a bake, a bought shopping list, a stocktake, an adjustment, a line coming off - and every one of them used to simply **overwrite the number**. Your question was *"i dont see a journal button, maybe there is never price movement"*. The journal now opens in two parts: **PRICE**, as before, and **STOCK** - every movement, in order, with what caused it.

It is called the Stock card now, and every ingredient has one. You asked for it twice (*"i want stock card"*, after *"cannot find it"*) and you were right: the thing existed but was hidden on the ingredients that happened not to show it, and it led with the price. **Stock comes first on it now**, and an empty card says what it is for rather than being left out.

The stock card is a real table, with three columns - **WHEN · WHAT HAPPENED · CHANGE** - and each line carries a small **x** to take that one line off, after asking. On a wide screen it opens wider than a phone column, so the words do not have to be cut.

Shopping lists now work one shop at a time. Each shop's list prints on its own (the day's ordinary Print is untouched), each shop's heading carries its **own Bought** press, and **Not buying** closes a shop you did not buy from - no stock, no money. Pressing Bought now puts only **that shop's** packs on your stock, where before it put every ingredient on the whole list on.

And Bought can be undone. Once a shop was marked bought there was no way back - you found that gap yourself. A bought shop now carries an **Undo** beside it, and it touches **only that one shop**: Undo Mydin and the other shop stays bought. It is called Undo rather than Delete because it is not removing a document, it is taking back a decision. The stock card is told: the packs coming back off are written as their own line, and **stock never goes below zero** - if a bake has already used those packs, the number stops at nothing rather than going negative. Two kinds of list deliberately do not offer it (one bought before it existed, and the paperwork behind it).

Deleting a shopping list now says what it leaves behind - it names the exact money figure before you agree, when there is one. The money stays on purpose. **And a row of buttons now wraps instead of running off the side of the phone** - your words: *"i cant find the delete button"*, and you were right: the row measured 403px across on a 360px screen, and Delete sat entirely off the edge.

Also: ticking a bake day on the Purchase Order page no longer throws the whole page back to the top.

Nothing to run, no SQL, no upload, no setting to change on either phone.

8 Oct 2026 - engine v382 -> v384, TWO VERSIONS (ONE LINE OF SQL TO RUN AGAIN - see the end of this entry).

A customer's review photo is now private until YOU publish it. This is a fix, and worth knowing about. When a customer added a photo to a review, it went straight into a part of your storage marked **public** - so **anyone who had its link could fetch it the moment it uploaded**, before you had ever seen the review. The rule on that folder said only approved pictures were readable; it wasn't true. Now the picture is uploaded somewhere **no anonymous reader can reach, not even with the exact link**, and when you press **Publish** your app moves it across to the part your homepage reads - in the same breath as publishing it. If that move fails, **the review is**

not published and the card tells you why, rather than putting a review on your homepage with its picture missing.

Taking a review down now really takes it down. Hiding it used to stop the homepage showing it while the file stayed reachable by anyone who kept the link. Taking it down now deletes the public copy too - and because the original is kept privately, publishing it again brings the **same picture** back without asking the customer for it twice.

Two smaller things from the same fix. A picture that cannot be shown now says so rather than leaving a blank where a photo should be - that card exists so you can judge the picture before publishing it, and a silent blank reads as "no photo". And both folders now refuse anything that is not a picture, and anything over 2 MB (your shop shrinks a photo to a small file before uploading, so the ceiling never turns a real one away).

And a phone that is behind now updates itself before the password screen (v383). Before this, a phone could still be running an older build at the moment you typed your PIN, and the update strip only appeared after you were already in. It now brings itself up to date first.

Before you push - the one thing you must do. In your Supabase project -> SQL Editor, run **supabase/reviews.sql** again. It creates the **private folder** the shop now uploads a customer's picture into, and sets the two folders' size and picture-type limits. **Until it is run, a customer's review photo cannot be uploaded at all.** Safe to run as many times as you like.

Nothing to upload, no Edge Function, no setting to change on either phone.

8 Oct 2026 - engine v369 -> v382, THIRTEEN VERSIONS (ONE LINE OF SQL TO RUN AGAIN - see the end of this entry).

A refund you choose. The refund press used to be all-or-nothing. It now asks **how much** and **what it was for**, and the refund card shows **what was charged** beside it, so you can see the figure you are refunding part of. The amount is recorded with the receipt - and once recorded, a later press that carries no amount cannot wipe it.

One invoice over a period - More -> Consolidated invoice. Pick a scope - **a day, a week, a month, or a single customer** - and get ONE invoice covering every order in it, with its own reference. It lays out as a filing list, says what was ordered, and **says plainly why a number is missing** when one is, rather than leaving a gap you have to guess at. It is built for a desktop screen, so it prints and files properly.

A real "forget me". Removing a customer's details now does what it says: their name and number come off the orders **and off the cards the shop publishes**, with an honest report of anything it could not finish - *"3 orders cleaned of their details - but 1 tracking card was not cleared"* - rather than a promise half kept. (Before this it was counting **items** rather than orders in that question, which is fixed too.)

The cost of an order is now frozen onto it. Until now, changing an ingredient's price rewrote the cost of sales on **every past month** - your own question: *"when i change the ingredient cost, for age orders, will its COS change?"* It did. Each order now carries the cost it had at the time it was sold, and you can **adjust** it if you know better, order by order.

Every order number is now a door. Press the order's code - on the receipt register, in a journal, on a customer's history - and the order itself opens. (On a customer's card it closes the card first, then opens the order, so the press never looks like it did nothing.)

Before you push - the one thing you must do. In your Supabase project -> SQL Editor, run **supabase/receipts.sql** again. It has changed: the refund now carries an amount, and until the script is run **the refund press will not work at all**. Safe to run as many times as you like.

Nothing to upload, no Edge Function, no setting to change on either phone.

8 Oct 2026 - engine v368 -> v369, ONE VERSION: the colours a customer actually reads (nothing to run, nothing to upload - pushing this one is the whole of it).

Why. Every screen was measured in a real browser for the first time, against the readability floor this app set itself in v313. That earlier fix had only ever been applied to the **grey small print** - the amber, green and red

had never been measured **where they actually sit**. Several were under the floor: some in your app, and the worst ones **on your shop**, in the three lines a customer reads most - the tracking-page step labels, the line naming the law at the foot of the privacy notice, and the line under Place order.

What changed on your shop. Those three quiet lines now use the same grey as everything else (they had been given their own, written by hand, which is why no check could see them). The **brand terracotta did not move** - on your instruction - so instead the fix was to stop using it for words: colours that were too light for text are now the darker shade, and your brand colour is still your brand colour wherever it is a button or a band rather than a sentence.

And one colour was never there at all. The two rules that draw the **day you can still order** and the **current step of a tracked order** pointed at colours the shop's own palette had never defined. With nothing to fall back on they simply took whatever colour was around them - so the one line a customer stares at while waiting on an order lost its colour, and the calendar's "still room" green had been drifting on a value nobody chose. Both are now the same colours your app uses, and the calendar green is a shade darker on purpose: the old one measured below the floor.

A check now catches that for good. The contrast test only ever looked for one named colour, so a second one walked past it. It now fails if either of your two stylesheets reads a colour the other one declares and it does not - which is what would have caught this before it shipped.

Nothing to run, no SQL, no upload, no setting to change on either phone.

8 Oct 2026 - engine v364 -> v368, four more versions (ONE LINE OF SQL TO RUN AGAIN - see the end of this entry).

A register for your receipt numbers. More -> Money now holds a **Receipt register**: every receipt number the app has issued, in order, and - the point of it - **any number missing from the run**. That is the page a business over RM150,000 has to be able to show, and it answers the question from outside rather than from inside the app.

And the gap in that run is closed. The way a number was drawn had a race in it: if both your phones drew one at the same moment, the loser's number was **lost and skipped**, leaving a hole in a run whose whole purpose is to have none. It is now drawn under a lock that only holds up that one order, so no number is ever skipped.

One number is said out loud, and two of them are not. If an order is paid but its number could not be drawn - a phone with no signal - the app now says exactly that, rather than leaving the line blank or blaming the order. An order that was **removed** says its number is gone with it; a number missing from the register is **named**, not silently passed over.

Clearing coupons, done safely. More -> The shop -> Bring a friend can now clear out coupons that are sitting on **no order at all** - the ones causing a stale "Apply coupon" on an old order. It is set apart under its own heading and dressed as a danger press, and it **deliberately cannot touch a coupon that is already coming off an order**: that coupon *is* the discount on that order's Total, so removing it would quietly put the customer's price back up. It says so on the card before you press it.

Every screen now checks itself on every change. A new pass builds **every screen in the app** - twenty-seven of them - twice each, once on a fresh install and once with real data in it, and fails if a screen crashes or draws nothing. A screen that opens blank and one that opens empty look identical to you and mean opposite things; now neither can slip through. It reads the list of screens out of the app itself, so a screen added later cannot quietly miss it. **What it cannot see is whether a screen looks right** - spacing, contrast, whether the words are ones you would use. That still wants your eyes on a phone.

Before you push - the one thing you must do. In your Supabase project -> SQL Editor, run **supabase/receipts.sql** again. It has changed (the skipped-number fix lives in it), and it is safe to run as many times as you like. **If you have not yet run supabase/tracking.sql from the previous update, run that too** - until it is in, the customer's tracking page does not work at all.

Nothing to upload, no Edge Function, no setting to change on either phone.

8 Oct 2026 - engine v344 -> v364, TWENTY VERSIONS IN ONE GO (ONE LINE OF SQL TO RUN FIRST - see "Before you push" at the end of this entry).

Why so many at once. This is the biggest single jump this app has taken. Twenty versions were built on the bakery first, and they all arrive here together. Three of them are different: they started with **your** report on this shop, were fixed here, and then went the other way to the bakery. Those three are marked below.

The one that matters: a customer's order had been thrown away. Your app would only accept a shop order if **every single item** on it was something you currently sell. You had paused one product, so an entire customer's order was refused - silently, with nothing telling you, and it would have been refused for ever. Meanwhile your shop had already counted it. **That is why the shop and the app never tallied.** It now takes in every item it can, writes the ones it could not onto the order itself with their quantity and price, and **says so on your Orders screen** - an amber note above the New-orders box, naming what the order is for and why it could not come in. It clears itself the moment the order can be read.

The day's count. Your shop, your Orders calendar and your Delivery Dates card each worked the same number out for themselves, and **your app's copy skipped any order whose product you had deleted, or whose recipe you had not filled in yet** - so a day with a real order on it could read **0/12** while the shop correctly showed it booked. One routine answers it now and all three read it. An over-booked day still shows **negative** on your app and **nothing left** on the shop: you need to know, a customer should never see a minus.

The product stamps. "Only N left" is worked out the same way, and now follows the same rule.

Your customers can no longer be read by anyone who asks. The order-tracking page asked the database for one order - but the permission it was allowed to read under said *every row is public*, and the key the shop uses is public by design. Anyone who knew that could have asked for the whole list and received **every customer's name and delivery address, and the delivery driver's name and phone**. The reading has moved inside the database, which now answers one code at a time and nothing else. **Nothing looks different on the page.** *(This is what the SQL line below is for.)*

Your shop now states how it uses a customer's details. A quiet line under Place order, opening into the full notice - what is used, who receives it, how long it is kept, how to ask for it to be deleted - in English, Chinese and Malay, closing on the law it is given under. Written to be read, not to impress a lawyer.

Every paid order can draw its own serial receipt number, and a refund is **marked on the order** rather than the number being reused - so the run of numbers never shows a gap where money moved.

Smaller repairs. The shop's fallback menu and fallback number now hold **what you really sell and your own number** (they held a sample product nobody delivers and, after the update, the bakery's number); applying a coupon no longer looks as though it did not take; and the van's price no longer asks you for the same pickup time twice.

Before you push - the one thing you must do. This release changes the database as well as the app, and **a push runs no SQL**. Open your Supabase project -> SQL Editor and run **supabase/receipts.sql** (the new receipt numbers) and then **supabase/tracking.sql** again (the new one-code-at-a-time reading - both files are safe to run twice). **Until tracking.sql has been run, the page a customer uses to track an order will not work**, so this one really does have to go in before you push.

Nothing to upload, no Edge Function, no setting to change on either phone.

7 Oct 2026 - no new engine (still v344), THE SHOP AND THE APP NOW AGREE ON HOW FULL A DAY IS (nothing to run, nothing to upload - pushing this one is the whole of it).

Why. You told me the shop showed one quantity for the 9th and the app showed another. You were right, and the cause was that **the same number was worked out three separate times** - once for what the shop publishes as "N left", once for the Orders calendar's chip, and once for the Delivery Dates card - and **two of the three were wrong in the same direction**, so the shop always showed the day fuller than the app did.

And the real cause, found from your own data: a customer's order your app had thrown away. While chasing the two numbers I asked your database one question - is there a shop order sitting unaccepted? - and

there was. **A real order from 7 October, for the 9th: 1 x Chicken Jerky (Taster), 1 x Pork Jerky (Taster), 1 x Pork Jerky, RM64.** It had been refused and forgotten from the moment it arrived, and the reason was one product: **you had paused Chicken Jerky (Taster)**, and the app would not take in a shop order unless **every single item** on it matched something you currently sell. One paused item threw away **the whole order** - quietly, with no message, and it would have kept throwing it away forever.

That is why your shop and your app disagreed: the shop had already counted that order, and your app had never heard of it.

What it does now. The app takes in **every item it can**, and writes the rest onto the order in plain words - its quantity and its price - saying it is not in the total below and to add it by hand if you can still make it. An order can no longer be lost because of one paused or renamed product, and you can no longer fail to be told.

What was wrong.

- The app's chip counted a day's orders with the routine written for **costing**, and that routine **skips an order whose product has been deleted, or whose product has no recipe filled in yet**. So a day with a real order on it could read **0/12** on your calendar while the shop correctly showed it booked. It also carried a note to itself saying the skipped order "was skipped", which is exactly the sort of quiet line that never reaches you. Where **two calendar records shared one day**, the chip counted only the record you tapped while the shop counted every record carrying that date.

What it does now. One routine answers "how many are booked on this day", and the shop, the Orders chip and the Delivery Dates card all read it. **An order is work you have to do that day: it counts, whoever it is for** - including a product you have since deleted, and one whose recipe you have not written yet. You took that order; the day has to carry it.

One difference that is deliberate and stays. The day's *capacity* was always one calculation and is untouched. If a day is booked past its capacity - you took more than the day can make - the app's chip goes **negative** and says so, because that is information you need; the shop publishes **nothing left** (never a negative), because a customer cannot be shown minus two left.

What did not change. No capacity, no limit, no day adjustment, no order, no price - and no figure was re-tuned. A day that was already reading correctly is still reading correctly. The shop's own number was the right one all along; it is the app that was undercounting.

And the same reading, on a product's stamp (added after your second message). A day was not the only number counted record-by-record. The quantity the shop stamps on a **product** - "Only N left" - is worked out from that product's share of the day, and the app's own product screens were still counting **one** calendar record where the shop counts the whole day. So the same order could sit on the shop's stamp and not on yours. It is now the same rule everywhere: **the whole day, counted once.** It changes nothing unless a day has been written down twice, which happens when both your phones add the same date before they have synced.

And one thing found while checking, fixed in the same push: the Guide was showing its own marking. The Guide's own cards are written with a pair of asterisks round the words that should stand out, and **nothing in the app ever turned that pair into anything** - so every card that used them printed the asterisks themselves, on your screen, for as long as the Guide has existed. It reads properly now: the asterisks are gone and the words they wrapped are simply bolder, as they were always meant to be. A check now holds it that way - it renders the whole Guide and fails if any paragraph prints its own marking, and it was bitten (the renderer removed on purpose made it fail, exactly as it should).

07 Oct 2026 - engine v344, HOUSEKEEPING: A DEAD LINE REMOVED (no database step, nothing to upload - pushing this one is the whole of it).

Nothing you can see has changed, and this entry is here so the version list stays complete rather than leaving a gap.

What went: the pickup reminder still held a line that said *"Packed and will be sent for delivery on the day it was ready"*. **It could not be reached by any order** - it has been dead since v340, when you decided a courier

customer should be told nothing at Packed at all - and it also named the **bake** day, which is the thing v337 to v343 spent a day and a half taking off every other surface. **It is deleted, so it cannot mislead anyone reading the code later, and a test now holds the rule it belonged to:** the pickup reminder says the order is READY and never that it is being sent - that belongs to the shipped message, at Collected / Shipped.

Your data is untouched. No SQL, no Edge Function, no upload. The suite is **2,857 tests, all green**, and the new guard was bitten: put the line back and it fails by name.

06 Oct 2026 - engine v343, A RUN IS ON THE DAY THE VAN COMES (no database step, nothing to upload - pushing this one is the whole of it).

Your words: *"the delivery run should not be on delivery date only, for the case of 7th delivery date order deliver 8th, his order should be appear only on date 8th. SO after his order, other order not specifing specific delivery will be on delivery date 6th"*

1 · A run's day is the day the VAN comes. It used to be the delivery date, always - so an order posted on the 7th with the van typed as the 8th still sat on the **7th's** run, which is the same confusion the last three versions have been unpicking everywhere else. **Now it is the day you typed on the order, and the order's delivery date only when you have typed none.** Your example, exactly: an order posted the 7th and delivered the 8th appears on the **8th's** run and on no other, and an order with nothing typed stays on its delivery date.

2 · A van day that is no delivery date gets a run of its own - the "not on delivery date only" half. Type a van day on a morning you are not baking and it appears in the day list as its own run, with its own stops.

3 · The run's own field says what it is. It read *"The delivery date"*; it is now **"The day the van runs"**, because it is the one control deciding which day's work you are looking at.

4 · The Run (N) badge on a delivery date counts the same way, so a card promising three stops opens a screen holding three. Where the van's day is no delivery date there is no badge for it - a van day has no bake-day card - but it is still reachable from the run screen's own day list.

AND YOUR CORRECTION ON v342 IS IN THIS VERSION TOO.

Your words: *"for the v342, you miss underrstood me, what i want is the courier booked details like the one we see after pressing GET A DELIVERY PRICE, i shows LALAMOVE is on this order, with check the trip, status, bookedz:3:25pm and lalamove link"*

You were right twice over. v342 unfolded a summary of the **order** - who it is for, the items, the money - which is not what you open a booked row to read. **And the thing you actually wanted was hidden by my own caution:** the trip's card lives inside the price section, under a flag I had turned off to be safe, so pressing the section's own button drew no trip at all. Two fixes:

- **See this order** is now **See the trip**, and what it unfolds is **the courier's own record of that booking**, saying the courier is on this order, with **Get a delivery price** right there. Open that and the trip's card is waiting - the vehicle, when it was booked, where it has got to, the customer's share link, **Check the trip** and **Cancel trip**. **The order summary is gone**, because it was the wrong thing.

Nothing on that panel can be edited, and nothing is asked of the courier until you press - a repaint of the screen never spends a quote.

Your data is untouched. No SQL, no Edge Function, no upload. The suite is **2,857 tests, all green**, with two more rules bitten: key the run by the delivery date again and the day test fails by name; put my "safe" flag back and the trip card's guard fails.

06 Oct 2026 - engine v342, THE RUN'S WARNING OPENS THE ORDER (no database step, nothing to upload - pushing this one is the whole of it).

Your words: *"on delivery run, it indicate an order which its lalamove already active, can clicking a button inside the red ribbon, drop down its full detail, as what is shown in edit order, get a delivery price."*

Done - and you chose for it to unfold in place, under the row rather than in a pop-up.

The button. A row whose courier trip is already booked wears the warning it always has, and that block now

carries a second press: **See this order**, beside *"Call off the trip and add to this run"* rather than instead of it. It folds the order out underneath its row; **Hide the order** folds it back.

What drops down:

- **who** it is for, and their number; **what** they ordered - each line as it was sold, with the quantity and the price that line went out at; **where** it goes: the address, or the Point it collects from; **when** - the delivery date, the day the van comes, and **your pickup time**, each on its own line; **their note**, when they left one; **what it comes to** - the same rows the Edit card and the invoice show; and at the foot, **Get a delivery price**, which asks the courier what that one order would cost.

Not one figure on it is worked out fresh. The money is drawn by the same renderer, from the same money function, that the Edit card, the Note / tracking card, the invoice and - most importantly - the customer's own message and tracking page all read. **A summary written only for this screen would have been a second figure, and a second figure is one that can disagree with the message you have already sent.** The days use the same wording those messages use, so the delivery date and the van's day are said apart here exactly as they are there.

Nothing on it can be edited. This is the screen you work on while a van is out, and every control that changes an order already lives in one place - the Edit card this panel deliberately mirrors.

Your data is untouched. No SQL, no Edge Function, no upload. The suite is **2,854 tests, all green**, and the two rules that protect it were bitten: move the detail inside the row and it fails naming the trap; let its open state live in the drawing rather than beside it, and it fails too.

06 Oct 2026 - engine v341, A PICKUP TIME IS NOT A PROMISE (no database step, nothing to upload - pushing this one is the whole of it).

Your words: *"for courier lalamove, there is no delivery window open and delivery window closes promise, it is more for self collection point, and for delivery run that courier have to go many stop, and delivery time have to be a window reasonably. Pickup time is something user should specify."*

You are right, and this corrects what I built this morning. A single van delivery is not promised to a customer as a window. A window is the right promise only where a van has **many stops** (the run) or where a **Point's own hours** are the promise - and both of those already exist. What you fix on a courier order is the **pickup time**: when the van collects from you.

So the courier half of the card now asks both, and they are kept apart:

- **Pickup time** - new. The time the van collects from **you**. It fills in the **Delivery run's** own "the day and time the driver collects", so that screen no longer opens on a time the app thought of. **Delivery window (opens / closes)** - kept, and now plainly the **customer's** promise. It is set for one of two reasons: a **run with several stops** needs one, or **the customer themselves has told you the hours they are available** - *"from 2 to 5"*, say. Left blank, the message promises to confirm the time.

The pickup time never reaches a customer. A van collected from your kitchen at nine is at the customer's door later than nine, so nine is not theirs to be told. It is stored on the order, remembered, and used by the run - deliberately not published in the confirmation, in the messages or on the tracking card. A test pins exactly that, and it was bitten: let it leak and the test fails showing the customer being told , 09:00.

All three doors to a courier order ask it - **+ New order**, **Edit**, and **Note / tracking** - so whichever one you open, the same question is there and the same answer comes back.

Your data is untouched. No SQL, no Edge Function, no upload. The suite is **2,851 tests, all green**.

06 Oct 2026 - engine v340, A COURIER ORDER IS TOLD NOTHING AT PACKED (no database step, nothing to upload - pushing this one is the whole of it).

Your words: *"to make it simpler, can we do like this, when status changed to PACKED ... 2. for courier order, we take away the SEND SHIPPED MASSAGE. When status change to SHIP/COLLECT - Send ship message button, by that time, courier already collected, lalamove link can be shared, no more confusion."*

Done - and it removes a message that was saying something untrue. At **Packed** nothing has left your kitchen, yet the row offered a courier customer **"Send shipped message"**: *"your order is on its way"*, with a share link, before any driver had taken the job. That is the same confusion the tracking card was fixed for in v338.

Packed now offers:

- **self collect** - **Send pickup reminder**, unchanged. The treats really are ready, and this is what says so and where to fetch it. **courier** - **nothing at all.** There is nothing true to tell them yet.

Collected / Shipped is where the shipped message lives now - the one place it is offered, for a van order and a posted parcel alike. By the time you mark it there the courier has the order in hand: the message is true, its Lalamove link works, and the customer's own tracking page shows that link too.

Your data is untouched and no screen moved. No SQL, no Edge Function, no upload. The suite is **2,846 tests, all green**, and the new rule was bitten - put the button back at Packed and it fails by name.

06 Oct 2026 - engine v339, DELIVERY DATES MOVES TO THE SHOP (no database step, nothing to upload - pushing this one is the whole of it).

Your words: *"i think the delivery dates should not be at logistic, it should be in the Shop."*

You are right, and the reason is worth writing down. Logistic is the list of ways an order LEAVES your kitchen. A delivery date is not a journey - it is **the day the shop is open**: the day the storefront's own calendar offers a customer, and the only days it will take an order for. So **Delivery dates** now sits under **The shop**, at the top of the group, because which days you are open comes before what you are advertising.

Logistic keeps the four that belong to it: Delivery run, Send a van, Self collection Points, Parcel couriers.

Nothing else moved, and no address changed - every bookmark, #/deliveries included, still opens the same screen. The suite is **2,844 tests, all green**, and the guard that catches a screen going missing in a regroup was bitten: take the row out and it fails, naming the screen that can no longer be reached.

06 Oct 2026 - engine v338, THE VAN GETS ITS OWN DAY AND TIME (no database step, nothing to upload - pushing this one is the whole of it).

Your words: *"and after we fix the courier delivery date and time in +add order or edit order, the card should remember"* - and, about the tracking page: *"the lalamove link should not be there because the driver might not be confirming, it only create more confusion if they were to click the link. Lalamove link and Delivery: Finding a driver, should not be send at this stage."*

This finishes the job that began with your customer asking whether his delivery was Wednesday or Thursday.

1 · The order now carries the van's own day and time, typed by you. + New order and **Edit** each gain three boxes in the courier half: **Courier delivery date**, and a **delivery window** (opens / closes). **They always open EMPTY.** Nothing is ever worked out from the delivery date - the bake plan is a plan and not a schedule, as you told me in September.

2 · And they are remembered. This is the half that was missing. Until now an order had nowhere to write the day the van comes, so the app assumed the delivery date. The day and the window are now stored on the order itself, and reopening **Edit** brings both back exactly as you left them. Blanking a box takes it back off the order rather than leaving an empty word behind.

3 · The customer's tracking card stops gluing the van's window to the delivery date. For your case - posted Wednesday, van Thursday morning - it used to read Wed, 7 Oct ... 9-11 am, **a time on a day the van does not come.** It now names both days and puts the window with the van's own day; the message line is also called **delivery** rather than "pickup window", which is what your run screen has always called that value. **An order with no van day on it reads exactly as it always has**, so nothing already in your records changed.

4 · The Lalamove link and "Finding a driver" are not sent at that stage any more. Your words, and you were right - a link the driver may never take up only creates confusion. While a booked van is still **finding a**

driver, the customer's card now says nothing about the trip at all: no link, no status. From the moment a driver is on the way, both appear as before. **Nothing is thrown away:** the order keeps its link and its status, your own screens are unchanged, and only the sending stops. A parcel is untouched - its consignment number is exactly what a customer needs.

5 · The Delivery run no longer hands you the delivery date. Opening a run used to fill the **"day the driver collects"** box in for you with the delivery date - so for Mr Tan's Thursday van it read **Wednesday**, written in as though you had chosen it, and that is the day that would have been booked. It now comes from **the day you already typed on the order**, and is empty when you have not typed one. The price box inside an order does the same.

One thing deliberately left alone: the run's **time** box still opens on your own dispatch time from Settings. That time is **when the van collects from you**; the time on the order is **when the treats reaches the customer** - different by the length of the journey - so seeding one from the other would send the van to your kitchen before the treats had left it. If you would rather it opened empty each time, that is a one-line change.

Your data is untouched. No SQL, no new column, no upload: the day and window ride on the order, and the customer's card carries them inside the text it already publishes. The suite is **2,843 tests, all green**, and three of the new assertions were bitten - put the old behaviour back and they fail by name.

06 Oct 2026 - engine v337, THE MESSAGE NO LONGER CALLS THE DAY ONE THING AND MEANS ANOTHER (no database step, nothing to upload - pushing this one is the whole of it).

This version is one repair and one guard. The repair is the one that reached a real customer.

1 · The confirmation message called the delivery date a delivery. You asked which day a courier order is on, and the answer is: **the day it is made and posted** - the van goes whenever you book it, which may be the next morning. So a van booked for **Thursday morning** was announced in the message as *Delivery: Wed, 7 Oct* - and that is why a customer had to ask you whether it was the Wednesday or the Thursday. **The message now says the two apart:**

- a **posted** order reads *Posting day: Wed, 7 Oct - Post (nationwide)*, and on its own line *Your courier delivery time will be confirmed separately* - because at that moment the app does not know the van's window yet, and it was stating one anyway; once you **have** booked the van, its window goes on **its own line** - never glued to the day it was posted; a **collection** order is unchanged (*Delivery: Wed, 7 Oct*), because your kitchen or a Point is genuinely where and when it changes hands.

2 · The app's own day keeps its own name. The bakery renamed its day to "bake day" in this version. **This app did not follow that** - your admin still calls it a **delivery date**, which is the word you have used here all along, and the storefront still calls it a **posting day**. Nothing about the screens changed for you.

3 · And a guard that finds a signpost pointing at a screen that no longer exists. When a screen is renamed, any sentence that names it becomes a dead end - and it was found by *building the guard*, not by looking. The new test/screen-names.test.js reads every **"More -> X"** in the app and checks it against the names that really exist, taken from the app's own route table and menu. It caught several stale signposts in this app's own Guide the first time it ran, and they were fixed rather than the guard being loosened.

06 Oct 2026 - engine v336, YOUR OWN REWARD COUPON NOW COMES OFF YOUR ORDER (no database step, nothing to upload - pushing this one is the whole of it).

Your words: *"when i refer a friend and get a coupon, but redeem that coupon will not reduce my order price"* - **and you were exactly right, again.**

What was wrong. There are **two kinds of coupon** in the app, and only one of them was ever being read:

- **The friend's** - RM3 off *their* first order. That is the one I fixed in v322. **Yours** - the RM3 you earn for bringing someone in. **Nothing ever took that one off anything.**

So when you pressed "Apply coupon" on your own order: the coupon was used up, the app said *"already taken off this order"* - **and the price did not move.** The coupon was spent and gave you nothing.

Why the two behave differently, and it is a real difference. The friend's coupon is earned and spent **on the same order**, so the order is written on it. Yours is earned on **the friend's** order and spent on **one of yours, later** - so the order number written on it is the wrong one to look for. **The app was looking for the friend's kind of coupon and never for yours.**

Now fixed. Pressing **Apply coupon** on your own order marks the coupon *and records which order you spent it on* - so the money function can find it, and **the Total comes off.** Your receipt and the customer's message also now call it a **bring-a-friend reward** rather than "the one you were sent", because it is your own coupon on your own order.

Careful about two things it must NOT do, and both are tested: a coupon you are **holding but have not applied** comes off nothing - it is yours to spend when you choose; and a coupon you spent on **one order** cannot come off a second one.

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,825 tests, all green**.

05 Oct 2026 - engine v335, THE SELF-CHECK NOW SEES EVERY KIND OF CONTROL (no database step, nothing to upload - pushing this one is the whole of it).

It left screens out, and finding out why taught more than the check itself. It was blind to three kinds of control, not three screens:

1 · Tick boxes and switches. The bring-a-friend on/off, the day ticks on the Purchase Order, every product toggle. The check walked straight past them - and a switch whose handler breaks is exactly the kind of fault it exists to find.

2 · Number boxes. The amount fields, the daily limits, the discounts. Same - skipped.

3 · The screens that fill themselves in. The Reviews screen asks the cloud for the waiting reviews and draws its Publish and Delete presses only when the answer arrives. The check looked the instant the screen was drawn, saw an empty card, and said "nothing to press" - when what it was really looking at was a screen that had not answered yet. It waits one turn now. **That is a whole class**: anything that arrives from a fetch behaves this way.

All three were reproduced here before being believed, not taken on trust. Blinding the check to tick boxes drops it from 6,046 presses to 4,824 and makes the whole Purchase Order screen read as having nothing to press - every control on it is a day tick. Removing its one-turn wait makes the Reviews screen read as empty.

05 Oct 2026 - engine v334, THE APP NOW CHECKS ITSELF BEFORE IT REACHES YOUR PHONE (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your question, after a day of faults, in your own words: *"you work is not checked, how can you improve?"* The honest answer is that the checks covered what was BUILT, not what you DO. Three faults that day were the same shape: a remove button asserted to be present and never pressed; a fault tested green on a made-up state; and a stray "null" found by a screenshot rather than by a test.

What it does now. It opens every screen the app has - the list is read out of the app's own route table, so a new screen is covered the moment it is given a route - and on each one it presses every button, link and dropdown, types into every text box, and then presses whatever those presses reveal, including a confirmation's Yes. **Anything that throws is a failure, and it names which screen, which control, and what it was doing when it broke.**

And the proof it works: a fault was put back to see whether the check would catch it. It did, and this is what it said:

/orders · revealed -> BUTTON "x" (after INPUT "Find an order - name, #code, phone...") · g is not defined

That is a dead button, named, on the screen it was on, with the press that revealed it and the error it threw. It is now found automatically, every time. Here it is **26 screens and 6,046 presses, in about five seconds.**

One thing it is not, said plainly. It is not a browser: it catches a control that breaks when used, never whether

a thing looks right or moves.

05 Oct 2026 - engine v333, THE REMOVE BUTTON I ADDED YESTERDAY DID NOT WORK (no database step, nothing to upload - pushing this one is the whole of it).

You asked the question that mattered: *"have you tested it really can delete?"* - and the answer was no. I had checked that the x was *there*, and that the row opens. **I had never pressed it through to a deletion. So I did - and it was broken.**

What was wrong. The remove button I put on a search result was wired to a variable name that does not exist in that part of the code - the name belongs to the New-orders box next door. Pressing it **threw an error and removed nothing at all.** A button that does nothing is the thing this app treats as a bug everywhere else, and I had shipped one.

Now fixed and properly tested - by pressing it. The test now **presses the x, answers the confirmation, and checks the order is actually gone**, from **both** doors: the New-orders box and a search result. It also fails with the old mistake put back, so it is a real check and not a hopeful one.

And it was worth asking for a second reason: the New-orders box's own x has been there since the box was built and had **only ever been checked for existing** - never pressed. It works, and it is now covered too. **Two buttons call the same removal; testing one proves nothing about the other.**

On v332's other half - nothing was lost. Opening an orphaned order was working, and still is. **No order was ever removed by the broken press**, because a press that throws removes nothing.

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,821 tests, all green**.

05 Oct 2026 - engine v332, AN ORDER WITH NO DELIVERY DAY CAN BE OPENED - AND A SEARCHED ORDER REMOVED (no database step, nothing to upload - pushing this one is the whole of it).

Two of your reports, and they were the same dead end met from two sides.

What an "orphan" is. When you delete a delivery day that still has orders on it, the app asks first and promises: **"The orders are kept in your delivery history."** You agreed, so they are kept - **but the day they belonged to is gone, and until now that left them stranded.** The row could not be opened (no arrow, no jump), so **the one thing you could do with one was delete a real sale** - and if you did not want to delete it, you could do nothing at all.

Both halves are fixed:

1 · The row now opens the order. Tap an orphan in the New-orders inbox, or a search result, and its **own Edit card** opens - the one with the **Delivery day** calendar in it. Pick a day, press Save changes, and the order is back where it belongs. **Nothing is deleted.** And that row now says what it is: *"its delivery day was removed - tap to put it on one"*.

2 · A search result can be removed. Your words: *"C2FDA5 i search this order, but no button to delete it."* **A search result had no controls at all** - no arrow, no x - so the one place you go to find an order by its code was the one place you could not act on it. **It carries the same x the New-orders inbox has always had**, and it still asks before it removes anything.

You chose this route yourself, and it is the safe one: **every orphan can be opened, put back on a day, or removed one at a time - and nothing leaves your sales or your Profit statement unless you remove that one order yourself.**

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,819 tests, all green**.

05 Oct 2026 - engine v331, A CODE SEARCH FINDS THAT ORDER AND NO OTHERS (no database step, nothing to upload - pushing this one is the whole of it).

Your report: *"C2FDA5 why when i type this 17 order found?"*

What was wrong. The search box has a second way of matching, added for a phone number typed with dashes or spaces - "*012-345 6789*" cannot match a number stored as "*60123456789*" character for character, so the

box strips everything that is not a digit and looks for those digits in the order's number. **An order code was falling into that same path.** "*C2FDA5*" carries two digits in it, so stripping the letters left "*25*" - and the search then found **every order whose phone number contains 25**. In a Malaysian book that is most of them: **seventeen, on yours.**

And it is worse than a long list. The one row you asked for is buried inside it, so a search that returns seventeen is a search that has stopped working.

Now fixed. A query with a letter in it is matched as text and nothing else - and a code never needed the number path anyway, because "*#C2FDA5*" is already in what the search reads. **Typing a code now finds exactly that order, with or without the #, in any case.**

And the number path still does its own job - "*012-345*", "*0123456789*", "*60123456789*" and "*016 555 7777*" all still find their order, exactly as before. **Both halves are pinned as a test**, so neither can be broken by fixing the other.

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,818 tests, all green**.

05 Oct 2026 - engine v330, THE BRING-A-FRIEND DISCOUNT SHOWS IN THE WORKING (no database step, nothing to upload - pushing this one is the whole of it).

Your report, and you were right about both halves: *"the discount dnt show in the total adding in edit, probably other place?"*

What was wrong, and it was worse than a missing line. v322 took the bring-a-friend discount off the order's Total and named it on the customer's message - **and left it out of the Edit card and the Note / tracking card altogether.** Those two do not read the order's Total; **they work the total out themselves** from the lines you are editing. So **the Total on the Edit card was the total BEFORE the discount** - and the same figure was missing from your receipt and from the invoice.

That is the figure you read while editing an order and quote to a customer. It now reads:

Items total - RM 30.00 · Bring-a-friend discount - -RM 3.00 · Total - RM 27.00.

Both cards, the receipt and the invoice now show the discount as its own line, right above the Total it moves - and the Total itself has it taken off.

The other place, answered. I swept every screen that shows an order's money. The discount was already named on the customer's WhatsApp message and on their tracking page. **The Edit card, the Note / tracking card, the receipt and the invoice were the ones missing it - and they were the ones that list the money as a sum you can check by adding it up.** The two pop-ups now price the discount through the same single rule the rest of the app uses, so **a figure she reads while editing and the message the customer gets can never be two different figures.**

AND THE SWEEP FOUND A SECOND FAULT, which I have also fixed. If a coupon was worth more than the order - the rare case, and a hand-given one could do it - the receipt said it had taken off the whole amount while the Total stopped at zero, **so the lines did not add up and the figure claimed more off than the order was worth.** It now takes off only what is really there: **a coupon can never make an order cheaper than nothing, and the line and the Total always agree.** That fix reaches the customer's message and their tracking page too, because all of them read the one money function.

And the rule is now a test, not a hope. The receipt's own check **adds up every line and compares it with the Total** - on a plain order, with a code, with a courier charge, with a COD charge and with a discount - so **the next time anything comes off an order and is not given a line, the suite goes red instead of the number quietly not adding up.** That is the check that found the second fault above, within seconds of being written.

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,816 tests, all green**.

05 Oct 2026 - engine v329, THE DAY THAT WOULD NOT OPEN: THE ACTUAL CAUSE, FOUND AND FIXED (no database step, nothing to upload - pushing this one is the whole of it).

The message you read out was the whole answer: "cur is not defined".

What it was. When I added the bring-a-friend discount on **v322**, one line that names the discount on the order's row used a value called cur - **and I never declared it there.** It was declared in the two blocks *next to* it, not in the one that used it. So the moment a day held an order that had been given a bring-a-friend coupon, drawing that day's panel **crashed**.

And that is why it looked like what it looked like. The calendar square is repainted a moment before the panel is built, so a crash while building the panel left **the square red on the day you tapped and the panel still showing the day before.** Exactly your recording, and your three words for it - **"the day like hang"**.

And it was never about 5 October or 7 October. It was about **any day holding an order that had been given a bring-a-friend coupon.** Only 7 October had one, which is why only 7 October refused. **Every day with such an order had the same fault** - and now none of them do.

I have to be straight with you about the road here. My first two answers (v326, v327) were guesses about your two days sharing an internal id. **They were wrong, and they cost you two pushes.** What finally worked was **v328 making the app report the error itself** - one tap and you handed me the exact line I could not see from the outside. **That is the lesson I am keeping.**

What stands from v326 and v327, and it is kept because it is good on its own: the day you are on now travels in the page's address (so a bookmark or a shared link opens the right day), the screen remembers which day each id meant, and a day whose records ever do share an id is separated at load, losslessly. **None of that was your fault** - but none of it is harmful either, and two of the three are small improvements in their own right.

And v328 stays, and it earned its place: if a day ever cannot be drawn again, the square will not turn red and the reason will name itself on the card, in one line you can read out. **It is the reason this fault is fixed at all.**

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,813 tests, all green**.

05 Oct 2026 - engine v328, THE DAY THAT WILL NOT OPEN NOW SAYS WHY (no database step, nothing to upload - pushing this one is the whole of it).

Why, and it is an honest one. You pushed v326, then v327, and told me both times that 7 October still would not open. **Two versions of fixing it from the outside got nowhere - and that is on me, because I was guessing.** What I could see was the shape of the fault from your recording: the day turns red and the panel underneath never changes. What I could not see was **why**, and the reason is in that one day's own records, which I cannot look at.

So this version stops guessing and makes the app speak.

The day is now built BEFORE the red mark moves. If it cannot be built:

- **the red mark does not move** - because a red day sitting over a different day's panel is exactly the fault you have been looking at, and it should never have been possible; and **the reason takes the day's place**, in the app's own words, with a line to read out to me.

What you will see. Tap 7 October on the new version and you will get one of two things:

1. **The day opens normally** - a plain square with its heading, its count and its orders. Nothing to do; it is fixed.
2. **A card headed "This day could not be opened"**, naming the day and ending with **"Tell whoever is at the shop this message: ..."**. **Read that sentence out to me and I will have the answer in one step** instead of another round of guessing.

Whichever it is, nothing is changed and nothing is lost - the card says so itself. 7 October stays in your list either way, and no order, product, price or posting day is touched.

If you get the second one, the message is what I need - not a description of it. The words after "this message:" are the whole answer.

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,812 tests, all green**.

Added 05 Oct 2026, the same day: you tapped 7 October and it said "cur is not defined" - and that line was the answer. See v329 above: a missing declaration added in v322, crashing the day's panel

whenever an order had been given a bring-a-friend coupon. This version is what surfaced it.

05 Oct 2026 - engine v327, THE DAY STAYS PUT WHEN THE SCREEN REBUILDS ITSELF (no database step, nothing to upload - pushing this one is the whole of it).

Why. You pushed v326 and told me plainly: *"the 7/10 still not working right."* **You were right, and my fix had a hole in it.** I found the hole by writing the test I should have written the first time, and it failed with your bug in one line: expected 'Thu, 10 Sep 2026' / actual 'Mon, 7 Sep 2026'.

What I missed. v326 made the day you tap travel with the tap - that part was right, and it is why the calendar square turns red correctly. But **the address the app writes carried only the day's internal id**, and the Orders screen **rebuilds itself from that address** every time the cloud answers, a pull lands, or the app regains focus. On that rebuild there is no tap and no remembered day - only the address. So the screen came back to the id, the id is shared by two of your days, and the panel opened the other one again.

And that is why you never saw it flicker: the rebuild happens in the same instant as the tap, before the phone has drawn anything. **The wrong panel was the only picture that ever reached your screen** - which is exactly what your recording showed: the red mark moves, and the panel does not change by a single pixel.

Now fixed twice over:

1 · The address carries the day, not only its id - #/orders?date=...&day=2026-10-07. A rebuild reads it back and opens the day you tapped. **A bookmark or a link you share now opens the right day too.**

2 · The app remembers which day each id meant, for as long as you are on the screen - so every other rebuild does the same: changing an order's status, saving a day's availability, and anything else that redraws the screen. **Thirteen separate places rebuild it; all thirteen are covered by that one memory.**

And this fix does not depend on the v326 repair having run - it holds whether or not your two days still share an id.

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,811 tests, all green**.

Added 05 Oct 2026, the same day: this did not fix it either, and the cause was never what this entry says. The real fault was v329 - a missing declaration from v322 crashing the day's panel whenever an order had been given a bring-a-friend coupon. Read v329 above. **The id-repair and the day-in-the-address described here are kept because they are small improvements in their own right, not because they were the answer.**

05 Oct 2026 - engine v326, A DAY YOU TAP IS THE DAY YOU GET (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your report: *"why the order calander not able to select 7/10/26?"* and then, exactly: *"that day shown 2/12, clicking that date, the date turn red, but the SET day's avaibility not changing to 7/10/26."* **You were right, and your recording showed it happening - four times in thirty seconds.**

What was happening. Each day in the calendar is drawn from that day's own record, so the **red mark followed your tap** - it always did. But the panel underneath was looked up a different way: by the record's **id**, and that lookup answers with the **first** record holding it. **Two of your delivery days are carrying the same id**, so tapping one of them turned the day red and then opened the *other* one. Pressing **5 Oct** worked and **9 Oct** worked because they are not the pair - **7 Oct** was the day it happened to.

Two fixes, and you get both:

1 · The panel now opens the day whose cell you tapped - the day's own date rides along with the tap, so the calendar and the panel under it can never be two different days again.

2 · The two days are given their own ids when the app loads - nothing is deleted, every day and every order and price survives, and **each order follows its own day** (its own saved delivery date is what tells the two apart). Your **7 Oct becomes openable, editable and deletable again** - and so does anything else the shared id was quietly affecting.

AND IT IS FIXED THE SAME WAY ON EVERY PHONE. The replacement id is worked out from the old id and

the date rather than rolled fresh, so both of your phones reach the **same** answer. Two different answers would have put the pair back with the next sync.

How it got that way I cannot say - I could not find anything in the app that makes two days share an id. What I can say is that the fault it caused is now impossible to repeat from your side, and if it ever happens again the screen will simply follow your tap.

Your data is untouched. No SQL, no Edge Function, no key. The suite is **2,809 tests, all green**.

Added 05 Oct 2026, the same day: this version did not fix it, and v327 is the rest of it. Making the day travel with the tap was right, but the app writes only the day's id into the address and then **rebuilds the screen from that address** - and on that rebuild the day was lost again. **v327 carries the day in the address and remembers it, and the fault is closed. Read v327 above.**

05 Oct 2026 - engine v325, THE FRIEND'S NUMBER COMES OFF THE NEW CUSTOMER'S ADDRESS (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your question, and it was a good one: *"for new customer clicking link from his friend, after he place an order have you remove his page linking his friend phone number?"* **I had not.**

What was happening. A referral link is /store/?via=60123456789 - **the friend's own phone number.** The shop reads it once, stamps it onto the order as *"this one came through their link"*, and then **the number just stayed there**: in the new customer's address bar, in their browsing history, and in anything they copied out of the address to pass on. After the order, it is only a phone number being carried around.

Now it comes off the address the moment the order is placed.

AND THE MOMENT MATTERS, which is why it is not simply done when the page opens. A customer can arrive by the link and browse for ten minutes before ordering - taking the stamp off on arrival would lose the referral altogether. So it goes when **the order really landed**, which is the same branch the app already uses to remember the order was placed. **An order that fell back to WhatsApp keeps its stamp on purpose** - nothing was recorded, they may try again, and the message they send carries it.

Only via is removed. If the same customer is also looking at an old order, track is a separate key handled by its own rule, and each removes only itself.

Your data is untouched. No SQL, no order, product, price or posting day moved. The suite is **2,805 tests, all green**.

05 Oct 2026 - engine v324, A TRACKED ORDER STOPS OWNING THE PAGE, AND YOU CAN FORGET A STRAY CUSTOMER (no database step, nothing to upload - pushing this one is the whole of it).

Two things you asked for, and they ship together.

1 · THE ORDER CODE COMES OFF THE ADDRESS BAR. Your words: *"when a customer track his order, his store version became associated with that order code. Anyway to take it off automatically?"*

The tracking link opens as /store/?track=CODE, and **that code used to stay in the address bar.** So the next time that page was opened - a bookmark, a history entry, a link re-shared - it landed back on **that one order's card** instead of on the shop. **The page belonged to an order rather than to the shop.**

Now the code comes off the address the moment the card appears, and the card stays up for that visit. The link still does exactly what it is for. The cost, said plainly: **a refresh now lands on the shop** rather than on the card - which is the direction you asked for, and their code is in their WhatsApp if they want it again.

AND ONE THING THAT WOULD HAVE BROKEN IN SILENCE: via is left alone. That is the friend's referral stamp, and a customer can arrive **by** a referral link and **then** track an order - stripping the whole address would have quietly broken bring-a-friend for exactly that person. Only track is removed, and that is a test.

2 · FORGET - for the strays. Your words: *"i need a button to delete a customer as well, i found there is few stray customer."*

READ THIS BIT, because it is why the button is where it is and not everywhere. Your customer book is **built from your orders.** Someone you added by hand has never ordered, so **their row *is* the record you typed**

- forget it and they are gone, name, number, reward and note, with a question that names exactly what goes and says **"nothing else in your book is touched."**

A customer who HAS ordered cannot be deleted from here, and it is not an oversight. Their row is their **order history**. Removing the record would leave the row standing - it comes straight back from the orders - while quietly throwing away their reward, their note and their dog's name. **A button that did half of what it said would be worse than no button**, which is your own rule about dead controls: two rows that look alike must behave alike. **For two records of one person, use Merge** - that is what it is for.

Your data is untouched. No SQL, no order, product, price or posting day moved. The suite is **2,804 tests, all green**.

*(One note: this commit carries two version numbers - v323 and v324 - because the tracking-card line and these two were built back to back and both were still uncommitted. The Engine on your More screen reads **324**.)*

05 Oct 2026 - engine v323, THE TRACKING CARD SAYS WHY THE TOTAL IS LOWER (no database step, nothing to upload - pushing this one is the whole of it).

Why. You asked for it after the last version: the customer's own tracking page was showing the **correct lower total with nothing saying why.**

What they see now. On the tracking page, under the items and the courier charge, a line in the customer's own language:

Bring-a-friend you were sent: -RM 3.00

- sitting where the promo line sits, so the figures above it still add up to the Total below. In In Mandarin it reads the friend-referral discount line and then -RM3.00, and in Bahasa Malaysia *Bawa rakan yang menghantar anda: -RM3.00*. It is the same sentence your WhatsApp confirmation uses, so the two agree word for word.

And it needed no database step, which is worth explaining because it nearly did. The obvious way was a new column on the tracking table - and the app's own code warns against exactly that: *"a missing column kills publishing for EVERY order silently."* **So it reuses the discount column that is already there**, which is honest because **a code and the friend's coupon can never both apply** - that column holds *the* discount on the order, whichever it is.

The small piece of cleverness, and why it is safe. The card tells the two apart by the code being **absent**: a discount with no code beside it can only be the friend's, because the app refuses to let a promo code be labelled without a name. If that ever stopped being true, a code would start wearing the friend's words - **so that rule is a test.**

Your data is untouched. No SQL, nothing to upload. The suite is **2,799 tests, all green**.

05 Oct 2026 - engine v322, THE NEW CUSTOMER'S DISCOUNT ACTUALLY COMES OFF NOW (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your two reports: *"the bring a friend discount used but not really create a discount for that new customer"*, and then *"there store front copy still hold the discount in cache."*

You were right on both, and the first one was worse than a wording slip.

What was happening. When you pressed **Give coupon**, the app wrote the friend a coupon - *"First order - via Jienny's link"* - and then **nothing ever read it**. The order's Total, the confirmation and the three later messages are all worked out by one function, and that function knew about promo **codes** and nothing else. So the coupon was recorded, the order was priced as though it did not exist, and pressing **Apply coupon** told you *"already taken off this order"* about a figure **nothing had ever taken off.** An app telling you it did something it did not do is the worst kind of fault, and I wrote that sentence.

What you see now.

- **The RM3 comes off the order's total by itself.** The customer pays the lower figure, so your money and your message agree. **The message says so, in its own line** - *"Bring-a-friend you were sent: -RM3.00"* - sitting with the items and Courier charge so the lines add up to the Total beneath them. A discount the customer

cannot find in the message is one they will ask you about. **The order says it too** - *" Bring-a-friend - RM3.00 already off this order's total"* - so the lower figure is never unexplained on your own screen either. **It cannot be used twice.** The coupon is tied to the one order it was born on, so their next order is priced normally no matter what any cached copy of a page happens to hold.

And your second report was a real gap of its own. Everywhere else in the app, anything that changes what a customer sees republishes their tracking page. **The Give coupon press did not** - so the customer's own card went on showing the price they were quoted *before* the discount. That is fixed: giving a coupon now republishes their card, exactly like changing the items or the address does.

Her own rule is honoured, unchanged: one discount per order - and if a customer typed a **code**, **the code wins** the total, because that is the one they typed and can see. **When that happens the friend's coupon is not spent** - it stays theirs for the next order, rather than being quietly burned.

One thing left, and I would rather say it than let you find it. The customer's **own tracking page** now shows the correct lower Total, but it does not yet say *why* - because adding that line there needs one small database step. **Their WhatsApp confirmation, which is where they are asked for money, already explains it in full.** Say the word and I will do the tracking-page line as its own version, with the one-line SQL for you to run first.

Your data is untouched. No SQL needed for this version, no order, product, price or posting day touched. The suite is **2,796 tests, all green** - including the one that would have caught this on the day it was written: not "does the coupon say the right thing" but **"does it actually come off the total"**.

05 Oct 2026 - engine v321, THE OFFERS WERE NEVER BROKEN - YOUR MAC ASKED FOR NO ANIMATION (no database step, nothing to upload - pushing this one is the whole of it).

THE ANSWER, AND IT CAME FROM YOUR RECORDING. Six seconds of your screen, one turn pulled out frame by frame at a tenth of a second: the message changed **between two frames**, with no half-way picture in between. The offers were changing the whole time. **What was missing was the animation - and the reason is a setting on your Mac.**

Your Mac has "Reduce motion" turned on. That is an accessibility setting in *System Settings -> Accessibility -> Display*. When it is on, the app is not allowed to animate, so it was showing you each new message **instantly, with nothing to see** - which is exactly what "no effect" looks like. The message *was* changing every 2 seconds; it just arrived with no movement at all.

AND THAT IS WHY I KEPT GETTING IT WRONG - I HAVE TO TELL YOU THIS PROPERLY. The very same setting is switched on in the browser I test in. So every check I made was made under your exact condition, and every one of them said "correct" - because the app *was* correct. I was reading where things ended up and calling it verified, and the one thing your setting removes is precisely the thing I was never able to see. **Four versions went by on that mistake.** I am sorry.

What changed - and it means you get an effect either way. The house rules say: when someone asks for less movement, take away the **movement**, and allow **a short fade - up to a fifth of a second - instead of nothing.** The app was taking away both. Now the new message **fades in over 0.18 seconds** rather than appearing in a single frame. No travel, no spinning, nothing to make anyone dizzy - but you can see it change.

IF YOU WOULD RATHER HAVE THE FULL SLIDE, where one message scrolls up and the next follows from below:

1. Open **System Settings** 2. Go to **Accessibility** -> **Display** 3. Turn **Reduce motion** off

The offers will then slide exactly as designed. **With it on you now get the fade instead of nothing**, so both ways round you will see something.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,789 tests, all green**.

And one thing said plainly rather than buried: I still cannot *watch* the animation from my side. What I can say for certain is what the code computes - the panel that arrives is in its place and fades from nothing to full - and that the setting which was hiding it is named above. **You are the one who can see it, and you have the**

?debug=offers line if anything ever looks wrong again.

05 Oct 2026 - engine v320, A FINGER CAN NO LONGER STOP THE OFFERS FOR GOOD (no database step, nothing to upload - pushing this one is the whole of it).

First, plainly: I owe you an apology on method. I told you twice that this was fixed, and both times I had never actually watched it move. Every check I made was on a browser pane that reports itself as **hidden**, and **a hidden tab runs no animation at all** - so all I could ever read was where things *ended up*. That is how two versions went to you broken. It will not happen again in this particular way, because there is now a way to ask the strip itself (below).

AND I HAVE FOUND THE ONE THING IN THE CODE THAT CAN STOP THE OFFERS FOR GOOD.

The strip holds still while your pointer rests on it - that was your own ask, so you can finish reading a message. The app remembers *"the pointer is on it"* as a little flag, and it clears that flag when the pointer **leaves**. **On a touch screen, that "leaves" may never arrive.** A finger landing on the strip sets the flag, and if the matching leave never comes, the strip is held **for the whole life of the page**: the timer keeps ticking, everything thinks the pointer is still there, **and the messages never change again.** That looks exactly like a broken strip - and it is the only mechanism in the code that can stop the turning dead, which matches what you are seeing.

What changed. A pointer can now hold the strip only where a pointer can hover - that is a mouse. **On a phone the strip simply carries on**, because on a touch screen there is nothing to rest and so nothing to hold. On your Mac, resting the mouse on it still holds it, exactly as before.

AND A WAY FOR YOU TO ASK IT DIRECTLY. If the offers are ever not turning again, add **?debug=offers** to the end of your shop's address, like this:

https://munchies.com.my/store/?debug=offers

One small line appears under the strip and tells us exactly what is going on - how many offers it can see, whether the timer is running, whether it thinks your pointer is resting on it, whether your phone can hover at all, and whether the tab is hidden. **No customer ever sees it: it only appears with that ?debug=offers on the link.** Next time, you read it to me and I will know, instead of guessing.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,788 tests, all green**, including two new ones: that a touch can no longer hold the strip, and that the debug line exists and can never reach a customer.

05 Oct 2026 - engine v319, THE FOLLOWING MESSAGE WAS BROKEN ON A TWO-OFFER SHOP (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your report, three words: *"there is no effect."* **You were right and I had broken it** - and the way it was broken is worth reading, because your shop is exactly the case it broke on.

WHAT WAS WRONG. When a message is replaced, the app has to know WHICH one is leaving so it can send it up through the top. It worked that out by counting backwards from the one showing: *the previous one*. That was inherited from the old 3D flip, where two panels leaving by opposite doors made sense, and it is fine when **three or more** offers are running.

WITH EXACTLY TWO OFFERS, "THE PREVIOUS ONE" IS THE MESSAGE THAT IS ARRIVING. So the message about to come in was parked **above** the window - where the one that had just left sits - and it then travelled **back down** into place. **Nothing was ever left waiting below**, so "one message following another" could not happen at all. A two-offer strip is not an edge case; it is the ordinary one, and it is what yours runs.

How it hid, and that is the uncomfortable part. With three offers running it looked perfect, so the page I checked and the harness I built both showed the effect working. **And 2,784 tests were green while it was broken on your phone.** Nothing was watching the two-offer case, which is the case that matters most. **That is now the one the test exists for.**

MEASURED, ON THE REAL SHOP, AT 375, TWO OFFERS:

	at rest	after a step
before	[0, -183] - **nothing waiting below**	the arriving message came from

above | | **now** | [0, +183] - **the next waits below** | it comes **up from below**, following the one leaving |

And the second half of the fix. A message that has just left through the top has to be back at the bottom to arrive from below next time - and moving it there *with* an animation would slide it the whole way **through the window**, sweeping the message you just read back across the strip in the wrong direction. So every waiting message is now put back in its place **instantly, with no animation**, before the movement starts. A jump paints only where it lands.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,786 tests, all green** - two of them new, and the one that matters asserts the two-offer case by name.

05 Oct 2026 - engine v318, EACH MESSAGE IS ITS OWN BOX AND THEY SCROLL UP (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words, and they were the whole spec: *"Maybe you box up each message, when 1st message start to scroll up, the 2nd message is following, So effectively you see 2 message, one follow by another, it scroll up, stop 2sec, scroll again until mouse over."*

What I had wrong. v316 sent them **sideways** - I read "scrolling off" as off to the side. You meant **up**. Both readings of *"box up each message"* went onto a page side by side, you picked the one where **every message is its own box**, and that is what is built.

What you see now.

- **Each message is its own amber box.** The box used to belong to the strip - one box that never moved. It has moved onto the messages: the strip is now just the **window** you look through, with no box of its own, so nothing is drawn twice. **They scroll UP.** The message being replaced leaves through the top and the next follows from below, with a **small gap passing between them** as they go - that gap is what makes it read as one message following another rather than one block of text sliding. **Two messages are visible during the move**, which is exactly what you described. **Then it stops for your 2 seconds**, and **rests while your pointer is on it** - unchanged.

And the rule that keeps your page still survives again. Measured on the live shop at 375: **206.6 pixels, identical at every step.** (It was 210.6 before - four pixels shorter, because the box moved off the strip and the strip stopped adding padding of its own.)

AND THREE MOTIONS HAVE NOW COME AND GONE, ALL THREE MINE: a fade (v292), a 3D flip (v296), a sideways slide (v316) - each with reasoning I wrote into the code about what "reads as a glitch". You are the one who watches this every day. The code and the test now say so, so none of them quietly comes back.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,784 tests, all green**, and the rewritten test holds every part of this: the motion is up, each message carries its own box, the strip carries none, the gap between the two, and no fading.

05 Oct 2026 - engine v317, EACH OFFER HOLDS FOR 2 SECONDS (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words, right after the scroll went in: *"maybe the scrolling is too fast and hardly see the results."*

YOU WERE RIGHT, AND IT WAS NOT MARGINAL - I measured your own shop before changing anything:

- your long offer runs to **38 words** (the code's line **plus your own sentence in the chalk hand**) - about **11 seconds** to read at a comfortable pace the short one (*AZQKJ*) is **14 words** - about **4 seconds** each of them was getting **1.5 seconds**, of which **0.4 is the move itself**

So the long one had about **a tenth** of the time it needed, and the short one about a quarter.

BUT THE SCROLL WAS NOT WHAT WAS TOO FAST. The move is 0.4 of a second, which is already the **top of what the house rules allow** (250-400ms for a change like this) - slowing the animation would have made it draggy and bought you **not one extra second of reading**. What was too short was the **stillness**: 1.1 seconds before, **1.6 seconds now**.

The pace is 2 seconds an offer - your number. It was 1.5s, chosen back when the line was mostly a short code; your offers have grown since, which is what changed the arithmetic.

One thing said plainly rather than left for you to find: 2 seconds is still under the 11 your long offer wants, so it will still turn before you have read all of it - and the move now takes a fifth of each turn. **Watch it on your phone with two codes running.** If it still turns too soon, **it is one number and nothing else needs touching** - say the word and I will put it wherever you like.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,784 tests, all green**, and the test that pins the pace now records that 2 is *your* number, chosen with the arithmetic in front of you.

05 Oct 2026 - engine v316, THE SHOP'S OFFERS SCROLL INSTEAD OF TURNING (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words: *"the promo code, can it be like scrolling off, and new code follow, that kind of animation."* The offer line under your shop name used to **turn** - a quarter-turn flip, the old offer tipping away over the top while the next rose from below.

What you see now. The offer being replaced **slides off to the LEFT** and the next one **follows it in from the right**, at the same 1.5 seconds an offer. The strip is clipped, so nothing shows outside it. Everything else about it is untouched: the dots, the pointer resting on it to hold it, the tab going quiet when you leave it, and **nothing turns at all when only one code is running**.

AND THE FLIP WAS MY CHOICE, NOT YOURS - that is worth saying plainly. Back in v292 I picked a fade, then turned it into a 3D flip, and I wrote the reason into the code: that sliding one line of message sideways "reads as a glitch". You are the one who looks at this every day, and you asked for the scroll. The argument loses.

ONE HONEST TRADE, so it is not a surprise. During a scroll **two offers are briefly on screen at once** - the tail of the one leaving and the head of the one arriving. That is exactly what makes it read as a scroll rather than a swap. The flip never showed two, so if a long offer like *"Free delivery within Malaysia on RM200.00 and above - use code FREEDEL"* looks busy mid-move, that is why.

AND THE ONE THING THAT MUST NEVER REGRESS WAS RE-MEASURED. v295's whole point was that the strip is exactly as tall as your tallest message and never changes height, because the page under it used to jump up and down every 1.5 seconds. **Measured on the live shop across three turns: 210.6 pixels, 16 samples, never once different.** A sideways slide is a transform, just as the turn was, so the guarantee carries over exactly.

Two corrections to the guide while I was in it. It described the old turning motion, and it said *"on a phone a tap holds it still for about twenty seconds"* - which **stopped being true in v297**, when we removed exactly that because it trapped anyone who touched the strip. Both now match what the shop actually does.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,784 tests, all green**, including the rewritten strip test: the flip coming back, a fade creeping in, an unclipped rotor, and a move slower than the house 250-400ms band each turn it red.

05 Oct 2026 - engine v315, TWO MORE CARDS LEAVE SETTINGS, AND ONE OF THEM IS RENAMED (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your two instructions: *"move Courier(Lalamove) and Mailing labels(courier), both to Logistic?"* and *"move Message Style to the shop"*. Both are the same idea as v314 - a thing belongs where its work is, not under "defaults, backup, transfer".

What moved.

- **Send a van** is now its own screen under **Logistic** - your own door for the driver, and which Lalamove account this phone is talking to. Same card, same pickup pin, moved whole. **Message style** is now under **The shop**, beside Bring a friend, Promo codes and Reviews. How your words reach a customer is the same subject as the offers and the reviews.

Two words I did NOT use, and you should know why. The first choice was to call the new Logistic row **"Courier"** - and it would have sat directly above **Parcel couriers**, which means something else entirely. A parcel is something you **post**; a van is a trip you **book**. v309 already gave those two their own words on the order cards, so the screen is called **Send a van** and the card inside still names **Lalamove**, asked from the registry rather than typed here.

AND THE THIRD CARD I DID NOT MOVE - because the name was the real problem, not the address. "Mailing labels (courier)" holds **one address**, and I went and checked what actually reads it. **Four things do:**

- the **FROM block on a parcel label** the **door the courier collects from the letterhead on every invoice** the **from line on the wish-list email**

So it is not a courier setting at all - it is one address typed once that four screens read. Filed under **Logistic**, an invoice with the wrong heading would have sent you looking under deliveries. You chose **"Rename it, keep it in Settings"**, and that is what it is now: **Your address**, with the four jobs named in the card itself, and a way to it from the **Send a van** screen so the trip is still one tap when it is a delivery you are fixing.

What Settings keeps: Delivery settings, App password, **Storefront**, Your address, Website & developer, Live availability, Shared data, Backup & safety, Danger zone. **Storefront stays there deliberately** - your call, and recorded so no later version proposes moving it again.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,784 tests, all green** - and the v312 route guard caught **both** new screens by name when I briefly took them out of the menu.

05 Oct 2026 - engine v314, BRING-A-FRIEND MOVES TO THE SHOP, AND THE REWARD IS A COUPON (no database step, nothing to upload - pushing this one is the whole of it).

Why. Two of your own, in one message. First: *"can be brought to The Shop, rather than in Settings."* Second: *"can we make to more seamless with other promo?"* - and when I asked what seamless meant, you chose **"One place, read as a family."**

What you see now. More -> The shop carries three rows instead of two: **Promo codes**, **Bring a friend**, **Reviews**. The scheme itself has not changed at all - same switch, same three numbers, same links on a customer's card, same Give-coupon press on an order. It simply lives where you would look for a customer offer, instead of beside "defaults, backup, transfer".

And each half now names the other. The Bring a friend screen carries a card called **"A link, or a code"**, which says the difference in the terms you gave me: a link **costs nothing to issue and travels** - forward it and it still works; a code and a label is for a partner who prints brochures. Promo codes says the same thing from its side and points back.

BUT THE IMPORTANT PART OF THIS VERSION IS YOUR SECOND SENTENCE. You said: *"if we state only credit of ringgit, there might be confusion of how much credit to apply, but we can state, only one coupon apply for each purchase."* **You were right, and it was worse than you described - it was two places, and both of them argued with themselves:**

- On a customer's card holding **two** RM3 coupons, the chip above read **"2 ready"** while the line beneath it read **"unused = you still owe RM 3.00 off an order"** - because that figure was taken from the **first** coupon only. Two numbers, disagreeing, on one line. On an order with two ready, the line read **"RM 3.00 credit available on this order"** beside a button reading **"Apply credit (2)"** - and the press spent exactly **one**. So the screen showed one coupon's money next to a count of two, and did something else again.

The fix is not arithmetic. It is your rule, written down. No screen shows a running balance any more. The customer's card now reads **"2 ready - one per order"**; the order reads **"RM 3.00 coupon ready for this order - one per order, 2 more after it"**; and the message a customer forwards to their friend now promises *"a RM 3.00 coupon for a future order - one coupon per order"* so nobody has to guess. The rule is stated in all four places you would meet it.

AND WE NOW CALL IT A COUPON, IN YOUR WORDS. Eighteen strings changed - Give **coupon**, Apply

coupon, **Coupons**, Add **coupon**, the customer's card, and all three WhatsApp messages (English, Mandarin, Bahasa Malaysia). **Your decision on the two schemes is unchanged and I have kept to it: both stay, and the difference between them is what you hand over. AND ONE THING DELIBERATELY DID NOT MOVE WITH THE WORD: the stored name.** Every coupon already given out lives under state.credits, and renaming that would have orphaned all of them - so the words changed and the storage did not. It is a test.

ALSO NOT RENAMED, because they are a different thing entirely: "Insufficient Credit" and the EasyParcel wallet line on the parcel block. That is the parcel money, not your reward.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,783 tests, all green** - and one of the new ones is the v312 route guard, which **caught this new screen by name** the moment I briefly took it out of the menu, exactly as it was built to.

05 Oct 2026 - engine v313, THE SMALL GREY LINES ARE READABLE NOW (no database step, nothing to upload - pushing this one is the whole of it).

Why. Every small second line in the app is drawn in one grey - the line under a menu row, under an order row, the hint under a box, a collection Point's address in the shop. You looked at it against the darker candidate and said do it.

IT WAS UNDER THE FLOOR, AND YOUR OWN APP HAD ALREADY ADMITTED IT ONCE. Measured against the card behind it the old grey came to **3.50:1**. The standard asks **4.50:1** of text. And in v309, when I drew the **Post a parcel / Send a van** headings, I deliberately dodged this exact grey for the line beside them and wrote the reason into the code: *"at 12px it is only about 3.5:1 - under the 4.5:1 floor that applies to text."* So the app knew. It had only ever fixed that one spot.

AND THIS IS THE PART WORTH READING: THE COLOUR IS SET BY THE WORST PLACE IT LANDS, NOT THE BEST. The value you approved on the page measured a comfortable **5.02:1** on a plain card - and only **4.24:1** on the soft grey behind a used promo chip or a "courier paid" tag. Five more backgrounds - the striped band, the warm block, the green, amber and red chips - came in between **4.27 and 4.47**, all under the floor. So the colour is a step darker than the one on the page: **#7b6552**, which is **4.56:1 on the worst background there is** and **5.40:1 on a card**. One colour has to clear the worst thing it is drawn on, or you have only moved the problem to the chips.

What changed, exactly: one value, in two files. The backoffice and the shop each keep their own copy of the grey, and both moved together - a shop whose quiet line is a different grey from the backoffice's is the same fault as two rows that look alike and behave differently.

AND ONE THING DELIBERATELY DID NOT MOVE. The 2px rule inside the **Post a parcel / Send a van** headings is an *edge*, not text, and an edge answers to 3:1 rather than 4.50:1. Left at the new darker text colour it would have stopped being a join inside a card and started reading as another box. It keeps the old, lighter value under its own name - **measured on the real card: still exactly rgb(154, 132, 113), unchanged, 2px.**

And the calendar's quiet day stayed put too. That colour was built as a halfway mix between the old grey and the ink, precisely because the old grey was faint. The grey is not faint any more, so re-deriving it would have pushed a quiet day *toward* a delivery day and narrowed the very gap it exists to keep - for no gain. It stays at 6.71:1 against a delivery day's 13.45:1.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,772 tests, all green**, including a new guard that measures this grey against **every background it can land on** and fails by name if it drops under the floor - the test that would have caught 5.02-on-a-card.

05 Oct 2026 - engine v312, THE MORE SCREEN GROUPED BY THE WORK (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words: *"when i work on Products, i have to alway go into Others to find, ingredient, unit, category"* - and *"Everything about delivery should be group under logistic."*

You were right on both counts. Seventeen screens sat in ONE list under a single heading called "Manage", in roughly the order they were built. Nothing was wrong with any row; the problem was that its neighbours told you

nothing. Categories, Units and Ingredients were at rows 13, 11 and 14 - two of them separated by other screens - and the four delivery screens were at rows 7 to 10, mixed in among the money screens.

What you see now. Seven headings, in the order you reach for them:

- **Logistic** - Delivery run · Delivery dates · Self collection Points · Parcel couriers
- **Products & ingredients** - Categories · Units · Ingredients
- **Buying** - Purchase Order · PO history · Suppliers
- **Money** - Money · Profit
- **The kitchen** - Production line · Scenario planner
- **The shop** - Promo codes · Reviews
- **Settings & this app** - Settings · Change history · Developer contact

Logistic is your word, and it carries all four ways an order leaves the kitchen. **Products & ingredients is the one your sentence asked for**: the three things you reach for while working on a product, together, under the name of the screen they belong to.

And the trip out of the screen is gone. The Products screen now carries the same three at its top, under the same heading - Categories, Units and Ingredients, one tap from where you are editing. It sits BELOW the + New product fold, so the thing you came there to do is still the first thing on the screen.

"About" is gone as its own section, because Settings and the change history are the same subject - the app itself - and two headings for one subject was one too many.

Nothing moved that has to be found twice. All seventeen addresses are exactly what they were, so an old bookmark still lands where it always did, and every row keeps the words it had underneath it; only the heading above it changed. **Two of those second lines now say the moment rather than the mechanism** - "Which days you deliver, and who is on each" instead of "Set and manage delivery dates", and PO history reads "Your saved shopping lists".

AND THE ONE THING WORTH KNOWING: A ROW DROPPED IN A REGROUP IS SILENT. The screen still exists and its address still works - it has simply become unreachable from the menu, and nothing goes red. So the new test reads the app's own route table and walks it against this menu, and fails by name if a screen is missing. It was watched going red before it was trusted, and it caught a real fault on the way in: the first cut passed the rows as raw lists and drew four empty headings with nothing under them.

Your data is untouched. No SQL, no order, product, price or posting day touched, and **no database step**. The suite is **2,769 tests, all green**.

Measured at 375 wide, same phone, same data: 1,994 px became 2,263 px. Grouping costs height - about **+269 px**, a third of a screen of extra scrolling. Finding beats scrolling here, but it is a real trade and it is worth knowing about.

05 Oct 2026 - engine v311, THE ADDRESS BOX SAYS WHY IT ISN'T SUGGESTING (ONE STEP FOR YOU, BELOW - the app side is a push, the reason needs a redeploy).

Why. Your words: *"why sometimes in add order or edit order, the address is not auto complete, and the map dont show?"* - and then, precisely: *"the suggestion list never appear"*. **It was true, and the app never said why. It knew, and stayed quiet.**

WHAT WAS HAPPENING. The suggestions are a Google feature that has to be switched on, and the server deliberately answered a switched-off one with **a plain "no" and no error** - its own words: *"this reaches her as a box that simply does not suggest, which is exactly what it did before this version existed."* That reads well until the feature *does* exist and **never works**: a box that never suggests and never explains itself is indistinguishable from a broken box.

What you see now. When the suggestions cannot work, the box says so, in the place the list would have been:

Address suggestions are switched off - Google refused the request. The Google key needs Places API

(New) enabled and allowed.

AND IT SAYS IT ONCE, THEN STOPS ASKING. A line under a field she is typing in must not flicker, and a phone must not spend a request per keystroke to be told the same thing. And the box now **marks the difference between the two kinds of failure**: something you must fix (not signed in; the Google key not allowed to use Places) is **said out loud**; a signal dropping out is **not**, because that one passes on its own and the suggestions come back by themselves.

THE ONE THING THIS NEEDS FROM YOU: the *reason* travels in the server's answer, so the courier function has to be uploaded again. The exact line is in the commit notes below. **Until it is uploaded the box stays exactly as it is today.** Nothing else changes, and nothing is broken by uploading it.

AND THE SCREENSHOT FOUND A SECOND FAULT THAT HAD ALREADY SHIPPED - a real one, mine, and this one is worth reading. Looking at the real card, the EasyParcel block was printing the **literal word "null"** under the weight box and **three more beside its buttons**. The block builds its own contents as a list with optional parts in it, and **replaceChildren(null) does not skip a null - it inserts a text node reading "null". Every one of the 2,758 tests passed while it was on your screen**, because the test shims quietly filter nulls for you. **A forgiving stub hid a real fault** - the same lesson as v226, word for word. It is fixed, and the real-browser check that would have caught it is now in place and was watched going red.

Your data is untouched. No SQL, no order, product, price or posting day touched. The suite is **2,758 tests, all green**, and the real card now runs **41 checks** on the Orders screen and **9** on the address box.

04 Oct 2026 - engine v310, THE SAME DIVIDE ON EVERY CARD (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words, right behind v309: *"the drawing of line between parcel and lalamove should be consistent over the app"*. You were only half-served - v309 put the two headings on the **Edit** card and nowhere else.

THREE CARDS CARRY BOTH KINDS, SO ALL THREE NOW SAY THE SAME TWO THINGS. The **Edit** card, the **Note / tracking** card, and the **+ New order** card each draw **Post a parcel** over the parcel fields and **Send a van** over the van fields - **from one definition**, so the words cannot drift apart. **Two headings on each is not the check; the same two is**, and that is exactly what the new test compares.

AND THE WAY I GOT IT WRONG FIRST IS WORTH KNOWING, because it is the kind of thing that would have looked fine on paper. I tried putting the heading *inside* the parcel block, so that no card could ever forget it. **That put it BELOW the consignment number on the Edit card** - the number is a field the card itself draws just above that block - so the heading landed in the middle of its own group and left the number stranded above it. The test caught it. **The words can be shared; the position cannot**, because only the card knows where its own fields are.

Your data is untouched, and there is nothing to run. No SQL, no upload, no key. No order, product, price or posting day is touched, and no field moved on any card. The suite is **2,756 tests, all green**, and the real Orders screen now runs **40 checks** - including that each heading sits above its own fields in document order, that neither one says "kind 1", and that **all three cards show the same two.**

04 Oct 2026 - engine v309, THE TWO WAYS AN ORDER LEAVES ARE SAID APART (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words: *"what i see is easy parcel is kind 1, lalamove is kind 2, but the interface din draw a clean border between them, so it is quite confusing for user"*. **The first half of that sentence is the proof.** You had the two the wrong way round - which you could only do because the card never told you which was which. Four labels sat next to each other - **Parcel carrier**, **EasyParcel**, **Courier charge**, **Get a delivery price** - with nothing saying what belonged to what.

What changed. The card now draws a **rule with a name on it**, twice:

Post a parcel *Nationwide, a few days* - the consignment number, the parcel carrier, and EasyParcel

Send a van *Today, locally* - the courier charge, who paid it, and **Get a delivery price**

AND THE NAME SAYS WHAT IT DOES, NEVER "KIND 1" OR "KIND 2". Those are this app's own words for the two kinds of courier and they mean nothing on your screen. What you need to know is that one of them is a van today and the other is a parcel over a few days - so that is what it says. **Nothing moved:** every field is exactly where it was, and each heading went above the fields it already sat on, so a heading can never end up announcing the other one's controls.

TWO THINGS I MEASURED RATHER THAN GUESSED, and both would have failed quietly. My first rule used the card's own line colour - **1.25:1 against the surface**, a faint tint that disappears on a phone in Malaysian daylight, which is exactly where you read it. **A boundary you cannot see would not have answered you.** It is now **3.5:1**, which clears the 3:1 the standard asks of an interface edge. And the small grey beside the name - *"Nationwide, a few days"* - was **3.5:1**, **under the 4.5:1 that applies to any text**; it is now **5.69:1**. Both were measured in the browser by reading the rendered pixels, not worked out on paper.

Your data is untouched, and there is nothing to run. No SQL, no upload, no key. No order, product, price or posting day is touched, and no field moved on any card. The suite is **2,756 tests, all green**, and the real Edit card now runs **36 checks** - including that each heading sits above its own fields in document order, and that neither one says "kind 1".

04 Oct 2026 - engine v308, THE EASYPARCEL BLOCK READS AS OFF, NOT AS BROKEN (no database step, nothing to upload - pushing this one is the whole of it).

Why. You looked at EasyParcel's top-up offer and found the thing that decides this: **only an account topped up by RM500 enjoys integration.** For a shop that posts the occasional parcel that is the wrong money, so **you decided not to sign up** - and you were right. This version is the consequence: the EasyParcel block now says so plainly instead of sitting there looking broken.

What changed. The block asks the server once, when it opens, whether EasyParcel is set up. If it is not, you now get one short paragraph rather than a set of controls that can only ever fail:

EasyParcel - Not set up yet - and nothing here is needed to post a parcel by hand. Record the carrier above and type the consignment number, exactly as before. EasyParcel is not set up yet - its key has not been saved on the server.

AND IT OFFERS NO PRESS AT ALL IN THAT STATE. No weight box, no "Check the price", no "Book it". A button whose only possible answer is the same sentence every time is the shape of a dead control, and this app treats a dead control as a bug - so when it is off, it is off.

The important half is the reassurance, not the apology: nothing is missing. Posting a parcel by hand is how this app has always posted one - record the carrier, type the consignment number - and that is completely unchanged. **The API was only ever going to save you the retyping.**

AND A REAL FAULT THIS FOUND, WHICH IS THE PART WORTH KEEPING. The first attempt had the block call the pop-up's own repaint when its answer came back. **That repaint arrived at an unpredictable moment while she was working further down the same card - and stranded the door block's own pending look-up.** A test caught it (a pin the card had just found was thrown away). **Every other block in this app paints ITSELF and leaves the card alone** - paintCourier, paintParcel, paintPoint - and this block now does the same. One repaint of the whole card, on the booking, because that one changes the tracking box above it.

Your data is untouched, and there is nothing to run. No SQL, no upload, no key, no account. No order, product, price or posting day is touched. The suite is **2,756 tests, all green**, and the real Edit card now runs **29 checks** - including five that can only be answered with the key absent, which is how it will be for you.

04 Oct 2026 - engine v307, THE EASYPARCEL SEAM (no database step in the app - but ONE one-time step for you, below).

Why. You asked to compare Ninja Van and SPX, and the honest answer turned out to be that **you do not integrate with a carrier - you integrate with an aggregator.** Ninja Van's own API is not self-serve (you must have an account, ask their sales team by email for even a sandbox, and production keys come only after an "integration audit" with sample orders). SPX has no public direct API at all for someone who is not a Shopee

seller. **EasyParcel carries both** - along with J&T, Pos Laju and DHL - and its API **is** self-serve, with a free demo environment. That is the same choice you made back on 28 September: *"record first, the booking API after / EasyParcel behind that API"*.

WHAT YOU GET: ONE PRESS, EVERY CARRIER, PRICED FOR THIS PARCEL. On an order's **Edit** card there is now an **EasyParcel** block. Type the parcel's weight, press **Check the price**, and every carrier they use comes back with a price for **that parcel, to that postcode, at that weight** - cheapest first, with the delivery time and whether they collect or you drop off:

J&T Express - RM6.20 · J&T Standard · 3-5 working day(s) · drop off · they collect · cheapest **SPX - RM7.10** · SPX Express · 3-4 working day(s) · drop off · they collect **Ninjavan - RM9.40** · Ninja Van Standard · 2-3 working day(s) · they collect

That is the comparison you asked for, done per parcel instead of guessed at from a blog post. The cheapest is marked but **never chosen for you** - which carrier to use is your call, and the reason to show several is that the cheapest is not always the one you want.

AND YOUR WALLET IS WARNED BEFORE, NOT AFTER. EasyParcel is **prepaid**: a booking with too little credit fails **with the parcel already packed**. So the balance is shown beside the price, and when it is short it says so plainly - *"Your EasyParcel balance is RM4.20 and this parcel costs RM6.20 - top it up before booking, or the booking will fail with the parcel already packed."* When a booking does come back refused, **their own words are shown** - "Insufficient Credit" - rather than a generic failure.

BOOKING SAYS THE PRICE AND THEN PAYS IT. Pressing Book asks first, naming the courier and the amount - *"Book this parcel with SPX for RM7.10?"* - and only then books and pays in one call. **The consignment number is SAVED AT ONCE**, not left waiting for a Save that may never come: the same rule your booked trips already follow, because a parcel that has been paid for must not be discardable by closing a card. It lands in the tracking box, where every screen already reads it.

WHAT THIS DOES NOT DO, said plainly rather than discovered.

- **It does not write the consignment number onto the customer's card by itself** - it goes in the tracking box, exactly as a parcel you booked by hand does. **It does not pick your carrier record for you.** EasyParcel's name for a service is not your own list's entry, so booking fills in the number and leaves the carrier box alone - one press above if you want it named. **It is not required.** Posting a parcel by hand still works exactly as it did - record the carrier, type the number. This is offered beside that, never instead of it. **Dry and sealed only, unchanged.** Fresh focaccia and anything frozen are still not parcels.

ONE THING ONLY YOU CAN DO, and it is a signup, not a key in chat. Sign up at easyparcel.my, complete the account verification, and register the key for API access - their three steps. Then the key is stored **on the server** (never in the app, never in a browser) with two Terminal commands, which are written at the top of the new function. Until that is done the block says so in words and nothing else changes. **Set it to DEMO first** - the demo host books a parcel nobody collects, which is how you can try the whole thing before a sen is spent.

Your data is untouched, and there is nothing to run in the app. No SQL, no upload. No order, product, price or posting day is touched, and every screen behaves exactly as it did. The suite is **2,756 tests, all green**, including **thirty-six** new ones - the wire format against EasyParcel's own 55-page document, the address reading, and the seam end to end on the real card.

04 Oct 2026 - engine v306, THE SMALLEST BASKET A POINT WILL TAKE (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your ask: *"the per-point minimum order, this should be switchable"*. It is exactly that - **per Point, and switchable**, on the Point's own card.

THE SWITCH, AND WHERE IT COMES FROM. On a Point you now get **Minimum order** with two choices - **No minimum** and **Only on a basket of at least** - and the second one opens a **Smallest basket (RM)** box. That is the SAME switch the Promo codes screen has used since v269, because a minimum is a minimum and learning a second shape for one idea is how two screens come to mean two different things by one word. **You**

picked ringgit, out of the two units offered, so "a basket of RM30" means the same thing everywhere in the app.

EVERY POINT STARTS WITH NO MINIMUM, which is where they already are and what you asked for when this whole feature was being discussed - *"keep it as simple as possible, say no minimum for self collect order"*. The card says so plainly: **No minimum order - one loaf still goes.**

On your card. Each Point's row now reads **Minimum order RM30.00**, or the sentence above.

In your shop. A Point whose smallest basket the customer has not reached is **PARKED, NOT HIDDEN** - it stays on the page with the reason in its own line:

Farlim, Air Itam Needs a basket of RM30.00 or more - yours is RM15.00 so far

Tapping it says the same sentence rather than silently doing nothing, and **the moment their basket reaches RM30 the Point opens up - on the same repaint, with no reload.** Add a loaf and watch it; take one back and it parks again, **and a Point they had already chosen falls back to your kitchen**, because the shop cannot post an order to a Point whose basket is not met.

THIS IS YOUR RULE, SO THE SHOP HONOURS IT - and that is not the same as a gate. The thing your standing instruction forbids is a rule **the app invents** (a closed day, a sold-out line) standing between you and a sale. A smallest basket is a rule **you typed on your own Point**, so enforcing it is the shop doing what you asked. **Nothing on your own side is blocked:** an order you take over the phone for one loaf at Farlim is yours to take, and the app will not argue.

AND ONLY WHAT THE SHOP NEEDS LEAVES YOUR APP. The Point's smallest basket is now published along with its id and name - **the receiver, their phone, the fee and the address still never do**. A smallest basket is the opposite of private: it is exactly what the customer has to know *before* choosing, and without it the page could only take an order the Point does not want.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. Every Point you already have carries no minimum, so **every one of them behaves exactly as it did yesterday** - proved rather than asserted: the shop's own v299 checks still pass, 20 of 20, with Points published that have no minimum at all. The suite is **2,720 tests, all green**, including eight new ones, and every one was watched going red with the fault put back.

04 Oct 2026 - engine v305, THE HOURS ON THE RUN ROW (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your ask, and it is the one number the Delivery run was missing. The collection hours you set on a Point decide **when the treats has to BE THERE and handed over** - so a trip booked for the wrong part of the day should be visible on the screen where you spend the money on a van, not discovered a day later.

What you see. A Point's row on the run now reads:

Farlim, Air Itam - 2 orders collecting here · Lebuhraya Thean Teik · **collect 2-6 pm** · Focaccia x2 · Focaccia x1 · Focaccia x3

Where, then when, then what - the address, then the hours, then the treats.

And it says nothing when you have not set any. The card already tells you a Point has no collection window; repeating it on every run row would be noise on the screen you read while working. A customer's own doorstep never claims hours either - a doorstep is not a place with opening times.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. A Point with no hours behaves exactly as it did yesterday, a doorstep's row is unchanged, and the load line is untouched. The suite is **2,712 tests, all green**, including three new ones, and every one was watched going red with the fault put back. **One of them taught something worth keeping:** the check that a doorstep never shows hours stayed GREEN under the first fault I tried - because reading a null Point already gives nothing - so the fault was not the one I had chosen. Re-pointed at the mistake that would really do it, *taking the hours from whatever Point happens to be first rather than from this row's*, it turns red by its own name.

04 Oct 2026 - engine v304, THE COLLECTION WINDOW (no database step, nothing to upload - pushing

this one is the whole of it).

Why. This is the piece we left open, and you chose it from three: **the window belongs to the PLACE.** You type it once on the Point - *"Farlim: collect 2-6 pm"* - and every order collecting there is promised it. Not the van's arrival window, which is a different thing entirely.

What you see, on More -> Self collection Points. Every Point's card gains **"Customers can collect from"** and **"and until"** - two time boxes, the same pair the Delivery run fills in for the van, so a window means one thing in this app and is read by one piece of code. And the row itself now says which of the two it is: **Collect 2-6 pm**, or **No collection window - customers are told the day only.**

What your customer is told. The confirmation, the payment reminder and the pickup reminder all name the place **and the hours**: *"Self collect at Farlim, Air Itam, collect 2-6 pm."* Leave both boxes empty and they are told the day and nothing else, which is a promise you can keep rather than one that reads as open all day.

THE ONE THING TO GET RIGHT, and it is your judgement rather than the app's: SET THE HOURS FROM WHEN THE TREATS IS THERE, NOT FROM WHEN THE SHOP OPENS. The van arrives during the round, so a window starting at opening time can have a customer standing at the counter before their order has been delivered. The app deliberately does **not** work this out for you - the same way it never works out the fee - because a time the app derives is a time it can get wrong.

AND THE VAN'S OWN WINDOW IS NEVER QUOTED TO THEM. When you book a round, the trip's window is stamped on every order it carries - and for a collection that is when the **treats reaches the Point**, which is your business, not the customer's. Telling them both would be telling them two different times in one message. So a collecting customer gets the place's hours or nothing at all, and never the van's.

A hole that fixing this found, and it was mine. The **pickup reminder** said only *"Packed and ready for pickup on ..."* and named **no place at all** - so a customer collecting at Farlim was told their order was ready and never where to go, while the confirmation, the payment reminder and the shipped message all named it. v299 claimed all four later messages named the Point; three of them did. It now reads **"Packed and ready to collect from Farlim, Air Itam on ..."**, and a collection from your own kitchen keeps the words it has always had.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. A Point with no hours behaves exactly as it did yesterday, and every order already placed is unaffected. The suite is **2,709 tests, all green**, including eighteen new ones - the model, the one place that decides whose window a customer is told, the card's own time boxes, and the three messages a collecting customer actually receives, driven through the real builders rather than through the helper behind them. **Every one was watched going red** with the fault put back.

Also in this version, and it is housekeeping you will never see: the window stopped being the courier's own and became a small module both the courier and the Points card read, so **a window cannot mean one thing on one screen and another somewhere else**, and its own tests moved with it.

04 Oct 2026 - engine v303, A POINT YOU SET UP AND HAND OUT BY HAND (no database step, nothing to upload - pushing this one is the whole of it).

Why. Two things you found while watching v301 and v302 land, and they are the same gap: everything about Points worked for a **customer** ordering from your shop, and almost nothing worked for an order **you** take.

1. + New order and Edit now let you say where it collects from.

Your words: *"how about + new order, and add edit order?"* You were right, and it is the important half - you take a great many orders over the phone and in chats, and until now an order you keyed in yourself **could not be a Point order at all.** It could not go on the Delivery run, the Point's fee was never counted for it, and the customer was never told where to collect.

- **A "Collect from" picker**, under **Fulfillment** - **My kitchen** first, which is the default and is what every order you have ever taken already means, then your open Points in your order. **It appears only when there is something to choose.** With no Point open, or on a Courier order, it is absent rather than sitting there doing nothing. **The Point's name is frozen onto the order** the moment you take it, so renaming or ending a Point

later leaves every order that already went there still saying where it went. The same rule that keeps a sold price on an order. **Choosing your kitchen clears it.** An order collecting from your kitchen carries no Point at all, which is exactly what it has always meant.

2. A Point's address box now asks Google as you type.

Your words: *"there is no address auto complete for collection point?"* - and then the better question, *"why not make the point consistent with the customer card?"* Both fair. The suggestion box was written for the order's delivery address and had never been given to this one, so the address a **driver** is sent to - and the address the pin is looked up from - was the one address you had to peck out in full on a phone.

- **The same behaviour as the order's address box, because it is now literally the same code**, moved into one place so the two boxes cannot drift apart. **Nothing is ever blocked by it:** a lookup that fails shows nothing at all, and whatever you typed is what gets saved.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. An order taken before today carries no Point, which is what "collect from my kitchen" has always meant, so nothing needs migrating and no order is rewritten. The suite is **2,691 tests, all green**, including twelve new ones - nine driving both order cards, and three driving the Point's own address box through the real suggestion channel. **Every one of them was watched going red** with the fault put back before it was called done.

04 Oct 2026 - engine v302, AN ORDER COLLECTED AT A POINT REACHES THE RUN (no database step, nothing to upload - pushing this one is the whole of it).

Why, and it is a fault I found while starting the collection window you asked for. The run was not carrying Points at all. Not rarely - **never**, and it said so quietly: a day where customers were collecting at Farlim read **"Nothing to run yet"**.

WHAT WAS WRONG. Choosing a Self collection Point in your shop **never changes the Self collect / Courier choice** - that is by design, a Point IS a collection. So the order is stored as a *collection*, and the run screen used that word to decide what needs a vehicle and **skipped every collection**. Every Point order was thrown away **before** the row work v301 added could ever see one. v301 built the right rows and they were unreachable.

The rule now, and it is the one that was always meant: a courier order needs a vehicle, and a collection AT A POINT needs one too - the treats still has to reach the Point. A collection from **your own kitchen** still needs none, because you hand those over yourself. That half is the one a careless fix breaks, and it has its own test.

What you see now. A day of collections at Farlim **offers the run**, the day's own line reads **"1 stop"** rather than "2 courier orders", and the Delivery dates screen's **Run (N)** button appears on that day counting the same stops. **A Point is ONE stop however many customers collect there**, so two customers at Farlim still read **1 stop - 6 items**.

AND ONE RULE NOW LIVES IN ONE PLACE, which is why it went wrong. The run screen and the Delivery dates screen's **Run (N)** button both have to answer *"does this order need a van?"*, and they each answered it themselves. **Two readings of one rule is how they came apart**, so the rule and the stop's own name are now asked of one function in courier_job.js and read from there.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. An ordinary courier day behaves exactly as it always has, and a collection from your kitchen behaves exactly as it always has - **a run with no Points on it is unchanged**. The suite is **2,679 tests, all green**, including eleven new ones: six on the rule itself, three driving the run screen with a **real** Point order, and two on the Run badge. **Every one of them was watched going red** with the fault put back - the gate, the kitchen half, the shared stop name and the badge - before it was called done. One of them is worth naming: the test fixture itself had been modelling a Point order as a courier order, **which the shop has never produced**, and that is the reason nothing caught this. **A fixture that cannot happen is not a test.**

04 Oct 2026 - engine v301, THE DELIVERY RUN CARRIES POINTS (no database step, nothing to upload - pushing this one is the whole of it).

Why. This is the piece you asked for: the Delivery run carrying Self collection Points.

A POINT IS ONE STOP, AND THAT IS THE WHOLE VERSION. A courier charges **a base fare plus a fee for every extra stop**, so the run screen's oldest rule is *one stop per customer* - a customer who bought three things is one doorstep, not three. A **Self collection Point is the next version of the same idea**: four customers collecting at Farlim are **one place the van goes**. A run built one-stop-per-customer would send the same driver back to the same shop and **bill you a stop fee each time**.

What you see on the run now.

- **A Point gets a row of its own**, listing **how many orders are collecting there** - *"2 orders collecting here"* - and its own address. The customer's name is deliberately **not** on it: the treats is going to Farlim and the customer is meeting it there. **The load line counts STOPS**, so two customers at one Point read **"1 stop · 6 items"**, not two stops. Every line of treats is still counted. **A Point you have not pinned offers "Pin the Point"** right there, because the run cannot price a trip to a place with no coordinates. **A Point and a doorstep on the same run are two stops** - the mixed run you described.

Three things it is careful about, and each one is money.

- **THE VAN GOES TO THE POINT, NEVER TO THE CUSTOMER'S HOUSE.** A customer who chose to collect at Farlim is **not at Farlim** - and she may still have a doorstep of her own pinned from an earlier delivery. That door is **ignored** for a collection order. Sending a driver to her house with four other people's treats would be the most expensive way to be wrong on this screen. **ONE DROP ON THE WIRE, and this is the one that shows up on your bill.** The price request is read back in the test on the **bytes that would leave your phone** - because a trip built one-drop-per-customer is priced for a journey you are not taking. Two customers at one Point: **one drop. A POINT YOU PAUSE STILL SENDS THE VAN.** Pausing decides what is **offered**, never what an order already promised - the customer was already told to go to Farlim, and their treats still has to get there. And a Point you have **deleted** has no pin to give, so that order falls back to the customer's own door rather than a van sent to coordinates nobody has any more.

And the double-booking guard still sees everyone. A Point is treated as already on a trip if **any** of its customers is - so a Point can never be quietly swept onto a second van, which is the fault you reported in v242, at a place instead of a door.

AND A POINT YOU ADD NOW ACTUALLY REACHES YOUR SHOP. You reported this while the run was being built - *"the point added still not able to appear on store?"* - and **you were right, and the fault was mine.** Your Points travel to the shop inside the **storefront row** the shop reads, and **the Points card was not republishing that row.** Every other screen that changes something the shop shows - Promo codes, Products, Categories, Settings - ends its save with a republish; the Points card only ever saved to your phone. So a Point you added **stayed on your phone and never went out**, and your shop went on offering only the Points it had last been told about. The test proves it on the **request itself**: adding a Point **publishes**, pausing one **republishes**, and **pinning one does not** - because the pin is deliberately never published, so republishing for a drag would be a request that changes nothing.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. **A run with no Points on it behaves exactly as it always has** - every order that is not collecting at a Point keeps a row, and a stop, to itself. The suite is **2,668 tests, all green** - including two new ones that fail if a Point change ever stops reaching your shop. Proved live on the real Delivery run at a phone width, and on the wire: **two customers collecting at one Point price ONE drop, at the Point's own address**, and the mixed run prices the Point and the doorstep as two. The Points card was driven at a phone width too - **40 checks, all green** - including the three that read the publish request itself, and every one of them was **watched going red** with the fault put back before it was called done.

04 Oct 2026 - engine v300, EVERY POINT GETS A PIN ON THE MAP (no database step, nothing to upload - pushing this one is the whole of it).

Why. You asked for the Delivery run to carry Points, and said a Point should **"get like what customer is getting location pin"**. This is that pin - and it is the piece the run cannot work without, which is why it comes

first.

A courier is not given an address. It is given a point. Lalamove wants **"5.41405,100.31408"**, and "Farlim, Air Itam" is a guess about one - the courier itself says so when it cannot place an address. So **a Point with no pin is a name you can read and a place a van cannot be sent to**, and the run cannot price a trip until every end is a real point.

What a Point has now, on More -> Self collection Points.

- **A line saying where it is - Farlim, Air Itam · 5.41405, 100.31408**. **Both the name and the two numbers**, because a name alone cannot be checked and a pin in the wrong place is only ever noticed by looking at the numbers. **"Put the pin on the map"** under every Point, opening **the same map a customer's doorstep is placed with**. It is the same act, so there is nothing new to learn - and your Point's own address is handed to the lookup, so if you have already typed where it is you may not have to drag anything. **An unpinned Point says so plainly**: *"Not pinned yet - a van cannot be sent to a name alone."* It never pretends to have a door. Once it is pinned, the press becomes **"Move the pin"**.

Two things this version is careful about.

- **Correcting a Point never un-pins it.** The pin is not a field of the form - it is placed on a map - so an edit had to be made to carry it across deliberately. Without that, fixing a spelling would have silently taken the door away, and you would have found out when a driver was sent nowhere. There is a test that edits a pinned Point and checks the pin is exactly where it was. **The pin never leaves your app.** The shop is still told only the Point's **name** - a customer chooses a Point by name, and the driver is the only one who needs the door.

And a half-typed pin is not a pin. Anything malformed - one number and no other, a latitude of 91 - reads as **unpinned** rather than as a point in the sea. A row half-synced between your phones can never send a driver somewhere absurd.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. Points you have already defined simply start unpinned. The suite is **2,656 tests, all green**. Proved live on the real card at a phone width: **37 checks**, including that an unpinned Point says so, that the press opens the map picker, that a pinned Point shows its numbers, and that correcting a Point leaves the pin where it was.

Next, and it is the thing you actually asked for: the **Delivery run** carrying Points. It is its own version because the run screen is the one that spends your money, and it needs changing carefully - a Point's orders have to become **one stop** on the trip rather than one stop each, and the screen that ticks them has to say so.

04 Oct 2026 - engine v299, THE CUSTOMER CAN COLLECT FROM A POINT (no database step, nothing to upload - pushing this one is the whole of it).

Why. v298 gave you the card that defines your Self collection Points. This is the other half: **your customers can now choose one**, your order carries which one, and **the customer is told where to go**.

What your customers see, in the shop.

- **"Collect from"**, under the Self collect / Courier choice. **Your kitchen is first** - it is not a Point and needs no record - and **your open Points follow, in your order. The whole thing only appears once you have a Point open.** With none, the shop is **byte-for-byte the shop it was**, which is how you said you would do it: one at a time. **The chosen row takes the brand's orange edge**, the same "this one is on" the buttons already use. **A Point paused or deleted while a customer has it chosen falls back to your kitchen.** The shop cannot post an order to a place you have stopped offering - and the kitchen is always there.

What your customers are told. The confirmation, and all four later messages, say **"Self collect at Farlim, Air Itam"** and give the **address of the place**. That wording is written **once** and read by both message builders, so a customer cannot be told one thing in the confirmation and another in the reminder.

Three things this version is careful about.

- **Only the Point's NAME ever leaves your app.** The receiver's name, their phone, the fee and even the address **stay in your app**. The shop is a public page with no login, and publishing the receiver's number would

put a private person's phone on a page anyone can read. The address is withheld for a plainer reason: the message that tells a customer where to go is built from **your** copy. **The name on an order comes from YOUR record, never from the page.** The shop is public, so a name it sent could be anything at all; the order is matched against **your own Points** and an id you do not have falls back to the kitchen. This is the same rule the promo codes follow. **A Point you PAUSE is still honoured for orders already placed.** Pausing decides what is **offered**, never what an order already promised - the customer was already told where to go. Exactly the rule that keeps an ended promo code coming off the order it was placed on.

And your own screens say it too. The order row and the packing label read **"Self collect · Farlim, Air Itam"**, so whoever is packing a bag can see where it is going. A kitchen collection reads exactly as it always has.

Your data is untouched, and there is nothing to run. No database step, no upload, no key - the Points travel in the storefront row your shop already reads. **Every order already placed is unaffected**: an order with no Point IS a collection from your kitchen, which is what it has always meant, so nothing needs migrating. The suite is 2,650 tests, all green. Proved live in the real shop at a phone width: **20 checks**, including that the kitchen is first and carries no id, that a paused Point cannot stay chosen, that with no Points open the shop is unchanged, and that a long Point name does not push the page sideways.

04 Oct 2026 - a correction to this entry. This entry says your Points travel in the storefront row your shop reads, and they do - **but only once your phone has actually republished that row**, and the Points card was **not** republishing it. Adding a Point kept it on your phone and **never sent it to the shop**, so the shop went on showing only the Points it had last been told about. You reported exactly this - *"the point added still not able to appear on store"* - and it is fixed in **v301**: every change to a Point now republishes, which is the same call the Promo codes, Products and Categories screens have always made. **Pinning a Point still does not republish**, on purpose - the pin is never published at all. Nothing else in this entry changes.

04 Oct 2026 - engine v298, SELF COLLECTION POINTS - THE CARD (no database step, nothing to upload - pushing this one is the whole of it).

Why. You asked to get the system ready for collecting from places that are not your kitchen - Sg Ara, Farlim, Chai Leng Park, Bukit Mertajam - and to discuss it first. We did, and this is the first piece of it: **the card**, which is the piece you scoped yourself.

Her words: *"we just need to have a card for points"*

What you see now, on More -> Self collection Points.

- **A "New Self collection Point" card**, and your Points listed under it. **Five things per Point:** its name, the address, **who receives**, **their phone**, and **the fee per order** - what *you* pay whoever receives there. **Pause · Edit · Delete** on every row. A paused Point sinks to the bottom and goes pale, and **Resume** brings it back. **Every Point is its own record**, so opening one changes nothing about the others - which is how you said you'd do it: *"open collection point one by one... and not likely will open all point one go."*

Three things the card is careful about, and each is one of your rules.

- **Your kitchen is NOT a Point.** Collecting from Sg Ara is what your shop already offers - free, no minimum, always there. It gets no record, no fee and no provider, because it has none of those. A Point is a *third* thing beside it. **Delete never rewrites where an order went.** An order keeps the Point's **name frozen onto it**, so an order that went to Farlim still says Farlim after the Point is deleted - the same way an order keeps the name and price a product was sold at. **Pause is a normal ending, not a failure.** You said most Points you open will end this way, so pausing is one press and completely reversible.

WHAT THIS VERSION DOES NOT DO YET, said plainly rather than discovered. This is the **card only**. Your shop does **not** offer Points to customers yet, an **order does not carry which Point** it went to, and the **Delivery run does not include them**. Those are the next pieces. So today: you can define your Points and see them, and nothing else changes.

And the fee is yours to set, with no arithmetic done for you. You said *"allow me to manually set it fee x2 or x3, but default x1"* - so the app never measures an order's size. There is no product-size field and no volume

sum anywhere.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. One new list is added and starts empty; no order, product, ingredient, price or posting day is touched. The suite is 2,646 tests, all green. Proved live on the real card at a phone width: **26 checks**, including that a nameless Point is refused with a reason, that pausing one Point leaves every other alone, that deleting one leaves the order still saying where it went, and that nothing scrolls sideways at 375px.

04 Oct 2026 - engine v297, THE OFFERS REALLY FLIP, WITH DOTS, AND A CLICK NO LONGER FREEZES THEM (no database step, nothing to upload - pushing this one is the whole of it).

Why. Three things in one message, and the third was a fault of mine. Her words: *"I dont like the flip, the flip should be 3D flip, and also i came with a flaw, once we put mouse over it or click it, the flip stop, there should be 2 dot if there is 2 message, 3 dot if 3 message. move the mouse outside the window, the flip should be back"*

1 - The flip is properly 3D now. v296 turned both panels through the **same** arc - the old one out one way, the new one in from where it had just left - which mirrors them the whole time and reads as a vertical **squash**. Now the offer being replaced tips **away over the top** while the next rises from **below**. Two panels turning through opposite arcs is a card turning over; that one difference is the whole of it.

2 - One dot per offer. Two messages, two dots; three messages, three dots; **one message, no dots at all** - the same rule that already leaves the turn unarmed, because one offer is a statement. Pressing a dot goes straight to that offer, and **does not stop the turn**.

3 - The flaw you found, and it was mine. v292 paused the flip for **twenty seconds on any click**. It was meant as the only pause a phone had, but it was a **clock, not the pointer** - so clicking the strip trapped it for twenty seconds, and moving the mouse away could not release it. That is exactly the symptom you described. **The pointer being over the strip is now the whole of the pause**, so moving the mouse out - of the strip, or out of the window entirely - always starts it again.

4 - I read your own design skill, which I should have done before touching this, and it caught four things I had wrong:

- **The dots were 22 pixels across. WCAG 2.5.8's floor is 24.** They are 28 now - the dot you see is still the same size as the ones on your homepage, and it is the button around it that carries the size. **They had no focus ring**, so anyone tabbing to them could not see where they were. They have one. **The turn used the same easing both ways.** Your skill says *ease-out coming in, ease-in going out* - that is what it does now, and it is the difference between a turn that settles and one that just stops. **The turn was 420 milliseconds**, just outside the 250-400 band your skill names for a change like this. It is **380** now.

One thing I did NOT change, and it is yours to call. Your skill asks for body text to be pushed toward **7:1 contrast** because your customers read it on a phone in Malaysian daylight. The offer line sits at **5.3:1** - comfortably past the 4.5:1 that is required, but short of that goal. Getting to 7:1 means darkening the amber text noticeably, which is a change to how the strip looks. Say the word and it is one line.

What I could not check from here, said plainly. The preview pane was off-screen the whole time, and **a hidden page does not run animations at all** - so I could verify *where* each panel ends up (it is the opposite side, which is the 3D fact) but **I could not watch the turn itself**. Please look at it on your phone. Everything else was measured: the strip holds **169.4 pixels** and the page below it **380.1**, identical on every single turn, so the dots have not brought the jumping back.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. The shop's page and nothing else. The suite is 2,635 tests, all green. Proved live on the real shop page at a phone width: **26 checks**, including that a click no longer freezes the turn, that a pointer leaving the window starts it again, that pressing a dot goes to that offer without stopping the turn, and that the two panels sit on opposite sides.

04 Oct 2026 - engine v296, THE OFFERS FLIP RATHER THAN FADE (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your question, straight after v295 landed: *"can the flip be an animation"*. Until now the offers cross-faded

into each other. Now they actually **turn**.

What you see. Each offer is a panel that **turns a quarter-turn into place**: the one leaving tips away from you while the next swings up to face you. Straight on when it settles, so it stays perfectly readable - it is the change between them that moves, never the text you are reading.

How the two halves stay out of each other's way. Both panels travel through the same angle as they swap, so if they faded at the same rate you would catch them **both half-turned and half-visible in the middle** - which reads as a smudge rather than as a turn. So the one leaving fades out quickly, in about a sixth of a second, and is gone before it is half-way round; the one arriving holds its fade back until the turn is nearly finished, then comes up. Two timing rules, no timer in the script.

The height fix from v295 is untouched, and I checked rather than assumed. A turn is paint-only - it cannot change how tall anything is - but the same measurement was run again with the animation in place, taken **mid-turn** rather than at rest: the strip holds **135.4 pixels** and the page below it stays at **346.1**, exactly as before. So it flips without the page moving.

If a customer has asked their phone for less movement, the offers still change - they simply arrive facing them, with no turn. That setting has been respected here since v292 and still is.

One thing to watch, and it is yours to tune. The turn takes about four tenths of a second out of every 1.5 seconds, so the words sit still for about a second. Whether that is the right balance is much easier to judge on the real shop than in a description - if it feels busy, say so, and either the turn gets quicker or the 1.5 seconds gets longer.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. The shop's page and nothing else: no order, product, ingredient, price, posting day or promo code is touched. The suite is 2,631 tests, all green. Proved live on the real shop page at a phone width - 14 checks, all passing, measured mid-turn - and the turn itself was frozen part-way round in a browser and looked at, to be sure it is a real turn and not a squash.

04 Oct 2026 - engine v295, THE SHOP'S OFFER STRIP STOPS MOVING THE PAGE (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your report, two codes running: *"when the message switch, the page is like jumping up and down repeatedly... The window should be fix, base on the tallest message."* You were right, and it was mine to fix - **v292 caused it.**

What was wrong. The strip is only as tall as the message it is showing. When one of your codes carries a sentence of your own and the other does not, those are **two different heights** - so every time the strip turned, it grew or shrank and **everything below it moved with it**. Measured on the fault, on a real browser at phone width: the strip swung between **40.8 and 135.4 pixels** and dragged the whole page up and down by almost a hundred pixels, every second and a half, for as long as a customer stayed on the page.

What it does now.

- **The strip is exactly as tall as your TALLEST message, and never changes.** Every running offer is drawn at once and stacked under each other; only the current one is lit. The box is sized by the longest, so a short offer simply sits inside it. Measured again with the fix: the strip holds **135.4 pixels** and the page below it stays put at **346.1**, through turn after turn. **It turns every 1.5 seconds**, your number. **Pointing at it still stops it**, and a press on a phone still holds it - both unchanged. **The slight dimming is gentler** now that the offers come round twice as fast, so the words are readable for most of the time they are up.

One thing worth knowing about the 1.5 seconds. A two-line message - the offer plus a sentence of yours - gives a customer about a second and a half to read it before it goes. That is quick. It is exactly what you asked for and it is one number to change, so if it turns out to be too fast once you watch it with real customers, say so.

One more thing it fixes, quietly. The lit offer now says which one it is to a screen reader, and the others are marked as behind it - previously, with all the offers in the strip, a reader could have taken all of them in turn as though they were one message.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. This is the shop's page and nothing else: no order, product, ingredient, price, bake day or promo code is touched, and no code's terms change. The suite is 2,630 tests, all green. Proved live on the real shop page at a phone width: 14 checks, including that the strip's height is identical on every turn, that the page under it never moves, and that the box measures the same whichever offer is lit. The fault was then put back on purpose - the strip immediately swung 94.6 pixels and the checks went red - which is what proves the measurement can see it.

04 Oct 2026 - engine v294, THE INVOICE NUMBER IS THE ORDER'S OWN CODE (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words, the day v293 shipped: *"for the invoice, i think we can use the order code as invoice number"*. You were right, and it turns out to be **simpler and safer** than the running number v293 gave you.

What an invoice is numbered with now. The order's own code - **Invoice #A3F9C2** - which is the same six characters already on that order's row, in the confirmation message, in the payment reminder and on the customer's tracking card. **One order, one reference, everywhere.**

Three things that got better, and they are the reason this is a real improvement and not just a preference.

- **No two phones can ever take the same number.** The order code is unique by construction. The running number v293 used had one case where that was not guaranteed, and it had to be explained and accepted. **Pressing Invoice now writes nothing at all.** v293 stamped a number onto the order the first time you opened its invoice. There is no number to stamp now, so the press only reads - nothing on the order moves, and there is nothing to keep in step between your two phones. **There is no number for you to look up.** A reprint is the same invoice because the number was never stored in the first place.

The trade-off, said plainly rather than discovered later. The code is **not sequential**: nothing on an invoice says how many you have issued, or which of two invoices came first. And two different orders could in principle draw the same six characters - that is the same risk the order tag has carried everywhere since the shop was built, and it is unchanged by this version.

Please read the v293 entry below with this in mind. Everything in it still holds except the number: the letterhead, the items at the price they were sold at, the courier charge, the code taken off, the Total, Print and Share. Where that entry describes a running number that goes up one each time, **this entry replaces it.**

Your data is untouched, and there is nothing to run. No database step, no upload, no key. This version **removes** the two fields v293 could write onto an order, and writes neither of them - an order you invoice is not touched at all, and no order, product, ingredient or price is rewritten. The suite is 2,625 tests, all green. Proved live on the real Orders screen at a phone width: 21 checks, including that the invoice number is the order's own code, that making an invoice writes nothing onto the order, that opening it twice is the same invoice, and that the paper carries your letterhead with your shop's name said once.

04 Oct 2026 - engine v293, AN INVOICE FOR A CUSTOMER (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words: *"And customer need an invoice"*. Asked what it should carry, you chose *"One order, one invoice"* and *"Yes - name, address, a number"*.

What you see now, on Orders -> any order -> Invoice (beside Edit and Note / tracking).

- **A card opens with the invoice on it**, and **Print** and **Share** underneath - the same two presses every other book in the app has. Share hands it over as a **PDF file**, so it goes into WhatsApp as a document the customer can keep. **It carries your real address at the top.** That address already exists - it is the one you typed for the Mailing labels in Settings, the same block that prints as FROM on a parcel. So an invoice and a parcel can never show two different addresses for one shop, and **there is nothing new to type**. **One order, one invoice.** The number is given the first time you open it and is never given again: open the same order next year and it is the same invoice, with the same number and the same date. ***(Superseded by v294, 04 Oct 2026: the number is now the order's own code - Invoice #A3F9C2 - so there is no number to give, nothing is written onto the order, and the two-phone caveat below no longer applies at all. Read the v294 entry at the top of this***

page.) **The order's own code prints beside the number** - *Invoice 0007 · Order #A3F9C2* - which is what a real invoice does, and it means two invoices could never be confused for one another. ***(v294: the code is no longer beside the number - it IS the number.)***

What the paper says. Your name and address, then **Invoice 0007 · Order #A3F9C2 · the date the order was placed**, then one row per item as **4 × Focaccia** with the line total in the money column, the **courier charge** when the customer bears it, the **code taken off as a minus row** named after the code, and finally the **Total**.

Why the figures can never disagree with anything else. Every figure on the invoice is read from the one function your confirmation message, your tracking card and your order rows already read, and the item names and prices are the ones **frozen onto the order** - so an invoice for an old order never shows a product you have since renamed or today's price. An invoice cannot state a sum the rest of the app contradicts.

The one limit, said plainly rather than hidden. Your two phones hold one shared copy of your business, and a running invoice counter kept in Settings would be **overwritten by whichever phone saved last** - numbers would repeat or skip without warning. So the number is written **on the order itself**, which is a record of its own and cannot be lost. The consequence: **two phones issuing an invoice in the same instant could take the same number.** Nothing is lost when that happens - both invoices exist, each on its own order - and the order code tells them apart. With one person and two phones it is vanishingly unlikely, and the alternative would be an invoice you cannot write without the internet, which is worse for a home business. ***(Superseded by v294, 04 Oct 2026: the order's own code IS the number now, so this whole limit is gone - there is no counter to share, and pressing Invoice writes nothing at all. Read the v294 entry at the top of this page.)***

What is deliberately NOT on it. Your customer's per-item note is not printed - it is a production instruction, not a line item. Say the word if you would rather it were, and it is a one-line change.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. Two fields are added to an order **only when you press Invoice on it** - an order you never invoice is not touched at all, and no order, product, ingredient or price is rewritten. The suite is 2,627 tests, all green. Proved live on the real Orders screen at a phone width: 22 checks, including that the Total is the customer's own total to the cent, that the number is written on every row of the cart, that opening the invoice a second time is the same invoice, that Print really reaches the printer, and that your address block never says your shop's name twice.

04 Oct 2026 - engine v292, THE SHOP'S OFFERS TURN INSTEAD OF HIDING (no database step, nothing to upload - pushing this one is the whole of it).

Why. Your words: *"the public code shown in shop, if more than one hide behind each other, we need a carousel, where it message turn, put the mouse over it stop rotate."* And you were reading it exactly right. The shop's offer line took the **first** live public code and quietly dropped every other one - so a week when you were running three offers, your shop was advertising one of them and hiding two behind it.

What your customers see now, at the top of the shop.

- **Every code you have running takes its turn.** About six seconds each, then the next one - the same pace as the reviews carousel on your homepage. **The words fade; the strip does not.** The amber box stays exactly where it is and only the line inside it changes. The whole box blinking off and on would read as a broken page rather than as a second offer. **Point at it and it stops.** It waits while the pointer is over it and starts again when you move away. **On a phone, a tap holds it still for about twenty seconds** - long enough to read a long offer - and then it carries on by itself. A tap can never leave the strip stuck on one offer. **A background tab stops it**, and it starts again when the tab comes back. **With only one code running, nothing turns at all** - no movement, and literally no timer running. One offer is a statement, not a one-slide carousel. **And if a customer has asked their phone for less movement**, the words still change so they see every offer - they simply change without fading. The setting is about movement, not about hiding things from them.

Which code shows first, and what counts as running. The first one you published. A code is advertised only if it is public, switched on, inside its dates and not used up - asked through the same rule the code box already uses, so the standing line and the box can never disagree about what "still running" means. A **personal** code is never advertised; being unadvertised is the whole of what personal buys. If you pause or end a code while

someone is looking at the shop, the strip picks up the change on its own and does not leave them reading an offer that has gone.

Your data is untouched, and there is nothing to run. No database step, no upload, no key. This version changes the shop's page and nothing else - your orders, products, ingredients, prices, posting days and promo codes are exactly as they were, and no code's terms are changed by it. The suite is 2,611 tests, all green. Proved live against the real shop page: 50 checks across three states - three codes running, one code running, and three codes with reduced motion - including that the turn wraps at the end, that hovering and pressing both stop it, that it starts again by itself after a press, that a hidden tab stops it, and that the amber strip itself is never faded.

04 Oct 2026 - engine v291, A REWARD YOU CAN ACTUALLY HAND OVER (no database step, nothing to upload - pushing this one is the whole of it).

Why. You asked the question that made this version: *"how do we exercise their reward, if the reward is only written text?"* Until now the reward was a sentence on the customer - you read it and settled up by hand, and the app neither reminded you nor remembered what you had given. Nothing said **when** a reward had come round.

What you see now, on More -> Customers -> open a customer.

- **The reward is a card of its own on their record**, in amber so it does not read as the green bring-a-friend ledger below it. **It says what they brought in, and what is due.** *"9 brought in · every 5 · 1 due"* - the count is from your own orders, recounted every time, so nothing can double-count. **A Given button records the hand-over.** Their card then says *"1 given"* with the date, so the next time you open them you can see what you have already settled. **Undo last takes a press back**, if you tapped it twice or they returned it.

Their reward, and its number, are two separate boxes. Your sentence stays exactly as you wrote it - *"a free loaf for every five friends"*. Beside it is a number box, **"Given every ... customers brought in"**. Nothing is ever read out of your sentence: a parser that misread "every five" would tell you a partner is owed a loaf you never agreed to. Leave the number empty if it is not a set figure - the app still counts what they brought in, it just never calls one due.

What counts as "brought in" is one number, whichever way they work. A friend with a share link and a partner with a code are counted the same way, and a person can be both - so the count is the union of the two, never their sum. A cart that carried both a link and a code counts **once**. And a friend who had **already ordered from you** is not a new customer here, exactly as the Give-credit button has always judged it.

Every hand-over is its own record, and that is deliberate. Both your phones hold one shared copy of your business. A "rewards given" tally kept as a single number would be **overwritten by whichever phone saved last** - one phone's hand-over would vanish in silence. One record per hand-over cannot lose an update. It is the same reason your credit ledger is a list.

Nothing is ever blocked. The Given button is offered whether or not the app's arithmetic says a reward is due - you may settle a favour early, or hand one over for a reason the app cannot see. The count is information, never a gate.

Your data is untouched, and there is nothing to run. No database step, no upload, no new key. This version adds one new list and touches nothing you already have: your orders, products, ingredients, prices, posting days and saved plans are exactly as they were. The suite is 2,607 tests, all green. Proved live at a phone width: 29 checks, including that a cart carrying both a link and a code counts once, that a second hand-over is recorded rather than replacing the first, that Undo takes back exactly one, and that the number box really reaches the customer's record.

04 Oct 2026 - engine v290, YOU CAN ADD A CUSTOMER YOURSELF, AND READ YOUR REMARK ON THE LIST (no database step, nothing to upload - pushing this one is the whole of it).

Why. Two things, and they arrived together. **A partner who has never ordered from you had nowhere to live** - until now a person existed only by placing an order, so a partner you recruited to hand out labels could not be

named as a promo code's owner. And you wanted to **read your note about someone without opening them**.

What you see now, on More -> Customers.

- **A "New customer" card.** Add someone by hand - a partner, or a friend who sends people your way. **A name or a number is enough**; everything else (their reward, their note, a photo) is written exactly where it always was. **They appear under their own heading: "Added by hand - no orders yet"**, at the bottom of the list, with a count. Their card opens like anyone else's, and fine-tuning their reward works there too. **Your remark now shows on the row itself**, so you can read it without opening anyone - in your own words, on one line, whichever is longest clipped with the full note still one tap away.

The important part, and why they are in the same list rather than a separate one. Your customer list stays what it was - **people who have ordered**, with their spend and their last order - and the people you added simply sit underneath, grouped. They are counted, they export to the CSV, they can be messaged in bulk, and **they are offered by the name box when you take an order**. That last one matters most: if they had been kept in a list of their own, the day your partner finally ordered the app would have offered you nothing, you would have typed the name and number by hand, and that one person would have become **two records** - one keyed by name, one by number, for good. **The moment they order they move up into the list proper, by themselves.**

Two things that would have read as faults. A hand-added person has no orders, and their row would have said **"0 orders · 0 units · about RM 0.00"** - the shape this app uses for a broken screen. It now says what is true: *"Added by hand - no orders yet"*. And opening their card said *"This customer's orders were removed"*, which is untrue of someone who never had any, and reads as lost data. It now says they have not ordered yet.

No database step and nothing to upload - admin/ only.

04 Oct 2026 - engine v289, A NAMED REWARD, AND A CODE THAT NAMES ITS PERSON (no database step, nothing to upload - pushing this one is the whole of it).

Why. Two schemes, one missing half each. The **bring-a-friend** link is for a casual advocate and costs you nothing to hand out - but its reward could only ever be a flat **RM3**, the same for everyone, set once in Settings. And a **promo code** was anonymous: its orders and its label's opens were counted, but not *whose* they were, so a partner's label couldn't be told from anyone else's.

Both stay, and the difference between them is the thing you hand over - a *link* for a friend, which travels and costs nothing; a *code and a label* for a partner who prints brochures and runs their own marketing.

What you see now, in two places.

- **On a customer's card** - a line you write yourself: **Reward**, e.g. *"a free loaf for every five friends"*. Not a number, because a partner may be owed a loaf, a favour, or an arrangement of their own. It sits in the profile block, above the bring-a-friend block, so it shows **whether or not that person has a WhatsApp number** - the casual friend-to-friend advocate is exactly the one who may not. The RM3 and 90-day settings are untouched and still the defaults. **On a promo code** - a new choice: **Whose code is this**. Pick a customer and their name appears on the code's row as *Aunty Bee's code*, beside "public - shown in the shop". Their label now tells itself apart from anyone else's, and the code's own count and opens are their tally.

THE THING TO KNOW: their name is never published. The data your shop is given is readable by anyone holding its public key. A code's person is kept in your own app - **never** sent to the shop, and **never** printed on the label. The label stays impersonal, which is right for a piece of paper handed to whoever walks past.

What the reward is, and is not, said plainly. It is a **label you apply by hand**, exactly like the credits. The app names it and counts what that person brought in; it does **not** total what you owe, and it does not track what you have already given. That is the trade for being able to write *"a free loaf"* instead of a figure.

No database step and nothing to upload - admin/ only.

04 Oct 2026 - engine v288, HOW MANY TIMES EACH LABEL WAS OPENED. THIS ONE HAS A DATABASE STEP - run supabase/promo_visits.sql once, see the box below.

Why. You could already see what a code **sold**, worked out from your own orders. What you could not see is whether the label was **picked up at all** - and those are different problems needing different answers. *"Nobody followed the link"* means print more cards, or hand them out somewhere else. *"Forty people followed it and two bought"* means the card is fine and the offer needs work.

What you see now. On **More -> Promo codes**, each code's row shows **how many times its link was opened**, and under that a strip of the **last 28 days, one bar a day** - so a label going cold is visible at a glance rather than something you have to work out from a date list. A day with no opens is a faint stub rather than a gap, so "quiet" never reads as "no data". A code the app *did* get an answer for and which nobody has opened yet says **"Not opened yet"** in words - that zero is real and worth knowing.

What the number is, and what it is not - please read this part.

- It counts **opens, not people**. You chose that: a reload counts again, and a phone that leaves the page open counts once. It is a **pulse for alive-versus- cold**, not a headcount. **Sharing the link in WhatsApp or Facebook adds an open with nobody behind it** - those apps fetch a link to draw the preview. And your own testing counts. **A customer who types the code at the shop, with no link, is not counted.** This measures the CARDS, not the code in general. The app will not show a figure it does not have. If Supabase cannot be reached, the row is left **exactly as it was** rather than showing a zero - because a zero here is a claim ("nobody opened your label") and it must not be made on the strength of a request that never came back.

The one step, and it is yours: run supabase/promo_visits.sql once. Open **Supabase -> your project -> SQL Editor -> New query**, paste the whole file, press **Run**. It is safe to run more than once. **You can run it before or after you push** - if you push first, nothing breaks: the shop's page simply fails to record opens until the table exists, quietly, and the Promo screen shows no count. (Your own browser's console will show a **404** while the table is missing. That is the missing table, not a fault, and no customer ever sees it.)

What was built, and why it is shaped this way. One row is stored per open. The shop page records it in the background and never waits for it - a visit that cannot be recorded must not slow a customer down or change a word on the page. **The shop counts an open once per page load**, which matters more than it sounds: that page re-reads its settings every 30 seconds while it is open, so without that rule a single customer leaving the tab open would have added an open every half-minute and a label that sold nothing would have read as a triumph. Measured live: the settings were re-read **7 times in one minute and the open was recorded exactly once**.

No SQL runs on its own and nothing is uploaded - the Edge Functions are untouched.

04 Oct 2026 - engine v287, EVERY CODE HAS A LABEL, AND PRINTING NO LONGER FREEZES IT (no database step, nothing to upload - pushing this one is the whole of it).

Why. In your words: *"instead of printed card, i think something like what we have in shop and code is more useful, we have QRs, some active some retired, when a promo code come together with a QR, when you tab on label, you are allow to copy, print."* Until now the only QR in the app was drawn on a **separate printed-card page**, reached by a **Print it** press that was a deliberate point of no return - it froze the offer permanently. There was no copy, and no QR anywhere on screen.

Every code now has a label, on its own row. Under **More -> Promo codes**, each code carries **its own QR**, so the list reads as a set of labels with the life chips saying which are still going. **Tap the label** and it opens: the QR drawn large enough to hold another phone up to, the link in words, **Copy link**, and **Print it**. The square is built from the very address the printed card uses, so a label and a card always point at one place - and **Copy link** pastes straight into WhatsApp, where the shop opens with the code already in the box.

A retired code keeps its label. An **ended** code still shows its QR and still offers **Copy link** - a dead link is worth being able to look at - but it is **not offered a print**, because a label with an ended code in it would not work. It says so rather than simply hiding the button.

Printing no longer freezes anything. Until this version, the first print pinned the offer for good: the amount, who it is for, the smallest basket, what it cannot sit beside and the name all stopped moving, an end date could only be moved later, a ceiling could only be raised, and the code could not be deleted at all. **All of that is gone.** Print

and copy as often as you like, change the offer whenever you like, and **retire a label by ending the code** - which is what ending was always for: it stops new uses and leaves orders already placed with what they were promised. There is no new thing to learn; **End** is the retirement, and **Pause** is the reversible version of it.

The one thing to know, and the app says it on the label itself. A label already in someone's hand is honoured at whatever the offer says when the customer **orders**, not when they picked it up. So if you print a hundred cards and then change the offer, those cards give the new offer. That is the trade, and it is better to read it here than to find out from a customer.

What has NOT changed. A code still needs a **name** and a **cost ceiling** before it can be given a label - a label carries no number and no end date, so the ceiling is the only thing bounding what it can cost you. Editing a code still cannot reach back into an order that already used it: every order keeps the code as it was written when the customer typed it.

No database step and nothing to upload. This is admin/ only - no SQL, no Edge Function, no key. Pushing it is the whole of it.

03 Oct 2026 - engine v286, A SUGGESTED PROMO CODE YOU CAN READ OFF A CARD (no database step, nothing to upload - pushing this one is the whole of it).

Why. You asked me to compare your promo codes with the printed-label codes munchies used to run and see if there was anything worth taking. There was one thing, and it is small: a code is read by eye **twice** - you type it when you make it, and the customer types it off the printed card. The pairs people get wrong doing that are **0 and O**, and **1 and I or L**. Your codes were allowed to contain any of them.

What you see now. On **More -> Promo codes**, the **New code** card has a **Suggest one** button beside the code box. Press it and the box fills with a code such as PCX68 - five characters drawn from an alphabet with no **0**, **O**, **1**, **I** or **L** in it, so there is nothing on the card to misread.

Nothing is chosen for you. The box is still yours to type in, and the button only fills it - type over it, or ignore it and type your own code as you always have. **Every code you already have keeps working exactly as it did**, including ones with those characters in them: a code that is already printed cannot be renamed, and none of them were touched.

It will not hand you a name you already use. Before suggesting, it checks every code on the list and never offers one that is spoken for - two codes sharing a name is the one thing the shop could not recover from, because it would take the wrong amount off.

The button leaves the room to the box. The code box asks for a sensible minimum width; if the screen is wide enough they sit on one line, and if it is not the button drops below and the box takes the whole width. Either way the box is never the part that gets squeezed - measured at a 375-pixel phone screen the box is 181 pixels beside the button, and on anything narrower it is 208 or more.

Where this came from. Munchies' printed-label codes carried the same idea, and their comment names the reason exactly: *"a customer or the owner may type the code by hand off a printed label, and those pairs are the ones people get wrong."* This is the only piece of theirs worth taking; the rest of theirs is a printed-label pipeline for shops, which you do not have. **No database step and nothing to upload.**

03 Oct 2026 - engine v285, THE SHOPPING LIST CAN BE CORRECTED AT THE SHOP, AND AN INGREDIENT KEEPS ITS PRICES (no database step, nothing to upload - pushing this one is the whole of it).

Why. In your words: *"PO, say it is created an base on the po we go shopping, same supplier price change and we decide to buy more, i would like to change the price and the qty, i need the PO to be amendable, ingredient price journaled, ingredient price updated accordingly. So an ingredient need a journals."* A saved shopping list was frozen the moment it was saved. So when you got to the supplier and the price had moved, or you decided to take more, there was nowhere to put it - and the app's idea of what that ingredient costs stayed wrong until you remembered to go and correct it on another screen.

The list is now amendable - until you tap Bought. On a saved list you now have an **Amend** press. It opens

the list with two boxes per line: **how many packs** you took, and **what each pack cost**. Change either and the line's total and the list's total follow as you type. You can also **Remove** a line you didn't take, **add a line** for something you picked up that wasn't on the list, and - if the shelf already covered something - press **buy some** on it to decide to stock up anyway.

Once you tap Bought, the list is what happened. That is deliberate. Bought has already put the packs on your shelf and asked you what you paid, so correcting the list afterwards would mean unpicking both. Amend is offered only while the list is still un-bought; the correction belongs at the shop, which is where you are when you discover it.

Your ingredient follows the price you actually paid. Change a price on the list, save, and the app writes that price onto the ingredient - so your recipes, your product costs, and your next shopping list all use it, with nothing else to set. The app tells you it did it rather than doing it quietly.

And every price move is now recorded. Open an ingredient and, once its price has moved at least once, it carries a **Journal** press. It lists **only the moments the price moved, newest last**, each row saying what it moved to and what it was before - and with **the price you are on now at the top**, because a list of changes on its own never says where you ended up. Buying the same thing again at the same price is stock, not news, so it is not listed. The journal prints and shares like every other book in the app, including as a PDF.

Why a journal matters here. Your Profit screen reads Cost of sales from your recipes and the ingredient prices as they stand **today** - so changing a price moves months that have already closed. That has always been true and the screen says so; what was missing was any record of **when** a price moved, and this is it.

A fault found and fixed while building this. The **"What did you pay?"** box you get after tapping Bought was **never pre-filled** with your list's total. It had been reading the total from the wrong place since the box was built, so it always opened blank and its sentence never named the figure. It is now filled in from the list's own total, which is also what makes amending worth doing.

What did not change. Nothing that was already saved was rewritten, no figure moved on its own, and the money is still recorded exactly where and when it always was - at Bought, from the account you choose in that box. A list's own totals still pre-fill that box. **No database step and nothing to upload.**

03 Oct 2026 - engine v284, A LONG COURIER LINK NO LONGER RUNS OFF THE CARD (no database step, nothing to upload - pushing this one is the whole of it).

Why. A booked trip came back with a long tracking link - one unbroken run of about 150 characters, with no spaces anywhere in it. Your customer's card draws that link as a **rounded button**, and a button cannot break a word it cannot find a space in. So the link stayed one enormous line and grew straight out of the card and off the screen.

What you see now. The link stays **inside** the card. It wraps onto as many lines as it needs, and the button grows downwards instead of sideways. Nothing else about the card moved: the wording, the order of the lines, the money and the progress line are all exactly as they were, and the link still opens the courier's own page in a new tab.

A short link is untouched. Measured on the card at a 375-pixel phone screen: a short link such as https://track.jt.com.my/A3F9C2 is still exactly the **38 pixels** tall it has always been, because the button's own floor and its centring are unchanged. Only a link long enough to need a second line behaves differently.

Measured before and after, on an order. The order's link is 131 characters. Before: it ran past the card's right edge and off the screen. After: it wraps to five lines, its right edge sits at 344 against the card's 361, and the page no longer scrolls sideways - the page's own scroll width equals the phone's width. **Your backoffice card was never affected**: the booked-trip card on your own screen already carried this rule, so both of your screens now stop a long link the same way.

Where else that link is drawn - checked, not assumed. You asked me to check everywhere it lives, so I did. The link is stored once, on the order, and reaches your customer in three places: the **customer's track card** (the one that was broken - fixed here); the **shipped WhatsApp message**, which carries it as plain text that

WhatsApp wraps in the chat itself, so it was never at risk; and your **own booked-trip card**, which was already right. The delivery-run screen only mentions it in a passing message, and the orders list draws it as a number to read out rather than a link, so neither of those can overflow.

No database step, and nothing to upload. This is one style rule in the shop's stylesheet - no SQL, no Edge Function, no key, and your data is untouched. Pushing it is the whole of it.

03 Oct 2026 - engine v283, SHARE SENDS THE JOURNAL AS A PDF FILE (no database step, nothing to upload - pushing this one is the whole of it).

Why. You pressed Share on a journal and the share sheet opened **without WhatsApp in it**. Nothing was broken - a phone is handed a journal as ordinary text, and WhatsApp does not offer itself for a bare block of text. It does offer itself for a **file**. You said to make it share by PDF, and that is what this is.

What you see now. The **Share** button does the same thing it always did, and what comes out of it is different: the journal arrives in the chat as a **PDF document** - Cash journal.pdf, Profit and loss.pdf - which you can open, keep, forward and print again. It is listed for Mail, Notes and Save to Files exactly as before, and WhatsApp is on that list now.

Worked out with your phone, in this order. The app asks the phone what it can take rather than assuming:

- The phone can share a file - you get the **PDF**, which is what happens on yours. The phone says it can share, but not a file - you get the journal as **text**, so it still leaves the app rather than nothing happening. The phone has no share sheet at all - the **PDF is saved to the phone** and a short message says so. You send it from WhatsApp yourself. Even saving is impossible - the journal is **copied** instead, with a message saying so.

If you open the share sheet and change your mind, closing it does nothing else. Nothing is copied and nothing is saved behind your back, at any of those four steps. A cancel is your decision, not a fault.

What is on the document. Your shop name as a letterhead, the journal's name and the stretch it covers, every row and every total in the same words and the same order as the screen, the note that explains the figures, and along the bottom From More -> Profit · printed 3 Oct 2026. Money that went out reads -RM 12.00. A long journal runs onto a second page, which says at the top what it continues. A4, black on white, 20 mm margins. It is about 3 KB - small enough for any chat.

Print has not changed. The Print button still opens the phone's own print sheet, and **Save as PDF** is still a choice inside it.

What did not change. No figure moved and no rule about your money changed. The screen is identical. The document is a fourth rendering of the same journal the screen, the paper and the sent message already shared, so the figure in the file is the figure on the screen, to the cent.

03 Oct 2026 - engine v282, EVERY JOURNAL CAN LEAVE THE SCREEN: PRINT IT OR SEND IT (no database step, nothing to upload - pushing this one is the whole of it).

What you asked for. You were reading a journal and said: *"those journals in profits and other journals should be printable and able to be shared"*. Until now a journal existed only inside the pop-up you were scrolling. You could read it, and there was no way to hand it to anyone or keep it.

What you see now. Every journal in the app wears the same two buttons, in the same order, under its last line:

- **Print** - the phone's own print sheet opens with the journal on it. That is also where **Save as PDF** lives, so a journal can leave as a file. **Share** - the phone's own share sheet opens with the journal as plain text, ready to go straight into WhatsApp, Mail or Notes. A phone with no share sheet - or a share that fails for any other reason - **copies the journal instead** and says so in a short message. You paste it wherever you want it. The fallback carries exactly the same text the share sheet would have been handed. **If you open the share sheet and change your mind**, closing it does nothing else. The journal is not copied behind your back. A cancel is your decision, not a fault. *Superseded the next day by v283: the phone is handed a PDF file now, not plain text, because WhatsApp was not appearing in the share sheet.*

Where the two buttons are. Every book in the app:

- **More -> Profit** - the Sales journal, the Cost of sales journal, and every spending category's journal, including Total expenses. **More -> Profit** - the **Profit and loss** statement itself. You chose that the whole statement gets the pair and not only the journals: Sales down to Net profit, with its two notes underneath. A journal on its own is half a document, and this is the other half. **More -> Money** - one method's book, opened either from its row on the screen or from **Books**. In Books, only the book you have opened can print; the list underneath never does.

What the printed sheet says. Your shop name as a letterhead, then what the journal is and the stretch it covers; every row and every total, in the same words and the same order as the screen; a line in the journal's own words saying what the figures mean and where they come from; and along the bottom, From More -> Profit · printed 3 Oct 2026. The statement's sheet carries the Cost of sales note as well, because anyone reading Gross profit needs it. It is plain black on white.

One rule this build rests on. The screen, the paper and the sent message are three renderings of **one** description of the journal. They cannot disagree about a row or a total: whatever figure is on the screen is on the paper to the cent, and the same figure is in the message.

On the phone. Pressing Print opens the phone's real print dialog - the same one any other app uses - and **Save as PDF** is a choice inside it. Choosing a printer there is the only way to see the true paper; the sheet itself is built by the app.

What did not change. No figure moved, and no rule about your money changed. Every pop-up looks as it did; the two new buttons sit under the last line of each.

3 Oct 2026 - no new engine (still v281), THE "SHOPS & CODES" SCREEN IS RETIRED (nothing to run, nothing to upload - pushing this one is the whole of it).

Why. The More menu carried two code screens side by side - **Shops & codes** (the pet shops you hand samples to, the printed QR labels, the page a scan opened, and the scan counts) and **Promo codes** (the codes a customer types at the shop). You said the promo code was meant to replace the other and only one should survive. That is now so: **Promo codes stays exactly as it is; Shops & codes is gone.**

What went. The **Shops & codes** screen; the pet-shop sample list; the printed QR labels and the page a scan opened (the whole stand-alone landing page); the **Label visits** counts; the "Scan a label" camera; and the line an order carried when a customer had arrived by scanning a card. The shop page no longer states a scanned label's offer, because there are no labels any more. An order the customer typed a **promo** code onto is untouched by any of this.

What stayed, and it is the part that matters. Everything on **Promo codes**: making a code, what it gives, whether the shop may show it, pausing and ending it, the printed promo card with its QR, the "Have a code?" box on the shop, and the code coming off the total. Bring-a-friend, the delivery calendar, and every order's own record are untouched too - only the label/QR-label side has gone.

What you must do. Nothing but push. There is no SQL to paste, no Edge Function to upload and nothing to change on the phones. You confirmed no QR labels are out with any shop, so no card in anyone's hand breaks - the addresses those labels used to open are simply gone.

3 Oct 2026 - engine v281, THE STATEMENT SAYS WHAT KIND OF COST IT IS SHOWING (no database step, nothing to upload - pushing this one is the whole of it).

Why. You asked whether Cost of sales is really your recipe's ingredient cost - the theoretical figure - rather than money you had spent. It is, and the screen did not say so where the figure is. It only said a part of it in the small print underneath, and nothing at all about the part that matters most: that the cost is read from your recipe and your ingredient prices *as they stand today*.

What you see now, on More -> Profit. A short line sits under **Gross profit**, inside the statement, in the same quiet grey as the other notes:

- **"Cost of sales is built from the recipe and the ingredient prices you have recorded, read as they stand today - so editing a recipe or a price moves past months too. It is not what you actually spent.**

Gross profit is therefore a guide to your pricing, not your bank balance - the Money screen is where the cash is."

Two things that follow from it, now said on the screen rather than left for you to work out:

- **A month that has already closed can still move.** If you fix a recipe, or update what an ingredient costs, the Cost of sales of a past month changes with it, and so does that month's gross profit. The sold price never moves - that is frozen on the order - but the cost side is read fresh every time you look. **Gross profit and Net profit are a plan, not a bank balance.** What you really spent is cash, and the Money screen is where you check it. A gap between the two is normal and not a fault.

The small print at the bottom changed too, so the two do not repeat each other. It now leads with the cash half - a pack bought today is cash on the Money screen and stock on the shelf, and only becomes cost of sales as the treats made from it are sold.

What did not change. No figure moved. The Sales journal, the Cost of sales journal, the month arrows and every spending line behave exactly as they did in v280. This is words on the screen, nothing else.

3 Oct 2026 - engine v280, THE WHOLE STATEMENT OPENS: SALES AND COST OF SALES TOO (no database step, nothing to upload - pushing this one is the whole of it).

What you asked for. You were on More -> Profit, reading the statement, and asked: *"at the profit section, can the sales and cost of sales be clickable to reveal its journal"*. The spending lines had opened on a tap since 17 September; these two did not.

What you see now. Tap **Sales** and its journal opens; tap **Cost of sales** and that one opens. Both read the same orders - one from the side of what the customer paid, the other from the side of what your recipes say those treats cost to make - so the two can never disagree about which orders the month held. Each row is one order line: the day it is delivered, what sold and how many, and the customer's own name, with the figure on the right. A product you have since renamed or deleted still reads as the treat that was sold, because the name is the one frozen on the order. The rows land on the figure you tapped, exactly. A line your recipe prices at nothing is marked "no recipe cost" rather than sitting there as a plain RM 0.00, and the footer counts them - that is a profit reading too high, the one thing worth chasing here. A month with nothing in it opens and says so.

One thing that stays a figure, not a door. Gross profit and Net profit are worked out from the lines above them, so they have no rows of their own to open.

3 Oct 2026 - engine v279, THE PRINTED CARD: PRINT IT, AND THE OFFER IS FIXED (no database step, nothing to upload - pushing this one is the whole of it).

What you asked for. Of the eleven promo steps, step 6 - Print it - was the last one with nothing behind it, and it is the step that puts a code into a customer's hand. You asked for the printed card next.

What you see now, on More -> Promo codes. A **public** code's row has a **Print it** button, first ahead of Edit and Delete, and only where a card makes sense: on a public code, never on one that has **ended**, and never on one **already printed** - that one wears its "Printed - fixed" chip instead. A **paused** code can still be printed, because pausing is a break. Pressing it asks first, and the question is honest: from here the amount, who it is for, the smallest basket and the name all go on saying what they say, because the card in a customer's hand cannot be amended; the end date may be moved later (never earlier) and that is all that stays yours. **It refuses a code with no cost ceiling, and says why** - a card carries no number and no end date, so the ceiling is the only thing left bounding what the card can cost you. A code with no name is refused too.

What the card looks like. A new tab opens on a print-sized page holding **four identical cards on one sheet of A4**, with a hairline guide to cut along. Each card carries your shop name and tagline, the offer in the customer's own words, the smallest basket if there is one, your own sentence when you wrote one, the rules that stay true ("First order only", "One per customer", "Not with the bring-a-friend welcome discount"), the code in large letters, and a square to scan. The square points at your shop with the code already filled in, and the same address is printed underneath in words - so a card that ever pointed somewhere wrong would be wrong visibly, on the paper, before anyone was handed it. It carries **no end date, no count and no ceiling figure** on purpose: those

three are promises paper cannot keep. Opened by itself the page never prints a blank sheet - with no code, or an unknown one, it says so and offers no Print button at all. Once a code is printed you can still raise its ceiling, extend its end date and end it; you cannot change what it gives, who it is for, or the smallest basket.

3 Oct 2026 - engine v278, A CODE'S LIFE: PAUSE, END, AND WHAT PRINTING FIXES (no database step, nothing to upload - pushing this one is the whole of it).

What you asked for. The Promo codes screen had a half-finished piece: the eleven management steps were a plain grey box, and the Pause and End controls were not wired to anything. You chose *"Finish it properly"*.

Every row now says what its code's life is. A chip on the name: **Paused**, **Ended**, or **Printed - fixed** (a live code wears nothing). **Pause** and **End** sit on the row under the offer, and they are different on purpose: Pause is a break - the shop stops offering and accepting the code, it switches back on any time, and nothing about the orders already carrying it changes; End is final - new uses stop and it cannot be switched back on. Both say what they are about to do first, including how many orders already hold the promise, and that they keep it either way, because ending stops new uses and never rewrites history.

A printed code is fixed. Once a code is on a card in someone's hand, the offer has to go on being true, so a printed code refuses any change to what it gives, who it is for, the smallest basket, what it cannot sit beside, or its name. Two things stay open, one way round only: the end date may be moved later or dropped, never pulled earlier; the ceiling may be raised or removed, never lowered. The Edit button on such a row reads **"Update the end date and ceiling"**. A printed code cannot be deleted either - the tap says why, and names End instead.

The eleven steps fold. The life of a promotion, in order, is a proper list under a fold line, and the fold line is the answer you came for: "1 live · 1 paused · 1 ended · RM 20.00 given away". Printing, step six, is the only one that cannot be undone, so it is the only shaded row, and it says **point of no return** in words rather than by colour alone.

3 Oct 2026 - engine v277, THE MONEY LINED UP IN ONE COLUMN (no database step, nothing to upload - pushing this one is the whole of it).

What you asked for. The day after the receipt you said: *"the format still not as clear as a receipt, the money have to align up"*. Asked which screen looked wrong you named the customer's tracking page and your own order list; asked what lining up meant, you picked **"Line up in one column"**.

The customer's tracking page. The figures now line up in one right-hand column, and the decimal points sit on that line. **On this shop the card keeps its own shape, and that is deliberate.** Your shop adds a flat nationwide postage to every posted order, and that fee is never published, so the card cannot work out an items subtotal from the total it is given - it would be wrong by the postage. So the card keeps the one line it has always drawn, "what they ordered - the total", which cannot be wrong because it names no subtotal at all. What the card *does* name, and now does name, is the code the customer used: the discount is published on the order itself, so no working out is needed. See the v272 entry below.

The order list. Every order row now ends with its own total, on a line of its own across the foot of the row, at the same right edge as every other row. Before this a row said "×2" and stopped - the one screen you work from all day was the one place an order's money never reached. Three things worth knowing: it is one figure worked out once, so the row, the receipt and the customer's message can never disagree; an order nobody has priced says nothing rather than "RM 0.00"; and the WhatsApp message is deliberately unchanged, because you were shown both ways and chose the plain lines.

3 Oct 2026 - engine v276, AN ORDER'S MONEY READS AS A RECEIPT (no database step, nothing to upload - pushing this one is the whole of it).

What you asked for. *"can the showing of promo be more streight forward, clearer, like putting them in an accounting format, clearly shown the working, how they add up."* So the promo showing was rebuilt the way the Profit statement already reads. On both screens that show an order's money - the Edit pop-up and the Note / tracking card - the one run-on sentence is replaced by a receipt: Items total, Courier charge, Promo FRESH10, a dashed hairline, then Total.

Four things this changed. A code that gave nothing is now a **line on the receipt reading RM 0.00**, not left out - before, an order that carried a code which paid nothing looked identical to one with no code at all. The reason sits under the figures in your own terms: the basket the code wanted, the basket this order was, and a plain statement that the rule is a **guide**, not a gate. The customer's WhatsApp message says the same thing in one quiet line, "Code FRESH10 not applied: basket below RM 100.00", where before it said nothing. And nothing about a missed code touches the money: the line explains, it never deducts. An order with no code is unchanged, line for line.

3 Oct 2026 - engine v275, A DISCOUNT NOBODY EARNED (no database step, nothing to upload - pushing this one is the whole of it).

What you reported. *"the arithmatic is not rigght, the fresh10 promo code discount 10 for order of 100, but my order only 16, it deduct 10 and customer have to pay 6 only. if thats the case bakery will broke."* You were right, and it was the worst kind of wrong: nothing crashed, nothing looked odd, and the app quietly took money off an order that had never earned it.

What happened. A code has two separate things about it, and only one was being asked. The shop's box has always asked "is this basket big enough for this code at all?" and refuses a code below its smallest basket. But the arithmetic that takes the money off only ever asked "what is this offer worth on these goods?" - a question about the offer, not about the sale. So once a code was on an order, the smallest basket was never looked at again.

The fix. The smallest basket is now asked wherever the money is worked out, which is one place - the same single figure the customer messages and your own screens all read. A code on an order whose basket never reached its smallest basket now gives nothing, and the order reads exactly as one with no code. The tally on your Promo codes screen counts it the same way, so such an order counts RM0 given and cannot eat a ceiling it never touched - though it still counts as a use, since the code did ride on the order. **What has not changed:** a code you have since paused, ended or used up still comes off the order it was actually placed on. Nothing here blocks an order or refuses a code you want to honour; it only stops the app doing arithmetic on a discount that was never earned.

3 Oct 2026 - engine v274, THE SHOP'S OWN SENTENCE IN THE HAND YOU PICKED (no database step, nothing to upload - pushing this one is the whole of it).

You picked the chalk hand - rounder and fatter, like chalk on a board - from five real versions drawn inside the shop's own strip. The amber strip at the top of the shop has two lines. The first, the offer and the code, is untouched: same lettering, same size, still first, because it is the part a customer has to act on. The **second line** - the words you wrote yourself about the code - is now set in the chalk hand, and a little larger than before, so it reads as your own note under the offer rather than a second announcement. **One limit, said plainly:** the hand is a Latin face and carries no Chinese characters, so a sentence you write in Chinese keeps exactly the plain lettering it has always had - the words are the same and nothing moves, it simply cannot wear the hand. English and Bahasa Malaysia wear it. The font travels inside the shop, in the shop's own folder with its licence, so nothing is fetched from a font company when a customer opens the page.

3 Oct 2026 - engine v273, YOUR MESSAGES IN A VOICE YOU CHOSE (no database step, nothing to upload - pushing this one is the whole of it).

One limit, said plainly. A font cannot be chosen in a WhatsApp message - WhatsApp carries no fonts at all, and its only four marks are bold, italics, strikethrough and monospace. So the closest thing that exists is making your opening line lean over. **Message style, on More -> Settings**, is a card with one choice, and it applies to all four messages you send a customer: **Plain** - exactly what you send today, the default, word for word the message that went out before this card existed - or **the greeting leans over**, where only the first line moves, and only into italics. One switch for all four on purpose: four separate switches are four ways for your messages to end up opening differently from each other. The bold Total is not affected, and the choice travels to your other phone.

3 Oct 2026 - engine v272, THE CODE COMES OFF THE TOTAL (ONE SMALL DATABASE STEP FIRST -

run it before you push, the order matters; nothing to upload otherwise).

What you reported. *"pushed. The whatsapp message still withhout the promo discount."* You were right, and this time it was a design that had gone stale: since the code arrived, it rode on the order and every discount was taken off by you, by hand, in WhatsApp. You have now said the app should do that arithmetic itself, and this is that.

The code now comes off the Total in all four places the customer reads it - the confirmation, the payment reminder, the "on its way" message, and their own track page - as its own line, named by the code, between the workings and the total, so the figure can still be added up by the person reading it: *Total: RM 30.00 · Promo FRESH10: -RM 10.00 · To pay: RM 20.00* (shown here in this shop's own words). On your own screen the two figures you read move with it - the amount the customer owes in the Note / tracking window and the Order total in the Edit window - and the "Still to collect" figure on your Money screen follows, because it was always worked out from the same number.

What does not change. An order with no code reads word for word as it did before. A code that has since been paused, ended or used up still comes off the order it was actually placed on: ending a code is a decision about future orders, and it never reaches back and re-prices one you have already promised. A code whose terms come to nothing on this order prints no line rather than -RM 0.00. A discount larger than the order leaves you asking for RM 0.00, never a negative. A percentage comes off the goods and not off the courier's charge. A free-delivery code has nothing to waive on a Collect order, so it says nothing.

THE DATABASE STEP - do this first, before you push. In your Supabase SQL editor, run the file **supabase/promo_track.sql** (Dashboard, SQL, New query, paste, Run). It adds two columns, promo_code and promo_rm, to the customer tracking table. **Order matters here:** your app publishes a whole tracking row in one call, and if those columns do not exist yet that call is refused as a whole and the customer's tracking page stops updating for every order, not only the orders that carried a code. This is the same trap the courier charge tables set. Run the SQL once, then push. It is safe to run twice.

One thing that has not changed, so it does not surprise you. Your sales and profit figures still count the full price of what you sold; the discount comes out of the money you actually collect, not out of the goods' own value. That is a real question about where a discount should sit in your books, and it is yours to decide.

3 Oct 2026 - engine v271, A CODE THAT KNOWS WHEN TO STOP (no database step, nothing to upload - pushing this one is the whole of it).

A card has no number on it and no end date on it - deliberately, because a date is a promise a card cannot keep. That left one real question: how does a code with a card stop being a good idea? Now you say so yourself, on the code screen. The new box, **"Stop after giving away (RM)"**, stops the code the moment it has given away that much money. It sits beside the order limit, and whichever runs out first is the end of it - so you can say "the first 5 orders, or RM50, whichever comes first". Leave it empty and the code has no limit at all. It works for every kind of code, including a percentage, which otherwise has no natural end.

Where the numbers come from matters. They are counted from your own orders, freshly, every single time - not kept in a tally on the code, so the figure cannot drift. A basket counts once: a customer who orders three things in one go used the code once. Your screen shows how far through its limits a code is, and a code that has finished says so in an amber banner. The customer never sees the figure or the reason: they get the one sentence, "that code has been fully claimed". A code that has given away everything you allowed stops being advertised at the top of the shop - so after you push, **open the app once on your phone with a connection** and the shop will be told.

3 Oct 2026 - engine v270, THE CODE THAT WAS THERE ALL ALONG, AND A PROMO SCREEN WITH SOMETHING TO SAY (no database step, nothing to upload - pushing this one is the whole of it).

What you reported. *"i dont see the promo code fresh10 send over to app together with the order."* You were right: the code had been travelling with the order since v269, but no screen ever showed it - and a code that arrives and cannot be read is a code that did not arrive. An order placed with a code now wears a small amber **FRESH10** tag beside its order number, on every row you read an order on: the delivery day's list, the unread

inbox at the top, the search results, and the Edit order window. It shows the code itself and not the word "promo", because you take the money off by hand, so which code it was is what tells you whether it is RM10 or RM5. An order placed without a code looks exactly as it always did.

The shop's line now says the whole offer - the smallest basket it works on and, when you have set one, the day it runs out: "Today: RM10.00 off on RM30.00 and above, until 31 October - use code FRESH10". And you can add your own sentence to it, in English, Chinese and Bahasa Malaysia, with a **Translate** button that fills the two other languages and never overwrites anything you typed. The code screen is now a full one: who it is for, when it runs, the smallest basket, how often it can be used, what it cannot be used with, and who can see it - all optional, all defaulting to the widest, simplest answer. **The shop now says exactly why it will not take a code** - nine plain sentences, including "that code ended on 30 September", which names the day so it does not read as a glitch. Two of those are stated rather than refused: the shop is a public page with no sign-in, so "once per customer" and "first order only" can only ever be a good guess, and a rule on the website must never block or hide a sale you take by hand.

3 Oct 2026 - engine v269, PROMO CODES, THE FIRST SLICE (no database step, nothing to upload - pushing this one is the whole of it).

What is new for you. A new screen, **Promo codes**, at the bottom of the **More** menu. You make a code there: the code itself, what it gives, and whether the shop may show it to everybody or you are giving it to one person. Three kinds of offer are in this first version - a ringgit amount off, a percentage off, or free delivery.

What is new for your customers. The shop page has a **Have a code?** box. A customer types the code and presses **Use it**. If it is a code you made, the page says what it gives and tells them you will take it off when you confirm the order. And when you are running a code the shop may show, the top of the page carries a line naming it - for example *Today: RM10.00 off, use code FRESH10*.

The most important thing about it. The total on the shop page **never moves**. A code does not change what the customer is charged; it is a note that travels with the order, and you take the money off by hand in WhatsApp exactly as you already do for the bring-a-friend credit. An order that used a code carries that code into your app. A code you made on one phone also reaches your other phone. One thing worth knowing about "personal" codes: a personal code is never shown on the shop page and never advertised, but the shop's own data is readable by anyone who looks at the page, so personal means **never shown** and not **secret**.

3 Oct 2026 - engine v268, FIVE PLACES THAT DISAGREED WITH EACH OTHER (no database step, nothing to upload - pushing this one is the whole of it).

What you said. *"pushed. when i schange the courier delivery to self pickup, the courier chages tag still there, just wondering are details well taken care of?"* - and then the audit you asked for: *"pls run whole system audit for the well being of the system later"*. v267 had fixed the tag you could see; reading the whole system after it turned up four more places where one screen had been told the truth about a self-collect order and another had not, and one button that made a claim it could not know. They are all the same fault underneath: a fact is written on the order, and only some of the screens that read it ask the companion question that goes with it.

Five things, fixed together. (1) The customer's track card was still being sent the courier's half of a self-collect order - a driver's name, a plate, a phone number, "on the way", a waybill - and it now publishes none of it. (2) The trip card counted a parked charge, and now counts only the orders actually going by courier. (3) Picking **Cash** or **TNG transfer** on an order wrote down how they paid and stopped there, so the row could wear a TNG tag while the day's till, the Money screen and the customer's card all still counted it as owing - it now records the payment itself, with **Not recorded** as the way back. (4) **Print label** vanished the moment the order was packed; it is now on every stage from Baked to the end. (5) **Send confirmation** turned the order green before the message was sent - it opens WhatsApp but does not press Send - so the button now only drafts the message, and a small **I have sent it** beside it is what turns Confirmed green. One extra tap on each confirmation, in exchange for never marking something sent that was not.

3 Oct 2026 - engine v267, A COURIER CHARGE GOES QUIET WHILE THE ORDER IS A SELF COLLECT (no database step, nothing to upload - pushing this one is the whole of it).

What you said. *"when i schange the courier delivery to self pickup, the courier chages tag still there, just wondering are details well taken care of?"* and *"confirmation message still include courier charges"*.

What was happening. Switching an order from Courier to Self collect never deleted the courier charge, on purpose - you might switch back. But nothing on the reading side ever asked whether the order was still going by courier. So a self-collect order went on wearing a **Courier RM 8.00** tag on its row, went on adding that RM8 into the customer's total in the confirmation message, in every WhatsApp message and on their track card, and went on being counted in **Still to collect** on your Money screen.

An order that is a Self collect now carries nothing for a courier to charge for. The charge is parked, not thrown away: it is still written on the order, so switching **Fulfilment** back to Courier brings the whole thing back exactly as it was, with nothing to type again. Both courier charge cards say in words that the charge is recorded but not added to the customer's total while the order is a self collect, and that switching Fulfilment back is how it goes back to work.

3 Oct 2026 - engine v266, A BUTTON SAYS WHAT HAPPENS IF YOU PRESS IT, AND WHAT HAPPENS IF YOU DON'T (no database step, nothing to upload - pushing this one is the whole of it).

What you said. *"when i see the button, i might self have to ask, i dont know what will happen or what will happen if i din press that button, these create confusion"*.

What was happening. Two places put a choice in front of you without saying what the choice cost. On the **shop page**, the line above the pin buttons read "Optional: drop a pin where the courier should stop" - which tells a customer the pin does not matter without telling them what they get if they do it, or lose if they skip it. It now answers both halves: skip this and the driver goes to the address you typed, which is fine for most houses; pin it only if the address alone will not find your door; and either way put the block and unit number in the address above. Written in all three languages. In your own **pin window**, the **Coordinates** box with its **Use these numbers** button stood open under every map; it now comes out only when the map cannot load, so the sentence that names it points at something really there. On a working map it waits behind one small button reading **Have a Google Maps link?**, because a Google Maps link is the most accurate point a customer ever sends you.

3 Oct 2026 - engine v265, THE CONSIGNMENT BOX IS A PARCEL'S, SO IT IS DRAWN WITH THE PARCEL (no database step, nothing to upload - pushing this one is the whole of it).

What you said. *"after i get price from lalamove, click use this fee, it closes the window, but user might confuse as at that page there is stick fields that user have not fill in"*. On the **+ New order** card the courier half drew the **Courier tracking number** box on every courier order, whatever kind of courier it was - and a consignment number is a parcel's, not a Lalamove trip's. So on a trip that box was empty, about nothing, and stood beside a finished price: that is what read as an order with something still to fill in. The box now belongs to the parcel, so it is drawn with the parcel and nowhere else. Nothing became unreachable: the parcel carrier picker is still on the card, saying **Not a parcel - nothing recorded** until you pick one, and a number you had already typed is never hidden. The Edit window and the Note / tracking box keep the layout they had.

3 Oct 2026 - engine v264, A TAP ON THE MAP DOES NOT MOVE THE PIN (no database step, nothing to upload - pushing this one is the whole of it).

What you said. *"click on the map should not move the pin, only dragging the pin will"*. A tap on the map no longer moves the pin; only dragging the pin itself moves it - on both maps you use to place a pin, the order form's pin map and the pin window. Why it matters on a phone: that map is a 200-pixel strip inside a card you scroll, and before this any accidental touch moved the pin to wherever your finger landed and pulled the map onto that spot, two things at once from one touch you did not mean. The one place a tap still works: a map with nothing on it yet has no pin to move and nothing to drag, so there a tap places the first pin. The shop's own customer map was deliberately left alone - for a customer the map is often the only way to place their pin at all.

3 Oct 2026 - engine v263, A DAY YOU DO NOT DELIVER IS DARKER STILL (no database step, nothing to upload - pushing this one is the whole of it).

What you said. *"can make the quite day more solid?"*, asked straight after seeing v262. A day you do not deliver

is now a solid, dark day - v262 had taken the fade off it, but it was still faint. It is now very nearly twice as strong again. The colour sits exactly halfway between the app's quiet grey and its ink, so it reads as a real day rather than a whisper, while still looking nothing like a day you deliver. A day you deliver is still twice as dark as a day you do not, so the two can never be mistaken for each other. The past grey itself is untouched, and one case is deliberately left faint: on a product's availability calendar, a day you have marked as selling but which you do not deliver is kept faded on purpose, because it means "this mark cannot become an order".

3 Oct 2026 - engine v262, A DAY YOU DO NOT DELIVER IS SOLID, AND THE PAST IS ONE GREY AT LAST (no database step, nothing to upload - pushing this one is the whole of it).

What you said. *"I still feel that the greyed and the non grey contrast is not big?"* - and then, when offered the choice, make the non-grey more solid. v261 had put the shop's grey in the right place, but it left the older half-transparent rule standing on a day you do not deliver. That half transparency is applied to whatever colour the day ends up with, so the past was still being drawn in two shades - and worse, a past day you do not deliver came out fainter than a future day you do not, which is backwards. With the fade gone, a day you do not deliver is a solid, plainly readable day, and the past is now genuinely one shade: a past day you deliver and a past day you do not are the identical colour on screen. The shop's past grey itself is untouched, and anything on a past day that is telling you something - a delivery day's green pill, a product's green tint - keeps its own colour.

3 Oct 2026 - engine v261, EVERY CALENDAR NOW GREYS OUT A PAST DAY THE SAME WAY YOUR SHOP DOES (no database step, nothing to upload - pushing this one is the whole of it).

What you asked. *"Is all calendar passed day grey out same thru all calendar? I like the one at store."* It was not. On the Orders calendar a day that had gone by came out in two different greys side by side in the same row - one shade for the days you deliver, a slightly different one for the days you do not. On the other calendars there was only one grey, but it was the app's own faded grey, not the shop's - and the shop's is the one you picked. A day already gone is now drawn in the shop's own quiet grey, flat and one shade, whether or not you deliver that day: the Orders calendar, the calendar inside Delivery Dates, the small calendar you open when you pick a delivery day, and the days you mark on a product's availability. Nothing ahead of today is touched, and anything on a past day that is telling you something keeps its own colour.

3 Oct 2026 - engine v260, YOUR MAILS NOW COME FROM YOUR OWN SHOP'S NAME, NOT THE APP'S (two functions must be uploaded - pushing alone does not change this one).

What a customer saw. The two mails this system sends - the wish-list mail you send out, and a customer's suggestion coming back to you from the shop page - each arrived with the name at the top of the inbox list reading **"BakeAdmin wishes"**. BakeAdmin is the app's own internal name: a word that appears nowhere on your homepage, nowhere in your shop, and on no label you print. It was the first thing a customer read from you, before they opened anything.

What they see now. Munchies Furkidz - your own name, taken from your homepage title, which is where your brand is written down. The address the mail is actually sent from has not moved: it is the same verified address as before, and only the name in front of it changed.

One thing to check, because it can silently win. If the project has a saved setting called RESEND_FROM, that setting is used instead of the name in the code, and this change would make no difference to what a customer sees. Run supabase secrets list in Terminal to see whether it is set.

Why pushing this one is not enough. These two mails are sent by the two functions that live in Supabase, not by the app on your phone. Uploading the code - a push - leaves Supabase running the old copy. Both functions must be uploaded separately, from the repository folder: wish-mail and shop-feedback. This changelog cannot do that part for you.

No database step. Nothing was added or changed in the database, and no existing setting moved.

1 Oct 2026 - engine v259, AN ORDER'S EDIT CARD NO LONGER CLOSES ITSELF WHEN YOU LOOK AN ADDRESS UP AGAIN (no database step, no redeploy, one push).

The fault, and it is one word. Pressing "Look this address up again" inside an order's Edit card found the address, wrote it onto the order, and then saved and closed the card - dropping you on the Orders list with no chance to ask for a delivery price. The card declares its own button called "Save changes", and that name silently took over the app's own saving step for the whole of that card's code. So the one press did two jobs: it saved, and then it closed.

The saving is kept apart from the button now. The address is written down and the card you are working in stays open, with the price press still there to use.

How the cause was pinned down. The tracking pop-up's own look-up carries the same saving step and was never broken, because it has no button of its own to shadow it. That difference is what named the fault rather than another guess at it.

1 Oct 2026 - engine v258, THE APP NOW TELLS YOU WHEN YOUR PHONE IS RUNNING AN OLD BUILD (no database step, no redeploy, one push).

This is the real answer to a fault you reported five times. The fix was in the app; your phone was still running an older copy of it. The site that serves your app lets a phone hold each file for ten minutes, and the app is a single page you leave open, so a phone could keep yesterday's build for as long as it stayed open. Worse, the Engine number on More reads a different file from the one carrying a fix, so it could vouch for a build that was not the one running - which is why the fault kept coming back after a fix had been published.

Two halves fix it.

- **The app re-checks its own files with the site before using its saved copy.** An unchanged file costs almost nothing to check; a changed one is fetched whole. Pictures are left alone. **An amber strip above the tab bar when the two differ.** It says which build is on the phone and which the site is serving, in plain words, with a single "Update now" press.

It is never a dead control. If the strip is still there after you press Update now, it stops offering the button and tells you to close the app completely and open it again.

30 Sep 2026 - engine v257, A PRESS ON A CONFIRMATION NO LONGER FOLDS THE CARD YOU ARE WRITING (no database step, no redeploy, one push).

This was reported three times: landing back on the Orders list with a half-written order gone. The + New order card folds away when you press anywhere outside it, which is meant, so a stray press never traps you in it. But the confirmation the card itself opens is drawn on a layer beside the page rather than inside it, so pressing "Reset the pin" on that confirmation counted as a press outside the card and folded the one you were writing - while the door's own look-up carried on and wrote the order.

A press landing inside the card's own confirmation now counts as a press on the card, so the card stays open. A press on the page itself still folds the card, exactly as before, and a layer that is not showing shields nothing.

v255 and v256 both changed the pin picker's "Look it up". The reports you sent were about the door block's "Look this address up again", and this is that fix.

30 Sep 2026 - engine v256, THE LIST OF MATCHING ADDRESSES FLOATS, SO A LOOK-UP STOPS THROWING THE CARD ABOUT (no database step, no redeploy, one push).

The pin picker's list of other matches was sitting in the card itself, above the map. With several matches the list was taller than the room it had, so everything under it was pushed down - carrying the box you were typing in, and the button you had just pressed, up off the top of the screen. That is the "it exits the page" you described.

The list floats under the button now and takes up no room at all. Nothing below it is shoved anywhere: the button stays exactly where your finger left it, the card does not scroll, and only the one answer line moves. The list's height no longer matters - it behaves the same with two matches and with four.

Picking a row now closes the list, so it stops covering the map you most likely wanted to look at next. The

door block's own "Look this address up again" was re-measured at the same time and was found not to have this fault.

30 Sep 2026 - engine v255, LOOKING AN ADDRESS UP NO LONGER THROWS THE PIN CARD AROUND (no database step, no redeploy, one push).

Your report: "when i say look this address up, why the interface jump out of the page?" The press you meant was the pin picker's own "Look it up". With four matches the list of other addresses is taller than a phone's screen, and it sat in the card above the map, so the map - and the "Use this spot" button under it - were shoved a whole screen down and off the page. The button you pressed did not move at all, which is exactly why it read as the page jumping rather than as the list pushing things down.

The pin card now holds its place while the address is looked up, using the same hold-this-still rule the delivery-price card already had: the row of buttons you are working with keeps its screen spot while the words below it extend downwards.

That rule now lives in one shared place, so every screen that grows while you watch uses the same one and they cannot drift apart. Both of the app's scrolling areas - the page itself, and the inside of a pop-up - are handled.

30 Sep 2026 - engine v254, THE NEW-ORDER CARD HOLDS STILL, AND THE BUTTON NOW SAYS PLACE ORDER (no database step, no redeploy, one push).

Three faults from your own report, all in the + New order card.

- **The card grew under your finger on a pin reset.** A reset can change the card's height in eleven ways at once, and nothing held the screen still. The row of buttons under your thumb is now what the card is pinned to, so the row keeps its place and the words below it extend downwards. **Maps left running after their card had gone.** Each rebuild of the price block left a live map behind on a background listener, and one of them could paint over the confirmation box. They are now swept away once their card has left the page. **The button that finishes a new order now reads "+ Place Order".** The card's own title is still "+ New order", the "+ New order" button on Home is untouched, and an order's Edit card still ends on "Save changes".

Measured, not guessed. At a 375-pixel phone the button row held to the pixel while the page scrolled; a press that made a map appear used to move everything below it by 210 pixels. And the overlap with the confirmation box measured 16 spoiled rows of 24 before the fix, none after.

30 Sep 2026 - engine v253, THE FEEDBACK MAIL NOW READS IN THE SAME ORDER AS YOUR WISH-LIST MAIL (no database step, but the shop-feedback function must be redeployed).

The heading block moved to the top, and the customer's words now come last. v252 put the words first and the heading underneath a short rule. You asked for the wish-list mail to be the reference, and the wish-list mail does the opposite: it opens with what the mail is, then who it is about and when, and puts the content underneath. So the feedback mail now does exactly that, with no rule in between and no closing line:

New feedback for the shop page (Engine v253).

Project: munchies.com.my/store/

Sent: 2026-09-30 17:31

Written in English.

the words the customer typed

The language is written out as a word. v252 printed the short code it was handed, so the line read "Written in en." The shop asks its questions in three languages, so the line is worth having - but "en" is not a word anybody reads. It says **Written in English.**, **Written in Malay.** or **Written in Chinese.** now, and it is left out entirely when the page did not say which language the customer was reading.

The subject line and the Project line are unchanged from v252: the subject is still Shop feedback · Engine v<n> · <date>, the project is still the live address the customer was reading, and the time is still on your own clock in Penang. Only the order of the body and the wording of the language line moved.

One step on your machine. The part of this that builds the email lives on your Supabase project, not on GitHub, so pushing is not enough on its own. After pushing this version, redeploy it once:

supabase functions deploy shop-feedback project-ref ircwozniiyywsowamixy

30 Sep 2026 - engine v252, THE FEEDBACK MAIL SAYS WHICH SHOP AND WHICH BUILD, AND THE SHOP PAGE NOW SAYS WHICH BUILD IT IS RUNNING (no database step, but the shop-feedback function must be redeployed).

A customer's message now arrives with the same kind of heading your wish-list mail has. Before this version the email was the customer's words and nothing else, so a message saying "the cart is confusing" could have come from either of your shops and from any build of the page. Every one of these emails now carries:

- **Project** - the address the customer was actually reading, for example munchies.com.my/store. It is read off the live page rather than typed in anywhere, so your two shops can never be confused for one another, and the second half names which page of that shop the words were written on. **Sent** - the date and time on **your** clock, Penang time, so "16:52" means the same thing on the email, on your phone and in the backoffice. **Engine** - the build the customer was looking at, both in the subject line and in the line above the project, so you can tell whether a comment about the shop came from a phone that is still running yesterday's copy. **Written in** - the language, when the customer was reading the shop in Malay or Chinese.

And the shop itself now says which build it is running. At the very foot of the shop page, under "Website by" and the developer's WhatsApp and email links, there is a new small line:

Engine v252

It is the same number your app shows on More, and the two are read from one file rather than kept as two copies that could drift apart. That is the point of it: when a customer tells you something looks wrong, you can compare the number at the foot of their shop page with the number on your own More screen and know in one glance whether they are seeing the same page you are. It is the smallest type on the page and the same muted grey as the credit above it, in the developer's own corner, out of the way of everything a customer came for.

One step on your machine. The part of this that builds the email lives on Supabase, so it must be redeployed once, exactly as before:

supabase functions deploy shop-feedback project-ref ircwozniiyywsowamixy

Nothing else to set up - no new secret, no new database table, no new key. The email reuses the same Resend account your wish list already sends with, and the same published settings row the shop page reads for its "Website by" line.

30 Sep 2026 - engine v251, THE SHOP'S QUESTION IN YOUR OWN WORDS (no database step, no redeploy, one push).

The line at the foot of the shop now reads exactly as you wrote it. "Webmaster: Like the User Interface? Tell me & I will improve it!" - your words, your punctuation, with "Tell me" back where you put it.

It is a measurement, not a rewrite, and this version takes the measurement in both directions. A phone gives that box about 323px of room, and "the User Interface" spelled out costs about 150px of that on its own. Your sentence runs to about 377px, so on a phone it takes **two lines** - and the guard built for exactly this opens the box to show the whole question rather than cutting the end off it. Nothing is clipped on any phone: at 320px the box simply opens a little further, and the page never scrolls sideways.

The Malay is your own sentence and holds one line - as long as it keeps the shape you wrote it in. It carries no "Webmaster:" prefix for that reason: the prefix would add another 76px and push it onto a second line. The English and Chinese still say who is asking. The Chinese needed no change at all - it already used the everyday word for a screen's interface.

30 Sep 2026 - engine v250, THE SHOP'S QUESTION SPELLS OUT "USER INTERFACE" (no database step, no redeploy, one push).

Shorthand is out. You looked at the built page and said it plainly - "UI should be user interface, ui is not a

laymen term" - so the line at the foot of the shop read "Webmaster: Like the user interface? I'll improve it", with the Malay and Chinese equivalents.

And it stayed one line on a phone, which is what made this a measurement rather than a rewrite. A phone gives that box about 323px of room, and "user interface" spelled out takes about 150px of it on its own. The full sentence you first wrote runs to about 373px - wider than the box - so its tail would have been cut off. Everything you asked for was kept: who is asking, the question in full words, and the promise that it gets improved. Only "Let me know" went, because the question mark and a box waiting to be typed in already say it.

30 Sep 2026 - engine v249, THE BOX AT THE FOOT OF THE SHOP SAYS WHO IS ASKING (no database step, no redeploy, one push).

The one quiet line at the foot of the shop page now introduces itself. It reads "Webmaster: Like this UI? Tell me, I'll make it better" - the customer types over that sentence exactly as before, and the whole of it still goes out as their own words when they send, with nothing to press but Enter.

And if a phone is narrow enough that the question does wrap anyway, the box now opens itself to show the whole question instead of hiding the rest. That is the one thing this version can do that a shorter sentence alone could not: on any phone, in any of the three languages, the customer always sees the entire question rather than its first line.

30 Sep 2026 - engine v248, NOBODY HAS TO PRESS ANYTHING, AND NOBODY LOSES WHAT THEY WROTE (no database step, no redeploy - the same one function, one push).

Two more things you asked for on the little box at the foot of the shop.

Closing the page is now a send. A customer who writes a sentence and then closes the shop, or taps a link out of it, has had their say - the words go as they leave, without them pressing Enter at all. **Nothing goes out while they are still on the page**, so a sentence somebody is midway through writing is never mailed off behind their back. Pressing Enter is still a send, and still shows your reply straight away.

And nothing they wrote is lost on the way. If they step away from the box with words in it, or the page is closed before anything could be sent, the sentence is **kept on their own device** and is waiting in the box the next time they open the shop - it goes only once it has been sent. If a send does not go through, their words are put back, so a failure nobody was around to read is never a sentence lost. And **with no internet, nothing is sent at all**: the words simply wait on their device for the next visit instead of going down with the page.

And the Bahasa Malaysia has been rewritten to read the way Malaysians actually write. Several lines on the shop were correct textbook Malay but nobody writes that way on a Malaysian website - "pembangun" for your developer, "telah" where anyone here would say "sudah", "Menghantar..." where a shop would say "Sedang dihantar...", and a reply that read like a letter from an office rather than from you. The words are the same promise, in the voice a Malaysian customer expects.

The Chinese is now Malaysian Chinese, which is what your customers read. Malaysian Chinese read simplified characters exactly as mainland China does, so the characters stay as they are - what changes is the words: the items count used the mainland measure word rather than the one a Malaysian uses, the basket was called a shopping bag, and the two arrows on the delivery calendar were in **traditional** characters, which is simply the wrong script for your customers. Taiwanese Chinese was not used: it would mean changing every character to traditional, and your customers do not read that.

30 Sep 2026 - engine v247, THE SHOP FRONT NOW ASKS THE CUSTOMER WHAT THEY WOULD CHANGE (no database step, one new function to deploy once, one push).

You asked for a way for a customer to tell your developer what they would improve about the shop page, in their own words, with a reply straight away.

A single line now sits at the foot of the shop, under the "Website by" line. It is printed with your own question, and the customer types over that sentence and presses **Enter**. There is no Send button, because you asked for none: Enter is the send. The line is one line tall and opens further only when the words no longer fit, so at rest the whole thing is a single quiet row at the foot of the page. Once there are words in the box a small grey

line under it says **"Press Enter to send"**, so the one key that does something is never something they have to discover by accident.

The moment the send lands, the box is replaced by your reply - **"Your idea is well taken care of. New updates soon!"** - so they see the answer without doing anything else. They are never asked for an email address, and nothing is created on your side for them.

The words come to you as an ordinary email, at the developer address you already set in **Settings -> Website & developer** - the same address the shop's "Website by" line shows. It runs on the **same email service your wish list already uses**, so there is no new account, no new key and no new DNS record. Each message says which page it was written on, in which of the three languages, and when it arrived.

There is no box until you have set that address, because the box has nowhere to send to without it. That is on purpose: a box that quietly collects sentences nobody will ever read is worse than no box at all.

A send that did not go through says so, plainly. If the email could not be sent the customer is told, in words, to try again or to use the WhatsApp link that is already drawn just above the box - and **their words stay in the box**, exactly as typed, so nothing they wrote is lost. A thank-you drawn over a message that never left would stop them trying again and nobody would ever find out, so that is the one thing this never does.

It speaks all three languages. The question, the line that says how to send and the reply are written in English, Chinese and Bahasa Malaysia, and a customer who switches language keeps whatever they had half-typed and whatever reply they had already been given.

What you have to do, once. This needs one small new function deployed in Supabase the same way the wish-list function was - the file is supabase/functions/shop-feedback. Until it is deployed the box still appears and still says plainly that the send did not go through. Nothing else is required: no database step, no new key, no new setting - it uses the developer email you have already saved.

30 Sep 2026 - engine v246, THE NOTES ON THE ITEMS ARE NOW UNDERLINED ON THE COMPACT LABEL TOO (no database step, no redeploy, one push).

You spotted this one: on the Compact label a note like **no nuts** was printing as ordinary words, while the same note stood out on the Full and Mailing labels. It was the last place a customer's own words went unmarked, and it was easy to miss - Compact puts every item on ONE line, so the note sat in the middle of that line with nothing to pick it out.

Each noted item's words are underlined on that joined line now, on the item they belong to, so a note on the second of four items is underlined on the second item and not somewhere else on the line. **If two items carry the same note, each one gets its own underline** - the line is not allowed to mark one item twice and leave the other plain.

The line itself has not changed at all. It reads exactly as it did - the items in order, the notes in brackets beside the item they belong to - and a label for an order with no notes on it is byte-for-byte what it was. Nothing else about any label moved: the Full, Mailing and Name-only styles print exactly as they did, and the delivery note on the Compact label is untouched by this.

30 Sep 2026 - engine v245, THE DELIVERY NOTE IS UNDERLINED TOO, AND IT NOW PRINTS ON THE COMPACT LABEL (no database step, no redeploy, one push).

You made the point that the **delivery note** is not a courier thing - it applies to a self-collect order just as much, so it should stand out and be printed wherever it belongs.

It is underlined now wherever it is drawn. The order row carries it as Aunty Bee · 012-345 6789 · deliver after 3pm, with the note itself drawn under, exactly as a note on a single item already was. The same on the printed label: the Note: line keeps its label plain and underlines the customer's words.

The Compact label prints it too. Compact used to join every item onto one line and leave the delivery note off entirely - so a note about the doorstep, the gate code or a collect time disappeared the moment you picked the denser label, on a self-collect order as much as a courier one. It now prints in the same place the Full label puts it: after the items, before the posting address.

Everything else is unchanged. The Mailing and Full labels read exactly as they did, the note box is still offered on both self-collect and courier orders, and an order with no note on it looks and prints exactly as before. Name-only stays a bag tag - code and name, no fields to read.

30 Sep 2026 - engine v244, A NOTE A CUSTOMER LEAVES ON ONE ITEM IS NOW UNDERLINED, SO IT IS HARD TO MISS (no database step, no redeploy, one push).

You asked where the words a customer leaves actually reach you. The answer turned out to be uneven: the **delivery note** on the whole order pings your phone the moment the order lands, but the **note on a single item** - "no nuts", "write Happy Birthday" - only ever lived inside the app, sitting quietly in grey beside its item where it is easy to skim past. You decided that is enough for it to be **underlined**, so it reads as something you were meant to notice.

On the Orders list, an item that carries the customer's own words now has those words underlined - Chicken Jerky 100g ×2 (no nuts) with the brackets drawn under. It is the same line in the same place, just marked. Nothing moves, nothing is added, and an order with no notes on it looks exactly as it did.

On the printed label, the note beside its item is underlined the same way, on both the Full sheet and the Mailing sheet - so what you hold in your hand while you kit an order is as easy to read as the screen.

The line itself is unchanged. Only the way it is drawn changed, so the row, the label and the search still read the very same words - the underline cannot quietly disagree with what the line says.

30 Sep 2026 - engine v243, THE DELIVERY-DAY CALENDAR IN THE ORDER SCREENS NOW FOLLOWS TODAY, THE SAME AS YOUR SHOP (no database step, no redeploy, one push).

Your words: you wanted the calendar in the + New order card to behave like the one on your shop.

It does now, and it is the **same calendar in all three places it appears** - the + New order card, the strip at the top of Orders, and the day picker in the Edit-order pop-up. One behaviour in three places, so a day sits in the same square wherever you meet it.

What was wrong with it. It was a **month** - the whole of September, then the whole of October - exactly the calendar v231 replaced on your shop, for the same reason. On the 30th, most of what it drew was days already gone; a delivery day on the 29th could not be seen at all; and both arrows were drawn even at the ends, so a greyed button sat there inviting a press that would do nothing.

From this version it is five whole weeks anchored on today. The first row is the week just gone, the second row is this week with today ringed so you can see where "now" is, and the three rows under it are the weeks ahead, which is where your delivery days actually live. Today is always in the second row. **Every square is a real date now** - no blanks, no padding, so a delivery day at the start of next month is visible and openable from the moment you publish it.

Your delivery days are unchanged on it. Still circled, still carrying how booked each one is under the number, still FULL in red where a day is at capacity, and a day already gone still dimmed but **still open** - you backfill and review old days. A day you do not deliver still answers a tap, and still names where it gets added, so a holiday falling on a day you do not deliver is never the one square with nothing to say.

The arrows move one whole week, and are gone where there is nowhere to go. A dead control reads as a bug, so at the ends of the days you have set there is simply **no arrow** - nothing greyed out, nothing to press that would do nothing. The window still stops where your shop's stops for the same list, so the two calendars agree about where the far end is. A new week arrives **sliding up** going forward and **coming down** going back - the direction you are travelling in - and only when you pressed an arrow: an ordinary redraw never replays it.

The marks you made on Delivery Dates are on it too - the same single-day boxes and the same see-through bands, in their own colours and their own depths.

It opens on the day you are working on, not on today. The Edit-order pop-up opens on the day that order is actually on, so moving an order a season out does not leave you looking at a week with nothing to do with it. And picking a day that is already on screen **does not move the window**, so the grid never slides out from under your finger.

One thing I deliberately left alone. v231 ended with "nothing in the backoffice calendars changed; this is the shop's calendar only" - that was your instruction then, and this version is you asking for the opposite, for this one calendar. The **look** of the past days in the backoffice is untouched, and so are the other admin calendars: the free order-date field, Delivery Dates, Profit and Deliveries. What changed here is this one calendar's behaviour.

30 Sep 2026 - engine v242, A CUSTOMER WHO ALREADY HAS A COURIER BOOKING IS NEVER QUIETLY PUT ON A SECOND VAN (no database step, no redeploy, one push).

Your report: if a customer's order already has a courier booked, the delivery run should not tick that order, and it needs to say so, so that a delivery is not booked twice.

Here is what was happening. A booked trip is recorded on the order's own row, and the delivery run did not look at it when it built the day. Every customer on the day opened already ticked, the Tick them all press ticked them all, and a booked customer looked exactly like any other. The only thing that refused was the app's own sentence at the Book press - so you could tick a customer a driver was already on the way to, price the run, and press Book, and the refusal arrived afterwards as a message rather than as anything you could see while you were choosing.

From this version, a customer whose trip is already running opens OFF the run, with a line under their name saying so. The line names the courier and its own word for where the trip has got to, and it says plainly that ticking them and booking the run would send a second vehicle to the same door. The head of the list counts them too - it will read something like "Who is on the run - 3 of 5, 1 already booked", so the number you tick and the number of rows in front of you never disagree.

It is not a gate. The tick is still live and still yours. If you tick a booked customer by hand the app honours it, and the refusal comes at the price instead, in the same words as before. That is deliberate: the tick is a decision you make, and the app's job is to make sure you can see what you are deciding.

And the row offers the way out, rather than going grey. Under the warning there is one press: Call off the trip and add to this run. It asks you first, in the open, because calling a trip off is the half that cannot be undone - the driver stops being sent, and the customer's tracking box keeps its link but nothing will update it any more. If you say yes, the original booking is cancelled with the courier, the order's own row records that it was you who called it off and when, the warning disappears, and that customer is put on this run so their delivery can be consolidated with the rest of the day. If the courier refuses to call the trip off, its own reason is shown and nothing is written.

A trip that has already finished - delivered, or cancelled earlier - is not a barrier at all. Those customers open ticked like anyone else, with no warning, because a delivery that has to be done again still has to go on a run.

Orders going out by parcel carrier are untouched by this version. They were never on the run list in the first place, so there is no second van for them and nothing here to warn you about.

No database step, no redeploy, no new key, and no new setting to switch on.

30 Sep 2026 - engine v241, THE RESET NOW ANSWERS ON EVERY CARD, NOT ONLY A FRESH ONE (no database step, no redeploy, one push).

You pushed v240 and you were right again: the press worked, but the card could still stay silent. This version closes the last way it could.

Here is what was happening. The answer your press writes - the pin moved, or it found the same spot, or the address could not be reached - has two possible places to be said. One is the line under the buttons on the door card itself. The other is the line inside the delivery-price section. That section folds away, and its line folds away with it. v240 made the press work before the price section existed at all, which was the right fix - but the moment you opened **Get a delivery price** even once, the card handed every later answer to the folded-away section and stopped using the door card's own line.

So the sequence that caught you was this: open an order, press **Get a delivery price**, close it again, then reset the pin. The look-up really ran and the pin really moved, but the sentence saying so was written inside the

section you had folded away, and the door card said nothing at all. And where the look-up answers with the same spot as before, nothing moved on the map either - so the whole press looked exactly like a press that had never happened.

From this version the answer is always said on the line you can actually see. While the price section is open it is said there, exactly as it always was. While it is folded away - or if you have never opened it at all - it is said on the door card, right under the buttons. The two lines are never both used at once, so they cannot come to disagree about what your press did.

The same fault was on the pin you drag. Moving the pin by hand ends through the same sentence about the price section's numbers, and that sentence had the same problem: with the price section folded away, a drag you had just made reported into a section you could not see. It now says what it did on the door card in exactly the same way.

Nothing else changed. No database step, no redeploy, no new key, and no new setting to switch on.

30 Sep 2026 - engine v240, THE RESET BUTTON NOW ACTUALLY PRESSES (no database step, no redeploy, one push).

You pushed v239, saw the reset button at last, and pressed it - and nothing happened. That was exactly what your report said, and it was exactly right. The button was on the card, but its press was not connected to anything yet.

Here is what was happening. The reset press was wired up at the same moment as the delivery-price half of the card, and that half is only built the first time you press **Get a delivery price**. So on a card you had just opened, the button was drawn and waiting but its press led nowhere: no look-up, no message, no movement. The instant you happened to open the price fold once, the button started working - which is why it looked like an ordinary working button that simply ignored you.

From this version the reset press is connected the moment the card opens. It never waits on the delivery price again. You can open an order, see the pin, and reset it without touching the price button at all - which is the order you would naturally do it in anyway, because you reset a stale pin when you call the customer, not when you ask for a price.

And the press now always tells you what it did, on the card itself, right under the buttons. It says it is looking the address up, then it says one of the three things that can happen: the pin moved, or the look-up found the same spot and there was nothing to move, or the address could not be reached and the pin has been left exactly as it was. Those words used to appear only up in the price section, which is another reason a press on a fresh card looked like it had done nothing at all.

If a reset is about to replace a pin a person chose - the customer's own pin, or one you placed by hand - it still asks you first, exactly as v239 described.

Nothing else changed. No database step, no redeploy, no new key, and no new setting to switch on.

30 Sep 2026 - engine v239, THE RESET NOW SHOWS UP WHERE YOU ACTUALLY WORK (no database step, no redeploy, one push).

This is the fix for v238. You pushed it, opened an order, and the reset was still not there. You were right, and the reason is this: v238 offered the reset when the pin was the customer's own, and hid it when the pin was one **you** had placed by hand.

But placing the pin by hand is the only thing that card ever offered you. So a drag is what you did, and a drag is exactly what then hid the reset. Every customer whose pin you had ever corrected by hand still showed **Move this pin** and nothing else. v238 changed nothing at all for the doors you had touched yourself, which is why it looked like it had not arrived.

From this version the reset is offered wherever there is a pin to replace. If a pin exists on the order and an address is typed, the press is on the card - the customer's own pin, a pin you dragged by hand, a pin an older look-up wrote, or one a reset already wrote. The only case with no press is an order that has no pin yet, where there is simply nothing to replace; the price button looks one up on its own.

Where it would replace something a person chose, it still asks first. That is the customer's own pin, and it is also now your own hand-placed pin. The wording names which of the two it is about to replace. Over your own pin it says this is the door you placed on the map by hand, that a look-up may only find the road and can be a step back from a door you already had right, and that you can drag the pin again afterwards. Over the customer's pin it says, as v238 did, that this is their pin and you can switch back to it. Either way the confirming button says **Reset the pin**, and Cancel changes nothing at all.

Where it only replaces the app's own guess, it does not ask - a look-up's answer, or a reset of one. That is what this press has always done, so nothing new is put in your way there. The button says **Look this address up again** on those, and **Reset the pin from the address** on the two that will ask.

Nothing else changed. No database step, no redeploy, no new key, and no new setting to switch on.

30 Sep 2026 - engine v238, RESET THE PIN, AND LET THE MAP ZOOM (no database step, no redeploy, one push).

This one is yours, word for word: when you call a customer, their address may already have changed, and the only thing on offer was to move the pin. The map would not zoom out, and dragging a pin onto the right rooftop by thumb was slow and easy to get wrong. You asked why there was no way to reset the pin. There wasn't one. There is now, and the map zooms.

There is now a reset, on the pin card, where you can see it. When the pin the driver is sent to is the customer's own - the one they dropped on your shop page - the button under the map reads **Reset the pin from the address**. One press asks your address service for the address on the order again and puts the pin where that answer lands. That is the case you described: the pin was right when they dropped it, and they have moved since.

It asks before it replaces their pin. The customer's own pin is a fact they gave you, so a button that quietly overwrote it would be the pin moving on its own again - the fault several versions of this card have been spent ending. So the press asks first, in plain words: this is their pin, resetting replaces it with a fresh look-up of the address, and a look-up may only find the road. The confirming button says **Reset the pin**. Cancel changes nothing at all, not even a look-up.

And you can put their pin back. Once a reset has replaced their pin, the same card offers **Use the customer's pin instead**, one press, back to their exact point. The reset is not a one-way door.

A reset yields to a pin they drop afterwards. If that customer pins a new spot on your shop page after you reset, their new pin wins again straight away - the same rule as before. A reset replaces one specific stale pin, not every pin that customer will ever drop.

The map zooms the whole time now. The plus and minus buttons are drawn on the pin card, and pinch, double-tap and a desktop box zoom all work - before you press anything. Only moving the pin is still behind **Move this pin**: the map will not pan under a pin you are reading, and the pin will not take a drag, until you say so. That split is the point of this version. Looking and correcting used to be one action; they are two again.

Nothing else changed. Your prices, your delivery calendar, your messages, the order card, the packing slip and the label sheet are all exactly as they were. Adding an order is still the one pass v237 made it, and a pin you placed by your own hand is still not offered up for replacement.

On two phones. If your second phone has not been reloaded yet and is still running v237, it reads a reset as the older kind of pin and will price to the pin you replaced until it is reloaded. Reloading it fixes it, and there is nothing to run in Supabase for any of this.

30 Sep 2026 - engine v237, AN ORDER IN ONE PASS, AND THE DAY ON ONE LINE (no database step, no redeploy, one push).

This one is yours, and it came from a complaint about your own way of working. Adding an order used to be two steps: fill in the New order card, press Add order, then find the order again and open the Edit window to finish it. You said that was redundant, that the flow was not smooth, and that it started in the wrong place.

A posted order is now one pass. The reason it was ever two steps is that four things lived only in the Edit

window and not on the card: the courier charge, the tracking number, the parcel carrier, and the Get a delivery price block. They are all on the card now. Choose Post (nationwide) and they unfold underneath it, in the order you need them: the address with the map pin, the charge, the tracking number, the parcel, and the price. Take the order, price the trip and record what it cost, all before you press Add order once.

The price is on the card. The booking is not. This is the way you asked for it. You can ask for a price and take the fee onto the order from the card, but Book this trip stays on the order itself, where it has always been. Booking a real vehicle against an order you have not finished taking would be the wrong thing to put under your thumb one press away.

The day is one line now. The card used to open on a whole month of calendar, even though you had already picked the day on the screen behind it. It now reads as one line - Delivering Fri, 2 Oct - with the calendar unfolding underneath only when you tap it. Nothing about how a day is chosen has changed: tapping one still switches the screen, so the products and the day's limits are right.

The items come first. After the day comes what the customer wants, then the customer. It is the order things are actually said to you on the phone: they tell you what they want, then who they are. Everything else on the card is unchanged and in the place it has always been.

Your own note box is now the delivery note. Your app and your shop both used to call the order's note box simply Note. Now that every item carries its own note, that box was left doing a different job, so it says what it does: it is the delivery note, for the gate code, the landmark and the time you should arrive. The box on your shop and the box in your app now show a grey hint saying exactly that, so a customer knows what belongs in it without you having to tell them.

Nothing is written until you press Add order. The card was, and still is, a draft. The charge, the tracking number and the parcel are written onto the order at the same moment the order itself is created, so an order you decide against leaves nothing behind it. And if you type a charge but do not say who paid it, the card refuses in words and changes nothing, exactly as the Edit window does.

Nothing else changed. The order's own field order in the Edit window, your prices, the delivery calendar, your WhatsApp messages, the packing slip and the label sheet are all exactly as they were.

30 Sep 2026 - engine v236, A NOTE ON EACH ITEM, SWITCHED ON PER PRODUCT (no database step, no redeploy, one push).

This one is yours, word for word: you wanted to switch a note box on or off on a product card, so a customer can leave a note on each thing they order - and switch it off on the products where a note makes no sense. And you asked for the same box in your own app, so a note you are told over the phone can be written down in the same place.

The switch lives on the product card. In your app, open a product and you will find a tick that reads **Ask the customer for a note on this item**, sitting beside the other selling switches. Tick it and the customer is offered a note on that item; leave it alone and they are not. It is off until you tick it, so nothing on your shop changes until you decide it should.

On the shop, the note is quiet. A customer who puts an item in the basket sees a small **Add a note** link under it. Nothing is opened on their screen until they tap it, so your menu still looks exactly as it looks today. Tap it and a box opens on that item, with a short example in grey: no nuts, or a name to write on the pack. Whatever they type stays with that one item and rides on that item's line when the order comes in.

The note belongs to the item, not to the order. This is the whole point of it. If someone orders two things and writes "no nuts" on one of them, that note comes through on that item's line only. The other line is untouched, and the order's own note box at the bottom still works exactly as it did for anything that is about the whole order.

A note in your own app is offered on every line. Your New order card, and the Edit order window, both give every line its own small note box - whichever product it is, and whether or not you have switched the customer's box on. The switch decides what you ASK a customer for; it must never decide what you are allowed to write down. A phone order of "no nuts on the chicken jerky" has to have somewhere to go.

You see the note where you work. On the orders list it appears in brackets beside the item it belongs to, so it sits next to the thing it is about rather than in a note line at the foot of the order. The same is true of the packing slip and the label sheet: it prints beside that item and never as the order's own note.

Nothing about a note blocks an order. An empty box and no box at all place exactly the same order - there is no such thing as a half-filled note to chase. Switching a product's note box off after the fact never hides or loses a note you have already collected, and clearing a note you wrote puts that line back exactly as it was.

Your WhatsApp messages do not change. The note stays in your app, which is what you asked for. Every message you send reads exactly as it reads today, and your customers' tracking page is untouched by this version.

Nothing to set up. There is no database step and nothing to redeploy. The switch travels to your shop with the rest of the product details, and a note travels with the order from one of your phones to the other on its own, the same way the rest of the order already does.

Nothing else changed. The order's own note box, the prices, the delivery calendar, the basket and the place-order bar are all exactly as they were.

29 Sep 2026 - engine v235, A BOOKED TRIP NOW SHOWS WHAT IT COST AGAINST WHAT YOU CHARGED (no database step, no redeploy, one push).

This one is yours, and it started as a worry rather than a request. A courier price you pick when you take an order can turn out to be higher or lower by the time the trip is actually booked, and nothing was watching that gap. You asked to see it rather than be protected from it: a real cost makes you aware, leaves room for a promotion later, and lets you decide not to collect the delivery at all.

So the booked-trip card now does the sum. Under the line that tells you when calling the trip off stops being free, the card compares what the trip actually cost against the courier charge on the order, and says which way the difference fell. If the trip cost more than you charged, it says so and tells you that much came out of your own pocket. If it cost less, it says that too, and that the difference stayed with you. Neither direction is treated as the bad one.

It shows both directions on purpose. A gap that only spoke up when you were short would be an alarm; you asked for a reading, so a surplus and a shortfall are worded the same way, in the same plain sentences.

A trip you decided not to charge for says so. If you have recorded no courier charge at all, the card tells you the whole cost of the trip is your own, rather than staying silent. That is the free-delivery case you named, and the one number worth seeing is the one you chose to give away.

A charge that matches the trip says nothing at all. When the two agree there is no line, because a difference of nothing is not a difference, and an "RM 0.00" row on every card would be noise you would learn to skip.

A delivered trip keeps the reading. The calling-off deadline disappears when a trip is over, because there is nothing left to call off. This line stays, because a cost is money that has already been spent and the whole point is to learn from it afterwards.

This is on your screen only. Nothing here goes near a customer. The price a customer was quoted does not move, their messages do not change, and their tracking page is untouched by this version.

Nothing else changed. The van you pick, the prices, the five-minute life of a price, the booking buttons, the charge box and the calling-off deadline are all exactly as they were.

29 Sep 2026 - engine v234, A BOOKED TRIP NOW TELLS YOU WHEN CALLING IT OFF STOPS BEING FREE (no database step, no redeploy, one push).

This one started as your own question: if you book a trip and then need to call it off, when does it start costing you money? The answer is a rule rather than a feeling. Lalamove lets you call off a scheduled pickup at no charge up to 45 minutes before it, and it may charge a fee after that. That rule was living in your head, so this version puts it on the page. Under the status line on a booked trip, the card now says **Free to call off until 30 Sep, 10:15 am - Lalamove may charge a fee after that.**

It turns over by itself when the window shuts. Leave the card open and the sentence changes on its own to **Lalamove's free calling-off window shut at 10:15 am, so a fee may apply from here.** You never have to work out which side of the deadline you are on, because the card says it for you.

A trip booked for as soon as possible is told apart honestly. On an immediate booking the free window runs from the moment a driver takes the job, and Lalamove never sends that moment back to the app. So rather than inventing a time that could be wrong, the card states the rule instead: **Booked for collection as soon as possible, so there is no pickup time to count back from. An immediate trip is free to call off only for a short while after a driver takes it - check with Lalamove before you count on it.**

Nothing is claimed that the app cannot stand behind. Every sentence says the fee may apply rather than that it will, because it is Lalamove's own terms that decide it and not this app. A trip that has finished, or one you have already called off, shows no deadline line at all, since there is nothing left to call off.

Nothing else about booking changed. The vans, the prices, the five-minute life of a price, the customer's tracking box, the charge box and the buttons on this card are all exactly as they were. This is one line added to a card you already had.

29 Sep 2026 - engine v233. The order page has a way back to your homepage. No database step - nothing to run.

One small thing, and it sits on the shop page rather than in the app.

Customers reach your order page from plenty of places that are not your homepage - an Instagram bio, a link someone shared in a group, a bookmark saved on their phone. Until now that page had no route back to your front page at all, so a visitor who landed on it first could never get to the reviews, the gallery or your story: there was simply nothing to tap. From this version there is a small "Our homepage" link in the orange banner, on its own line just under your tagline and marked with a little house. It points at your homepage, and it reads in all three shop languages.

It sits on its own line rather than on the top row beside the EN, Mandarin and BM buttons on purpose. That row already fills a phone - the line beside it wraps to two lines at 375px - and a third item would have pushed the language buttons out of easy reach. The link is a full-size tap target in the same pill shape as those buttons, so it looks like it belongs there and is easy to hit. Nothing else on the order page moved, and nothing about taking an order changed.

29 Sep 2026 - engine v232. A translated line you delete stays deleted, and the greyed hint is readable. No database step - nothing to run.

One version, and both halves are the same card: the Chinese / Bahasa Malaysia card on a product's Edit screen.

A line you empty on purpose goes quiet. If you deleted the wording from a translated line because you wanted the English shown for that line, the machine translation used to come straight back into the box, greyed out - and the small right-pointing arrow that is meant to sit over those words had gone. The greyed words and the arrow are one single offer: the arrow is drawn on the words, and the words exist only because the arrow can take them. They were written in two separate places, so they could disagree, and the box ended up showing machine words that nothing would take. They are now written together in one place, so a line can never show words with no arrow over them. An emptied line stays empty, says "Left blank - English shows." so blank is clearly meant, and keeps its round arrow as the way back to a translation if you change your mind.

The greyed hint no longer runs off the edge. On a line you have not touched, the greyed hint read "e.g. <the translation>.....if blank, it will be filled with English". On a one-line box that tail ran past the right edge and was cut off mid-word, right where the arrow sits - so the one sentence meant to explain the line was the one you could not read. It is now just "e.g. <the translation>", the same shape as every other suggested box in the app, and the promise that blank means English is still made in full in the card's own line above.

Nothing you have already typed is touched. A line you typed is yours and is never overwritten, and an untouched empty line behaves exactly as it did before: greyed suggestion, with the arrow.

28 Sep 2026 - engine v229, v230 and v231. The delivery address box has room, and the shop's posting-day picker is five whole weeks. No database step - nothing to run.

Three versions in one go, and two of them are the same problem: an address does not fit on one line, and the box you type it into gave it no room.

v229 - the delivery address box has room for a whole address. It was a single-line box, so a real address - three lines, four, sometimes five - was cut off and you had to scroll it sideways to read back what you had just typed. It is now a proper multi-line box: four lines tall on the New order card and on an order's Edit pop-up, three on the pin card under "Put this doorstep on the map". You can also drag its bottom edge to make it taller while you type. Nothing about what the box stores, saves or pins changed - it is the same field, only taller.

v230 - and it takes the whole width of the form. An order form is laid out in two columns, and the address sat in one of them with an empty cell beside it. The address is the longest single thing you type on an order, so it now stretches across both columns. Nothing else on the form moved.

v231 - the shop's posting-day picker is weeks, not a month. Your customers' calendar used to be a month page, with the leading and trailing squares of the month padded out with dead numbers. It is now five whole weeks that follow today: the week just gone at the top, faded because there is nothing left to book in it, then this week with today ringed, then three weeks ahead - and every square on it is a real date, so nothing is greyed out for no reason. Small arrows page it a week at a time and the grid slides as it does. An arrow is drawn only when there is somewhere to go: at the start there is usually nothing earlier, and a later arrow appears once you have published a posting day further out. A control with nothing to do is left off rather than shown greyed out. A day you have not published is still a real square, and tapping it still tells the customer why it cannot be booked, in the same two sentences as before.

28 Sep 2026 - engine v226, v227 and v228. Parcels you post yourself, an address that fills itself in, and address suggestions as you type. No database step - nothing to run.

Three versions in one go, and they all answer the same kind of question: how a posted order finds its way to a door.

v226 - the parcels you post yourself have somewhere to live. Your flat postage and the courier you could book were already here; the third case, the box you take to a counter or book on a carrier's own website, was not. There is a new screen, **More then Parcel couriers**, that keeps your list of carriers. J&T Express, Ninja Van, Line Clear, Pos Laju and SPX Express are one press to add, and you can add or rename your own. On an order you pick the carrier, the app remembers it, and the consignment number goes into the **tracking box the order already has** - so it reaches the customer's page and the posted message through machinery that was there already, with no new slot and nothing to run. A product can also be ticked **"Can travel as a parcel"**, and that tick is deliberately not a switch the app obeys: it only changes the advice you are shown when you record a parcel on a courier order, and it never blocks, hides or delays a sale. Two things this version also repaired, both had been quietly broken for a long time: the app's own dropdown control had a change handler that had been dead for around a hundred versions (nothing depended on it until the carrier picker did), and the **Note / tracking card** on an order rebuilt itself from scratch every time it repainted, so it threw away the carrier you had just named and the number you had just typed.

v227 - the delivery address remembers your regulars. Type a customer's name into an order and, if you have posted to them before, their delivery address fills itself in from the order you sent them last. It helps the **returning** customer only, because it reads your own past orders - and it never overwrites an address you typed yourself.

v228 - the address suggests real ones while you type. For the customer you have never posted to, who has no history for v227 to read. As you type a delivery address, the app offers real addresses that match and a tap writes the whole thing in. It is on both order forms, the New order card and an order's Edit pop-up. It is deliberately quiet: it waits for you to stop typing before it asks anything, it ignores a fragment, and an answer that arrives after you have typed on is thrown away rather than painted over your words. This half needs one thing switched on in your Google project - the same key the map lookup already uses, with the Places service ticked

on - and until that is done the address box behaves exactly as it did before.

One day, three versions. The bakery and this app had differed for eight days over the bakery's bread-only Production line. That was settled earlier on 28 September, and v224 and v225 came the same day; v226 to v228 followed on that same day too, so the two apps read the same number again and will after every future sync.

28 Sep 2026 - engine v225. A product on the shop can always be hidden. No database step - nothing to run.

One button, in one place, and it fixes a rule that bit at the worst possible moment.

What was wrong. You published a product, looked at its row, and the only button there was **Delete**. Its **Hide** button was missing exactly when you were most likely to want it. The rule behind that was Hide-if-it-has-history, Delete-if-it-is-clean, and a product you have just published has no history yet, so the app counted it as clean and offered you the one button that throws work away. Delete takes the recipe with it. So the fresh product, the one you might simply want off the shop for a while, was the one that could only be destroyed. Your report: after publishing a product, only Delete is allowed, and it should not be.

Fixed. A product that is on the shop now always offers both buttons, side by side. **Hide** takes it off the shop and keeps everything - the recipe, the photo, the price, the category and its past orders - and **Unhide** on the Hidden list puts it back whenever you like. **Delete** is still right there beside it for the product you really do want gone for good, and it still asks you to confirm first. The older products that already carry orders are the exception, exactly as before: they keep Hide alone, because deleting one would break the orders and the sets that point at it. That is not a missing button, it is the app refusing to let you break your own books.

Nothing else about the Products screen changed - the same three lists, the same folded New product card, the same rows. This is one button in one place.

28 Sep 2026 (no new engine) - the address lookup in your back office now works. Nothing to run, and nothing to update on your phones.

Your back office had two address boxes that did nothing: you typed an address, pressed the button beside it, and neither a map pin nor a list of suggestions ever came back. One cause, two symptoms - those boxes ask a small helper that runs inside your own Supabase project, and that helper had never been put there, so the request went nowhere. It is there now, deployed by you on 28 September 2026.

The same helper also holds the price-and-book half for a courier, and that half stays exactly where it was: it needs a courier key you have not set up and do not need. So a courier price screen will still say a key is missing - that is the honest answer rather than a fault, and nothing about your orders, money, labels or postage is touched by it.

Two things worth knowing before you try it. First, the lookup sends your phone's sign-in along with it, so the phone has to be signed in to your shared data, or you will be told that instead of being given an answer. Second, the list of suggestions only appears when there is a real choice to make - two or more matches. Now that your lookups come back with a house number, most addresses will return one exact answer and therefore show no list at all, with the pin simply landing where it belongs. That is the design, not the old fault returning.

28 Sep 2026 - engine v224. The picture window is square, and a product's row shows only what a customer sees. No database step - nothing to run.

Two small changes, both about a product, and one tidy-up besides.

The picture window is square, and your photo is cropped to fill it. Until now the window kept whatever shape your photo happened to be: a tall photo showed whole with the window's cream colour at either side, a wide one filled across with cream above and below. You asked for the crop by name, so a product photo is now trimmed to a square as it is chosen and fills the window edge to edge with no cream at all. The square taken is the middle of your photo. The trade is honest: whatever the square leaves out is gone from that picture, so the only way to change it is to choose the photo again. A photo already saved keeps the shape it was stored in and is trimmed to the square's middle when it is shown, rather than being reshaped on disk. This reverses v220 to v223, where nothing was cropped.

A product's row on the Products screen now shows only what a customer sees. The row used to print the ingredient cost per unit ("RM 0.30 / unit") and the recipe's own ingredient lines under the sell price. Both are gone from the row, on your word - the row is your shop view, and what a packet costs you to make is not something a customer ever sees. Nothing was lost: the price per unit and the full recipe are still on the product's **Edit** screen, which is where you build them.

And one tidy-up. A product with no photo yet showed a small engraved bread loaf in the empty picture box - a leftover from the bakery system the app was built from. It is a paw print now. Nothing else changed about it: the box is the same size and a product with a photo never saw it.

28 Sep 2026 - engine v223. Your app has caught up with the bakery in one jump, and it now has the Production line.

This is the biggest update this app has ever had: ninety engine versions at once, from v133 to v223. Everything the bakery built since we last synced is now here, plus one new switch of your own that has nothing to do with the bakery at all.

Why one big jump rather than many small ones. Improvements are always built on your bakery app first, then copied here. The copying had gone quiet while you got Munchies Furkidz off the ground, so the gap had grown wide. Doing it in one go means one copy job, one rebuild of every document, and one engine number on both phones at the end of it - rather than a dozen tiny pushes you would have to do yourself.

Nothing you already had is lost. Every figure, every order, every product name and price is exactly as it was. This adds screens and fixes; it does not change your records.

THE PRODUCTION LINE AND THE SCENARIO PLANNER (engine v133 to v187).

Two new screens, both under More. More -> Production line is a planner for a production day: how many units you can make, which step is holding you up, and where the day's hours actually go. **More -> Scenario planner** is the wider screen next to it, where you build your day out of blocks - a block is a machine and the pair of hands that tends it, counted as one thing - and watch the day as a timing diagram. A block has its own cycle time, how many units one pass deals with, how many minutes of you it takes, and how often it repeats. You raise the numbers until something stops you, which is how you find your real ceiling.

These screens are the bakery's, with the bakery's own words. You asked for this one to be copied exactly as it is rather than reworded, and that is what has been done: the screens talk about ovens, pans, dough, mixes, trays and bake days. That is deliberate and not a mistake. The reason it is worth having anyway is that the arithmetic underneath is about people, machines and hours, and that part is not about bread at all - it works out how many of anything a day can make, given what you have and how long each step takes.

A block can be told to repeat, which is how a loop works. Nothing special is needed for a step you do several times: it is simply a block that repeats more often. A block you have two of is drawn twice. A block can be switched off and sits in the list reading "not in this scenario", so the screen answers without it - that is how you try a day with and without something you have not bought yet.

The day is drawn as a timeline you can read. The hour ruler has its lines drawn, the clock above your people sweeps as the day runs, every batch bar carries its own label, and the step that sets your pace wears a mark and one sentence saying why it is the slow one. A person's row is only as tall as the people in it, so three workers do not leave a tall empty box on screen.

Said plainly: at your scale this is a toy until you put your own numbers in. Every box it seeds is the bakery's, and a seeded number is a guess, not a measurement. The screens say "not timed yet" rather than quietly reading as free, so you can see which figures you have not given it. If a mix box reads 25 and you know better, type 28 - the figure the app keeps is the one you last typed, so a new seed only reaches a fresh phone.

Windows that open over the app are taller, and cards fold. Every pop-up in the app now sizes itself to what it holds and scrolls inside itself rather than pushing past the bottom of your phone. A long card on the planner folds away when you tap its title. Neither of these is a new feature so much as the reason the new screens are usable on a phone at all.

A NEW PHONE NO LONGER EMPTIES THE OTHER ONE (engine v181). Worth its own line, because it is about your records rather than a screen. A phone that had never seen one of your settings could previously arrive and hand its own empty version back to the cloud. Staying quiet about a thing is now correctly read as "no opinion", never as "delete this". This is the same rule that already protected your delivery dates, and it now covers much more.

COURIER WORK: PRICE IT, BOOK IT, FOLLOW IT (engine v188 to v193).

This is the part to set up later, not today. It needs a Lalamove key on your Supabase project before any of it does anything, and nothing about your app breaks or nags without it. You told me Lalamove is not suitable for Munchies as your main arrangement, and this is built to sit quietly as a secondary choice until you decide otherwise.

What it does, when you want it. On an order you are delivering, one press asks Lalamove what the trip would cost on every vehicle it runs and shows the prices side by side with the distance - the quotation lives five minutes, and the one you choose fills the courier charge box that was already there. A second press books it for real, and the share link the courier hands back goes into the tracking box your orders already carry. A third checks where the driver has got to, or calls the trip off.

One trip, several doorsteps, and what it saves. A day's courier orders can be grouped into a single trip. The app prices that trip against the same doorsteps sent one at a time and shows you the difference as two numbers rather than as a claim. The saving stays with you: a customer who bears a courier charge is charged what their own doorstep costs on its own, never a share of the combined fee, so consolidating does not quietly become their discount.

The customer can watch it. A booked trip's own progress shows on their track card, in their own language, with the driver's name, the number plate and a press to ring them. When the courier says the parcel is on the vehicle, the order moves itself on. One honest limit, found and left in place rather than papered over: the courier hands back ONE link for the whole trip, so a customer opening it can see the other stops. There is no way to split it, so the app now warns you instead of pretending otherwise.

THE SHOP'S MAP, THE PIN, AND THE ADDRESS (engine v194 to v218).

Your customer can now put their door on a map, and the app uses it. On the order page they type their address, the app suggests doors underneath as they type, and they pick theirs. That address is written straight into the delivery address box rather than making them type it twice. If nothing comes up they can drop a pin by hand instead.

The pin is a suggestion and never a fact. Nothing is priced, booked or sent to a driver from a customer's pin on its own; it is offered to you where you already look at a doorstep - on the order's charge section and on the delivery run under that customer's row - and you are the one who acts on it.

A pin and an address can no longer disagree. Before this they arrived as two bare numbers with nothing tying them to the address written beside them, so the two could say different places and your screen had no way to show you. Now the pin carries the customer's own typed address as its name, and the app will not let the two drift apart.

The order's Note / tracking box now opens with the door on it. For a courier order the top of that card tells you which door the driver would be sent to - the customer, the address, and a small map with the pin on it - and you can look at it without pressing anything, or move it if it is wrong.

Nobody's door is another customer's business. Said in the courier section above, and repeated here because it is a privacy matter rather than a convenience: the courier's own link covers the whole trip, so a customer following it can see the other stops. The app tells you this rather than leaving you to find out.

An address lookup that used to give up now says why. A Malaysian address leads with its house number, and the free map services the app asks often can only find the road. The screen now says so in words - "this is the road, not the house" - rather than leaving you with a pin in the wrong place and no explanation. An address with a unit number, which the lookup used to refuse outright, is now found.

PRODUCT CATEGORIES AND PICTURES (engine v219 to v223).

More -> Categories is a new screen, and it starts empty on purpose. This is where your shop's headings live, built the same way your ingredients are: press New category, give it a name, and it appears. Nothing is in it until you put it there, because your shop should say what you sell, not what a program guessed for you.

Your shop now lists by your headings instead of in whatever order the products happened to be stored in. That is the whole point. Under each heading sit the products you filed there, and the order they appear in is the order you set - not alphabetical, not by price. A heading can hold headings under it, as deep as you like, so "For Dog" and "For Cat" can sit at the top with Pork, Duck and Fruits under them.

A product with no heading yet is never hidden - it is shown last, under a plain heading called **More items**. Add something this evening and file it tomorrow: it is on sale tonight, it simply sits at the end until you say where it belongs.

A product can be in more than one heading, and you choose which one it is listed under by the order you tick. The box you tick first is the heading it is drawn under. Tick a second and the product does not vanish from the first, and it is not drawn twice either - one product appears in exactly one place on your shop. The other ticks are kept as a note to yourself, and the app's own product row spells out which ones they are.

Every product can now have a picture, and it is one standard window. Choose a photo from your phone in the product editor; your shop shows it at the left of the product's row and your own Products list shows it too, so you can see at a glance which products have one and which are still blank. Since v223 the window is the same size in all three places - your shop's card, the app's list, and the editor where you choose the photo - and it is bigger than it was.

Your photo went in whole (as of v223 - the v224 entry above reverses this). A tall photo and a wide photo both showed in full, with the window's soft cream colour filling the spare space beside them. From v224 the photo is cropped to a square instead. **A product with no picture is untouched** - no window, and its name and price read across the whole card exactly as before.

Pictures already saved keep the shape they were stored in. Re-choosing the photo is the step that stores a fresh one for the window as it now is.

Two things to know, said plainly rather than found out later. The homepage grid is untouched and still carries the photographs already written into it, so a picture set here reaches your shop and your own product list but not the front page. And a picture costs weight twice: it travels inside every backup and every export, and it is fetched again by every customer who opens your shop. That is why it is kept small.

Moving a heading, or a product inside one, is a drag. Each row carries a small handle at its left; press the handle and drag the row up or down. It is the first drag in the app, so it is the part most worth a second look on your own phone.

Deleting a heading is guarded, the way your ingredients and suppliers are. A heading with headings under it will not delete. A heading with products filed in it will not delete, and it tells you how many. An empty one simply goes.

THE POSTAGE SWITCH - flat fee, or quoted by courier (this app only; no engine bump).

Your ask: "add a switch whether a flat postage or quote by courier". On **Settings -> Storefront**, the Postage card now carries a switch: **Quote each posted order by courier instead of a flat fee**. Off is exactly what this app has always done - the flat fee you set (RM8 unless you change it) is added to the To-pay line on every posted order. On, the flat fee is taken away and nothing stands in its place: you find out what the courier charged and record it on the order (More -> the order's Note / tracking), and that figure is what the customer is told.

What the customer hears while the figure is unknown. They are told plainly rather than quoted a number that is not the real one: **"Postage: quoted separately - we'll message you the exact amount"**, with no To-pay line at all, because there is nothing extra to pay yet. The very same sentence is on their track card, word for word, so the two can never say different things. It goes away by itself the moment you record what the courier asked for,

and the ordinary courier charge line takes over.

The flat fee is kept, not thrown away. Switch back to a flat fee and your RM8 is still there and quotes exactly as before. A collect order, and an order whose charge you absorbed, say nothing about postage in either mode - they owe nothing for delivery, and they say so by saying nothing.

This choice syncs between your phones, like the fee itself and for the same reason: whichever phone you set it on last wins, so both quote the same. It is never shown to customers.

WHAT IS DELIBERATELY NOT HERE.

The bakery's v199 money layout is left out on purpose, and it is worth knowing why. From v199 the bakery's WhatsApp messages and track page set the money out as a small addition - "items total this, plus the courier's charge, reaching this total". That works on the bakery because the courier's charge is the only delivery cost it has. Here it is not: your flat RM8 postage is also inside the total and is deliberately never published to the shop page. A subtotal worked out from the published total would therefore be RM8 too high, which is worse than not showing the working at all. So this app keeps its own money lines - **Total**, then **Postage (nationwide)** or the recorded **Courier charge**, then **To pay** - which are already an addition the customer can follow, in the place where your postage actually lives.

WHAT YOU MUST DO BEFORE YOU PUSH THIS.

Two new database scripts, and one rule that matters. Run **supabase/courier_job.sql** and **supabase/postage_mode.sql** in your Supabase SQL editor before this build is live. Both are safe to run twice. The reason for the order is not tidiness: the app publishes the whole tracking row for an order in a single call, so a column that does not exist yet rejects that call as a whole - every customer's track page would stop updating, not only the charged ones. If you have not already run courier_fee.sql and courier_cod.sql, those are needed as well.

Two optional Edge Functions, for later. courier and shop-geocode sit in supabase/functions/. They are what talks to Lalamove and to the address lookup. Deploy them when you set the courier up, not before; until then the courier screens say plainly that no key has been added rather than blaming the build.

Nothing else to run. No new table beyond those scripts, no change to how you push, and **Engine v223** on your **More** screen once this is live.

27 Sep 2026 (no new engine) - Pear Chicken Roll is RM24. One card changed price.

Pear Chicken Roll was **RM25**; it is **RM24** now. The card's size line already said 50g pack, so only the figure moved.

No new engine - nothing changed on your phone. No SQL to run.

26 Sep 2026 (no new engine) - the homepage grid carries your new prices. Seven cards changed price, the three jerky now name both pack sizes, and one card's price came off.

What each card says now. Chicken Potato Puff, Carrot Chicken Biscuit, Pear Chicken Roll and Chinese Yam Chicken RM24 -> **RM25**. Apple Chicken Roll **RM24** and Okra Chicken Roll **RM24** are the prices you already had, so they are unchanged. (Pear Chicken Roll went back to RM24 the next day - see the 27 Sep entry at the top.)

The three jerky show two prices on one line. They sell in a 50g pack and a 100g pack, so the price line now reads **50g RM16 · 100g RM29** (Duck **50g RM18 · 100g RM33**, Pork **50g RM17 · 100g RM31**). Before this they carried a single figure. The two-price line is set a little smaller than a one-price line on purpose: at the full size it wrapped onto a second row on every screen and pushed the description down on those three cards only.

Egg Yolk Melts has no price on it now. You asked for the price to come off, and it is off - the card shows the name, its OUT OF STOCK pill, the weight and the description. Putting a figure back is one line whenever it returns.

Nothing here needs translating. A price is a plain number written into the page, not a dictionary entry, so English, Chinese and Malay all show the same figure and a language switch cannot show an old price beside a new one. This is the homepage's own copy: the live menu in the backoffice is a separate list, and its prices are

still yours to set there.

No new engine - nothing changed on your phone. No SQL to run.

23 Sep 2026 (no new engine) - the website says "pack", and the three jerky come in two sizes. Three wording changes on the homepage grid.

"Pack", not "pouch". Every card's size line used to end in "pouch"; the website now says **pack** throughout, and the note under the grid reads "Every pack is clearly labelled with its meat". This is the homepage's own wording - the shop menu in the backoffice is separate copy and was left alone, so the unit box on a product still holds whatever you typed there.

Chicken, Duck and Pork Jerky also come in a 50g pack. Their size line now reads **50/100g pack**, showing both sizes in one label. The other seven are unchanged - 80g for the puff, the biscuit and the Chinese yam chicken; 50g for the three rolls; 40g for the melts.

One small correction to the wording you sent. The Carrot Chicken Biscuit's description said "make into little dinosaur biscuits" and now reads **"made into"**.

The page is read in three languages, so Chinese and Malay were updated to match. The Chinese size lines now use the Chinese word for a pack rather than the word for a bag, and the Chinese note under the grid changed with them; Malay already said Pek for a pack, so only the weight changed there. A language switch can never show an old word beside a new one.

The card showed one price - the 100g price - at the time. A second size did not get a second price on the card until you asked; you asked on 26 Sep and both prices are now on the line (see the top entry).

No new engine - nothing changed on your phone. No SQL to run.

23 Sep 2026 (no new engine) - the homepage grid carries your new prices and the full treat descriptions. Every card in the homepage's "Our Treats" grid now reads the price and the wording you sent, for all ten treats (the weights at the time were the ones you gave - 80g puff, biscuit and Chinese yam, 50g rolls, 40g melts; the jerky gained a second size the same day, see above).

Prices. Four went up: Chicken Potato Puff RM22 -> RM24, Carrot Chicken Biscuit RM23 -> RM24, and all three rolls (Apple, Pear, Okra) RM20 -> RM24. Chicken Jerky RM24, Duck Jerky RM26, Pork Jerky RM28, Chinese Yam Chicken RM24 and Egg Yolk Melts RM22 are the prices you already had.

Wording. Each card carries your **full** description word for word - what it is made of, that nothing else goes in, and how it behaves in the hand - replacing the older short lines. The page is read in three languages, so the Chinese and Malay versions were updated to match: switching language can never show the old words beside a new price.

Egg Yolk Melts is marked out of stock. Its card keeps its price and wears a small "OUT OF STOCK" pill beside the name - the same pill style the PORK note already uses - in whichever language the reader has chosen.

The short one-line descriptions were not used. Your instruction was the full description on the homepage, so the short lines are kept aside rather than discarded - they are ready if you would like them somewhere smaller later (the shop menu, or a printed label). The treat photos are untouched.

No new engine - nothing changed on your phone. No SQL to run.

21 Sep 2026 (no new engine) - tapping a day you do not post is answered. On the order page, tapping a day on the calendar that is not one of your posting days used to do nothing at all: no highlight, no message, nothing moved. A customer tapping the day they wanted read that as a broken page rather than as an answer. The grid now answers - one line under the calendar, naming the day in words, in whichever of the three languages the customer is reading.

It says one of two things, because there are two different facts. A day you do not post gets *"Tue, 22 Sep is not a posting day - please pick a green day."* A day you **do** post, tapped after its 6pm window has shut, gets *"Orders for Mon, 21 Sep have closed - please pick a green day."* That second sentence matters more than it

looks: your orders close at 6pm **the day before**, so from 6pm on Tuesday that Wednesday is gone, and today's own day is always gone. Both of those are days you really do post, and the card right above the calendar names them as posting days - telling a customer "that is not a posting day" on your own posting day would have had the page contradict itself every single evening. So a day that has shut says **closed** instead.

Nothing else answers. Choosing a green day takes the sentence away with it, so it can never outlive the question. A day already past stays quiet - it *was* a posting day, so the sentence would be a lie on it, and it is dimmed besides. And a day you post whose window is still open, but which the shop has not put on offer yet, stays quiet too: neither sentence would be true of it. The sentence also disappears by itself if you later make that day one you post.

No new engine - nothing changed on your phone. No SQL to run.

21 Sep 2026 (no new engine) - the cut-off time is written out in words. The order page's info card used to read **"Order by 18:00 the day before"**. It now reads **"Order by 6pm the day before posting"**: the time the way you would say it out loud, and the word *posting* spelled out. Settings is untouched - the cut-off box still takes a time (18:00), because that is what a clock reading is for; only the customer's page changed. The other two places the page names the deadline changed with it, so the whole page agrees: the line under your business name ("Made to order · closes 6pm the day before") and the note a customer sees if they left the page open across the deadline ("Orders for this day close at 6pm the day before - please pick a new posting day."). The Chinese page and the BM page say it their own way too (the BM page reads **"6 petang"**), so a reader in any language gets words rather than a clock face, and the wording the shop uses to describe a posting day is now the same in all three.

20 Sep 2026 (Engine v124-v130, v132) - the courier's charge, and who bore it. An order can now carry a **courier charge** beside its note and tracking number. The **Note / tracking** button on the row opens the box for it, and the same box sits in **Edit**. You record what the courier asked for, and then **who paid it** - the customer or you. Because the same charge means two completely different things depending on the answer, it lands in two different places: a charge **the customer bears** is added to what they owe for delivery and named in their WhatsApp message and on their own track page; a charge **you bore** becomes an ordinary **Delivery & fuel** expense in your books, so it comes off your profit like any other cost and the customer owes no delivery charge at all.

One delivery charge per order - your rule, applied. This app already quotes a flat RM8 postage on every posted order, so a recorded charge and the flat fee were two answers to one question. As you decided, a recorded charge **replaces** the postage rather than joining it: a customer-borne charge is the **one** delivery line they are quoted, never two, and a charge **you bore** takes the flat fee away with it, so that order asks them for nothing on delivery at all. An order with **no** charge recorded is quoted exactly as it was before any of this existed, and a box half filled in - an amount typed with the payer left blank - changes nothing either. **The postage fee itself is still never published to the storefront** - but it is now inside the figure on the track page, so the customer's page and your WhatsApp message ask for the same amount.

Cash on delivery, if anyone ever asks for it. The same box can be ticked so the courier collects the charge at the door. The charge is still named in full, so the customer knows what to have ready, but it is kept **out** of the amount your message asks for - asking for it up front as well would be asking for the same money twice - and their page words it as a payment to the courier, not to you. This is entirely optional and off unless you tick it; an order without it behaves exactly as before.

And the customer's own card catches up properly (v132). Your phones have always agreed - an order is a synced record - but the customer's track page was only re-sent when the day, the tracking number or the charge moved. Edit the items, a price, the address or the name and their page carried on quoting the order it used to be. The app now compares the **whole card** it is about to send with the last one it sent, and re-sends it whenever anything on it differs. If a write is refused - most likely because a SQL step has not been run yet - it is not remembered as done, so it is retried rather than leaving that page wrong for good.

Two SQL scripts to run once each, before this build goes live: supabase/courier_fee.sql and

supabase/courier_cod.sql. Paste each into your Supabase SQL editor. The app publishes a customer's whole track card in a single call, so a missing column refuses that call **as a whole** - every customer's card would stop updating, not only orders with a charge. Both scripts are safe to run twice. As always, **a commit or a push never runs SQL**: publishing the file to your site does nothing to your database.

Left out on purpose - your bakery has one thing this app does not. The bakery gained a **Production line** (a planner for focaccia bake days) in the same batch of work. Posting pouches of jerky has no bake-day planner in it, so it was deliberately not copied across. That is the first time the two apps have genuinely differed, and it is why this app's engine number is now **its own** rather than a twin of the bakery's. Everything else is still the same code, and future improvements still arrive here the same way.

20 Sep 2026 (manual v123e) - a customer can put a code in, and the order tells you what to take off (no new engine number: built here first, so both your apps stay on the same number). The order page now carries a small **Have a code?** box at the top, on every visit. Until now a code could only arrive by scanning a label's square, so a customer who was told the code at the counter - or who copied the letters off a card by hand - had nowhere to put it. Typing a code **replaces** whatever the link brought, and clearing the box and pressing Apply puts the label's own code back. **A code the app never printed is still quietly ignored**, so a made-up code never lands in your books as a label that never existed.

The page now says what will happen to the code. Under the offer it reads, word for word: *"We'll take this off when we confirm on WhatsApp - the total shown is before the discount."* If the basket is below the code's minimum spend it says how much more is needed instead - *"Add RM8.00 more to use it."* **The total on the page never moves, and that is the whole point.** An order keeps the price it was sold at, so a discount quietly taken off on the page would rewrite the money you actually earned - and *new customers only* cannot be checked from the page at all, because that needs your entire order book.

So your app does the telling. An order that came in on a code now carries a promo line on its row and in its **Edit** box: the code, the name you gave that label, what it promised, and *"take it off when you confirm."* Two warnings appear when they apply - that the offer is for new customers only and this person has bought from you before, or that the order is under the code's minimum spend, naming both figures. Both warnings show only while the offer is still live. **The line never goes blank:** a code you have since retired, or even deleted, is still named, because the order records the label the customer actually came in on.

One word in this entry was wrong, and is now fixed. It said the promo line names "the shop or product it was for". It does not, and could not: a code keeps only the *ids* of the shop or product it was made for - their names are written for the customer's page and are never stored in your book - so the line names the **label**, which is what you typed beside the square, and the code itself when you left the label unnamed. That is the name you recognise on the shelf anyway. **Nothing you see on screen has changed** - the two look-ups that read the missing names never found anything and have simply been removed, which makes the code match the words that describe it.

One fix came with it. The offer shown on the order page now honours its own **Ends** date - a label whose offer has run out stops showing it, the way the landing page already did. The label itself keeps working; only the expired offer goes quiet.

Nothing in your books changes. The line states; it never subtracts. Money, Profit, the WhatsApp confirmation and the customer's track page all keep reading the price the order was sold at, and what you take off at confirmation is still your decision - exactly how a bring-a-friend credit already works. One small consequence: the counts on **Shops & codes** now also include orders where a customer typed the code rather than scanned it, which is right - it is the same printed label.

20 Sep 2026 (manual v123c) - your landing pages: one page per promotion, picked on each label (no new engine number: built here first, so both your apps stay on the same number). Under **More -> Shops & codes** there is now a **Your pages** card, and a **Landing page** dropdown on every label you make or edit. A page is a set of words for one activity - *"Sample card"*, *"Raya promo 2026"*, *"Clearance"* - so a promotion running on a dozen printed cards is written **once** and every card on it follows. Leave the dropdown on its first

choice, **the shared page**, and the label reads the words you already had.

The words now come in three layers, and each line is decided on its own. A customer reading a label gets, for the heading and for the paragraph, in each of English, Mandarin and Bahasa Malaysia: the **shared page** first, then the **page the label picked**, then anything that **label typed for itself** - the last one wins. So a page supplies the lines it has and everything it leaves blank still comes from the shared page, and a label on that page can still shout its own line over the top of both. Each language falls back on its own: a page with only English typed still shows the shared page's Mandarin, rather than going blank or mixing languages.

A page is live, and that is the point. Every printed label carries only its short code and its words are read at scan time, so editing *"Raya promo 2026"* changes what the cards **already in customers' hands** say, with no reprint - one edit, every card on it follows. A label that needs to differ types its own line, which always wins.

Deleting a page loses nothing. The labels sitting on it go back to reading the shared page, and any lines those labels typed for themselves stay exactly where they are. The confirm box tells you how many labels that is before it acts, and each page's row shows the count too.

20 Sep 2026 - printed labels with QR codes, and the page each one opens (no new engine number: this was built here first, so both your apps stay on the same number). A new screen, **More -> Shops & codes**, makes a code for anything you print: a pet shop's sample card, a short promotion, a bring-a-friend card, or a plain card that just says hello. Each code gets its own square - a **QR** you can download as a picture or print as a sheet of labels - and that same code's short letters, like **K3X9**, are printed small beside the square so you can tell two labels apart by eye.

One code is one printed label. A code can carry more than a promotion. It can name the **shop** that handed the card out (with its contact, its commission rate and how many samples you gave it), a **product and an offer** (a percentage or a ringgit amount off, an optional minimum spend, an end date), an **introducing friend**, or nothing at all. The screen keeps them in one list, so a new idea is one more row and not a change to the app.

What the customer sees. Scanning the square opens a page carrying that label's code. It says hello in your own words, states what that label offers - *"10% off · on RM30 and above · new customers only · valid until 30 Sep"* - asks whether the treat is for a **dog or a cat**, and sends them on to your shop **with the label still attached**, so the order that follows is counted against it. Once an offer's end date has passed the page stops promising it and says the offer has finished instead. **New customers only** means exactly what you described: a WhatsApp number that has never bought from you before.

One label can say its own words. Every label opens the same page unless you give it a page of its own, and the shared page's heading and sentence are what a customer reads - unless a label wants to say something the others do not. Open a code and tap **What this label says on the page**: a heading and a paragraph, in English, Chinese and Bahasa Malaysia, and a **Fill Mandarin / BM** button, exactly like the shared page's own copy. Anything left blank falls back to the shared page's line for that same language, and the greyed text in an empty box is the line a customer will actually get - so a label that says nothing for itself changes nothing at all. Every translated box also carries a small **Mandarin** or **BM** tag beside it, so you can always see which language you are typing into: on a label the greyed text inside the box is the shared page's line rather than the language's name, and two boxes whose shared line is blank would otherwise both read as the same English. And a line you have typed yourself stays yours - pressing **Fill Mandarin / BM** again after you change one line translates only the lines you left alone, and never overwrites what you wrote. That is how one promotion card can shout its own offer while a shop's sample card keeps the shop's words.

You can see what a label brought in. Every opening of a label's page is counted for you, along with the dog-or-cat answer, on the same screen under **Label visits** - how many visits, how many dogs and cats, and a line per label, busiest first. A shop's label also shows the orders it has produced. The counts live in your own cloud behind your login, so only you can read them; the public page can only add one. The order page's own banner states the offer too, so a customer meets the same sentence twice.

Hold a phone up to a printed label. Scan a label opens the camera so you can point it at a square and see at once which code it is, what it offers and how many orders it has brought. Typing the short letters always works

as well, and sits right beside the camera button. The camera works on Android and on a computer; an iPhone's Safari cannot read a QR inside a web page - that is Apple's rule - so on your phone, type the code. (A customer's own camera app reads the square fine; that happens outside the page.)

Your own words, and the money stays yours. The page's heading and sentence are yours to write on **More -> Shops & codes** - the shared page every label opens, and any single label that needs its own - in English, Chinese and Bahasa Malaysia, so you can change what a scanned label says without touching the app. **No discount is ever applied silently**: the app states the offer and shows you what to apply when you confirm the order on WhatsApp - exactly how the existing bring-a-friend credit works. Your recorded prices and profit are never rewritten by a promotion.

Your part, once. Run **supabase/taster_visits.sql** in your Supabase SQL editor. It makes the small table the visits are written into, and is safe to re-run. Until it is run, labels print and scan normally - only the visit counts are missing, and the **Label visits** card says so rather than going blank.

What did not change. Your prices, orders, customers, posting dates, the shop and the way messages are written are exactly as they were. A code changes nothing until someone actually scans one.

18 Sep 2026 - engine v123 (no database setup needed). The product list's middle section now groups sold out and not-sold-that-day together, and a product the shop does not sell on a day is named by the days it IS sold instead of being given a count for a day it was never on.

What was wrong. The middle section was called **Sold out** and read its count for the day you were adding to. But a product the shop does not sell on that day at all - a weekend-only treat on a Wednesday - was sitting in **On the shop**, and worse, could be labelled "sold out" from a count for a day it was never on the menu. Neither told you anything true.

What it does now. The middle section is **Unavailable**, and it holds both kinds together: a product that has **sold out** for the day you are adding to, and a product the shop **does not sell on that day**. The second kind is named by the days it IS sold - a weekend-only product on a Wednesday reads "Chicken Jerky 100g - only Sat & Sun". The three headings are now **On the shop**, **Unavailable**, **Taken down**.

Why they are one section, not two. They are the same thing to you: an active product you can still choose. You make to a plan of your own, and as you put it - every day the kitchen may make 12 pouches, ordering closes the day before at 6pm, and the 12 are not necessarily sold off, so some go to a walk-in or into the fridge. **The picker is a guide while you sell, never a gate**: nothing is blocked, and the order-by deadline you set on a product is deliberately not counted, because that deadline stops a stranger ordering - it says nothing about what you may sell by hand.

Where to see it. Orders -> **+ New order** -> Items, and Orders -> open an order -> **Edit**.

Nothing else moved - your prices, customers, posting dates, the shop and the way messages are written are all exactly as they were.

18 Sep 2026 - engine v122 (no database setup needed). The product list now shows the three kinds inside the list itself: each kind sits under its own heading, and on a newer phone each section is tinted in its colour.

What changed. Adding items to an order - in **+ New order** and in **Edit** - the product list now comes in three labelled sections instead of one long run of names: **On the shop** first, then the ones you cannot sell for that day, then **Taken down**. The headings appear on every phone, because they are part of the list itself.

The colour. On a phone with the very latest system - a new iPhone or a recent Android - each section is also tinted inside the open list: **green** for on the shop, **amber** for unavailable, **grey** for taken down. The list then reads as three blocks you can tell apart without reading a word.

On an older phone. The open list is drawn by your phone itself, and an older phone will not let a page colour it. You get the same three headings in the same order, and the closed box still wears the colour of whatever it holds - you lose the section tint and nothing else. Nothing is broken, and nothing has moved; only the colour is missing.

This replaces the v121 note below. v121 put the colour on the closed box alone, because the open list could not be reached. Newer phones can be reached, so the list carries the colour too.

Where to see it. Orders -> **+ New order** -> Items, and Orders -> open an order -> **Edit**. The list is otherwise unchanged: the same products, the same "(hidden)" marker, the same "12 left" count.

Nothing else moved - your prices, customers, posting dates, the shop and the way messages are written are all exactly as they were.

18 Sep 2026 - engine v121 (no database setup needed). The product list now shows you which kind of product each choice is, in colour, and lists them in the order you would look for them.

What changed. Adding items to an order - in **+ New order** and in **Edit** - the product list is now sorted into the three kinds you already know, in the order you would reach for them: what is **On the shop** first, then anything that has **sold out** for the day you are adding to, then whatever you have **taken down** (still marked "(hidden)"). A product still in Draft stays out of these lists entirely, as it always has.

The colour. The closed box wears the colour of whatever is picked - **green** for on the shop, **amber** for sold out, **grey** for taken down. On an order with several lines that means one glance down the list tells you which lines need a second look, without reading each one.

Why only the closed box. The open list is drawn by your phone itself, and iOS does not let a page colour the rows inside it. So the colour sits on the box you see while the list is shut, which is where you read it anyway.

Where to see it. Orders -> **+ New order** -> Items, and Orders -> open an order -> **Edit**. The list itself is unchanged: the same products, the same "(hidden)" marker, the same "12 left" count.

Nothing else moved - your prices, customers, posting dates, the shop and the way messages are written are all exactly as they were.

18 Sep 2026 - engine v120 (no database setup needed). One customer is one row. The app now recognises a phone number by its digits, so the same person can no longer appear twice just because their number was written with a + in front of it - and the duplicate already in your app joins itself into one the first time you open it after updating.

What was wrong. A customer used to be identified by their number written in exactly the same way every time. So **+60123456789** and **60123456789** looked like two different people, and the Customers list showed the same customer twice. Your messages always went to the right person - only the list disagreed. Numbers are now read by their digits, which means **012-345 6789**, **+60 12-345 6789** and **60123456789** are one person, the same way "012345" already found "012-345" in the finder.

It tidies itself once. The first time you open the app after updating, it joins any customer who was split this way, carrying their pet's name and photo, what they like and avoid, and their note across to the one that remains. There is nothing to do - you will simply find one row where there were two. If you are the careful sort, take a cloud backup first from **More -> Backups**: that is the way back if anything looks wrong, and it costs one tap.

New: "Join with another customer". Open a customer's history and you will find the button under their number. Tap it, pick the duplicate from the list, and confirm. Reach for it when two rows are split for some other reason - a misspelt name with no number beside it, say - which the tidy above cannot guess at. It moves the other person's orders under the name and number of the customer you were looking at, so open the row you want to KEEP. This one cannot be undone inside the app; your cloud backup is the way back, and the confirm box says so before it acts.

Two things worth knowing. Update both phones - a phone still on v119 can save a number the old way from its Profile card, and only the updated build does the tidy. And on the Profile card itself, the number you type is now stored in the one form every label, message and link already uses, so a "+" or a space can never split a customer again.

Nothing else changed - your prices, your posting dates, the shop and the way messages are written are all exactly as they were.

18 Sep 2026 - engine v119 (no database setup needed). Start typing a customer's name in an order form and the people you have already served appear underneath - tap one and their name and WhatsApp number fill themselves in.

How it works. Type two letters - "aun" is plenty - and a short list appears right under the name box. Each row shows their number, how many orders they have placed, and the item they buy most, like **012-345 6789 · 5 orders · usually Chicken Jerky 100g**. Tap the right person and the name and the number both go in; the posting day, the items and collect-or-post stay yours to set. One letter on its own is not a search - it would offer you half your address book - and a name you have never served offers nothing at all. Typing a number without its dashes finds them too, so "012345" works the way "012-345" does.

Both order forms do it - the **+ New order** card and the **Edit** pop-up, the same way.

Two things worth knowing. It draws on the orders already in the app, so on a brand-new setup it has nothing to offer until your first order is saved. And on the **Edit** form, saving a changed name has always carried that name to every other order belonging to the person it was - that is the one-saved-name rule from v68 and it has not changed, but picking a name is now much easier than typing one, so it is easier to do by accident. If you ever pick the wrong person while editing an order and save it, tell me and I will add a confirmation that asks first.

Nothing else changed - the Customers screen, its finder, your prices and the shop are all exactly as they were.

18 Sep 2026 - your phone pings the moment an order lands (no new engine; the phones did not change). Until now the app only found out about a customer's order when you opened it, so an order placed at night sat unseen until morning. Your database can now send the alert itself: **the instant a customer taps Place order, a notification arrives on your phone** - even with the app closed, the phone locked, and no computer switched on.

How it works. A rule in your Supabase project watches the orders table. The moment a new order lands it writes a short summary - the customer's name and WhatsApp number, the items and quantities, the posting day, the total, and any note or address - and hands it to the free **ntfy** app, which pushes it to whichever phones have subscribed to your private channel. Orders you add by hand in the app stay silent; only orders customers place on the order page ping.

Your part, once per phone. Install the free **ntfy** app, tap **+**, and type your own private topic (the 16 letters and numbers after 'furkidz-orders-' that your setup script showed you). Leave the server at the default **ntfy.sh**, then allow notifications. Every phone holding that topic pings at the same moment, so a helper's phone can hear it too.

The topic is a password. Anyone who knows it can read your alerts - so it is stored in your own database and deliberately kept out of the app and out of GitHub. The app cannot leak it. If it ever does leak, one line in the SQL editor gives you a fresh topic and you re-subscribe the phones.

A failed ping can never block an order. If the alert cannot be sent for any reason, the customer's order is still saved - and the reason is written down in order_alert_errors so you can read it later. The whole setup is one script, supabase/order_alerts.sql, which is safe to re-run. Since nothing on the phones changed, this is not a new engine version.

17 Sep 2026 - engine v118 (no database setup needed). The Paid step now stays in its place on every order, wearing an X until the money is in - and turns into a green tick when it is. This replaces what v117 did, on your instruction.

Your reasoning, and you are right. The customer's track page is the design: they must see the same steps in the same places every time, and your own app has to read the same way. v117 left the Paid step **off** an order that skipped it - five dots instead of six - which meant the two lines did not line up, and a missing step is itself a thing to puzzle over.

What happens now. A regular who pays when you collect still never passes through Paid. But that step keeps its place in the line, and it wears an **X** - amber, deliberate - instead of a tick, and it never turns green while the money is outstanding:

> New: done · Confirmed: done · **Paid: x** · Preparing: done · Packed: done · Collected / Posted: current

Press **Paid · Cash** or **Paid · TNG** when the money is handed over - at whatever stage the order has reached - and **the X becomes the tick**: the ordinary green tick, in the same spot, with the **Cash** or **TNG** tag beside the row. The order itself stays where it was: taking the money never drags it backwards.

Both lines change together. Your row's map and the **customer's own track page** draw the same six steps, so the X is on their page too, and it turns green there the moment you record the money. A step that is *deliberately gone past but not paid* now looks different from a step that has not been reached (grey) and from a step that is done (green) - which is exactly the distinction that was missing.

17 Sep 2026 - engine v117 (no database setup needed). A regular who pays when you collect no longer has a Paid step on that order - and the Paid · Cash / Paid · TNG buttons stay on until the money is actually recorded, wherever the order has got to. From your note about close customers paying by TNG or cash at pickup.

What was wrong. Move an order from Confirmed straight to Preparing - no payment - and the row's map still showed **Paid with a green tick**. It was claiming money you had not taken. The app assumed "past Paid means paid", which is fine for your older orders but false for a regular who settles up at the counter.

What happens now.

- **A bypassed order has no Paid step.** That route reads New -> Confirmed -> Preparing -> Packed -> Collected / Posted: **five dots instead of six**, and no tick for a payment that never happened. **The Paid · Cash / Paid · TNG buttons stay on** at every stage from Paid onwards - Paid, Preparing, Packed, and Collected / Posted - until the money is recorded. So the moment the cash is handed over at the counter, wherever the order has got to, you press the button right there: **the order stays where it is** (marking an order paid never drags it backwards) and the money is stamped with the day it landed, which is the day the Money screen counts it. **And the Paid step comes back once it is paid** - a green tick, in its proper place in the line, with the **Cash** or **TNG** tag beside the row. So the map ends up telling the truth about that order either way. The buttons disappear the moment the payment is recorded, and they never appear before the order reaches the Paid stage. **The customer's own track page follows the same rule** - same five dots, no Paid tick for a payment that has not happened, and the step appears there too once it is recorded.

One rule behind it, worth knowing: moving an order into Paid **or any later stage** without having pressed a Paid button is taken as *this order owes money*. That is what your older orders are protected from - nothing already in the app changes, because only orders you move now are marked.

17 Sep 2026 - engine v116 (no database setup needed). A Day one form: your opening balance - the cash in the tin, the money on your phone and what is on your shelf - entered once, in one place. From you telling me *"we need to enter opening balance"*.

What an opening balance is here. Four answers about the morning your books begin: what is in the tin, what is on your phone, what is on your shelf, and who still owes you. Until now the app could take all four, but in three different places and one ingredient at a time, which is a poor way to start.

The form. More -> Money -> the new **Day one** line (under **Books**) -> **Set**. It asks for:

- **Cash in your tin** and **Money on your phone (TNG)** - two boxes, and the day your books begin, which takes any date but should be the day you are starting from. **What is on your shelf** - your ingredients, one box each, already in the unit you use for that ingredient (kg for the meat, g for the oil - no converting). Ingredients you have marked as *not something I buy*, like labour, are left out: they have no shelf to count. **Who still owes you** needs no box - those orders stay unpaid and show under **Still to collect**.

Nothing is half-saved. Every box is checked before anything is written: if one has something that is not a number, nothing saves at all until you fix it. And **a box you leave empty is left exactly as it is** - so the form is safe to reopen later just to correct one figure, and safe to run again without wiping what you already had.

What it writes. The tin and the phone become ordinary money-in rows dated that day (so from then on the Money screen's **Net** is what you should really hold, and the weekly check works from the first week), and the

shelf becomes each ingredient's **On hand** - the figure your shopping lists subtract, so your first list buys only what you are genuinely short of.

What it deliberately does not do. Money you **owe** - a loan you took out for equipment, a supplier you have not paid - has nowhere to go: the app keeps a loan as a way of paying, not as a debt, so record what it pays for as it happens. Your own money in shows under **Capital you put in** on Profit, never as income. And the tin must not be entered as a sale: a sale has a customer behind it.

17 Sep 2026 - engine v115 (no database setup needed). The month arrows on Profit work both ways again. From you telling me *"the profit month can move earlier but cannot move later"*.

What was wrong. Open Profit - it starts on this month, where the forward arrow is correctly switched off, because there are no numbers after today. Press the back arrow to step back to August, and the forward arrow stayed switched off too, so you could not get back to September. You could walk backwards through the months but never forwards again: to return you had to leave the screen and open it again. The arrow's state was worked out once, when you opened the screen, and then reused on every step - so it kept answering for the month you started on rather than the month you were looking at.

Fixed. The arrows are worked out fresh on every step, so the back and forward arrows now move both ways between the months you have, and the forward arrow switches off again only when you are back on this month. The same fault did not exist on the other calendars - Orders, Delivery dates and the date pickers all recompute their arrows each time they draw.

17 Sep 2026 - engine v114 (no database setup needed). Every spending line in Profit & Loss now opens - including the ones reading 0.00 - and the lines are twice as tall, so they are easy to tap. From you asking *"in profit the expenses is not clickable, is that a bug?"*

You were right, and it was my mistake. The lines that open their journal were the ones with money in them. A line showing **0.00** was deliberately left dead, because there is no journal to show - but I made it look **exactly** the same as a live one: same colour, same font, no hint. So if your month has most categories at 0.00, nearly every line you tapped did nothing, and "not clickable" was the only sensible conclusion. A line that looks alive and does nothing is worse than no line.

What it does now. Every spending line opens: *Packaging*, *Utilities*, *Delivery & fuel*, the categories you have added, and **Total expenses**. A line with money in it shows its journal as before. An empty one opens and **says so in your own words** - *"Nothing recorded under Utilities in September 2026"*, In / Out / Net at zero, and a line telling you it will fill up on its own as you record spending under that category. The statement's own totals (**Sales**, **Cost of sales**, **Gross profit**, **Net profit**) stay figures rather than doors - they are not spending, and you did not ask for those.

Two more things found while fixing it.

- **The lines were only 17 pixels tall** - a small target for a finger, and easy to land in the gap between two lines and hit nothing. Tappable rows everywhere (the Profit statement, the Money screen's cash rows, the Books list) are now **36 pixels**, comfortably thumb-sized. **The Total expenses journal did not say which category a row belonged to.** It read *"2 Sep · boxes · Cash"* with no way to tell what "boxes" was for. It now names it - *"2 Sep · Packaging - boxes · Cash"* - while a single category's journal still reads short, since its title already says which category it is.

17 Sep 2026 - engine v113 (no database setup needed). Every way you pay now has a book you can always open - including a pocket that has been quiet - behind a new Books line on the Money screen.

What was wrong. The journal rows live on the money card, and that card only shows a pocket's row **when that pocket moved money in the stretch you are looking at**. The card opens on **Today**, so a pocket that paid for something last week had no row today - and no way into its book at all. Cash and TNG never had the problem: their four rows (Cash in, Cash out, TNG in, TNG out) are always there. The pockets were the only books that could go missing.

The Books line. Under the two money cards there is now a second line beside *Categories & ways to pay*:

> **Books** > Every way you pay · each opening into its own rows

Open lists **every** way you pay - Cash, TNG, Loan, a personal pocket you have added, and anything you add later - each with what moved by it in the stretch on screen, each opening into its own book **under the line you tapped**, so the list stays in front of you and you can step from one book to the next. The first method with anything in it opens ready; a quiet pocket says so plainly - *"Nothing moved this way in this stretch"* - with its In / Out / Net at zero.

Two things this keeps honest: a method you have since **renamed or deleted** still gets a line, because its money is still in the books and a figure you cannot open is a figure you have to take on faith; and every figure in the list comes off the Money screen's own rows, so the list and those totals cannot drift apart.

17 Sep 2026 - engine v112 (no database setup needed). You can pay a personal pocket back out of the till in one go, and the category list on the expense form is no longer cut off.

Pay back a pocket. On **More -> Money**, beside **+ Put money in**, there is now **+ Pay back a pocket**. It is for the case where a pocket of yours - a personal pocket you added, a loan - paid for something, so the money left you rather than the till; that pocket's line on the Money screen then reads **-RM 40.00**, which means *the till owes it 40*. Open the form and it comes up **already on the pocket that is owed**, with the amount filled in, and a line saying so. Say whether the money came out of **Cash** or **TNG**, check the day, and press **Pay back**.

It writes both halves at once, which is the whole point: the **till goes down** by that amount (it shows in Cash out and in the Net, and in the Cash journal with your note), and the **pocket's line comes back to zero**. One pocket can be paid back without touching another. Because it is your own money going back to you, the till's side is recorded as a withdrawal - so it never counts as a cost and your **profit does not move**. On the pocket's own list it reads *"Paid back by the till"*, not *"From my pocket"* - the two are different things and now say so. Paying it back in part is fine: pay RM 25 of the RM 40 and the pocket reads -15 after.

The category list was being cut off - fixed. Thank you for catching this. On the expense form, the row of categories ran off the side of the screen: it was laid out as one single line, so at phone width everything past about the sixth category - *Salary (you)*, *EPF / SOCSO*, *Marketing*, *Equipment & tools*, *Other*, *My own withdrawal*, and the **+ New category** chip - was off the screen and impossible to reach. Measured on a 375-pixel phone, the row was 744 pixels wide inside a 343-pixel box. The pills now **wrap onto as many lines as they need**, the same fix applies to the ways-to-pay row and to the new pay-back form, and the whole list is visible and tappable. Nothing about the categories themselves changed.

17 Sep 2026 - engine v111 (no database setup needed). Every figure on the books is now a door: a journal behind each line of spending in Profit & Loss, and a book for every way you pay.

Profit & Loss - tap a spending line and see what made it up. On **More -> Profit**, the running-cost lines (Packaging, Utilities, Delivery & fuel...) are now tappable, and so is **Total expenses**. Tapping one opens its journal for the month on screen: every expense behind that figure, oldest first, each reading *"<day> · <what it was for> · <how it was paid>"* and ending on the total the statement itself shows. So *"Packaging RM -58.00"* opens two lines - the boxes paid from a pocket and the bags paid in cash. A line with nothing in it that month is not tappable; there is nothing behind it. The header now says **Running costs · tap a line to see the spending behind it**.

Money - one line and one book per way of paying. The single *"Paid by loan / other"* row is gone. In its place, every way of paying that is not cash or TNG gets **its own line** with what moved by it, and its own journal behind it: **Paid by Loan**, **Paid by Personal Pocket ...** - whatever you have called them. Money of your own you put in that way reads as a plus on that line; what was paid out of it reads as a minus. Cash in, TNG in, Cash out, TNG out and the Net are exactly as they were, and none of these other ways is in the net - it never came out of your purse.

Why they appear by themselves. The lines are read off your own transactions, not off a fixed list, so a method you add tomorrow needs nothing from me: record one payment with it and it has a line and a book. And a method you have since **renamed** keeps a line for the name its older rows were written with - those rows are still in the books, and a figure you cannot open is a figure you have to take on faith. Each book ends on **In / Out /**

Net for that method.

One small correction while here: the note at the foot of the Profit screen now matches v110 - either pay yourself a **Salary (you)** expense (with EPF / SOCSO as their own category), or mark Labour as a not-bought ingredient and put the hours into the recipes. Count them one way, never both.

17 Sep 2026 - engine v110 (no database setup needed). An ingredient you never buy - your own labour, electricity, gas - can be marked as a cost, and it then never appears on a shopping list again.

What it does. Open **More -> Ingredients** and tap **Edit** on the ingredient (or fill in the New ingredient card at the top). Above the cost there is now a switch: **Not something I buy - it only ever costs. It still counts in a recipe's cost, but it never appears on a shopping list or a purchase order.** Turn it on for Labour, Electricity, Gas, your own time.

Where you see it.

- The ingredient's **card** stops showing an On hand and a Keep-at-least line, and says instead: *Not bought - a cost in your recipes, never on a shopping list*. Nothing else about the card changes, and the cost you typed is untouched. Every **shopping list** (the preview, the list you save, and an "orders changed" follow-up) leaves that ingredient out - and says so at the foot of the list: *Not on this list: Labour, Electricity - marked as not something you buy. Their cost still counts in the products that use them.* So it reads as deliberate, not as something the app forgot. The **recipe cost** is unchanged: a recipe using Labour at RM 8 an hour for 15 minutes still adds RM 2.00 to that product's cost, and that runs on into the Profit screen's cost of sales.

Switching it OFF restores the ingredient exactly as it was - the app removes the mark rather than storing a "No", so nothing that exists today changes unless you turn this on.

And a word on the books, since this touches Profit: if you mark Labour as not-for-purchase because it is your own unpaid time, it is already inside each product's cost of sales through the recipe - so do **not** also record it as a Salary expense, or the same hours get counted twice. The **Salary (you)** category is for when you actually pay yourself money out of the till.

16 Sep 2026 - engine v109 (no database setup needed). A journal for each way the money moves - the cash book, the TNG book - and a bug that was hiding in the TNG column.

A bug first, because it matters. Every order you marked **Paid · TNG** since v106 was landing in "Paid, no method" instead of the TNG column. The app was checking for the old way of writing it ("tng") while the paid buttons write the list's own label ("TNG") - so the TNG in figure read RM 0.00 with the money in your phone, and the same money showed as unaccounted for. Fixed, and there is now a test that would have caught it. If your TNG column has looked wrong since v106, that was it.

Tap a figure, see its journal. On the Money screen, **Cash in / TNG in / Cash out / TNG out** (and the loan line) can be tapped. Each opens that method's book for the stretch: every order paid that way with its code and customer, everything you spent out of it with its note, anything of your own you put in - in date order, ending on In / Out / **Net**, and a line saying what that is meant to be (your purse, your phone, or money that never went near either).

16 Sep 2026 - engine v108 (no database setup needed). Where the two lists live, and how to change one.

Why. v106 put them in a Settings card and you could not find it, and there was no way to change an entry once it existed - *"dont put the setting separately, it should be at where it suppose to be."*

They are on the Money screen now, as a line under the money lists reading "Categories & ways to pay · 11 categories · Cash, TNG, Loan" with an **Edit** beside it. Tap Edit and the lists open: every category with its kind (ingredients / running cost / your own money) and every way to pay, each line saying "edit".

Tap a line and it turns into the form: rename it, change what kind it is, or delete it - in place, with the rest of the list still in front of you. Renaming moves what you have already recorded with it, so a slip of the thumb does not split your history into "Packaging" and "Packing".

And the + chips sit where you are working: the Add an expense form and Put money in each end their pills with **+ New category** / **+ New way to pay**, which opens the same small form in place - the amount and the note you had already typed stay put.

No window inside a window. These forms expand in place instead of opening a pop-up of their own, the same rule the date field follows: the app has one pop-up layer, so a second one would wipe out the form underneath it.

16 Sep 2026 - engine v107 (no database setup needed). The item line on an order reads properly again.

What was wrong. When the selling price joined the line (v101) it took most of the row. Measured on a 375px phone: the price box was 183 pixels wide and the product dropdown was squeezed to **2** - so the name of what you had picked was pushed onto a line of its own and unreadable, and the whole line looked broken.

What it does now. Each item line is two clean lines: **the product across the top**, full width so its name reads, and its controls under it - how many, the selling price on the right, and the x to remove. The same shape in the + New order card and in the Edit pop-up, since both are the same row.

16 Sep 2026 - engine v106 (no database setup needed). A note on every transaction, and two lists you shape yourself.

A note on each one. Add an expense now takes a note ("Mydin run, 2 boxes") - the Put money in form already had one - and both lists on the Money screen show it beside the amount, so a row reads as what it was, not just a figure.

The categories are yours. Every category shows what it means to the accounts (ingredients / running cost / your own money), each has a x to delete it, and you can add your own - "Pet expo", "SSM licence" - choosing which of the three it is. Deleting one never touches what you have already recorded: those rows keep their label and still count, printed at the end of the statement's cost list rather than dropped.

And the ways to pay are yours too. The list is Cash, TNG and **Loan** - the third choice you asked for - and you can add your own, a **Bank OD** or a cheque, so a shopping run nobody would pay for out of the till can still be recorded honestly. The same list appears on the Bought prompt (where the receipt is still in your hand) and on both money forms.

One thing that follows from it. Money that never came out of your purse - a loan, an overdraft - no longer sits in the net. The Money screen leaves it out of the cash figures and names it on its own line, with a sentence saying why. Net now means exactly what it says: what should be in your purse and on your phone.

16 Sep 2026 - engine v105 (no database setup needed). The books: a profit and loss account, a month at a time. One new screen, More -> Profit.

Why. Money told you what was in your purse. It could not tell you whether the making made money - a RM250 meat run makes one week look like a disaster while the treats it went into sell the week after.

What it shows. More -> Profit, month by month (arrows either side), reading down like a set of accounts: Sales, Cost of sales, Gross profit, the running costs one line each, Total expenses and Net profit - with the gross margin beside the order count. Then, kept apart from all of it, your own money: capital you put in, drawings you took out, and what is left in the business.

The one rule that makes it accounting rather than a cash total. Ingredient cost is what the making cost, taken from your RECIPES as the treats sell - not the packs you bought. Buying and using sit in different places on purpose: a shopping run is money out on the Money screen and stock on your shelf, and it becomes a cost of sales here as the treats made from it go out. Nothing is counted twice, and a big stock-up week stops looking like a loss.

Your own money is neither income nor a cost. What you put in is capital; what you take out is drawings (including the "My own withdrawal" category). Both move cash - Money counts them - and neither changes profit. If you pay yourself a proper salary, record it as **Salary (you)**, with **EPF / SOCSO** as its own category, and those DO count as running costs.

The category list is now a chart of accounts: Ingredients & shopping (stock), Packaging, Rent, Utilities,

Delivery & fuel, Salary (you), EPF / SOCSO, Marketing, Equipment & tools, My own withdrawal, Other - the same list the Add an expense form offers, so every row lands on the right line of the statement.

Two things it will not pretend to know. Your own unpaid hours are not a cost - if your time is worth paying, pay yourself a salary and it becomes one. And a product whose recipe prices to nothing is counted as costing nothing; the screen says how many lines that was, so it is visible rather than flattering.

16 Sep 2026 - engine v104 (no database setup needed). Money you put in yourself, and taking it back out. One change, on More -> Money.

Why. Some spending happens before any order money arrives - a packet of meat paid from your own purse, a float of change for the day - and there was nowhere to put it. The Money screen only counted what customers paid.

Put money in. More -> Money now has a **Put money in** button beside Add an expense: how much, the day you put it in, cash or TNG, and a short line saying what it was for. It counts into the Cash in / TNG in rows, because your own money really is in your purse and those rows are what you check the purse against - and one line underneath says how much of the money in was yours: "of the money in, RM 100.00 was your own". The entry is listed in its own **Money in** card, so you can see where it came from, and take it off again.

Taking it out again. Add an expense has a new category, **My own withdrawal** - money you take back for yourself rather than a cost of making. It leaves through the same money-out list as everything else, so the net, the day lines and your backups count it with no new machinery.

16 Sep 2026 - engine v103 (no database setup needed). Money out: what you spend, beside what you take in. One change, on the shopping side of the app.

Why. v101 gave you the money coming in. Spending had nowhere to go: a purchase order worked out what a shopping run should cost, and tapping Bought put the packs on your shelf, but nothing recorded what you actually paid.

A shopping run now asks. Tap **Bought** on a saved list and, once the packs are on your stock, a small box opens: **What did you pay?** - pre-filled with the list's own total, with **Cash** or **TNG** beside it. Accept the guess with one tap, or type what the receipt really said. **Skip the money** leaves the stock added and records nothing, exactly as the app behaved before.

Everything else goes in by hand. More -> Money now carries an **Add an expense** form for what a purchase order never sees - packaging, delivery and fuel, utilities, equipment, or a market top-up - each with the day you paid it, so a receipt found in your bag later still counts on the right day.

The Money screen adds it up both ways. Cash in, TNG in, Cash out, TNG out and **Net** - what should be in your purse and on your phone for Today / This week / This month - with that stretch's spending listed underneath, each line showing what it was for and how you paid. What is still to collect stays out of the net: it is money owed, not money held.

One fix along the way. A payment taken just after midnight was being counted as the day before, because the app read the date off the stamp in UTC. It counts on your own day now.

16 Sep 2026 - engine v102 (no database setup needed). Swipe across the calendar to pick a run of delivery dates. One small change, on More -> Delivery dates.

The delivery-date calendar now works the way a product's availability calendar does: **drag across days and the whole run fills in under your finger**, then **Add selected** puts them on the calendar. It is the same gesture as dragging a sell period on a product. A single tap still picks one day (or puts it back), and a day that is already a delivery date is left alone by a drag - it comes off by tapping it, the way it went on.

That makes four ways in: tap dates, swipe a run, tap a weekday letter for every one of that day in the month, or **Generate the next dates** on your Settings pattern.

What I deliberately did NOT copy: the From / To boxes. On a product's availability card those two boxes are how a SEASON is said - a sell period genuinely has two ends. A delivery date is one day, not a stretch, so a

From/To here would only ever have been a slower way to do what swiping now does.

16 Sep 2026 - engine v101 (no database setup needed). A price you can change on an order, and the money recorded as cash or TNG. Three things, all about money.

A price you can change on an order. Every item line on the + New order form and in the Edit pop-up now carries its selling price. It opens on what the product costs; type over it and THAT order is sold at your price - the confirmation, the payment and pickup reminders, the receipt, the customer's own track page and every money number in the app follow it. Your menu price is untouched, and a menu price you change next week never rewrites a sale already made. Blank keeps "whatever the product costs", which is how an unpriced product has always behaved. Both forms also show the items total as you go.

Cash or TNG, written down when the money lands. The Paid button is now **Paid · Cash** and **Paid · TNG** - one tap each. The order remembers which, and when, and shows it beside its status. The Note / tracking box (now **Note / tracking / payment**) carries a **Paid by** box as well, for an order you marked paid before you could tell, or one to correct later. None of this reaches the customer: their page just shows Paid.

Two places to check it against. The delivery day's own header on Orders now carries a short till - "Cash RM 95 · TNG RM 120 · 1 to collect". And More -> **Money** is a screen of its own, with **Today / This week / This month**: cash collected, TNG collected, paid but no method recorded, and what is still to collect with the number of orders. Collected money is counted by the day it LANDED, so a transfer that comes in today counts today even when the order goes out on Friday; what is still to collect is counted by the delivery day, because that is the day you hand it over. An order paid before any of this existed shows under "Paid, no method" rather than being guessed at.

16 Sep 2026 - engine v100 (no database setup needed). Making delivery dates for a weekday is one tap again. One change, on More -> Delivery dates.

Why. You found the function missing: adding your delivery dates for, say, every Monday meant tapping each Monday by hand. Nothing had been removed - the app's only "do the dates for me" button lived on Home, inside the "No delivery dates yet" message. That message shows only while you have no upcoming dates, and pressing the button creates the very dates that hide it: it was a one-shot button that took itself off the page the moment it worked. So once you had dates, which is every day since your first week, it was never there again.

Three ways in now. On More -> Delivery dates: tap dates and **Add selected**, as before; or **tap a weekday letter** - tap **M** and every Monday in the month shown is picked for you, tap it again to put them back, and the letter underlines solid once that whole weekday is picked; or press **Generate the next dates**, which is always there, follows the delivery days in your Settings, and adds the next six dates that are not on the calendar yet. Generate moves the calendar to the month the new dates start in, so you can see what it just did.

One wrong line fixed with it. Home's "no dates yet" message used to say "the next Mon/Wed/Fri dates" whatever your delivery days really were. It names your own days now (and so does the line under Generate).

15 Sep 2026 - engine v99 (no database setup needed). The customer can tap the "Next available" line and be given that day. One change, on the order page.

Why. A product you keep on the shop, on a day it cannot be ordered, tells the customer when it can be had - "Next available: Sat 19 Sep". Until now they had to go and find that date on the calendar themselves.

What it does now. The line is a control: tapping it takes that posting day, so the product in front of them can be ordered straight away, and the page moves up to the calendar, where the chosen day is written out in words. It keeps its gentle pulse, and wears a small arrow so it reads as something to tap.

The one time it does not take the day. If the customer already has something in their basket, the line stays a plain label and tapping it answers instead: "Your basket is for Fri, 18 Sep. To order for another day, choose it on the calendar above." Taking a day then would silently move their whole order to a new date - and drop anything that does not fit there - which is not something a tap on a product line should ever do. Changing the day is what the calendar is for.

16 Sep 2026 - engine v98 (no database setup needed). The last status reads Collected / Posted, and a

placed order gains a Note / tracking button. Two small changes on the Orders screen.

One label for the last stage. v97 named it per order - Collected on one the customer fetches, Posted on one you mail. You asked for the pair itself instead, so the stage now reads **Collected / Posted** everywhere it is named: the row's status list, the row's own journey map, the day's status filter, and the map on the customer's track page (in all three languages). Which message the row offers is still decided by how the order leaves - a post order shows **Send posted message**, a Collect one **Send pickup reminder**.

A way in for the two fields you reach for most. Beside Edit, every order now carries a **Note / tracking** button. It opens a small box with exactly two fields - the order's **Note** and the courier's **Courier tracking number** - and a Save. Send, and the note lands on the order, while the tracking number also goes onto the customer's track card and into the posted message. Nothing else moved: **Edit** still opens the full form (the posting day, the customer, the address, the items, the quantities) whenever you need it. The tracking box that sat on the row itself in v97 is gone - the button is the quicker, tidier way to the same field, and it works on any order rather than only a posted one.

16 Sep 2026 - engine v97 (one database line to run once). The last status is named for how the order leaves, and a posted order can carry its courier's tracking number. One change, across the order row, the WhatsApp messages and the customer's track page.

The last status is Collected or Posted. The final step used to say "Delivered" for everything. It now reads **Collected** on an order the customer fetches from you and **Posted** on one you mail - the same step in the journey, named for what actually happened. That is true everywhere the status is named: the row's status list, the row's own journey map, the day's status filter, and the map the customer sees on their track page (in all three languages). Nothing about the step's behaviour changed, and every order already sitting at that step keeps its place - only the word is different.

A tracking number, typed where you post the parcel. A posted order shows a **Courier tracking number** box under its details from Packed onwards, and a **Send posted message** button beside the status. Type the number the courier gave you, press the button, and WhatsApp opens with the message already drafted: the order code, what was sent, and the tracking number. The box is in the Edit pop-up too, for a number you need to fix or read back over the phone.

The customer sees it. The number is published with the order, so a posted order's track page shows it on its own line under the delivery details - they can read it to the courier without messaging you. Collect orders show no such line. The posted message is only offered on post orders: a Collect order keeps its **Send pickup reminder**, which is the message that fits that hand-over.

What you need to run once. The tracking number needs one column added to the tracking table - supabase/track_no.sql (a single ALTER, safe to re-run). Until that runs, everything else works; the number just reaches the message and not yet the customer's page.

16 Sep 2026 - a stray "null" removed from two screens (no engine change; the phones did not change behaviour otherwise). Found while checking the new v96 pop-up, and fixed in the same pass.

What was wrong. The word **null** could be printed on the page under the day's total, and at the very bottom of Settings under "Delete all data". Nothing was broken behind it - no order, price or setting was ever wrong - but a stray line reading "null" is alarming to read, so it is gone.

Why it happened. The app builds its screens from pieces, and a piece that is only sometimes needed was being handed over empty in a way the browser prints as the word rather than skipping. The two places that could show it were the day pop-up (when every product on sale that day was counted, which is the usual case) and the Settings screen (whenever you have any product or ingredient - that is to say, always, once you are set up).

The guard. A new automatic check builds both screens and fails if the word "null" can appear on either, so a future update cannot quietly bring it back.

16 Sep 2026 - engine v96 (no database setup needed). "Set day's availability" now shows how the day adds up. One change, on the pop-up you already use.

Why. v95 made a day's total count only the products you sell that day, but the pop-up that sets those numbers still showed nothing about where the total came from - and a product you do not sell that day sat in the list looking as if it were part of it.

What it does now. Under the product rows, the pop-up ends with **How the day adds up:** one line per product on sale that day with what it contributes, and a final line with the total - **the same number the order page uses for that day**. Type a + or a - and the lines and the total move as you type. Underneath, the pop-up names what it left out ("Not counted: Duck Jerky - not sold on this day, so no order can go on them here") and says what is already booked, so you can read how much the order page can still take. A product not sold that day keeps its row so you can still set its numbers; it simply does not enter the total.

One number, one place. That sum and the day's capacity are worked out by the same code the shop's own numbers come from, so what you read on screen and what your customers get cannot drift apart.

16 Sep 2026 - engine v95 (no database setup needed). A day's total now counts only the products you actually sell that day. One change, on your side of the app, with one knock-on for customers.

What it was doing. The "x/y" on a day - the chip beside the date on Orders, the same number on the day in the month calendar, and the dials on Home - was adding up the daily limit of every product on your menu, whether or not that product can be ordered on the day you were looking at. So a day with one product on it (a limit of 12, say) still read **1/42**: thirty of those units were limits belonging to products that could never take an order that day.

What it does now. Only the products on sale that day count. A Saturday-only item adds nothing to a Wednesday, and a product kept on the shop as **Unavailable** adds nothing to a day it is not sold on. The same day now reads **1/12** - what that day can really take. Products with no sell marks are unchanged (they sell every day, so they always count), and a day where nothing on sale has a daily limit falls back to the **default capacity** in Settings, exactly as before - it does not drop to 0, which would make the app call the day Sold out.

The knock-on worth knowing. This same number is what the app tells the order page about a day, so a day now turns FULL at the moment every unit you are actually selling that day is booked, rather than at some larger number padded out by products that were never on sale. Past every real slot being taken, the order page stops offering that day - which was always the intent, and now happens at the right moment rather than late.

Nothing else moved. Your over-capacity warning, the "N ordered · N left" line on a delivery date, the shopping list and each product's own "N left" stamp all read the same number they always did for products that are on sale.

16 Sep 2026 - engine v94 (no database setup needed). One sentence on the customer's greyed card, reworded. Wording only - nothing behaves differently.

On a product you keep on the shop, the line saying why it cannot be ordered today now reads **Only available on Mon.** where it read "Only sold on Mon." The old wording was the seller's voice on a customer's card, and "available" is what the rest of that same card already says ("Only available for posting from 19 Sep"). The Chinese and Bahasa Malaysia say the same thing as before, in their own words.

16 Sep 2026 - engine v93 (no database setup needed). The track link now lands on the track card, lit. One change, on the customer's side of the shop.

Why. Your WhatsApp confirmation ends with a Track your order link. Tapping it opened the order page at the very top, with the tracking card somewhere below the fold - the customer had to scroll and hunt for the very thing they had just tapped, on a page they had never seen before.

What it does now. The page comes up with the Track your order card already in view **and gently glowing**, so it cannot be missed. The glow does not fade on a timer: it goes when their finger (or mouse) actually reaches the card, so it is still burning while they are looking around for it. That is the same rule your own app already uses when you tap an order in New Orders and its row flashes - the flash stays until you arrive. On a phone with Reduce Motion on, the card is lit steadily instead of pulsing.

Nothing else changed. Typing a code into the box on the page behaves exactly as before (the card is already

in front of them, so nothing moves), and a customer who simply opens the shop gets no glow at all - the link is the only thing that lights it.

15 Sep 2026 - engine v92 (no database setup needed). The New product card on Products folds away. One change, on one screen.

Why. The card that adds a product sat open at the top of the Products screen, and open it is a long form - it pushed your three lists (On the shop, Draft, Hidden) right off the bottom, so every look at what you sell began with a scroll past a form you were not filling in.

What it does now. The card arrives folded to a single line reading **+ New product**. Tap it and the form opens; tap the title again, or tap anywhere else on the screen, to fold it away. It stays open after you add a product - the toast still says it was saved as a draft - so you can type the next one straight away, and it starts folded again the next time you come to the screen. Nothing about adding a product changed: the form inside is exactly as it was, and it is still the same card that stays on the page while an Edit pop-up is open over it.

15 Sep 2026 - engine v91 (no database setup needed). The Orders calendar now answers a tap it cannot act on. One change, on one screen.

Tapping a day you do not post on. The month calendar at the top of Orders rings your posting days in green with how booked each one is, and draws every other day of the month plain. A plain day never did anything before - tapping it was silent, which left you guessing at the reason. Now the calendar answers: a warm line appears under the grid naming the day and saying it is not a posting day, and that posting days are added in More -> Delivery dates. A marked day says its usual name above the day at the same time, so a holiday that is not a posting date now tells you both what it is and why no order can go on it. Opening a real day, or tapping another plain one, replaces the line. Nothing can be ordered on a plain day, and nothing is created by the tap - the calendar simply says so now instead of staying quiet. A plain day already gone stays silent: there is nothing left to add to it.

Where it appears. The month calendar at the top of Orders, the day picker inside the + New order card, and the Delivery day picker in an order's Edit pop-up - every calendar in the app that offers only your posting days.

Nothing else changed. Posting dates are still made in exactly one place, More -> Delivery dates (or **Generate next posting dates** on Home). Tapping a calendar elsewhere in the app does not create one, and a tap that could open a day before still opens it.

15 Sep 2026 - engine v90 (no database setup needed). A product can stay on the shop on a day it cannot be ordered, with the date a customer can next have it. One new switch, off unless you turn it on.

Keep it on the shop. A product's Availability card now opens with a switch at the very top, above the sell-day calendar. It is for the two or three hot items customers come back looking for - if it is not listed they wonder whether you still make it - but leave it on for everything and the menu fills up, so it is a decision you make one product at a time. The card's own title carries the state too, so you can read it while the card is still closed.

What the shop shows with it on. On a posting day the product is not sold on, the card stays instead of vanishing: greyed, stamped **Unavailable** (a day you never sell it is not the same as a day it ran out), with the reason it already writes ("Only sold on Mon") and one more line - the next date it can be ordered. On a day it has sold out the card is exactly what it was, **Sold out** stamp and all, and gains that same line. The line reads **Next available: Sat, 19 Sep**, and names how many are left that day when a daily limit is set; it pulses gently so it reads through the dimming, and sits still - simply lit - if your phone asks for reduced motion. Nothing on a greyed card can be ordered: the + and - do nothing, it never enters a basket, and a line already in a basket still leaves with a note when the day changes. The switch never overrides **Draft** or **Hidden** - those still take a product off the shop entirely.

With the switch off, nothing at all changes. Off is the default and no product you already have carries the setting, so every product behaves exactly as it does today until you switch one on. One consequence worth knowing: on a day where every product is unavailable, a shop with kept items shows those greyed cards instead of the "nothing is on the menu" line.

15 Sep 2026 - engine v89 (no database setup needed). The Delivery dates screen trades its list for the calendar: a ticked day comes off when you tap it again, and the past dates fold away into one group. Three small changes, all on one screen.

Tapping a ticked day takes it back off. A posting date that was already on the calendar used to come off only from the list underneath it. Now tapping that green day takes it off on the spot - the same tap that put it there undoes it, so the calendar is the one place you both add and remove dates. If that day already has an order on it the app asks first ("2026-09-16 has 2 order(s) on it. Delete the date? The orders are kept in your delivery history."), because that is the one removal worth a second look; a day with nothing on it just goes, with a small "Delivery date removed" note. The date is all that is ever removed - the orders stay in your history either way.

The list of dates still to come is gone. The calendar already draws every posting date as a green tick, and now it lets you take any one of them back, so the list below was repeating what was already in front of you. That is most of why this screen reads so much lighter. Nothing else moved: the green tick, the coloured holiday tags and the "N ordered · N left" line are all as they were.

The past dates are one group, folded away. Dates already gone keep their green tick but cannot be tapped, so nothing on a past day can be changed by a stray touch. They now sit together under a **Past dates (12)** heading that starts folded - tap the heading to open the list, tap it again to fold it back. Every past date is in there now: the old list showed only the ten most recent and left the rest out of reach. Each one still opens its orders or can be deleted exactly as before, with the same "Del" button and the same question when orders sit on it.

14 Sep 2026 - engine v88 (no database setup needed). The holiday tint sits on the dates at Orders, and a mouse resting on a marked day names it. Two small things, both about the Orders screen and the bubble.

The tint now sits on the line of dates. On the Orders calendar each posting day carries its booking count, and that count sat *under* the date number, which pushed the number 5.5px above the middle of its row. The occasion wash is centred on the row, so on this one screen the tint sat lower than the numbers it was meant to be washing - and the dates themselves did not line up with each other. The count now sits at the foot of the day, every date number sits on one line, and the tint runs through that line. Nothing else about the calendar moved: the green ring, the chosen day's fill, and the counts all read exactly as before.

On a computer the name comes up on hover. v87's bubble needed a tap. With a mouse, resting the pointer on a marked day now shows its name on its own, exactly as the customer's page does. This is deliberately only for a pointer that can hover: a phone keeps the tap, so a finger sweeping a run of days across the Availability or Delivery Dates calendars never drags a bubble along with it.

14 Sep 2026 - engine v87 (no database setup needed). Tap a marked day and it tells you its name - on every calendar in the app. v85 drew your marks on all five of the app's month calendars, but only as colour: a wash and a band, with nothing to say which holiday it was. The customer's own page has answered this since its calendar arrived - tap a marked day and a small dark bubble above it says the name - so the app now does the same thing, with the same look.

A tap names the day and still does that day's usual job. The bubble appears above the day ("Malaysia Day") and goes away as soon as you tap anywhere else. Nothing else about the tap changed: on the Orders calendar it still opens the day, on a product's Availability calendar it still marks or unmarks the sell day, on Delivery Dates it still starts the range you are about to mark. A day that sits inside two marks takes the name of the shorter one - the same rule the colours already follow, so the name and the colour always belong to the same mark. A day already gone is never named, because no mark is drawn on a past day either.

Two kinds of day could not be read at all before, and now can. On the Orders calendar a marked day you do not post for was a quiet number with nothing to tap, and on the Delivery Dates calendar a marked day that is *already* one of your posting dates could not be tapped either - its tick is removed from the list below the calendar, never by tapping its square, so its tap was free. Both now say their name when tapped, and nothing else about them moved: a Delivery Dates square still never ticks or unticks a date.

The Order date boxes name their day above the date, not above a day of the month. Those two fields fold

their calendar the moment a day is picked, so a bubble drawn on the day would vanish with it. The name sits above the date the box is showing instead, which also means an order recorded on a public holiday says so for as long as that date is on the field.

Nothing was added, moved or removed, and the shop is not touched: this is the same bubble the shop's calendar has been showing, arriving on your own calendars.

14 Sep 2026 - engine v86 (no database setup needed). On the Orders screen your marked days now sit on their days, at the same depth as everywhere else. v85 put your marks on every calendar in the app. On the Orders screen the wash did not line up with the days underneath it. Two things were behind that, and both came from the same fact: the days on that one screen are taller than the days anywhere else, because a posting day there carries its booking count ("3/12") under its number.

A holiday band was stretched to fill its week row. Where every day is the app's usual height, a band came out the same depth a single-day box is, so the two shapes matched - which is how the rest of the app has always drawn them. On the Orders screen the rows are taller, so the band there came out deeper than a single-day box, and deeper in the weeks that held a posting day than in the weeks that did not, so one holiday's own band changed depth from row to row inside the same month. **A band is now a sheet of one depth, centred on its row** - the same depth a single-day box has, on every calendar, whatever height the days around it happen to be.

A plain day kept the app's usual height inside a taller row, so it sat at the top of the row rather than on its middle. A single-day mark was therefore drawn 6px higher on a day you do not post for than on a day you do: the same holiday, on a different line, decided by whether you post that day. **Every day of the Orders month is now drawn at one height**, posting day or not, so a mark sits on the same line as the day it covers, beside the booking counts.

One more thing fell out of that. Today's soft glow is drawn around a whole day on the app's shorter calendars, and around a taller day it would have become a box again - the very thing that glow replaced. **Today on the Orders screen now glows around its number**, the same round glow a posting day's number carries, so today reads the same whether or not you post that day.

Nothing else changed. The shop's calendar and your Delivery Dates calendar look exactly as before (their days are all one height, so they never had this), and no mark was added, moved or removed: this is the drawing only.

14 Sep 2026 - engine v85 (no database setup needed). Your marked days are now drawn on every calendar in the app, not just the one you marked them on. Until now a holiday only appeared on the customer's calendar and on the Delivery Dates screen you marked it on. Every other calendar in the app drew a plain month: the month calendar at the top of the Orders screen, the little **Order date** calendars in the new-order form and in the Edit-order pop-up, the posting-day picker inside that pop-up, and a product's **Availability** calendar where you mark the days it sells. So a holiday you were planning around was invisible on every screen you actually plan on.

Now all of them draw your marks, in the same two shapes and the same colours as the Delivery Dates calendar - a see-through band across a run of days, deeper the shorter the run, and a box for a single day. A mark on a day you do not post for is drawn too, on a dimmed day, since that is the only way a holiday on a quiet day can be seen at all. Nothing about the marks themselves changed: they are still only ever something to see, and a mark still never adds or removes a posting day or changes what a product sells. Two small pieces of the drawing were tidied while moving it: the green tint on a day a product sells is now a see-through wash rather than a flat fill, so a holiday band running behind it still shows through instead of the mark disappearing on exactly the days you sell (the tint itself looks the same as before); and a marked day that happens to be today now keeps the soft glow that says "this is today", the way the shop's calendar has always drawn it.

14 Sep 2026 - engine v84 (no database setup needed). A day you marked is now the same gentle wash everywhere, whichever calendar you are looking at. A marked day was drawn one of two ways, decided by a single question: is this a day you post for? On a day you post for the mark went pale, so the green "you can order" ring stayed the first thing the eye read. On a day you do not, the mark was drawn at full colour, as a solid

block with the number turned white. September's Malaysia Day sat on a posting day and read as a quiet tint; October's World Animal Day fell past the posting days published so far and read as a loud solid block - so the same kind of holiday looked like two different things, and a mark quietly changed its own appearance as the posting window moved towards it.

Now a mark is only ever a see-through wash of its own colour, and its depth follows how long the mark runs: a one-day holiday the deepest, a long school break the palest. The customer's calendar and your own Delivery Dates calendar read that depth from the same place, so a day wears exactly the same mark on both sides of the shop and in every month. Nothing is ever a solid block of colour any more, which also means the date number and the green posting pill always read on top of a mark, whether or not you post that day. Only the look changed: marking, removing, renaming and loading standard occasions all work exactly as before, and a mark still never adds or removes a posting day.

14 Sep 2026 - engine v83 (no database setup needed). Marking a holiday now tells the shop straight away. A day you mark on the Delivery Dates calendar is drawn on the customer's own calendar (engine v79 to v81). But saving a mark only ever sent your posting days to the cloud - it never sent the customer's page anything - so the marks reached the shop only by accident, whenever you next happened to save a product or a setting. Mark the holiday, open the shop, and nothing had changed. Found while you were looking for a holiday on the shop and finding none.

Now the same save does both. Marking a day, removing a mark, loading a set from the standard list, or renaming or recolouring one all republish the customer's page within a couple of seconds, exactly the way editing a product does. Nothing else about a mark changed: it is still only ever something to see while planning, it still never adds or removes a posting day, and a name you typed yourself still never leaves your phone.

One tap to catch up the marks you already have. The fix sends the marks from now on; the ones already on your calendar have never been sent at all, so open More -> Settings -> Storefront and tap **Publish now** once. After that the shop's calendar follows your marks by itself. Nothing to run in Supabase.

14 Sep 2026 - engine v82 (no database setup needed). A product's selling days are now marked on a little calendar inside the product's own screen - weekend, certain days, a season, or any mix of them. Until today a product could say only two things about *when*: a notice period, and one from-date and to-date for a season. You asked to set it freely instead, so the two date boxes have become a card you mark.

Open a product and tap the Availability card. It sits under Daily limit and stays folded until you tap its title, exactly like the product-text card you already know. Folded, its title tells you what is marked - "Every day", or "Sat & Sun", or "1-24 Dec 2026" - so the Products screen alone tells you a product's selling days without opening anything.

Inside, it is a month calendar and four gestures. Tap a weekday letter (M, T, W...) and every one of that weekday **in the month shown** is marked: tap S twice and the product sells on Saturdays and Sundays. Tap one day to mark just that day. Slide your finger across several days and the whole run is marked. Tapping or sliding over a day that is already marked takes it back. Everything you mark is added together - mark Saturday and Sunday, then a week in the middle, and the product sells on all of them.

Nothing carries over into the next month, which is the point of it: marks belong to the days you marked and no others, so a December-only set stays December-only and January starts with nothing marked. Page to another month with the arrows and mark that one too if you want to sell then. And a product whose card you have never opened sells on every posting day exactly as before, so nothing on your live shop moves until you mark it.

A period that crosses a month or a year is one mark with two ends you can stretch. The card lists your marks under the calendar; tap one and its **Starts** and **Ends** dates appear above it, and you can push either end as far as you like - a Sat & Sun run from 1 Dec to 4 Jan is one line, not two marks. Leave an end empty and the mark simply has no bound on that side ("from here on", or "up to here"). Type the two dates the wrong way round and they are put in order for you. A small x on a listed mark removes it whole.

On the order page, a day a product is not sold for is not a note - the product is simply not there. No

card, nothing to want and not have. If a customer already had it in their basket and then switches to a day it is not sold for, it leaves the basket and a short line says why. The one rule that keeps its note is the notice period: a product needing 14 days IS sold that day, it only has to be ordered earlier, so it stays on the menu reading "Orders close 14 days before the posting day - pick a later date" - the two are different questions, and only the second one hides anything.

Two things worth saying plainly. The old From/To season boxes are gone; a product that had them opens with that period already sitting in the card as one mark, so nothing you set before is lost. And a product whose selling days have all gone by keeps them, on purpose: your own rule is "if I have not indicated a selling date, it is not selling", so a mark you leave on says exactly that, and the way to put a product back on sale every day is to take its marks off with the x. No database setup, and posting days, capacity, value packs, ingredients, suppliers and everything else are untouched.

14 Sep 2026 - engine v81 (no database setup needed). A holiday you marked is now drawn on the shop's calendar exactly the way your own calendar draws it: a pale band across a run of days, a solid box for a single day. The marks were already reaching the shop; it was only their shape that differed, and a mark that looks different on the customer's page than on yours is a mark your customer has to learn twice.

On the shop, a marked day now wears the mark your own calendar gives it. The shop had been drawing each marked day as its own flat pale square, so a holiday running over a week read as a row of separate blocks where your own calendar shows one continuous shape. Now a holiday running over **several days** is a **pale see-through band** across those days - one unbroken rounded band per week row it crosses, in its own colour, and **deeper the shorter the run**, so a one-day holiday still stands out inside a long school break. A **single-day** holiday is a **solid box** in its full colour on its one day, a touch narrower than the band so a longer band still peeks out at its sides. The eight colours are your own calendar's eight, and the strength steps are the same three, so a marked day reads the same to your customer as it does to you.

The one place it gives way: a day you post for. Green is the colour the posting ring uses, and a solid box would swallow that ring - so on a day you post for, the day a customer can actually order, the box goes pale there and the **green ring stays the first thing the eye reads**. The mark is still there on that day, and the day is still named on a tap. This is the one guard on the shop's calendar, and it covers every mark colour rather than green alone, so no colour can ever hide the days that can be ordered for.

Nothing else moves. Tap a marked day - or, on a computer, rest the mouse pointer on it - and the same small bubble names it, e.g. "Malaysia Day"; tapping anywhere else puts the bubble away, and still nothing is listed under the grid. Today still breathes its soft glow. A run still stops at today rather than colouring days already gone, exactly as your own calendar leaves past days alone. And your own typed-in days still never reach the shop: only a day that is, by name and date, one of the built-in standard days can be published, exactly as before. No SQL, no schema change. The calendar the shop draws with is still a copy of your app's own, and a test now pins the two copies' mark rules together as well as their month grids, so a day can never again look like one thing here and something else there.

14 Sep 2026 - engine v80 (no database setup needed). Today glows instead of wearing a box, a holiday you marked is a soft tint you can tap to name, and the Orders screen's sideways date strip becomes the same month calendar the shop shows. Three small things, all pulling the same way: a calendar should read the same on both sides of the shop, and nothing drawn on it should look like something it is not.

On the shop, today no longer wears a little red rectangle. It always meant to be a ring around the date, but on a day you do not post for the app had no circle to draw that ring on, so it traced the day's number as a box instead - which is why it looked like a red square on the quiet days and only looked right on the days you post for. Today is now a **gentle glow** that breathes around the number, with no box at all, and it sits happily on a day you post for too: the **green ring** says you can order for this day, the soft brown glow says it is today. Under **prefers-reduced-motion** (a phone set to calm its animations) the glow holds steady instead of pulsing, so it is still there to find. The same fix reaches the app: the today marker on the Delivery Dates calendar, and inside every date picker, was a hard outline and is now the same soft glow.

A holiday you marked is now a soft wash, and its name comes out on a tap. The shop's calendar used to mark a standard day with a small coloured dot and then list the month's names in a "Holidays" line under the grid - which crowded the very line that line was there to keep clear, and made a holiday compete with the days a customer can actually order for. A marked day now wears a **pale wash** of its own colour over the whole day. The wash is deliberately neither a ring nor a filled dot, because a ring is what "you can order for this day" looks like, and the two must never be read for each other. Tap a tinted day - or, on a computer, rest the mouse pointer on it - and a small dark **bubble** above it names the day, e.g. "Malaysia Day". Tapping anywhere else puts the bubble away. The "Holidays" line under the grid is gone. Your own typed-in days still never reach the shop: only a day that is, by name and date, one of the built-in standard days can be published, exactly as before.

In the app the same holiday is named **beside the day** instead of in a list: open a posting day and, if it carries a mark, its name sits under the date in the mark's own colour. Nothing at all is drawn on a day you have not marked.

The Orders screen's date strip is now a month calendar. The row of date chips at the top of Orders grew a little longer every week. It is now the same **month calendar** the customer sees - one month at a time, with left and right arrow buttons either side of the month's name. The days you post for are ringed in **green**, and under each one is **how booked it is**: the same "3/12" the Home dials show, or **FULL** when the day has no room left. A day whose orders have already closed shows its count in **red**; a day already gone is dimmed but still opens, because you backfill and look back at old days. Every other day of the month is plain and cannot be tapped. Tap a green day and the app's view moves to it - that day is filled in on the calendar and its orders appear below - and tapping a day in the list below moves the calendar with it, so the two always agree. The arrows reach only the months your posting days fall in, so paging never strands you on an empty month.

The + New order card arrives folded, and leads with the calendar. It used to sit open on every posting day, which is not what that screen is for day to day. It now starts **shut**, showing just its title, "+ New order" - tap the title to open it, and tap the title again (or tap anywhere else on the screen) to fold it away. Opened, it reads in the order you think in: **the month calendar first**, the same one the shop shows, already sitting on the day you are looking at, so tapping another green day moves the whole screen before you have typed anything; **then the customer's details** - name, WhatsApp number, post or collect, a note; **then the items**, with "+ Add another item" and "+ Add order" at the end. It stays open after you add an order, so you can type the next one straight away, and coming back to the screen later starts it folded again.

The Edit pop-up now reads the same way. Its **Delivery day** control was a button you had to tap to reveal a calendar; the calendar is simply there now, already open on the order's current day. The pop-up then reads the day, then the customer's details, then the items - the same order as the card. Every day still to come is tappable, plus the order's own day even if it has passed, so an old order always shows where it is, and the soft notes re-read underneath the moment you pick a different day. Moving an order is unchanged in every way that matters: still one posting day per order, and capacity, the confirmation and the payment reminder, the track page and the WhatsApp messages are all exactly as they were. The button-style day picker the app used in these two places has been deleted along with its tests, because nothing calls it any more.

14 Sep 2026 - engine v79 (no database setup needed). The customer picks a posting day on a calendar, and so do you in the app. The shop used to show its posting days as a row of date chips laid out sideways. Now, under **Pick a posting day**, the customer sees a **month calendar** - one month at a time, with the arrow buttons either side of the month's name to look ahead or back. The days you are taking orders for are picked out in **green**; every other day of the month is plain and cannot be tapped. They tap a green day, and the day they chose is written out in words just underneath - **"Your posting day: Wed, 16 Sep"**. That is the one line of text below the calendar, exactly as asked: no dates floating outside the grid, and still **one posting day per order**. The first open day is already chosen for them when the page opens, so ordering can never be blocked by forgetting to tap. Nothing else about taking orders changed: a day you have cut off is still not shown at all, and a day that is full still greys out with a red **Sold out** badge and cannot be tapped.

Only the standard days you have marked reach the customer. If you have loaded days from the built-in list onto your own Delivery Dates calendar, those days now wear a small coloured dot on the customer's calendar,

and a short **Holidays** line under the grid names the month's ones (e.g. "16 Sep · Malaysia Day"). They follow the colour you gave them. A day **you typed yourself** - a birthday, a promo, a school break - never appears: only a day that is, by name and date, one of the built-in standard days can be published, so your own notes stay private to your phone. Mark nothing in the app and the shop's calendar simply carries no dots and no caption.

In the app, the same little calendar now stands wherever you name a date. Three places changed. In the **Edit** pop-up, **Delivery day** is no longer a drop-down list: it names the order's current day, and tapping it opens a small month calendar beneath it - every day still to come is tappable, plus the order's own day even if it has passed, so an old order always shows where it is. The **Order date** box in that pop-up is a calendar too, with a **Today** shortcut for an order you are typing in right now. And the **+ New order** card has gained a **Delivery day** picker at its top: tap another green day and the whole screen moves to that day, so the product list you are choosing from is always the one that is actually sellable for the day you picked. The soft notes that help you decide - the change/cancel window, a warning, a closed day, too little left - all still sit under the calendar, and none of them stop you.

The calendar expands in place under the button rather than opening in its own window, because the app's pop-ups share one layer and a calendar opened inside one would wipe out the Edit pop-up it was opened from; expanding in place also reads better on a phone. No database setup needed - nothing about how orders are stored, counted, bought for or messaged changed.

14 Sep 2026 - tapping a New Orders row now takes you all the way to the order (no new engine; the phones did not change). Tapping a row in the **New Orders** inbox opened that order's delivery date and stopped there. On a day holding a long list that is as good as landing on the wrong order: you then had to scan down the list to find the very order you had just tapped, and could easily open a neighbour instead. A tap now **scrolls the order to the middle of the screen and flashes it**, and it clears any status filter on the way so a narrowed list cannot hide it. The inbox's own hint line now reads "Tap a row to jump to that order on its delivery date and confirm it." Finding an order with the search box already did this - the two now share the same reveal.

14 Sep 2026 - the TNG QR is a WhatsApp payment code, and Settings now says so (no new engine; the phones did not change). The box in **Settings -> Storefront** that holds your TNG QR image link promised it was "shown on the customer's track page". It never was - and it should not be. Your QR reaches the customer **inside the WhatsApp confirmation and the payment reminder**, and your order page has always told customers you send it over WhatsApp; nothing on the shop has ever drawn a payment code. Only the wording was wrong. The box, the link it holds and the way you take payment are all unchanged, and the value still syncs between your phones through the storefront settings row - that row is how it travels, not a shop surface.

10 Sep 2026 - the back-office link is off the homepage (no new engine; the phones did not change). The homepage footer carried a small "Back-office login" line - the only public link to your app anywhere on the site - and it has been removed, so nothing on your public pages now points at /admin/. You open the app exactly as before: the **Home Screen icon** on your phone (open munchies.com.my/admin/ once in Safari, Share -> Add to Home Screen), and the login itself is unchanged - it still asks for your 4-digit PIN and your sign-in. The WhatsApp number under the footer link now also writes itself the Malaysian way (**+60 18-913 6389**) whatever you type in Settings, and it still follows **Settings -> Storefront** like the rest of the footer.

10 Sep 2026 - the app can no longer be handed a web page where it asked for a script (no new engine; the phones did not change). So the app can open without a connection, it keeps a background copy of itself. Whenever a file failed to download, that copy used to answer with the app's own **web page** - even when what had failed was a **script** or a **picture**. A browser handed a web page where it asked for a script cannot run the script, so whatever that script was meant to set up would quietly stop working, with nothing on screen to explain why. Now only a real page - opening or refreshing the app - may fall back to the stored copy; a script, a stylesheet or an image that fails simply fails, so a fault shows itself as a fault instead of hiding behind a page. Opening the app offline still works exactly as before. Your website has always run its app from /admin/, so it never had the leftover copy that caused the visible version of this fault elsewhere; the guard is in for good anyway, so it cannot start here.

10 Sep 2026 - the homepage's contact details now follow your Storefront settings (no new engine; the

phones did not change). The WhatsApp number, Instagram and Facebook links in your homepage footer were typed into the page itself, so changing them under **Settings -> Storefront** moved your order page and left the homepage still showing the old ones. The homepage now reads the same published settings as the order page, so the two always quote the same number and the same handles. Leave a box blank and the homepage keeps the link it already has, so an empty box never breaks a working link. (The homepage's contact **email** has no Settings box behind it - tell me if you want one.)

10 Sep 2026 - review photos load one at a time on the homepage (no new engine; the phones did not change). The **What customers say** section on your homepage used to fetch **every published review photo up front** - including the reviews a visitor never waited around to see. A photo is now fetched **as its review comes around**, one step ahead of time so it is ready before it slides in, and the reviews a visitor never reaches are never downloaded at all. On a phone that means less data and a quicker page, and it changes nothing about how the section looks or behaves.

A review photo is also shrunk a little harder before it is stored - to 1000 pixels across instead of 1600. The card on the homepage shows a picture no more than 340 pixels high, so the smaller copy looks just as sharp while being about **a third of the size**, which is kinder to the customer's mobile data and keeps your stored photos light. This applies to photos left from now on; the reviews already on your page keep the picture they were stored with. (Your homepage itself already loads its pictures as separate files - its page is only about 29 KB - so nothing needed changing there.)

v78 - The calendar's Add buttons now say what they do (14 Sep 2026)

On the **Delivery Dates** screen, **Mark an occasion** carried a small button reading "+ Add occasion". The name did not tell you it opens a ready-made list of Malaysia's holidays and fun days, all ticked and waiting - so you had to open it to find out. It now reads **"+ Load standard occasions"**, and the window it opens is titled the same.

Inside that window there were two buttons both called "Add": the big one at the bottom that files the days you ticked, and a smaller one in the "My own day" strip just above it, which files a day you name yourself (a birthday, a one-off promo). Reaching for "Add" and hitting the wrong one gave you "Type a name first", for a day you never meant to name. The smaller one is now spelled out as **"Add my own day"**, so the only plain Add in the window is the one that adds the days you ticked. Nothing else moved - the import picker, the Untick all / Tick all buttons, the heading tick boxes, the "My own day" box and the way a day is marked on the calendar are all unchanged. Engine v78, guide v78.

v77 - The translated-text card says what a blank line means (14 Sep 2026)

A wording fix on top of the fold-away card that arrived in v76. The greyed line in a product's translated-text card used to end "blank keeps English"; it now finishes **"if blank, it will be filled with English"** - stating what the customer gets, rather than what the line does. The card's own sub-line ends the same way.

Nothing else moves. Delete what is in a line and the greyed suggestion and its arrow come back exactly as before; the card still folds on a tap outside, the right arrow still takes the words, and the circular-arrow button still re-translates that one line. Engine v77, guide v77.

v76 - The translated text is one quiet card that folds away (14 Sep 2026)

Every product's translated-text corner was the busiest part of the screen: always open, eight lines each with its own "Translate this one" button, two "Fill all" buttons above them, and an "auto" tag in a word you had to decode. It is now **one quiet card that starts shut and stays shut** - it is setup, not daily use. Tap the title to open it; tap anywhere outside to fold it back.

Opened, each empty line shows its translation **already worked out** as the app's standard greyed suggestion, with the -> at its right edge - the same gesture as every other suggested box in the app. Taking it replaces the

arrow with a **circular-arrow button** that re-translates just that one line, in place. Typing your own words hides the circular-arrow button (the line is yours); a blank line keeps the English. The two "Fill all" buttons, the eight "Translate this one" buttons and the "auto" tag are gone.

A product you never open the card for still reads in all three languages on the shop - the automatic translation still runs after every save and publish. Engine v76, guide v76.

v75 - The green glow on a jumped-to order waits for you (14 Sep 2026)

Tapping a row in the **New Orders** inbox (or a result in **Find an order**) takes you to that order's delivery day and lights the order up green. That glow used to fade after under two seconds - no time at all while your eye is still travelling down a long day, so you were left hunting for a row that was no longer lit. The glow now **keeps pulsing until your finger arrives on the row**, and ends the moment you reach it.

If your phone is set to reduce motion, the same green ring holds steady instead of pulsing, so it still stays until you reach the row. Engine v75, guide v75.

v74 - The little arrow in an empty box keeps working (14 Sep 2026)

Engine v73 added the small right-arrow at the right edge of a box showing a greyed example: press **->** on a keyboard, or tap the arrow, and the example drops in as real text. On a phone the tap worked the first time and then stopped - once you had tapped into any box, tapping the arrow on the next one did nothing.

Two things caused it, both about the box moving under your thumb. Taking a suggestion focused the box, which opened the phone's keyboard and shrank and panned the page, so the arrow was no longer where the finger landed; and the strip that counted the tap was narrow and measured only as a position on the page. Taking a suggestion **no longer opens the keyboard**, the strip is about **half again as wide**, and a tap is judged against the box's own edge as well as its place on the page - either reading can take the tap, so the rule can only accept a tap the old one refused, never refuse one it took.

The drawn arrow is kept exactly as it was - same size, same place - a tap on the greyed example itself is still an ordinary tap that puts the cursor there, and the sign-in and cloud boxes still carry no arrow. Engine v74, guide v74.

v73 - Change windows, moving an order, Policies, and a one-tap suggestion (14 Sep 2026)

An order is not refundable, but it can be moved to another posting day - and a customer may ask until a set number of days before the day it is due. That number is now **per product**: a new **"Changes or cancellations (days before delivery)"** box sits beside the other date rules when you edit a product. It only ever tells the customer - **blank** means no window is stated, an explicit **0** states no advance limit, and neither ever blocks you, who move every order by hand. The customer reads the sentence on the product's card and again on their green **"Order received!"** card, and when one order holds products with different windows they see the strictest (largest) figure, so a mixed basket gives one clear number. The sentence is written in **English / Mandarin / BM** with the rest of the page. The order's own track card stays silent - it is read days later, when the window may have changed.

An order's posting day can now be **changed in the app** instead of deleted and re-typed - a deletion reads as a cancellation. The **Edit order** pop-up gains a **"Delivery day"** select of every day still to come; saving moves the whole order across in one step, and heals an order whose rows had somehow ended up on different days. Soft notes under the select tell you the order's own window, warn when the new day falls inside it, say when that day has already closed, and say when it is short an item - none of them block you. The destination day's availability is checked, and the customer's tracking page is republished so it shows the new day. The old date is left for the **Del** button on the Deliveries screen to clear.

A **Policies** box in **Settings -> Storefront** holds your own cancellation and refund wording: type it once in English, translate it to Chinese and Bahasa Malaysia with the same free machine translation the product text uses, and it is drawn on your shop under **Track your order**, in the visitor's language, with the line breaks you typed. Blank hides the section.

And app-wide: wherever an empty box shows the app's greyed recommendation, the **right arrow** - or a tap on the small arrow drawn at the box's right edge, for a phone with no arrow key - **accepts it as real text**, the same gesture as an AI chat prompt. It is offered only where the grey text really is a value: the Supabase email, key and password boxes and the sign-in fields carry no arrow, so a made-up login can never be accepted by accident. Engine v73, guide v73.

v72 - A sold-out note reads in the customer's language (10 Sep 2026)

Your order page switches language the moment a customer taps it, but two corners of it had stayed English whichever language they chose: the short note on a product that **cannot be ordered for the posting day they picked** (that it only opens from a certain date, only runs up to one, or that orders close so many days ahead - with the date written in the page's language), and the notes that appear above the menu when the page **has to change a basket** - something that just sold out, or a quantity trimmed to what is actually left.

Both now read in **English / Mandarin / BM** along with everything else. The reason a product is closed is handed to the page as a plain fact - which kind of rule, and the day or number of days behind it - and the page writes the sentence itself, so no English can slip into a Chinese or Malay page. An English visit reads word for word as it did. Engine v72, guide v72.

v71 - The order page's Mandarin / BM buttons answer instantly (10 Sep 2026)

On your order page, tapping **EN / Mandarin / BM** used to reload the whole page. That re-downloaded the page and fetched the menu, the "only N left" numbers and your saved order-page settings all over again, so each tap sat there for a moment before anything changed - and the slower the customer's connection, the longer the wait. A reload could also wipe a basket that was already half filled.

Now the order page switches **in place**, exactly like your homepage already did: the words change the moment they tap, and nothing is downloaded again. Whatever the customer has already done is left exactly as it was - the items in their basket, the posting day they picked, the name, WhatsApp number and postal address they typed - and if they are looking up an order, that card re-reads in the new language too. Nothing to set up.

v70 - An order keeps the name and price it was sold at (10 Sep 2026)

Until now an order did not remember what it was sold as - it just looked the product up afresh each time it was shown. So if you renamed a product or changed its price, **every past order changed with it**: last month's orders, the labels you printed, and even the amount a customer saw on their own tracking page all silently moved to today's wording and today's price. And if you later deleted a product, its orders fell back to "(deleted product)" with no price at all.

Now each order **freezes the name and price at the moment it is taken** - whether it came from your shop or you typed it in yourself - and every message, the tracking page, customer spend totals and the Home estimate read that frozen record. Rename or reprice a product and the past stays as it was; a product you delete later still shows what it was that someone bought.

Two careful edges: if you deliberately **swap a line to a different product** when editing an order, that line picks up the new product's price (you changed what is being sold), while a line you leave untouched keeps the price it was sold at. And orders taken **before** today are stamped just once with what they were already showing, so they stop moving from here on - a line whose product is gone, or has no price set, is left alone rather than guessed at. Engine v70, guide v70.

v69 - Either place updates the customer (10 Sep 2026)

Yesterday's fix put a customer's name and number back in one place - but it made the **customer card** the side that always won, so if the card and an order disagreed, the card's name was the one shown. That is no longer the rule: now it is simply **whichever you edited last**. Correct a customer's name with **Edit** on one of their orders and that is the name the list, the history pop-up and the "Hi {name}!" greeting use; open their card, type the name and save, and the card's is the one used. Either place updates the customer - neither is the boss. They normally read the same anyway, because each save writes through to the other side; this only decides the rare case where an old copy and a newer one are both sitting there. Nothing to set up; the phones pick it up with the next sync.

v68 - One name and one number per customer (10 Sep 2026)

A customer's name and WhatsApp number used to be held in two places at once - on their orders, and on their customer card - and the two did not always agree. If you opened a customer and typed their name on their card, the customer list could carry on showing the old one, because it read the name off the order. And renaming someone who has **no WhatsApp number** on file could lose them: the name is what ties a numberless customer to their history, so the new name slid off their orders and appeared nowhere at all.

Now the name and number you save on a customer are the ones that count, and fixing them once fixes them everywhere: saving their card writes the name and number onto **every order they have**, so the labels, the WhatsApp messages and the customer list all follow. It works the other way round too - correct a wrong number with **Edit** on an order and that customer's card updates as well, so the two can never drift apart again. Correcting a number carries that customer's bring-a-friend credits across with it, and renaming someone who has no number no longer loses them.

Leaving a box empty means "leave this as it is" - it never wipes a number your confirmations depend on. And if you renamed anyone before today and it never stuck, the app puts their orders right the first time this version opens. Engine v68, guide v68.

9 Sep 2026 - the wish list's automatic email went live (no new engine; the phones did not change). Adding a wish on **More -> Software wish list** now emails the **full wish list** - every wish, ticked or not, newest first, with the app version and the date - to the developer address(es) you set under Settings -> **Website & developer**, all on its own, whenever the phone is signed in and sharing data. Before today that automatic email needed a behind-the-scenes service that was not switched on yet, so only the **"Email the full wish list"** row sent it by hand; that row is still there as a backup any time.

v67 - Naming a "No name" customer (10 Sep 2026)

A customer who reached you without a name - a storefront order that carried none - used to stay **"No name"** in your customer book even after you opened them and typed a name. Now it sticks: open them, tap **Add details**, type the name (and the pet's name, what they like, and so on) and save, and their row shows that name straight away - in the list, the finder and the 'Hi {name}!' WhatsApp greeting. A tidy-up; nothing else about your customers changes. Engine v67, guide v67.

v66 - Products that translate themselves + Draft / On the shop / Hidden (9 Sep 2026)

Product text now translates itself into Mandarin and Bahasa Malaysia. When you save or publish a product, the app quietly fills Chinese and Malay versions of its name, description, selling unit and feeding tip - machine-made from your English, free, whenever you're online. Each translated box is tagged **"auto"**. Type over any box and it becomes yours, never overwritten again; and in **Products -> Edit** you can tap **Fill all Mandarin** /

Fill all Bahasa Malaysia to fill a whole language now, or **circular-arrow button Translate this one** for a single line. If you later change the English, the box is re-filled to match; delete the English and the translation is dropped. Shoppers reading in that language then see the product in their own words on your order page.

Products now have three states, shown as three lists - **On the shop** (live, orderable), **Draft - not on the shop yet** (a brand-new product starts here, fully built but not for sale), and **Hidden - taken down** (off the menu, history and recipe kept). Publish a draft to put it on the shop; Hide a live product to pause it. Drafts and Hidden products never appear on the storefront and can't be ordered, so building a new recipe can't accidentally go live.

The "Copy follow-up" referral message now has a language choice. In a customer's profile, before you copy the follow-up, pick **English / Mandarin / BM** - the whole check-in (greeting, the product's translated name and feeding tip, the scheme pitch, the link) is written in that language, ready to paste into WhatsApp. English stays the default. No SQL this version.

v65 - Reviews to publish, one at a time (9 Sep 2026)

More -> Reviews now walks you through the reviews **one at a time**, drawing each exactly as it will look on your homepage (photo, stars, message, name). The ones waiting for you to publish come first; **Publish** puts it on the homepage, **Take down** hides it again, **Delete** removes it for good - and every action moves you on to the next review. It deliberately does not auto-play, because you are deciding, not watching: the review in front of you stays until you move it with the arrow buttons, the dots, or a swipe.

Two places now tell you when reviews are waiting on you: **Home** shows an "N new review(s) to publish" card near the top (tap it straight into Reviews), and the **Reviews** row on More gains a green "N waiting" pill. The count refreshes every ~45 seconds while Home is open. No SQL this version.

v64 - Your site in Mandarin / English / Bahasa Malaysia + a website credit (9 Sep 2026)

Your **homepage and ordering page now speak three languages.** Tap **English / Mandarin / BM** at the top of either page and every word - navigation, product cards, the delivery-day and order form, even the track page - switches over on the spot, and each phone remembers the language it picked. Because the choice lives in the browser, one shop link serves every customer in their own language. Each product can also carry a Chinese or Malay **shop name** in Products (e.g. Jerky Ayam, or a Chinese name): it shows on the product card while the order and your records keep the English name.

The homepage **reviews** section is now a **swipeable carousel** - drag left or right through the reviews you have published, or tap the dots - and if your website was built for you, **Settings -> Website & developer** lets you name who made it. A small **"Website by ..."** credit then appears at the bottom of your homepage and order page, and **More -> About** adds a WhatsApp and an email row to reach them. That same developer is who the **software wish list** emails: add a wish on More and the whole list is sent to them automatically (quietly - if it can't send, a gentle note points you to the always-works **"Email the full list"** row underneath). The credit and the email links stay hidden until you type a name and an email in Settings.

v63 - Customers tab, wish list, holidays fix (8 Sep 2026)

Customers is a real tab of its own, swapped with **Purchase Order** (now under **More -> Purchase Order**). The tab stays your automatic customer list - who ordered, how much, their favourite - and now every person can carry a **profile** (the **Edit** button in their history pop-up): their pet's name and a small photo (auto-shrunk to a thumb), what they like, what to avoid, and a note. Profiles save into a synced customers collection, so both phones know the same people, and they build up quietly for a future AI chat about your customers. A **finder** box at the top of the tab filters the list as you type - name, number, pet's name, likes, notes - and says how many match, just like "Find an order" on Orders. A saved profile shows as a paw or photo beside the name and a small line under the row.

On **Home**, the "Upcoming holidays" card is fixed: it used to vanish whenever nothing was marked on the Delivery calendar. It is now always there - up to three marks at a glance, a scroll for any more, and a friendly empty state that still opens the calendar so you can add days.

On **More**, a small green **Engine v63** pill (in the header card) shows which build this phone runs, a **Software wish list** sits at the bottom and behaves exactly like your to-do list (add, tick, reword, remove - a tick stays ticked and never touches the weekly routine), and **Full change history** opens this history as a PDF kept on your website. No SQL this version.

v62 - "Not sharing" strip + read-only View (8 Sep 2026)

When a phone is not on the shared cloud - shared data off, never set up, or signed out - a thin amber strip at the very top of every screen reads **"Not sharing right now"** with a one-line reason and a **Fix it** tap straight to Settings -> Shared data. It never locks the app, and it disappears on its own the moment sharing is back on. (Set up once per phone: no SQL - the shared-data setup you already did covers it.)

Also, every backup copy in Settings -> **Backup & safety** now has a **View** button: a purely read-only look inside that copy - its orders grouped by delivery date (customer, product, quantity, status), the product price list and ingredient stock at that time. Looking at a June price confirms the detail without moving today: no rewind, nothing written, ever.

v61 - Cloud backups (8 Sep 2026)

Every time the app opens while signed in, it quietly saves a full **Daily** copy (first open each day), a **Weekly** copy (Mondays) and a **Monthly** copy (the 1st) to your own Supabase cloud, keeping the newest 7 daily / 4 weekly / 3 monthly. A "Back up to cloud now" button, and one "Before restore" copy saved before every restore, stay until you delete them. Each copy can be **Restored** (steps the phone - and the shared cloud - back to that copy; a "Before restore" copy is always saved first, so it is never one-way), **Downloaded** as furkidz-backup-YYYY-MM-DD-KIND.json (which re-imports like an Export), or **Deleted**. Lives under More -> Settings -> "Backup & safety". The copies never contain the per-phone app-login email/password or the app password. Setup: run supabase/backups.sql once.

v60 - Homepage customer reviews (8 Sep 2026)

Customers can leave a **What customers say** review on the homepage - name, 1-5 stars, a message in English / Mandarin / Bahasa Malaysia, and an optional photo. Every review lands *unpublished* in More -> Reviews, where **Publish** shows it on the homepage, **Take down** hides it, and **Delete** removes it - nothing becomes public until you tap Publish. Reviews appear on the homepage only. Setup: run supabase/reviews.sql once.

v59 - Occasion import tidy-up (8 Sep 2026)

Added **World Animal Day** (4 Oct) to the fun/pet-day import, plus whole-group ticking - tick a heading box, or use **Untick all / Tick all**.

v58 - Occasion import (8 Sep 2026)

A one-tap **+ Add occasion** import of Malaysia's days plus fun days (pet, people & kindness days, and more), and a **My own day** quick-add.

v57 - Private notes per product ingredient (8 Sep 2026)

Each ingredient line in a product can carry a short private description *for that product only* (e.g. which cut or brand of chicken that product uses). Typed and seen on the Products screens only - never on the shop, a label, or the product cards.

v56 - "Keep at least" reserve (8 Sep 2026)

Ingredients can set a **Keep at least** level. The order list tops an ingredient back up to that level in whole packs when your stock drops more than 10% under it, and low items ride along even when today's posting run doesn't use them. The Ingredients screen shows a red low-stock strip.

v55 - Time & length units (7 Sep 2026)

min/hr and **cm/m** are pre-loaded in Units (under More), like g/kg - so oven or dehydrator times and treat sizes can be measured and converted (1 hr = 60 min, 1 m = 100 cm). Stock is guarded so a time or length unit can never distort a gram count.

v54 - Ingredients on hand (7 Sep 2026)

Ingredients now have **On hand** stock. The PO buys only what you don't have (rows you already have drop out), "Bought" on a saved list adds its packs once, and marking an order **Preparing** (the engine step the bakery calls "Baked") subtracts its ingredients (undo restores them).

This history covers the two-phone cloud era (v54+). Earlier versions (pre-v54) were never recorded version by version, so they are not listed here rather than invented. Each new version is added here as it ships.